2027 考研 · 计算机专业基础

408 系统知识手册

从基础概念走向机制、算法与计算：按四科知识依赖重新组织讲解，保留关键表述、适用条件、步骤、例子与对照。系统学习与针对性纠错分开，阅读时可以交叉参照。

完整内容 · 2026-11-06

绪论与程序基础

数据结构 · 3 条

从数据关系到存储实现 001 先分清描述对象、逻辑关系和具体存储，再选择操作与算法。

逻辑结构、存储结构与数据运算

数据的逻辑结构描述数据元素之间的关系，不依赖具体存储方式。集合中元素之间仅有同属一个集合的关系；线性结构为一对一关系；树为一对多关系；图可表示多对多关系。

非空线性结构有唯一首元素、唯一尾元素；首元素没有直接前驱，尾元素没有直接后继，其余元素各有唯一直接前驱和直接后继。空结构需单独处理。

存储结构把元素及其关系映射到内存：顺序存储通过位置相邻体现关系，链式存储通过指针或游标体现关系；索引存储另建索引表，散列存储通过散列函数计算位置。

判断一个名称是否包含存储约定

- “栈、队列、线性表”只规定逻辑关系或运算限制；“顺序栈、链队、双向链表、静态链表、线索树”已经指定某种存储实现。
- 顺序存储支持按下标随机访问，但要连续空间；链式存储允许离散分配，结点额外保存链接信息。
- 索引能加快定位，但占附加空间，更新数据时也可能维护索引；散列查找通常高效，但必须解决冲突，不能将“散列”直接理解为绝对无冲突。

结构体、指针和动态结点 002 把类型、变量、地址和地址中的内容分别看清。

类型与地址

结构体可以把不同类型的成员组织成一种复合数据类型。typedef 是类型别名，不会创建变量；#define 是预处理阶段的文本替换，二者的作用不能互换。

指针变量保存地址。若 p=&x，则 p 指向 x，*p 访问 x 的内容；&x 是取地址。把地址赋给指针和给指针所指对象赋值是两个不同操作。

结构体对象使用“.”访问成员，结构体指针使用“->”访问成员；p->data 与 (*p).data 等价。解引用前必须保证地址有效。

链表结点的构造

链表结点由数据域和链接域组成。声明指针只得到保存地址的变量；malloc 申请结点空间后，指针才可以指向这片空间。分配失败时返回空指针，不能继续访问成员。释放结点前应保存后继地址，释放后不能再通过原指针读写结点。

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;
Node *p = (Node *)malloc(sizeof(Node));
if (p != NULL) {
    p->data = 0;
    p->next = NULL;
}
```

算法特性与复杂度分析 003 复杂度衡量规模增长带来的资源变化，不等于某次运行的秒数。

算法的五个特性

- 有穷性：执行有限步骤后结束；确定性：每一步含义明确。
- 可行性：每一步可以通过有限基本操作完成。
- 输入可以有零个或多个，输出至少有一个；输出可以通过返回值、参数或状态变化体现。

从基本操作次数估计增长阶

时间复杂度由基本操作执行次数 T(n) 的增长阶刻画。保留最高阶项并忽略常数因子，例如 $3n^2+10n+8$ 属于 $\Theta(n^2)$，也常记为 $O(n^2)$。必须说明讨论的是最坏、平均还是最好情况，不能把这三种情况混为一谈。

顺序执行的两段取增长更快的一段；嵌套执行则累计内层次数。只有各层次数独立时，才可直接相乘；若内层上界随外层变化，应求和。例如内层执行 1、2、…、n 次，总量为 $n(n+1)/2$。

循环变量倍增或缩减通常产生对数层数；递归需同时考虑分支数、子问题规模与合并工作。空间复杂度通常统计额外辅助空间，递归调用栈也应计入。

常见增长顺序

对足够大的 n，常见次序为：$1 < \log_2 n < n < n\log_2 n < n^2 < n^3 < 2^n < n! < n^n$。比较时关注渐近增长，不能用某个很小的 n 代替结论。

线性表

数据结构 · 5 条

线性表的抽象定义与操作接口 004 先规定元素序列和操作语义，再讨论顺序表或链表如何实现。

线性表的定义

线性表是 n 个相同数据类型的数据元素组成的有限序列。n 是表长，n=0 时为空表；逻辑顺序由前驱与后继关系确定，并不意味着物理地址也相邻。元素类型可以是基本类型，也可以是结构体。

操作与参数的含义

操作	语义与注意事项
InitList、Empty、Length	初始化空表、判断是否为空、返回有效元素个数。容量与表长是不同概念。
GetElem、LocateElem	前者按位序取元素，后者按给定值查找；位序通常从 1 开始，数组下标通常从 0 开始。
ListInsert、ListDelete	插入位置允许位于当前表尾之后；删除位置必须已有元素。删除通常通过输出参数返回被删值。
Clear、Destroy	清空使有效元素数变成 0；销毁还要终止对象或释放其资源。动态空间必须按具体实现释放，不能只改 length。
PrintList	按照逻辑顺序访问并输出所有有效元素。

顺序表的寻址、插入与删除 005 连续存储带来随机访问；中间插删则需要移动元素。

连续地址与长度管理

顺序表包含连续数组和有效长度 length。按位序访问第 i 个元素时，数组下标为 i−1，地址可由起始地址加固定偏移直接算出，所以按位访问为 O(1)。顺序存储本身不保证按值查找快，未排序时仍需顺序扫描。

静态分配的容量通常固定；动态分配允许扩容，但重新分配和搬移数据可能花费 O(n)。初始化 length=0 不代表分配的每个单元都成为有效元素。

插入与删除的方向

在第 i 位插入时，从尾向前搬移，避免覆盖尚未移动的元素，需移动 n−i+1 个元素；删除第 i 位后，从前向后补空位，需移动 n−i 个元素。插删平均和最坏时间均为 O(n)，在表尾操作可为 O(1)。

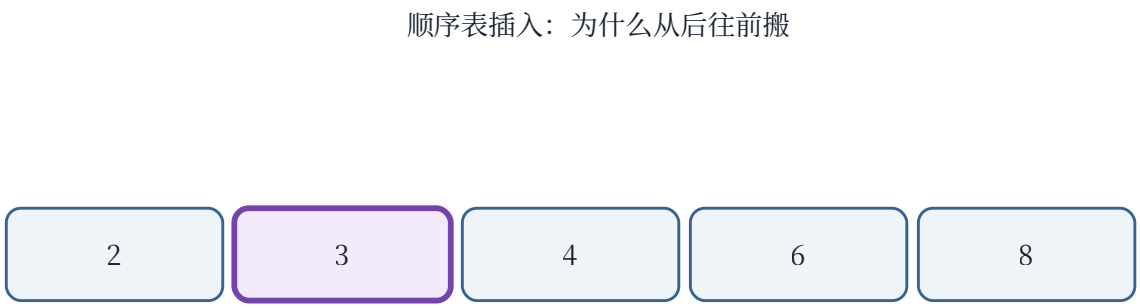
```
bool insert(List &L, int i, Elem e) {
    if (i < 1 || i > L.length + 1 || L.length == L.capacity) return false;
    for (int j = L.length; j >= i; --j) L.data[j] = L.data[j-1];
    L.data[i-1] = e;
    ++L.length;
    return true;
}
bool erase(List &L, int i, Elem &e) {
    if (i < 1 || i > L.length) return false;
    e = L.data[i-1];
    for (int j = i; j < L.length; ++j) L.data[j-1] = L.data[j];
    --L.length;
```

```
    return true;
}
```

按值查找与按位查找

顺序查找逐个比较，第一个等于目标值的元素可返回其位序，失败返回约定标记。未排序表按值查找通常 O(n)；已经有序且支持随机访问时，可以采用折半查找 O(log n)。

图解与逐步推演：顺序表插入：为什么从后往前搬



单独查看图片

顺序表插入：为什么从后往前搬

查看完整步骤（也可离线阅读）

1. 原表：在下标1插入3；先保证容量足够，当前长度4。
2. 移动8：从后向前搬移，避免覆盖尚未搬走的数据。
3. 移动6：把下标2的6搬到下标3。
4. 移动4：把4搬到下标2，下标1空出。
5. 填入3：写入3，长度增加为5；共搬移3个已有元素。

单链表的头指针、头结点与建表 006 链表以链接关系表达逻辑顺序，头指针和头结点不能混称。

头指针与头结点

头指针保存链表入口地址；头结点是可选的附加结点，不计入有效表长。带头结点时，首个数据结点为 L->next，空表条件通常为 L->next==NULL；不带头结点时，空表条件为 L==NULL。若接口允许 L 本身为空，访问 L->next 前还须检查 L。

结点可以分散存放，next 指向逻辑后继，末结点 next 为 NULL。随机访问第 i 个结点通常不能由地址公式得到，必须沿链查找。

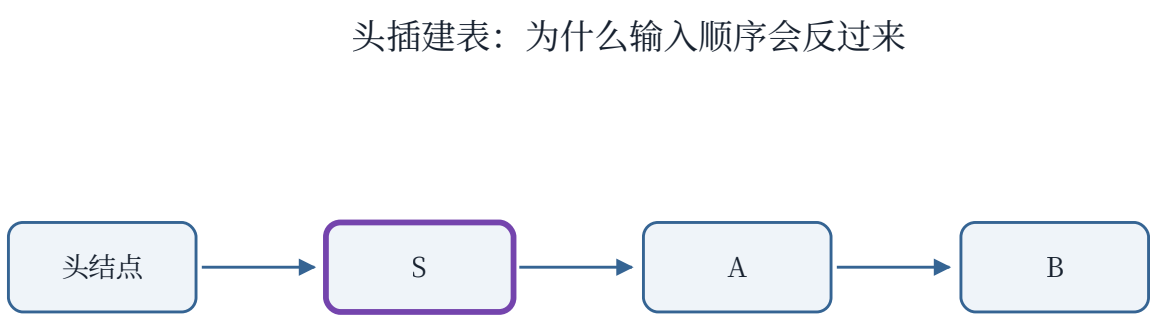
两种建表方法

方法	链接步骤	得到的逻辑顺序
头插法	新结点 s->next=L->next，再令 L->next=s。	与输入顺序相反；每次插入 O(1)，总计 O(n)。
尾插法	保持尾指针 r；令 r->next=s，随后 r=s，末尾置空。	与输入顺序相同；有尾指针时每次插入 O(1)，总计 O(n)。

空间利用与遍历

单链表增加指针域，存储密度低于只存数据的紧凑顺序表；优势是结点空间可按需申请、插删无需整体搬移。能否减少实际耗时取决于定位方式和实现，不能仅凭二者都为 O(n) 就断言某种结构总是更快。

图解与逐步推演：头插建表：为什么输入顺序会反过来



单独查看图片

头插建表：为什么输入顺序会反过来

查看完整步骤（也可离线阅读）

1. 带头结点的原链：head指向不存普通数据的头结点，原表A→B。头指针与头结点是两个不同概念。
2. 新结点S先接原首元：执行S.next=head.next，使S连到A。此时还没改head.next，原链仍完整。
3. 再让头结点指向S：执行head.next=S，得到S→A→B。对输入1,2,3连续头插时，最终数据次序会是3,2,1。

单链表的查找、插删与资源释放 007 把“定位结点”的代价与“修改链接”的代价分开计算。

查找与插删的边界

- 按位查找从首个数据结点开始累计位序，按值查找逐个比较 data；一般均为 O(n)。查找失败返回 NULL，不能继续解引用。
- 在已知结点 p 之后插入 s：先 s->next=p->next，再 p->next=s。若颠倒顺序，可能丢失原后继或形成自环。
- 删除 p 的后继 q：先确认 q 存在，保存 q=p->next，令 p->next=q->next，最后 free(q)。已知前驱时改链 O(1)，但按位删除仍包含查找前驱的 O(n)。

从表中摘除一个结点

表长统计只数数据结点，带头结点时从 L->next 开始。清空带头结点的表，应逐个释放数据结点并令 L->next=NULL；销毁整个表还应释放头结点，并按接口将头指针置空。

逐个释放时先保存 next，再释放当前结点；不能 free(p) 后再读取 p->next。把一个指针置 NULL 只会改变该指针变量，不会自动释放原来指向的空间。

```
Node *q = p->next;
if (q != NULL) {
    Elem removed = q->data;
    p->next = q->next;
    free(q);
}
```

图解与逐步推演：单链表插入：先接后继，再改前驱

单链表插入：先接后继，再改前驱



单独查看图片

单链表插入：先接后继，再改前驱

查看完整步骤（也可离线阅读）

1. 原链 A → B：已找到前驱p，先分配新结点s。这里只演示定位之后的链接修改。
2. s.next = p.next：先保存原后继，使s指向B；p仍指向B。
3. p.next = s：再让A指向S，形成A→S→B。若先覆盖p.next再用它找B，就会丢失原后继。

双链表、循环链表与静态链表 008 不同链接方式改变了可直接访问的邻居和判空约定。

双链表的双向维护

双链表每个结点含 prior 和 next，已知结点时可直接访问前驱、后继。在 p 后插入 s，应设置 s->next=p->next、s->prior=p，并修正原后继的 prior 和 p->next；非循环表遇到尾结点时，原后继可能为空，须先判空。删除 q 时要同时修正 q 前后两侧链接，再释放 q。

循环结构的终止条件

结构	空表与结束判定	操作特点
带头结点循环单链表	空表 L->next==L；遍历回到头结点结束。	末结点 next 指向头结点，没有普通的 NULL 终止标记。
只设尾指针的循环单链表	尾指针 r 的 next 给出头结点或首结点，取决于是否带头结点。	定位表头与表尾可 O(1)，但删除尾结点仍可能需寻找前驱。
带头结点循环双链表	空表时 head->next==head 且 head->prior==head。	双向均可循环遍历；正常结构中 p->next->prior==p、p->prior->next==p。

静态链表的游标

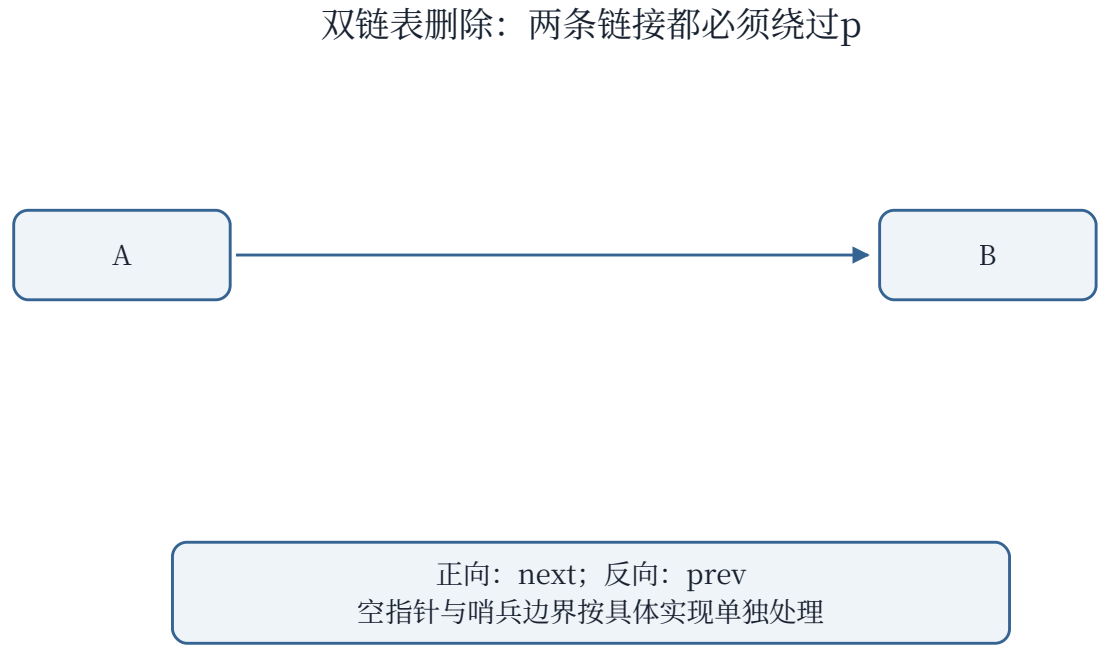
静态链表在预先分配的数组中存结点，每个单元保存 data 和 next。next 存的是数组下标（游标），而非普通指针地址；逻辑后继由游标决定，数据不必按逻辑顺序紧邻。

终止标记与空闲链表约定由实现决定，可用 −1，也可用保留下标 0；不能把不同教材的标记混用。插删通常只修改游标，不整体移动元素；但数组容量仍有界，需维护空闲单元。

静态链表的物理次序与逻辑次序

若数据逻辑序列为 1→2→3，可以让值 1 存在 a[1] 且 next=3，值 2 存在 a[3] 且 next=5，值 3 存在 a[5] 且 next=0。这里 0 作为结束标记，逻辑遍历按 1、3、5 这些下标链接，与数组物理相邻位置不同。备用单元可另串成空闲链，插入时取空闲单元，删除后归还空闲链。

图解与逐步推演：双链表删除：两条链接都必须绕过p



单独查看图片

双链表删除：两条链接都必须绕过p

查看完整步骤（也可离线阅读）

1. 原双链表：假设p不是头尾哨兵，前驱A和后继B都存在；反向链接为B.prev=p、p.prev=A。
2. 先改前驱的后继：执行p.prev.next=p.next，即A.next=B。不要先释放p，否则再访问p的链接是错误的。
3. 再改后继的前驱：执行p.next.prev=p.prev，即B.prev=A；两个方向都已绕过p，随后才能安全释放结点p。

栈与队列

数据结构 · 3 条

栈与队列的运算限制 009 它们都是线性结构，区别在于允许插入和删除的位置。

栈与队列的基本语义

结构	插入与删除	常用操作
栈	同一端进行插入、删除；栈顶可变，栈底相对固定，后进先出。	Push、Pop、GetTop；GetTop 只读取，不删除。
队列	队尾入队、队头出队，先进先出。	EnQueue、DeQueue、GetHead；GetHead 不移动队头。

链式实现

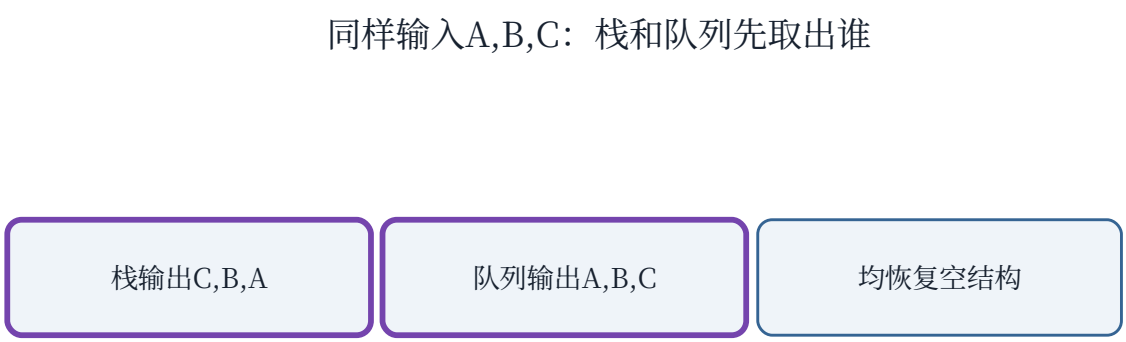
链栈通常把单链表表头作为栈顶，插入与删除栈顶为 O(1)。带头结点与不带头结点的空栈判断不同；出栈时先保存数据和后继，再释放原栈顶。动态链栈没有数组固定容量的“满”，但仍可能内存分配失败。

链队同时设 front 和 rear，分别定位队头入口与队尾。带头结点时，空队通常 front==rear，且头结点 next 为空；不带头结点时，空队通常 front==rear==NULL。入队接到 rear 后并更新 rear；删除最后一个数据结点后，还必须同步调整 rear，防止悬空。

初始化、清空与销毁

初始化建立与约定相符的空结构；清空删除有效元素但可保留头结点；销毁释放全部动态结点。初始化时“已分配头结点”与“两个指针均为空”属于不同实现，不能组合成同一段操作。

图解与逐步推演：同样输入A,B,C：栈和队列先取出谁



单独查看图片

同样输入A,B,C：栈和队列先取出谁

查看完整步骤（也可离线阅读）

1. 按A,B,C依次加入：先固定三次加入都完成，再比较后续取出次序，避免把交错入栈出栈情况混进来。
2. 第一次取出：栈只在栈顶弹出；队列从队头取出，加入在队尾。
3. 第二次取出：某一次相同不表示两者规则相同。
4. 全部取完：只描述本例“全部加入后全部取出”；栈允许输入与输出交错，可能产生其他合法序列。

顺序栈与共享栈的指针约定 010 每个判空、判满和读写顺序，都要由 top 的定义推导。

top 指向当前栈顶元素

容量为 M、数组下标 0…M−1、top 指向当前栈顶时：空栈 top=−1，满栈 top=M−1，元素数 top+1。入栈先检查未满，再执行 data[++top]=x；出栈先检查非空，再执行 x=data[top--]。逻辑出栈不要求清除原数组字节。

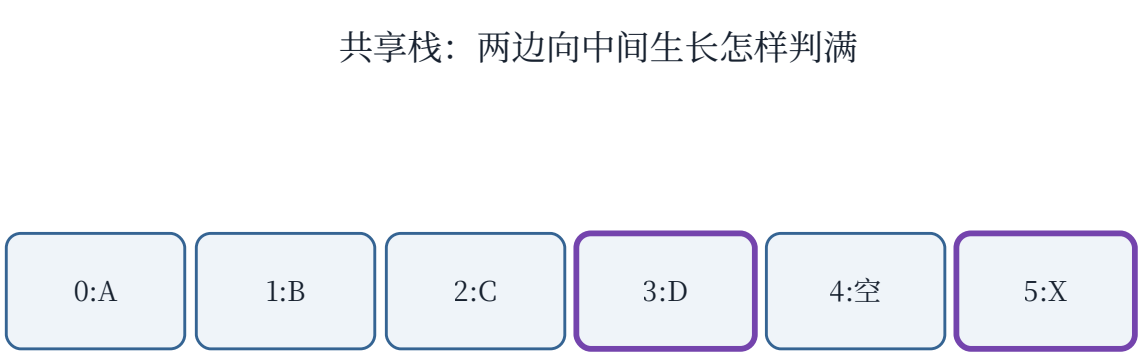
若 top 指向下一空位，空栈可为 top=0，入栈可写 data[top++]=x，出栈可读 data[--top]。指针型 base、top 的差表示元素数时，也要确认 top 是否指向下一空位。前置、后置自增表达不同取值时机，不能据此断言其中一种在所有编译环境下效率都更高。

两个栈共享一个数组

令 0 号栈由左向右增长，1 号栈由右向左增长，并令 top0、top1 指向各自栈顶。初始化 top0=−1、top1=M；两者相邻，即 top1−top0=1 时共享空间已满。0 号栈入栈先 ++top0，1 号栈入栈先 --top1。

共享栈能让两个栈动态利用同一片剩余空间，避免固定切半造成一边满、一边空。只要还有空间，基本入栈和出栈仍为 O(1)。

图解与逐步推演：共享栈：两边向中间生长怎样判满



单独查看图片

共享栈：两边向中间生长怎样判满

查看完整步骤（也可离线阅读）

1. 初始空栈：数组下标0～5；栈1从低端向高端，top1=−1；栈2从高端向低端，top2=6。
2. 两侧分别压入：栈1依次压A、B，栈2依次压X、Y；top1=1，top2=4。
3. 栈1再压C、D：中间两个空槽被栈1使用，top1=3，top2=4，满足top1+1=top2，共享栈满。

4. 栈2弹出Y：top2从4变5，下标4释放；此时任意一侧都可以继续利用该空槽。

循环队列的判空判满与长度 011 先标清 front、rear 各指向哪里，再选择容量约定。

从假溢出到循环使用

普通顺序队列出队后队头前面的数组单元闲置，rear 到达数组末端时可能出现“假溢出”。循环队列用模运算把下标首尾连接，使已释放单元可重新利用。

队头指向首元素、队尾指向下一空位

数组容量为 M，采用牺牲一个存储单元的约定：空队 front==rear；满队 (rear+1)%M==front；元素数 (rear−front+M)%M，最多保存 M−1 个元素。入队先写 data[rear] 再推进 rear，出队先读 data[front] 再推进 front。

也可以规定指针指向首元素之前或末元素本身，相应读写顺序和公式会改变。不能把“先移后存”的代码直接套入“rear 指向下一空位”的定义。

另外两种区分空与满的方法

- 增设计数器 count：空队 count=0，满队 count=M，每次入队、出队同步增减；此时可以使用全部 M 个单元。
- 增设 tag：front==rear 时由最近一次操作区分空满，例如插入置 tag=1、删除置 tag=0。必须确保每次操作都维护标记。

双端队列与受限形式

双端队列两端都允许插入、删除。输入受限表示只允许一端插入而两端删除；输出受限表示两端插入而只允许一端删除。判断可能输出序列时，应按允许操作逐步模拟，不能把双端队列等同于普通栈或普通队列。

图画推演与逐步动画

例：5个槽位，留空一格的循环队列

查看完整步骤（也可离线阅读）

1. 0 · 初始化：下标0～4，front指向队头，rear指向下一个写入位置。留一个空槽，所以最多存4个元素。初始front=rear=0。
2. 1 · A、B、C、D依次入队：每次先写rear指向的槽，再令rear=(rear+1)%5。四次入队后rear=4、front=0，(rear+1)%5=front，队满。
3. 2 · 出队A、B：每次先取front槽，再令front=(front+1)%5。现在front=2，逻辑队列为C、D。被释放槽位不再计入队列。
4. 3 · E入队：E写入下标4，rear从4回绕到0。rear小于front不表示队空或出错。
5. 4 · F入队：F写入下标0，rear推进到1。逻辑次序为C、D、E、F，长度(1−2+5)%5=4，又满。下标1保留为空槽。
6. 5 · 全部出队：依次从下标2、3、4、0取出C、D、E、F，front最终到1，与rear相等，队空。

数组与矩阵

数据结构 · 3 条

多维数组的地址计算 012 数组元素类型相同，地址映射由维度大小与存储次序共同决定。

数组与线性表的关系

数组是固定下标范围内、同类型元素的集合，可以看作线性表的推广。通常只讨论按下标取值与修改，不像一般线性表那样频繁改变元素数量。物理空间连续，按合法下标可随机访问。

二维数组的行优先与列优先

设 A 的行下界为 r_0、列下界为 c_0，共 R 行 C 列，每元素占 w 个存储单元，首地址为 base。行优先先存较小行号，同一行再存较小列号；列优先则交换行列角色。

计算偏移前先把实际下标减去对应下界，再乘前面完整行或列的元素数。多维数组同理：某维下标的权重是其后各维长度的乘积（行优先）。

$$\mathrm{LOC}(A[i][j]) = base + [(i - r_0)C + (j - c_0)]w$$

行优先；括号中是从首元素起经过的元素个数。

$$\mathrm{LOC}(A[i][j]) = base + [(j - c_0)R + (i - r_0)]w$$

列优先；行数 R 是相邻两列首元素的间隔。

图解与逐步推演：二维数组：行优先与列优先怎样映射到内存



单独查看图片

二维数组：行优先与列优先怎样映射到内存

查看完整步骤（也可离线阅读）

1. 行优先：C风格0起下标A[2][3]，每元素4B，首地址1000B。行优先把同一行元素连续存放。
2. 计算A[1][2]：偏移=(1×3+2)×4=20B，因此地址1000+20=1020B。先算前面有几个元素，再乘每元素字节数。
3. 列优先对照：同样的2×3数组若按列优先，偏移=(2×2+1)×4=20B；本目标恰好两种方式偏移相同，不代表布局相同。
4. 换查A[0][1]：行优先偏移4B，地址1004；列优先偏移8B，地址1008。布局差异用这个位置看得更清楚。

对称、三角与三对角矩阵的压缩存储 013 压缩存储必须同时记住保留区域、遍历次序和下标起点。

压缩的依据

对重复元素只保存一份，对规定的零元素可不分配位置，从而减少空间。特殊矩阵具有明确的分布规律；只要保留元素的位置能由公式求出，仍可由下标计算压缩地址。

对称矩阵与三角矩阵

n 阶对称矩阵满足 $a_{ij}=a_{ji}$，只需保存主对角线及一个三角区域，共 n(n+1)/2 个元素。以原矩阵下标从 1 开始、下三角按行存入 B[0...] 为例：i≥j 时 k=i(i−1)/2+j−1；i<j 时交换 i、j 后使用公式。

上、下三角矩阵的一侧严格三角区域为同一个常量 c 时，保存另一侧连同主对角线，再保存一个 c，共 n(n+1)/2+1 个位置；若题目规定零常量无需单独保存，应按题设处理。上三角按行存储时，先累计前面各行的 n、n−1、... 个元素，再加本行偏移。

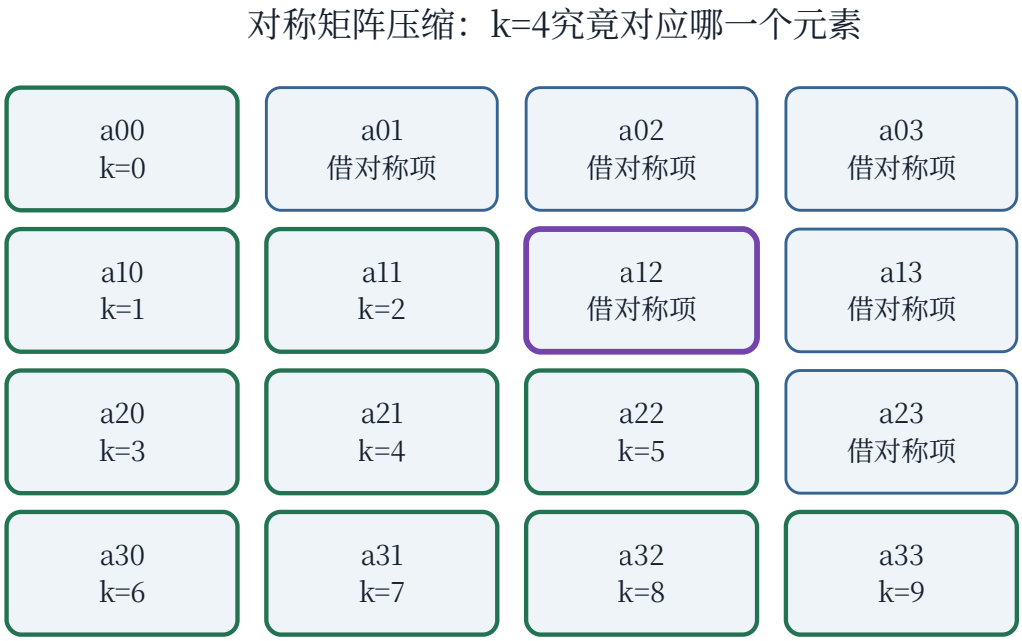
原矩阵下标从 1 开始、B 从 0 开始时，上三角按行存储的非恒定项（i≤j）映射 k=(i−1)(2n−i+2)/2+(j−i)，严格下三角常量统一映射到 B[n(n+1)/2]。下三角对应 i≥j 用 k=i(i−1)/2+j−1，严格上三角常量同样映射到最后一个位置；这与对称矩阵把另一侧交换下标的办法不同。

三对角矩阵

三对角矩阵仅主对角线及相邻上下两条对角线可能非零，非零带外满足 |i−j|>1。n≥2 时带内元素共有 3n−2 个。以 i、j 从 1 开始、按行压入 B[0...] 为例，带内元素下标 k=2i+j−3。

反求行列时要利用每行带内元素数：首末行各两个，中间行各三个。带外元素取零或指定常量，不能套用带内地址公式。

图解与逐步推演：对称矩阵压缩：k=4究竟对应哪一个元素



单独查看图片

对称矩阵压缩：k=4究竟对应哪一个元素

查看完整步骤（也可离线阅读）

1. 只保存下三角：4阶对称矩阵按行优先保存下三角含主对角线，共4×5/2=10个元素；原矩阵0起下标。
2. 定位a[2][1]：前两行共1+2=3项，本行之前还有1项，所以压缩下标k=2×3/2+1=4。
3. 上三角借用对称位置：a[1][2]=a[2][1]，统一先把上三角位置交换下标，再使用下三角公式，仍对应k=4。

稀疏矩阵的三元组、伪地址与十字链表 014 稀疏性节省空间，但普通三元组表通常失去直接随机存取能力。

稀疏矩阵与顺序表示

当非零元素数 t 远小于总元素数 m×n 时，可只记录非零元素。三元组记录（行、列、值），通常按行优先或列优先排序；矩阵总行数、总列数、非零元素数也应保留。根据任意 (i,j) 找元素通常需查找，不能像普通二维数组那样一次偏移直接取值。

伪地址表示把元素在展开矩阵中的相对位置与值成对保存。对 m×n 矩阵、下标从 1 开始时，行优先的 1 起始位置为 n(i−1)+j；如果位置从 0 开始，则再减 1。由伪地址可反求行列。

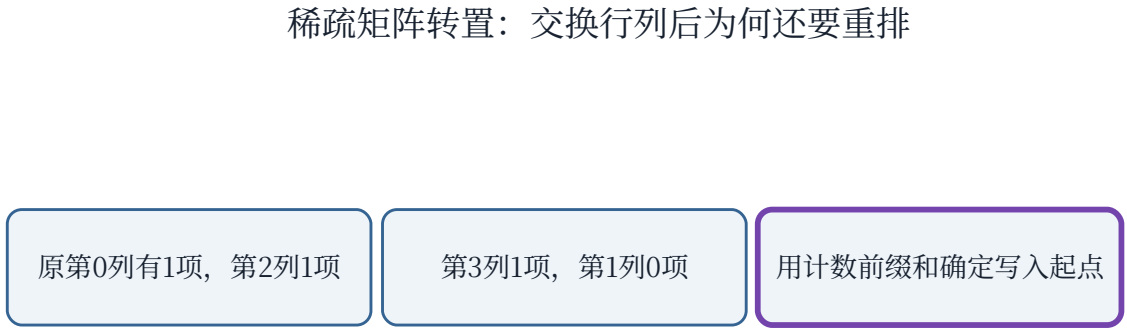
三元组的行号、列号本质为整数。若为了演示而把值、行号、列号都放入 float 类型二维表，取行列参与寻址时应转回整型；正式结构体通常给行列单独使用整数成员，避免隐式精度与类型问题。

链式表示与十字链表

按行建立链表时，每个非零结点保存列号、值和行后继链接；行号可由所属行表隐含。十字链表同时组织行表与列表，每个非零结点保存行号、列号、值，以及 right、down 两个链接域。

right 连接同行的下一个非零元素，down 连接同列的下一个非零元素。行、列头指针分别给出入口；插入或删除非零元素时，两条方向的链都要维护，不能只改行链。总表头可保存行数、列数、非零元素总数等信息。

图解与逐步推演：稀疏矩阵转置：交换行列后为何还要重排



单独查看图片

稀疏矩阵转置：交换行列后为何还要重排

查看完整步骤（也可离线阅读）

1. 原矩阵非零项：原三元组按行优先有序；三元组字段为行号、列号、值。
2. 先交换行列：转置后矩阵形状变成4行3列，值不变。
3. 恢复新矩阵的行优先次序：只交换字段还不保证新表按行号递增；需按新行次序组织记录。
4. 快速转置按列计数：快速转置可用原列的非零项数定位新行的起始位置，避免对每一列反复扫描整个三元组表。

栈与队列的应用

数据结构 · 1 条

括号匹配、表达式求值与递归 015 栈保存尚未完成的任务，队列保存等待按顺序处理的任务。

括号匹配的完整条件

扫描表达式：遇左括号入栈；遇右括号，先判栈是否非空，再检查栈顶是否为相同类型的左括号，匹配才弹出。右括号到来时栈空、括号类型不一致，或扫描结束后栈仍非空，都属于不匹配。

后缀表达式求值

从左向右扫描后缀表达式，操作数入栈；遇二元运算符，先弹出右操作数 b，再弹出左操作数 a，计算 a 运算符 b，将结果压回。减法和除法尤其不能颠倒顺序。扫描结束应恰好剩一个结果，栈下溢或剩余多个值都提示表达式不合法。

递归和队列的系统用途

递归把当前问题转成同类、规模更小的子问题，必须有终止条件。调用栈保存返回地址、参数和局部状态；递归写法简洁，但时间和空间代价取决于具体递推关系，不能笼统认为所有递归都低效。

树的层序遍历、图的广度优先遍历使用队列，把后续层结点按发现顺序等待处理。设备缓冲、请求排队等系统场景也利用队列协调速度差异；具体是否严格先进先出仍由调度策略决定。

图解与逐步推演：括号匹配与栈顶的变化





单独查看图片

括号匹配与栈顶的变化

查看完整步骤（也可离线阅读）

1. 读入 (：左括号入栈，等待匹配。
2. 读入 [：左括号继续入栈。
3. 读入]：与栈顶[匹配，弹出[。
4. 读入)：与栈顶(匹配，弹出(；输入结束且栈空，匹配成功。

串与模式匹配

数据结构 · 3 条

串的概念、比较与存储表示 016 串是字符序列，空串、空格串和结束标记必须分别理解。

字符、子串与字典序

串由零个或多个字符组成；长度为 0 的串是空串，一个或多个空格构成的串是非空的空格串。子串由主串中连续字符组成，其位置用第一个字符在主串中的位序表示。

串相等要求长度相同且对应字符全相同。比较按字符编码逐个进行，首个不同字符决定大小；如果一方为另一方前缀，则较短者较小。不能只比较长度，也不能把字符数组首地址相等当作串值相等。

三类存储方式

表示	特点与长度
定长顺序表示	为串分配固定容量，另存 length 或约定结束标记。超过容量可能截断，但截断是接口策略，不能误认为自动保留完整串。
堆分配顺序表示	运行时按所需长度申请连续空间，保存字符指针和长度，使用完后释放。连接与复制时须申请足够空间。
块链表示	一个结点可保存一个或多个字符，结点之间链接。每块字符越少，链接开销比例越大；最后一块可能不满。

基本操作的接口约定

赋值、复制、求长、判空、比较、连接、取子串、定位、清空和销毁都是串操作。字符数组不能用普通数组赋值符直接整体复制，但语言提供的字符串类型可能重载赋值，必须区分抽象串和具体 C 数组。

取子串前检查起点和长度是否越界；下标从 0 或位序从 1 开始会改变边界公式。若存储格式要求末尾 \0，连接两个串时除有效字符总数外还要为结束标记留空间，且 \0 不计入串长。

朴素模式匹配与 KMP 的指针移动 017 匹配失败后，主串是否回退，是两种算法的核心区别。

朴素匹配的逐次试探

从主串的一个候选起点开始，逐个比较模式串字符；全部匹配则返回该起点。失败时让主串候选起点后移一位，模式串回到开头，继续尝试。主串长 n、模式长 m 时，最多有 n−m+1 个候选起点，最坏比较量为 O((n−m+1)m)，通常简写为 O(nm)。

若 i、j 都采用从 1 开始的下标，当前失配时已匹配 j−1 个字符，原候选起点为 i−j+1；下一候选起点为 i−j+2，因此回退可写 i=i−j+2、j=1。若模式串已全部匹配，起点为 i−m。

KMP 利用相同前后缀

模式串某一前缀的最长相等真前缀与真后缀长度表示已匹配信息可以保留多少。KMP 失配后主串指针 i 不回退，只让模式指针按预处理表跳转，避免重新比较已知相同的片段。

使用 1 起始 next 数组时，next[1]=0；j≥2 时，next[j] 等于模式串前 j−1 个字符的最长相等真前后缀长度加 1。失配令 j=next[j]；当 j=0 时，统一推进 i、j，相当于主串移到下一字符并重新从模式开头开始。预处理 O(m)，完整匹配 O(n)，总计 O(n+m)。

部分匹配值与 next 不要混用

对 ABABA，每个前缀的最长相等真前后缀长度依次为 0、0、1、2、3。它描述的是各前缀的长度；1 起始 next[j] 使用的是 j 前面的已匹配部分。0 起始失败函数又有另一套下标约定，不能直接把数值平移后套进代码。

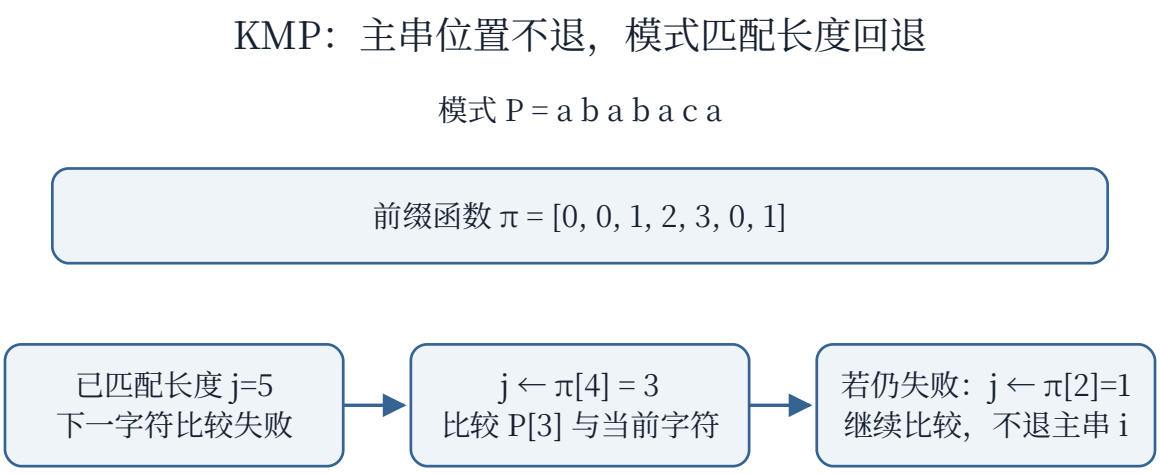
例如已匹配 q 个字符，能保留的前后缀长度为 ℓ，则模式串相对主串后移 q−ℓ 位。先明确 q、ℓ 的定义，再计算移动量，避免把 next 值本身当成后移位数。

1 起始 next 与 KMP 的完整配套代码

本代码假定模式串非空，S、P 的有效字符均从下标 1 开始，next 已按同一规则生成；0 为失败返回值。条件 j==0 或 k==0 必须放在字符访问之前，利用短路求值避免访问无效的第 0 个字符。

```
void getNext(const char P[], int m, int next[]) {
    int j=1, k=0; next[1]=0;
    while (j<m) {
        if (k==0 || P[j]==P[k]) { ++j; ++k; next[j]=k; }
        else k=next[k];
    }
}
int kmp(const char S[], int n, const char P[], int m, const int next[]) {
    int i=1, j=1;
    while (i<=n && j<=m) {
        if (j==0 || S[i]==P[j]) { ++i; ++j; }
        else j=next[j];
    }
    return j>m ? i-m : 0;
}
```

机制与步骤图解



此图使用 0 起始前缀函数 π；不要直接混用 1 起始 next 数组。

匹配成功后，主串位置和已匹配长度各前进一位。

单独查看图片

图画推演与逐步动画

例：主串abababaca，模式ababaca

查看完整步骤（也可离线阅读）

1. 0 · 约定下标：使用0起始下标；i指向主串当前待比较字符，j为已匹配长度，也是模式待比较位置。π=[0,0,1,2,3,0,1]。
2. 1 · 连续匹配5个字符：主串下标0～4和模式下标0～4同为ababa。此时i=5、j=5，下一次比较S[5]=b与P[5]=c。
3. 2 · 发现失配：b与c不同。朴素算法可能重新比较较早的主串字符；KMP使用已匹配前缀的前后缀关系，i仍保持5。
4. 3 · j回退到3：令j=π[j−1]=π[4]=3。已知aba可以复用，将模式对齐起点移到i−j=2；现在比较P[3]=b与S[5]=b。
5. 4 · 继续匹配：本次b匹配成功，推进i=6、j=4，继续比较两个a。模式整体右移，但主串指针没有退回到2。
6. 5 · 找到完整匹配：再依次匹配a、c、a，最终i=9、j=7。模式长7，因此匹配起点i−j=2，即主串第3个字符（1起始位序）。

nextval 的优化与手算检查 018 若跳转后的模式字符仍与刚才失配字符相同，就可以继续跳过。

递归跳过无效比较

在 1 起始约定下，若模式字符 P[j] 与 P[next[j]] 不同，可令 nextval[j]=next[j]；若相同，则令 nextval[j]=nextval[next[j]]，跳到更前的位置。nextval[1]=0。优化只是减少必然失败的重复比较，不改变匹配结果与 O(n+m) 的渐近复杂度。

模式串 abaabcac

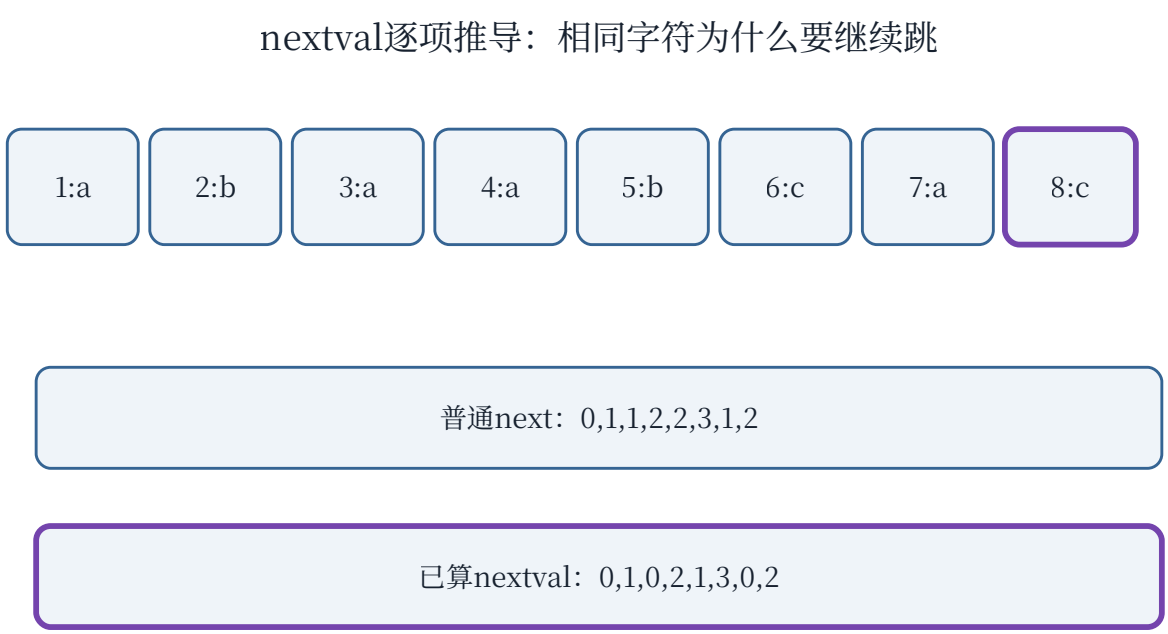
第 5 个字符 b 的 next 值为 2，而第 2 个字符也是 b，继续比较仍会失配，因此继承 nextval[2]=1。第 7 个字符 a 与第 1 个字符 a 相同，故 nextval[7]=0。

j	1	2	3	4	5	6	7	8
P[j]	a	b	a	a	b	c	a	c
next[j]	0	1	1	2	2	3	1	2
nextval[j]	0	1	0	2	1	3	0	2

检查顺序

- 先列出普通 next，再逐项比较当前字符与跳转字符。
- 先统一“下标从 0 还是 1 开始、首项是 −1 还是 0”的约定，再解释表值。
- 真前缀、真后缀都不能等于整个串；空前后缀的长度为 0。

图解与逐步推演：nextval逐项推导：相同字符为什么要继续跳



单独查看图片

nextval逐项推导：相同字符为什么要继续跳

查看完整步骤（也可离线阅读）

1. 计算nextval[1]：约定nextval[1]=0，0是边界标记，不访问P[0]。
2. 计算nextval[2]：P[2]=b与P[1]=a不同，保留普通next值1。
3. 计算nextval[3]：P[3]与P[1]同为a，跳到1后还会重复失配，继承nextval[1]=0。

4. 计算nextval[4]：P[4]=a与P[2]=b不同，保留普通next值2。
5. 计算nextval[5]：P[5]与P[2]同为b，跳到2后还会重复失配，继承nextval[2]=1。
6. 计算nextval[6]：P[6]=c与P[3]=a不同，保留普通next值3。
7. 计算nextval[7]：P[7]与P[1]同为a，跳到1后还会重复失配，继承nextval[1]=0。
8. 计算nextval[8]：P[8]=c与P[2]=b不同，保留普通next值2。

树与二叉树

数据结构 · 7 条

树的术语、度数与高度 019 树的每个非根结点只有一个双亲；树的度取所有结点度数的最大值。

树与森林

非空树由一个根和若干互不相交的子树组成，定义具有递归性。结点的度是孩子数，度为 0 的结点是叶子，度大于 0 的是分支结点；树的度是结点度的最大值。孩子、双亲、兄弟、祖先与子孙分别描述局部和跨层关系。

根通常位于第 1 层，树高是最大层数。若改用根深度为 0 的约定，所有高度公式相应少 1。路径是沿相邻结点构成的序列，路径长度按边数计；森林是若干互不相交的树的集合。有序树的孩子有确定次序，交换孩子可能得到不同树。

结点数、分支数与 m 叉树界限

n 个结点的非空树有 n−1 条边，也等于所有结点孩子数之和，所以 n=1+Σd(v)。注意这里求和的是出度／孩子数；若按无向图的度计数，则度和为 2(n−1)。

m 叉树指每个结点最多 m 个孩子；“度恰为 m 的树”还要求至少有一个结点具有 m 个孩子，因此二者不是同一限制。m≥2、根为第 1 层时，第 i 层最多 m^{i-1} 个结点，高度 h 的最大结点数为 $(m^h-1)/(m-1)$。

$$h_{min} = \lceil \log_m \left(n(m-1)+1\right) \rceil$$

m≥2、n≥1，求含 n 个结点的 m 叉树最小高度；来自 $n \le (m^h-1)/(m-1)$。

二叉树、满二叉树与完全二叉树 020 左孩子与右孩子是固定角色，不由是否有另一个孩子决定。

二叉树的有序性

二叉树可以为空；非空时由根、左子树和右子树组成。每个结点最多两个孩子，度只可能为 0、1、2。即使只有一个孩子，也必须区分左孩子还是右孩子。

度为 2 的有序树至少存在一个度为 2 的结点，并不等同于二叉树；二叉树可以只有一个结点甚至为空。

特殊形态

名称	定义与性质
满二叉树	按本资料采用的完美二叉树定义：高度 h、每层达到最大结点数，共 2^h-1 个结点；叶子全在第 h 层。
完全二叉树	与同高度满二叉树按层从左到右编号的前 n 个位置一致；最后一层从左连续排列，其上各层均满。
二叉排序树	所有左子树关键字小于根，右子树关键字大于根，左右子树也满足此规则；重复关键字另按题设。
平衡二叉树（AVL）	每个结点左右子树高度差绝对值不超过 1，同时满足二叉排序树次序。

完全二叉树的结构限制

完全二叉树中度为 1 的结点至多一个，且只有左孩子；编号大于 $\lfloor n/2 \rfloor$ 的结点均为叶子。遇到第一个不完整的分支结点后，后续层序位置都不能再有孩子。奇数 n 时无度为 1 的结点，偶数 n 时恰有一个。

二叉树计数公式与编号关系 021 先确定根层次和数组下标起点，公式才有意义。

通用二叉树计数

设度为 0、1、2 的结点数分别为 n_0、n_1、n_2。由 $n=n_0+n_1+n_2$ 和 $n-1=n_1+2n_2$ 相减，得到 $n_0=n_2+1$，不要求完全或平衡。

根为第 1 层时，第 i 层至多 2^{i-1} 个结点，高度 h 的二叉树至多 2^h-1 个结点；含 n 个结点的最小高度为 $\lceil \log_2(n+1) \rceil$，最大高度为 n。

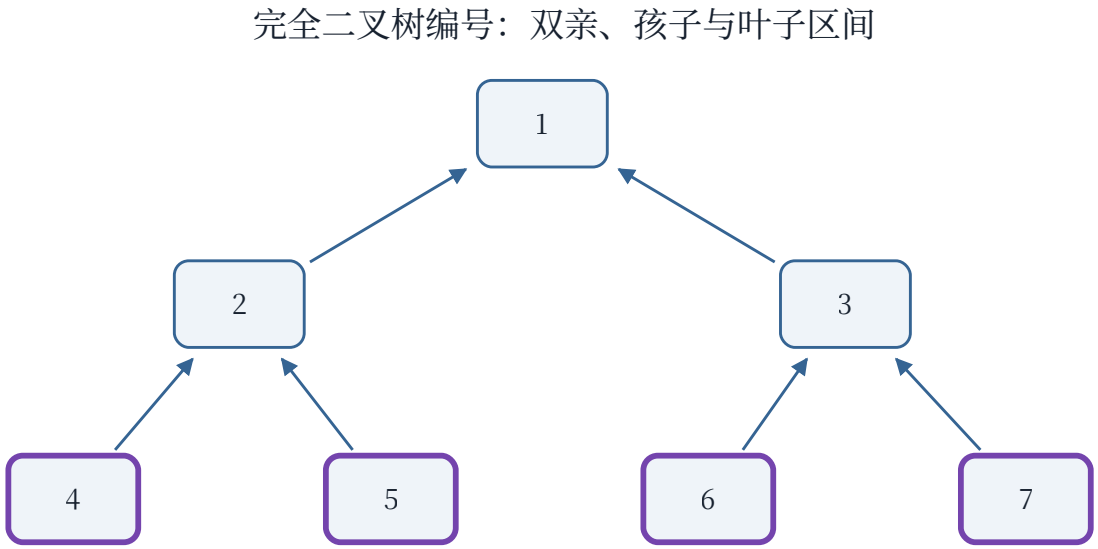
完全二叉树的 1 起始编号

对象	公式与存在条件
双亲	$i>1$ 时为 $\lfloor i/2 \rfloor$；偶数 i 为左孩子，奇数 $i>1$ 为右孩子。
左孩子	编号 2i，只有 $2i \leqslant n$ 时存在。
右孩子	编号 2i+1，只有 $2i+1 \leqslant n$ 时存在。
结点层次	$\lfloor \log_2 i \rfloor + 1$。
树高与叶子数	$h=\lfloor \log_2 n \rfloor + 1=\lceil \log_2(n+1) \rceil$；叶子数为 $\lceil n/2 \rceil$。

链式存储的空链接

二叉链表每个结点有左右两个孩子链接，共 2n 个链接域；非空孩子链接等于 n−1 条边，因此空链接域有 n+1 个。这一结论用于解释线索化可利用的空域，而不是说树有 n+1 个叶子。

图解与逐步推演：完全二叉树编号：双亲、孩子与叶子区间



单独查看图片

完全二叉树编号：双亲、孩子与叶子区间

查看完整步骤（也可离线阅读）

1. 完全二叉树从1编号：按层从左向右编号1～7。完全二叉树紧凑存储，不需要为中间空缺位置保留槽位。
2. 结点6的双亲：1起下标时parent(i)=floor(i/2)，结点6的双亲为3。图中箭头从孩子指向双亲。
3. 结点3的两个孩子：左孩子2i=6，右孩子2i+1=7；超出结点总数时表示该孩子不存在。
4. 叶子编号区间：总数n=7，叶子编号floor(n/2)+1～n，即4～7；前3个结点是分支结点。

二叉树的顺序存储、链式存储与遍历 022 选择存储方式时，要同时考虑空位置浪费和常用访问方向。

两类存储

顺序存储通常按照完全二叉树编号把结点放入数组。完全树紧凑，父子位置可直接计算；一般树需为空缺位置保留编号，深度 h 的右单支树可能用到编号 2^h-1，空间浪费明显。

二叉链表用 data、lchild、rchild 描述结点，适合不规则形态；若常需找双亲，可增设 parent 成为三叉链表。没有双亲指针时，不能仅凭任意结点的孩子链接 O(1) 找到其双亲。

三种深度优先次序

遍历	递归次序	根出现的位置
先序	根→左子树→右子树	子树序列的第一项。

遍历	递归次序	根出现的位置
中序	左子树→根→右子树	左右子树序列之间。
后序	左子树→右子树→根	子树序列的最后一项。

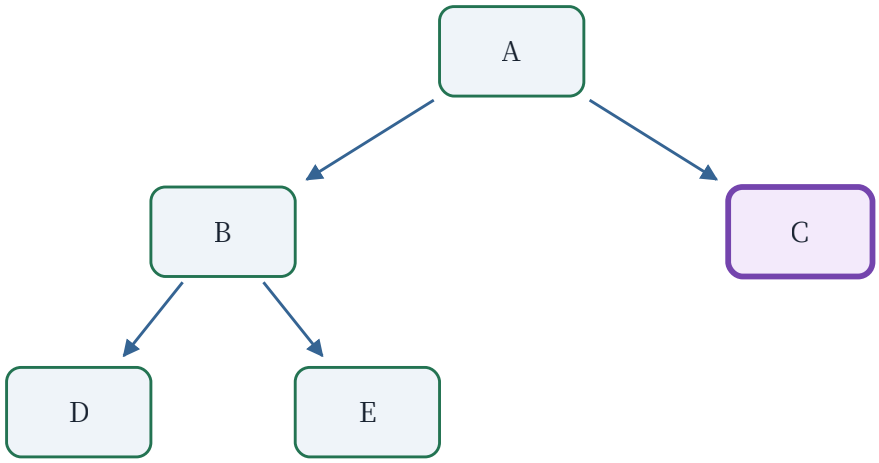
递归代码骨架与层次遍历

把 visit 放到左递归前得到先序，放到右递归后得到后序。三种遍历访问每个结点一次，时间 O(n)，递归辅助空间为 O(h)，最坏单支树 O(n)。层次遍历用队列：根非空则入队，反复出队访问，再依次将非空左、右孩子入队，直至队空。

```
void inorder(Node *p) {
    if (p == NULL) return;
    inorder(p->left);
    visit(p);
    inorder(p->right);
}
```

图解与逐步推演：一棵树逐步走完中序遍历

一棵树逐步走完中序遍历



单独查看图片

一棵树逐步走完中序遍历

查看完整步骤（也可离线阅读）

1. 访问 D：中序规则：左子树→根→右子树。这里的“访问”是输出结点，沿边经过不等于输出。
2. 访问 B：中序规则：左子树→根→右子树。这里的“访问”是输出结点，沿边经过不等于输出。
3. 访问 E：中序规则：左子树→根→右子树。这里的“访问”是输出结点，沿边经过不等于输出。

4. 访问 A：中序规则：左子树→根→右子树。这里的“访问”是输出结点，沿边经过不等于输出。
5. 访问 C：中序规则：左子树→根→右子树。这里的“访问”是输出结点，沿边经过不等于输出。

非递归遍历与镜像交换 023 显式栈保存递归中的待处理状态，压栈顺序要与出栈顺序相反。

先序与中序的显式栈

先序可先把根压栈，循环弹出并访问，再依次压右孩子、左孩子；由于栈后进先出，左孩子会先访问。中序则反复把当前结点及其左链压栈，遇空后弹栈访问，再转向弹出结点的右子树。

例如根 A、左孩子 B、右孩子 C，B 的左右孩子为 D、E：先序 A B D E C，中序 D B E A C，后序 D E B C A。必须按同一棵树检查三种序列。

后序的状态辨别

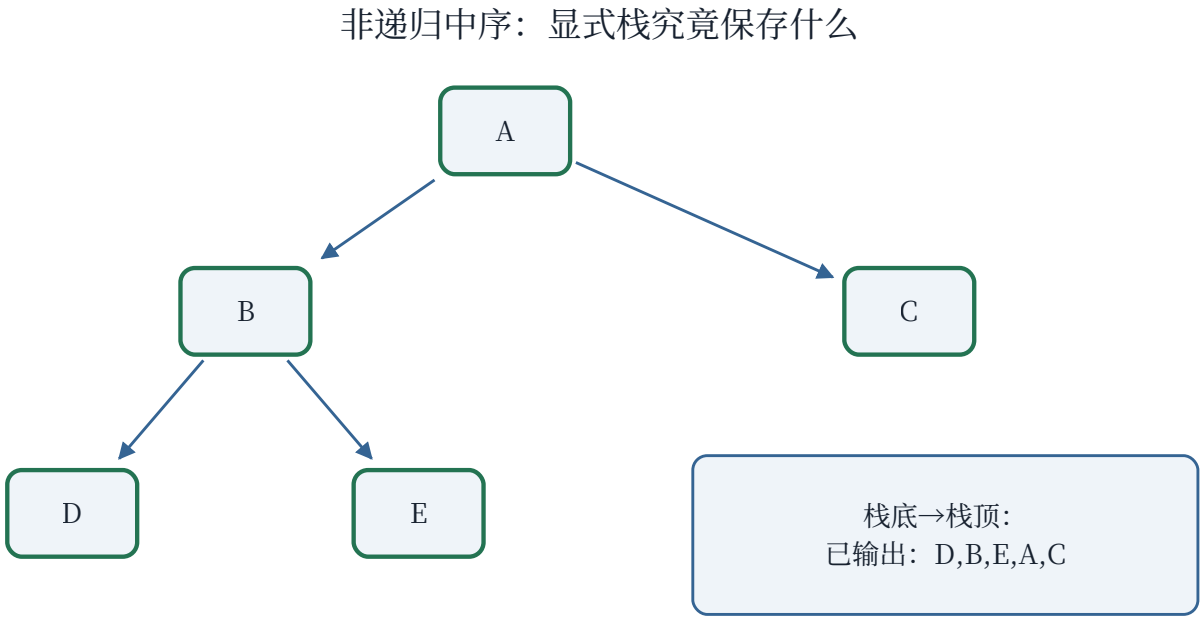
后序要在左右孩子都完成后才访问根。可用一个栈与 lastVisited 记录上次访问结点：沿左链入栈，查看栈顶；若其右孩子未访问则转入右子树，否则访问并弹出。也可先得到“根→右→左”的逆后序，再逆转序列，但需额外保存结果。

交换整棵树的左右子树

每个分支结点都交换，不能只交换根的两个孩子而保留子树内部不变。代码访问 n 个结点，时间 O(n)，递归栈空间 O(h)。

```
void mirror(Node *p) {
    if (!p) return;
    mirror(p->left);
    mirror(p->right);
    Node *tmp = p->left;
    p->left = p->right;
    p->right = tmp;
}
```

图解与逐步推演：非递归中序：显式栈究竟保存什么



单独查看图片

非递归中序：显式栈究竟保存什么

查看完整步骤（也可离线阅读）

1. 压入A：沿左链下降，压入结点保存“左子树处理完后再访问它”的任务。
2. 压入B：沿左链下降，压入结点保存“左子树处理完后再访问它”的任务。
3. 压入D：沿左链下降，压入结点保存“左子树处理完后再访问它”的任务。
4. 弹出并访问D：左侧已处理完，弹栈输出该结点，然后转向它的右孩子。
5. 弹出并访问B：左侧已处理完，弹栈输出该结点，然后转向它的右孩子。
6. 压入E：沿左链下降，压入结点保存“左子树处理完后再访问它”的任务。
7. 弹出并访问E：左侧已处理完，弹栈输出该结点，然后转向它的右孩子。
8. 弹出并访问A：左侧已处理完，弹栈输出该结点，然后转向它的右孩子。
9. 压入C：沿左链下降，压入结点保存“左子树处理完后再访问它”的任务。
10. 弹出并访问C：左侧已处理完，弹栈输出该结点，然后转向它的右孩子。

由遍历序列重建二叉树 024 中序序列负责分割左右子树，另一种序列负责找到根。

互不相同关键字条件下的重建

先序+中序：先序首项是根，在中序中找根，左侧是左子树、右侧是右子树；按左右结点数切分先序后递归。后序+中序同理，只是根取后序末项。层序+中序也可唯一确定：层序中在当前结点集合里最先出现的结点是当前根，再按中序左右范围分别筛出子树层序。

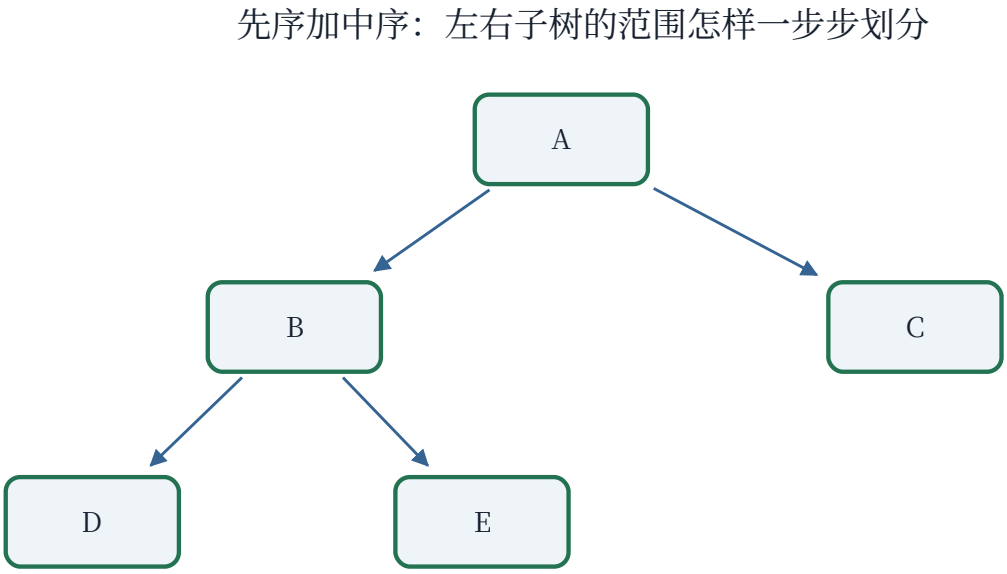
单独先序、单独后序或先序+后序一般不能唯一确定二叉树，例如一个孩子时无法判断是左孩子还是右孩子。若关键字重复，还需额外标识或条件，不能直接套用唯一性结论。

分割示例

先序 D A E F B C H G I，中序 E A F D H C B G I。先找根 D：中序分为左侧 E A F、右侧 H C B G I；左子树先序 A E F，所以根为 A、左右孩子 E 和 F。右子树先序 B C H G I，所以根为 B，左侧 H C 由 C 作根、H 为左孩子；右侧 G I 由 G 作根、I 为右孩子。

得到的后序应为 E F A H C I G B D，可用来反向检查，而不是任意挑选序列中的相邻字符作为父子。

图解与逐步推演：先序加中序：左右子树的范围怎样一步步划分



单独查看图片

先序加中序：左右子树的范围怎样一步步划分

查看完整步骤（也可离线阅读）

1. 先序找根A：先序A B D E C，中序D B E A C；根是先序第一个A，在中序中把左右子树分开。
2. 递归确定左子树根B：左先序B D E的首项B为左子树根，中序D B E把D与E分成左右子树。
3. 回读序列检查：重建后先序A B D E C，中序D B E A C，与输入一致；本例所有关键字互不相同。

线索二叉树的标志与前驱后继 025 线索利用空链接表达某一种遍历次序中的相邻关系。

线索与孩子链接的区别

普通二叉链表中无左孩子时，可令 lchild 指向遍历前驱；无右孩子时，可令 rchild 指向遍历后继。常用 ltag=0 表示左孩子、ltag=1 表示前驱线索，rtag 类似。必须先看标志再追指针，否则可能把线索当子树并陷入循环。

线索化要明确先序、中序或后序；同一棵树的前驱后继会随遍历次序改变。可增加头结点统一边界：头的左链接指向根，右链接指向最后访问结点，并让首结点前驱和末结点后继指向头。具体标志设定仍需按题设。

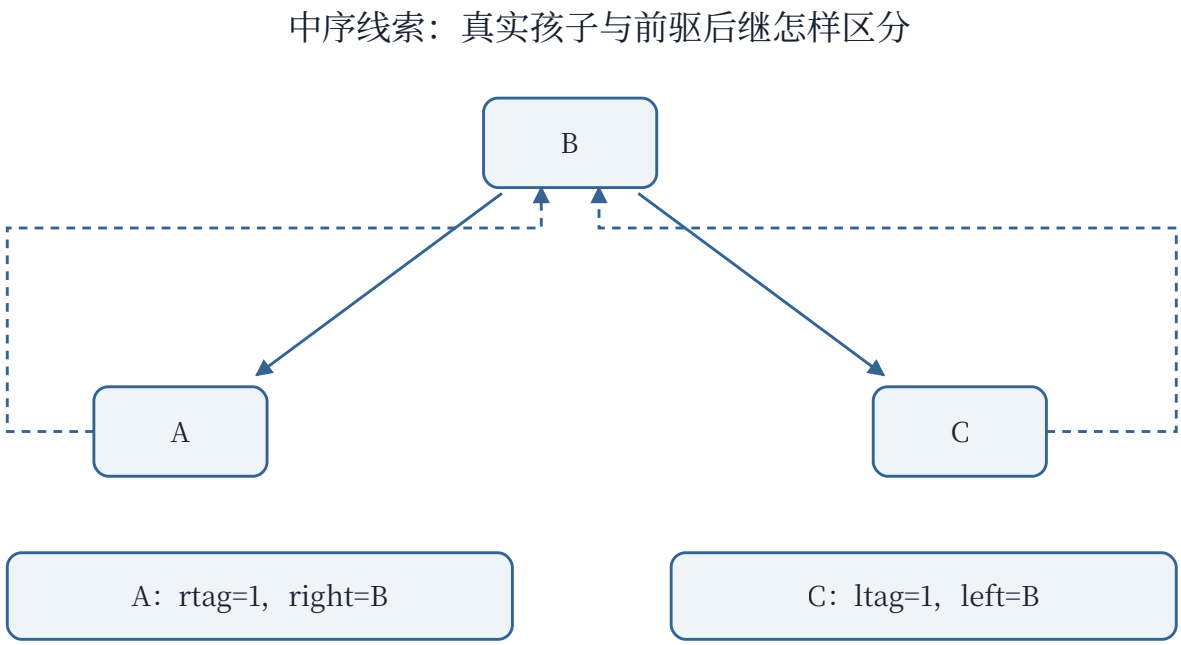
中序找后继

若 rtag=1，右链接就是中序后继；若 rtag=0，则转入右子树，沿着“有真实左孩子”的链接走到最左结点。中序前驱对称处理：有左线索直接取，无则到左子树最右结点。

后序后继往往需要双亲

根的后序后继为空；若当前结点是双亲的右孩子，或是左孩子且双亲无右子树，后继为双亲；若它是左孩子且双亲有右子树，后继为右子树的后序首结点。仅靠普通后序线索不一定能直接取得双亲，因此后序遍历的实现不能简单照搬中序。

图解与逐步推演：中序线索：真实孩子与前驱后继怎样区分



单独查看图片

中序线索：真实孩子与前驱后继怎样区分

查看完整步骤（也可离线阅读）

1. 原始二叉树：B为根，左A右C，中序顺序A、B、C；三个结点均无额外的实际子树。
2. A的空右指针指向后继B：把A的空右指针改作后继线索，rtag=1；虚线从A绕回B，不是新增真实右子树。
3. C的空左指针指向前驱B：C的ltag=1，左链接指向中序前驱B；根B本来有左右孩子，仍保留真实孩子链接。

树与森林

数据结构 · 2 条

双亲表示、孩子表示与孩子兄弟表示 026 同一逻辑树，可以为不同访问需求选择不同存储结构。

三种表示的访问特点

表示法	保存内容	优势与限制
双亲表示	数组中保存 data 和 parent 下标，根可用 −1 标识。	已知结点找双亲 O(1)；找全部孩子往往扫描数组。
孩子表示	每个结点对应一个孩子链表，头表保存结点信息和首孩子入口。	找孩子直接，叶子的孩子链为空；找双亲通常需扫描。
孩子兄弟表示	每个结点保存 firstchild 和 nextsibling。	两个指针即可表示任意度树，方便树与二叉树转换；找双亲仍可额外增设 parent。

左孩子、右兄弟的语义

孩子兄弟表示中，左链接不是“普通树唯一的左孩子”，而是第一个孩子；右链接是同一层的下一个兄弟。单棵树根没有兄弟，转换后的二叉树根因此无右孩子。必须保留原树兄弟次序，否则转换结果会改变。

树、森林与二叉树之间的转换和遍历对应 027 转换只是改变链接解释，结点集合保持不变。

转换步骤

- 树→二叉树：同一双亲的孩子依次连成兄弟链，只保留双亲到第一个孩子的边；按左孩子、右兄弟解释。旋转画图只是示意，不是新增边。
- 森林→二叉树：先转换每棵树，再把各树根看成兄弟，从第一棵根开始沿右链接串联。
- 二叉树→森林：从根开始沿右链分出各棵树根，再把每棵二叉树按左孩子右兄弟规则还原。

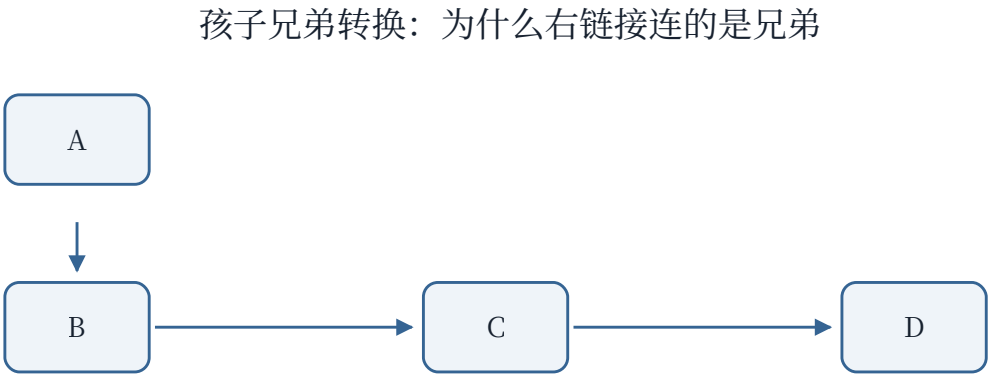
遍历次序的对应

原结构的遍历	对应二叉树的遍历
树的先根遍历：根先于依次各子树	先序遍历。
树的后根遍历：依次各子树后访问根	中序遍历。
森林的先序遍历	先序遍历。
森林的中序遍历（逐树后根次序）	中序遍历。

层次遍历单独理解

普通树层次遍历按层从左到右，用队列保存待访问结点；转换后的二叉树层次结构已经改变，因此不能把二叉树的层次遍历序列直接当作原树层次序列。

图解与逐步推演：孩子兄弟转换：为什么右链接连的是兄弟



单独查看图片

孩子兄弟转换：为什么右链接连的是兄弟

查看完整步骤（也可离线阅读）

1. 普通有根树：A有三个有序孩子B、C、D。普通树中这些边都表示父子关系。
2. 保留第一个孩子，其余用兄弟链接：A.firstchild=B，B.nextsibling=C，C.nextsibling=D。兄弟链接不表示原树中B是C的父亲。
3. 作为二叉链表解释：把firstchild看作左孩子、nextsibling看作右孩子，便得到二叉树表示。原树先根顺序A,B,C,D对应此二叉树先序。

树与森林的应用

数据结构 · 3 条

哈夫曼树的构造与带权路径长度 028 每次合并最小权值，使较大权值叶子尽量靠近根。

最优二叉树与 WPL

叶子带权路径长度是叶权值乘从根到叶经过的边数；树的 WPL 为全部叶子的这一乘积之和。给定正权叶子集合，使 WPL 最小的二叉树称哈夫曼树。叶子权值是原始数据，新产生内部结点权值等于孩子权值之和。

贪心构造

把每个叶子看成独立树，反复取根权最小的两棵树作为新根的左右子树，新根权为两者之和，再放回候选集合，直到只剩一棵。n 个叶子（n≥2）产生 n−1 个内部结点，总结点数 2n−1，没有度为 1 的结点。

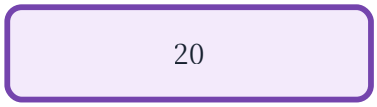
同权时选择不同或交换左右可能产生不同形态和编码，但最优 WPL 相同。把“较小权放左边”等作为画图约定即可，不是哈夫曼最优性的必要条件。

通过合并和计算 WPL

权值 2、3、7、8：先 2+3=5，再 5+7=12，最后 8+12=20。WPL 等于所有合并权之和 5+12+20=37；也可按叶深度算 2×3+3×3+7×2+8×1=37。两种计算可互相复核。

图解与逐步推演：哈夫曼构造：最小两棵树反复合并

哈夫曼构造：最小两棵树反复合并



单独查看图片

哈夫曼构造：最小两棵树反复合并

查看完整步骤（也可离线阅读）

1. 四个叶子：初始权值2、3、7、8，每次取出当前最小的两个树根。
2. 合并 2+3=5：每个叶子的权值在每上升一层时贡献一次；所有合并权值之和等于最终WPL。
3. 合并 5+7=12：每个叶子的权值在每上升一层时贡献一次；所有合并权值之和等于最终WPL。
4. 合并 8+12=20：每个叶子的权值在每上升一层时贡献一次；所有合并权值之和等于最终WPL。

哈夫曼编码与多叉推广 029 叶子路径给出前缀编码，多叉合并前要先检查叶子数条件。

前缀编码

沿根到叶路径分别以 0、1 标记左右边，叶子路径串就是其编码。由于叶子不会成为其他叶子的祖先，一个完整编码不会是另一编码的前缀，因此可即时识别边界。哈夫曼编码在给定权重和二元前缀码条件下使加权平均码长最小，码字本身通常不唯一。

定长编码每个符号使用相同位数；变长编码可让高频符号码字更短。不能把“不同长度”与“可唯一译码”直接画等号，前缀性是便于唯一译码的重要结构条件。

m 叉哈夫曼树

m 叉版本每次合并 m 个最小权根。满 m 叉树叶子数 L 与内部结点数 I 满足 L=(m−1)I+1，所以原始叶子数 n 不满足 (n−1) 能被 m−1 整除时，先补最少数量的 0 权虚拟叶子。

补齐后 I=(L−1)/(m−1)，也等于 ⌈(n−1)/(m−1)⌉。虚拟叶子不代表实际符号，编码时只输出原始符号。

并查集的初始化、查找和合并 030 用森林表达互不相交集合，用根结点代表集合。

集合与双亲数组

每个元素起初独立为一个集合。双亲数组 parent 保存森林：非根项存父结点下标，根项可用负数标识；根的负值也可编码集合规模。Find(x) 沿双亲链返回所属集合的根，两个元素根相同表示同属一个集合。

合并的前置条件

合并前先分别查根，仅当两个根不同才连接，否则不应再改变结构。把一个集合根接到另一个集合根是常见实现；按规模或按秩合并，可避免无谓增加高度。不能直接把任意非根元素的 parent 改向另一集合，否则可能只移动了原集合的一部分。

查找与路径压缩

这里根的负值表示负的集合大小，故数值更小的根代表更大的集合。路径压缩把查找途中结点直接接到根，不改变集合划分。并查集可用于动态连通性、Kruskal 选边时的环检测等。

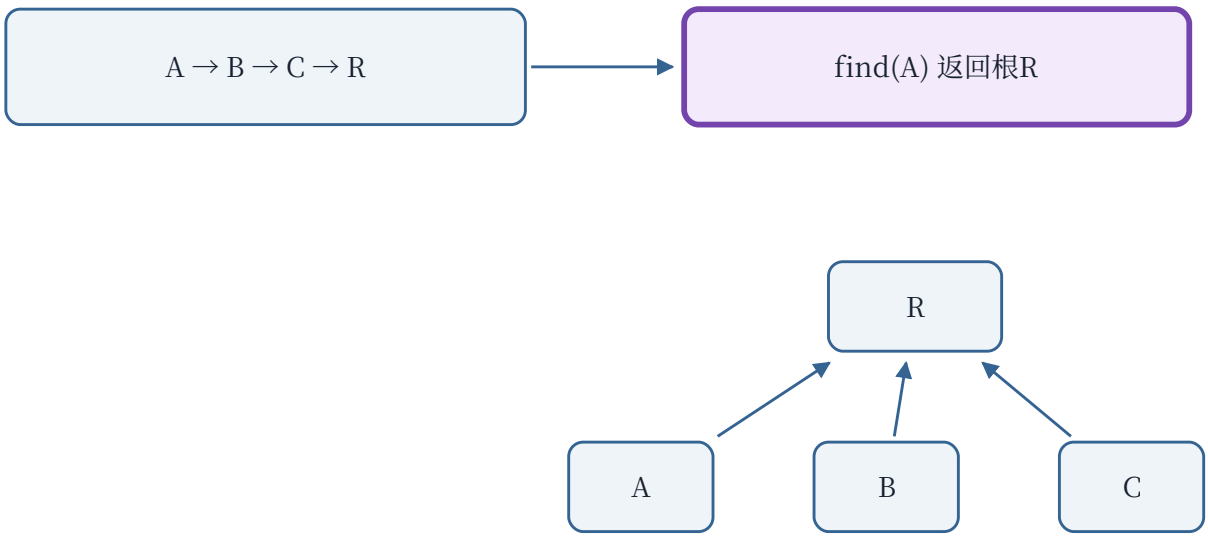
```
int Find(int x) {
    if (parent[x] < 0) return x;
    return parent[x] = Find(parent[x]);
}
void Union(int a, int b) {
    int ra = Find(a), rb = Find(b);
    if (ra == rb) return;
    if (parent[ra] > parent[rb]) { int t=ra; ra=rb; rb=t; }
    parent[ra] += parent[rb];
    parent[rb] = ra;
}
```

结构对照图：并查集：路径压缩只改父指针

假设原父指针链A→B→C→R，R为根。递归路径压缩的find(A)在返回时把沿途A、B、C直接指向R。

集合划分和代表根不因此改变；压缩后的图仍是一棵树。按秩合并中的秩在压缩后一般不等于当前精确高度。

并查集：路径压缩只改父指针



单独查看图片

图的基础与存储

数据结构 · 3 条

图的方向、度数、路径与连通性 031 区分顶点关系、可达关系和极大连通子图。

有向边与无向边

图记为 G=(V,E)，顶点集 V 与边集 E 分别描述对象及其联系。本资料采用顶点集非空、边集可以为空的约定。无向边 (u,v) 无方向，有向弧 ⟨u,v⟩ 从弧尾 u 指向弧头 v；简单图不含自环与重边。

无向图的度为与顶点关联的边数，度和为 2|E|；有向图中每条弧为弧尾贡献一次出度、弧头贡献一次入度，因此入度和=出度和=|E|。n 个顶点的简单无向完全图有 n(n−1)/2 条边，有向完全图有 n(n−1) 条弧。

路径、回路和连通分量

路径长度按边数计；带权路径长度按经过边的权值之和计。简单路径不重复顶点；简单回路只允许起点与终点相同，其余顶点不重复。

无向图任意两点有路径则连通；连通分量为极大连通子图，不能再添加同图中的顶点仍保持为同一连通块。有向图强连通要求每对顶点双向可达，强连通分量是极大强连通子图。一次从某点可达全部顶点，不足以说明有向图强连通。

生成树、极小与极大

连通无向图的生成树包含全部 n 个顶点和 n−1 条边；删任意树边都会断开，补任意非树边都会出现环，因此是极小连通生成子图。非连通无向图各连通分量生成树的集合是生成森林。

边数少于 n−1 的无向图必不连通，但边数达到 n−1 不保证连通。有向树还需满足根入度 0、其他结点入度 1，并从根可达全体且无环；只列入度条件不能排除另有环状分量。

图的基本操作与邻接矩阵 032 邻接矩阵用空间换取任意两点之间的直接判定。

图接口的语义

判断邻接、列出邻居、插删顶点、增删边以及读写边权是基本操作。FirstNeighbor、NextNeighbor 负责依次枚举一个顶点的邻接点；返回 −1 等标记表示枚举结束。删除顶点时也要删除所有关联边。

矩阵元素表达什么

邻接矩阵用一维表保存顶点信息，用 $|V|\times|V|$ 矩阵保存邻接关系。无权简单图可用 1 表示有边、0 表示无边；带权图可用边权表示有边、∞ 表示无边，对角线通常另设为 0。若允许权为 0 的边，就不能把所有 0 都当作无边。

无向图矩阵对称，可以压缩保存一侧；有向图第 i 行反映出边、第 i 列反映入边。对无权图数 1 可求度，对带权图应按“边是否存在”计数而非简单数非零项，且按题设排除对角线。

代价与矩阵幂

邻接判定与访问指定边权为 $O(1)$，枚举一个顶点全部邻居需扫描一行 $O(|V|)$，空间为 $O(|V|^2)$，适合稠密图。边数统计通常需扫描矩阵，无向图还要避免重复计数。

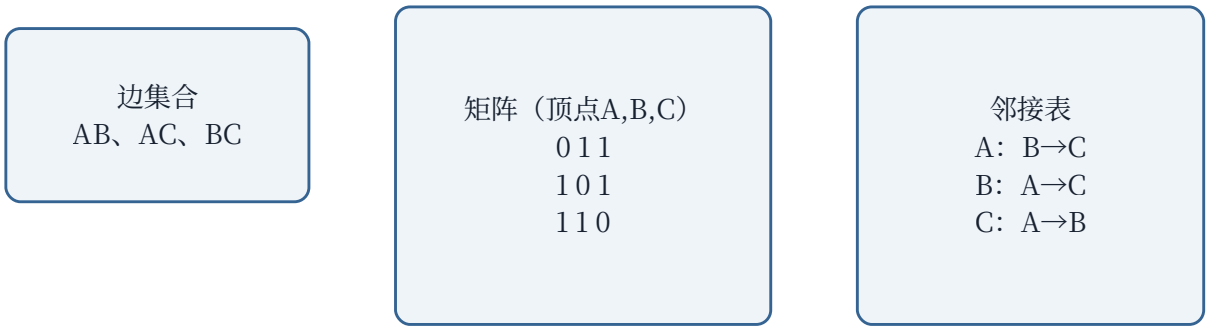
对普通 0/1 邻接矩阵 A，$(A^k)_{ij}$ 表示从 i 到 j 长度恰为 k 的游走数，允许重复顶点和边；不能一概称为简单路径条数。

结构对照图：同一无向图：矩阵与邻接表

无向简单图邻接矩阵对称，每条无向边在邻接表中对应两个邻接项。矩阵占$\Theta(V^2)$，邻接表占$\Theta(V+E)$。

使用邻接矩阵遍历全部邻边每个顶点要扫描一整行，完整BFS/DFS通常$\Theta(V^2)$；邻接表通常$\Theta(V+E)$。

同一无向图：矩阵与邻接表



单独查看图片

邻接表、十字链表与邻接多重表 033 存储中边结点出现几次，决定统计和删除时如何处理。

邻接表

邻接表由顶点表与各顶点的边链组成。顶点项保存数据和第一条边入口，边项保存邻接顶点下标、下一条边链接以及可选权值。无向图每条边通常出现在两个顶点的链中，共 2|E| 个边结点；有向图按出边存储时每条弧出现一次。

空间为 O(|V|+|E|)，适合稀疏图。无向图数顶点链长得到度；有向出边表数链长得到出度，求入度通常扫描全部表。邻居次序可因头插、尾插等实现不同而不同。

有向图十字链表

弧结点同时保存 tailvex、headvex 两端下标，tlink 连接同弧尾的下一条弧，hlink 连接同弧头的下一条弧。顶点项含 firstout 与 firstin 两个入口，从而都能直接枚举出弧与入弧；每条弧只对应一个弧结点。

无向图邻接多重表

边结点保存 ivex、jvex、ilink、jlink 和可选的 mark、info。ilink、jlink 分别接同一端点的下一条关联边，顶点 firstedge 给出入口。每条无向边只有一个结点，通过两个端点的链共同使用，适合需要按边标记或删除的处理。

图的遍历

数据结构 · 2 条

DFS 与生成森林 034 沿一条分支尽量深入，走不动再回退。

访问与回溯

从未访问顶点 v 开始，先访问并置 visited[v]，再逐个检查邻接顶点，对每个未访问邻居递归 DFS。当前结点所有邻居都已处理时返回上一层。visited 避免在环中重复访问。

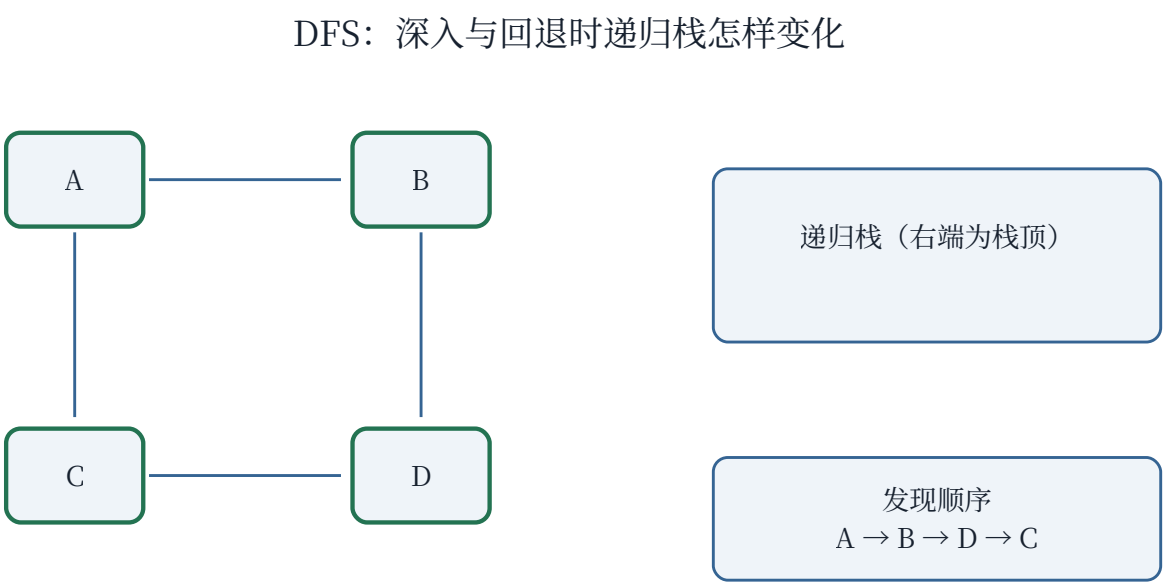
只调用一次 DFS，覆盖从起点可达的部分。遍历全部无向图应外层扫描所有顶点，遇未访问者重新启动 DFS；启动次数等于连通分量数。遍历中首次发现顶点的边形成深度优先生成树，各分量合起来是生成森林。

代价与不唯一性

邻接矩阵需要逐行扫描，遍历全部图时间 $O(|V|^2)$；邻接表每个顶点和各边项被检查常数次，时间 $O(|V|+|E|)$。访问数组及最坏递归栈为 $O(|V|)$。

起点和邻居访问次序可能改变 DFS 序列与生成树。固定顶点编号、邻接矩阵和扫描顺序后实现输出可以确定，但“图使用矩阵存储”本身不规定唯一起点或扫描顺序。

图解与逐步推演：DFS：深入与回退时递归栈怎样变化



单独查看图片

DFS：深入与回退时递归栈怎样变化

查看完整步骤（也可离线阅读）

1. 深入 A：从A开始，发现即标记；邻接访问顺序按字母升序。
2. 深入 B：A先访问B，保留A尚未完成的递归状态。
3. 深入 D：B的A已访问，转而深入D。
4. 深入 C：D的B已访问，再深入C；C邻居A、D均已访问。
5. 返回 / 结束：C结束，返回调用者D；回退不再次输出D。

6. 返回 / 结束：D结束，返回B。
7. 返回 / 结束：B结束，返回A；A的另一邻居C已访问，不再递归。
8. 返回 / 结束：A结束，递归栈清空；DFS发现顺序A、B、D、C。

BFS、最少边路径与可达性 035 按与起点的距离逐层扩展，发现时就标记。

队列控制层次

访问起点并置 visited 后入队；循环取队头，依次发现未访问邻居，立即标记并入队。若延迟到出队才标记，同一个顶点可能由不同前驱多次入队。遍历全部非连通图同样需要外层扫描未访问起点。

BFS 首次发现 v 时可记录 dist[v]=dist[u]+1、parent[v]=u。在无权图或所有边权相同的图中，得到从起点到 v 的最少边数；不可达顶点距离保持 ∞。对任意非等权图，BFS 不能代替带权最短路算法。

时间、空间与生成树

完整 BFS 用邻接表为 O(|V|+|E|)，用邻接矩阵为 $O(|V|^2)$；队列与标记最坏占 O(|V|)。首次发现边形成 BFS 生成树，但同层顶点的访问次序及最短路前驱可能不唯一。

无向连通图从任一点都能一次覆盖全图；有向图从某点覆盖全图只证明它能到达其他点，不证明其他点能返回它，不能据启动次数直接统计强连通分量。

图画推演与逐步动画

例：BFS发现时入队并标记，避免重复入队

查看完整步骤（也可离线阅读）

1. 0 · 图与访问约定：无向边A-B、A-C、B-D、C-E、D-E，从A开始，按字母顺序检查邻居。发现时标记，并放入FIFO队列。
2. 1 · A出队，发现B和C：A的邻居B、C尚未发现，将其标记并依次入队。d(B)=d(C)=1。
3. 2 · B出队，发现D：A已访问，不再入队；D首次发现，d(D)=2，前驱为B。队列由[C]变为[C,D]。
4. 3 · C出队，发现E：E首次发现，d(E)=2，前驱为C。D与E处于同一距离层。队列为[D,E]。
5. 4 · D出队，不重复加入E：D也能到E，但E在发现时已经标记，不能再次入队。E到A的最少边数仍为2，不改成经D的3。
6. 5 · E出队，遍历结束：队列为空，访问顺序A、B、C、D、E；距离依次0、1、1、2、2。无权图的最少边路径可由前驱恢复。

图的应用

数据结构 · 6 条

最小生成树与 Prim 算法 036 最小生成树优化全体连通的总边权，而非某个源点到各点的距离。

MST 的对象与性质

在带权连通无向图中，生成树总边权最小者为最小生成树。它有 $|V|-1$ 条边，最小权值总和固定，边集合可能不唯一。所有边权互不相同是 MST 唯一的充分条件，不是必要条件；原图若已是一棵树，生成树就是它本身。

Prim 维护已选顶点集合

从任一顶点加入集合 S 开始，每轮在一端在 S、另一端不在 S的边中选最小边，加入该边及新顶点。随后更新每个未选顶点连接 S 的最小边权及其前驱。直到所有顶点加入；若无法找到跨集合的有限边，则原图不连通。

普通数组实现每轮扫描未选顶点并更新连接代价，共 $O(|V|^2)$，适合稠密图；此代价对应具体实现。稀疏图用邻接表和优先队列可有不同复杂度，不能把 $O(|V|^2)$ 当作所有实现必然相同。

每轮选择的量

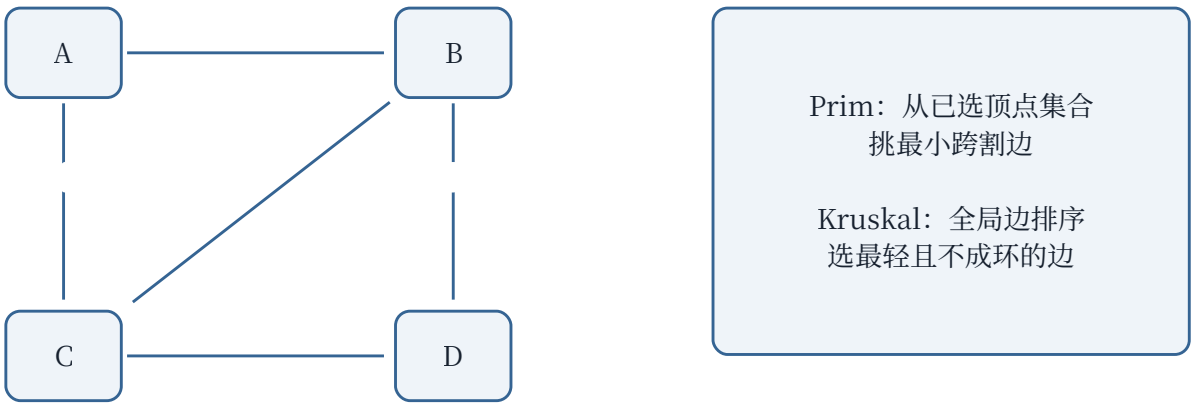
Prim 的 key[v] 是 v 连接当前生成树的一条最轻边；Dijkstra 的 dist[v] 是源点到 v 的整条路径长度。二者都逐步扩展集合，但更新公式和优化目标不同。

结构对照图：同一张图：Prim与Kruskal看边的方式

从A做Prim：先AB(1)，再BC(2)，再CD(3)，总权6。Kruskal按边权1、2、3、4、5排序，前三条已连接全部顶点。

这里所有边权不同，最小生成树唯一。两种算法都解决带权无向连通图的最小生成树问题；不是从起点到各点的最短路径。

同一张图：Prim与Kruskal看边的方式



单独查看图片

Kruskal 算法与并查集判环 037 按全局边权从小到大考察，只保留连接不同分量的边。

排序、判环、合并

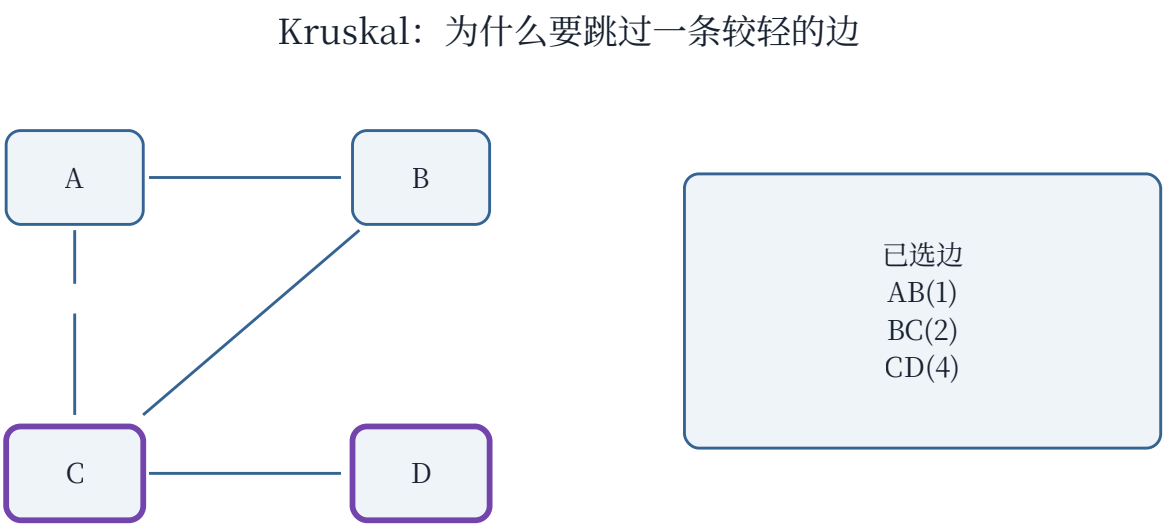
开始时每个顶点是独立分量，将所有边按权值非降序排序。依次检查边 (u,v)：Find(u)≠Find(v) 时选入并合并集合；若根相同，加入会形成环，跳过。已选 |V|−1 条边后可结束。

边排序通常 O(|E|log|E|)，并查集操作在优化后相对较小，因此总体常记为 O(|E|log|E|)。适合边较少的稀疏图。若遍历完仍不足 |V|−1 条边，则只得到最小生成森林。

与 Prim 的对照

三角形边 AB=1、BC=2、AC=4：Kruskal 先取 AB，再取 BC，AC 因两端已连通而跳过，总权 3。Prim 从 A 开始也先取 AB，再从跨集合边 BC=2 与 AC=4 中取 BC。两者构建过程不同，但都满足每步安全选边原则。

图解与逐步推演：Kruskal：为什么要跳过一条较轻的边



单独查看图片

Kruskal：为什么要跳过一条较轻的边

查看完整步骤（也可离线阅读）

1. 接受边AB，权1：两个端点属于不同连通分量，合并它们并选边。
2. 接受边BC，权2：两个端点属于不同连通分量，合并它们并选边。
3. 拒绝边AC，权3：A和C已通过A—B—C连通，再选AC会成环，故跳过。
4. 接受边CD，权4：两个端点属于不同连通分量，合并它们并选边。

Dijkstra 的松弛、定点与路径恢复 038 每轮固定当前距离最小的未确定顶点，前提是边权非负。

维护距离而非直接边权

令 dist[s]=0，其余为 ∞，重复选出未确定顶点中 dist 最小者 u，把它固定，再对其出边 (u,v) 松弛：若 dist[u]+w(u,v)<dist[v]，更新 dist[v] 和前驱。已确定的距离在非负权条件下不会再被更短路径推翻。

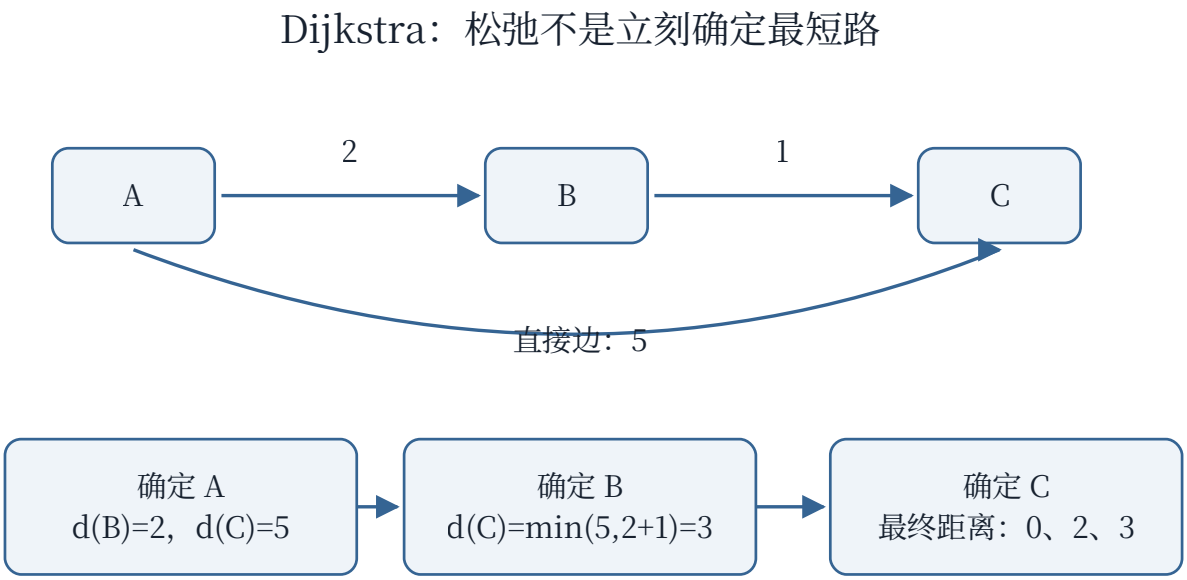
数组扫描选最小值的普通实现时间 $O(|V|^2)$；在此实现下，即便用邻接表，选点步骤仍可能主导。优先队列实现可改善稀疏图时间，需与具体数据结构对应。负边可能使已固定距离后来变小，标准 Dijkstra 不适用。

资料中的五顶点过程

从顶点 1 出发，资料图的初始可达距离为 d(2)=10、d(5)=5。先固定 5，经过 5 将 d(2) 改为 8、d(4) 改为 7、d(3) 暂为 14；再固定 4，把 d(3) 改为 13；固定 2 后，把 d(3) 改为 9。最终固定顺序可为 1、5、4、2、3。

最终路径依次为 1→5（5），1→5→4（7），1→5→2（8），1→5→2→3（9）。记录 predecessor 并从终点反向追到源点，再逆序，即可恢复路径。

机制与步骤图解



每轮选择未确定顶点中距离最小者；不能将本规则用于任意负权图。

单独查看图片

图画推演与逐步动画

例：从A到C，逐轮确定最短路径

查看完整步骤（也可离线阅读）

1. 0 · 初始化：有向非负边A→B=2、A→C=5、B→C=1。从A出发，d(A)=0，其他暂定距离为无穷。
2. 1 · 确定A并松弛A→B：未确定顶点中A的暂定距离最小，确定A。比较0+2与∞，将d(B)更新为2，记录前驱A。
3. 2 · 松弛A→C：比较0+5与∞，将d(C)更新为5，前驱暂为A。这只是暂定值，还不能宣布A→C为最终最短路。

4. 3 · 选定B：未确定的B、C中，B的2小于C的5，所以接下来确定B。不能按顶点名称或发现先后直接选C。
5. 4 · 经B再次松弛C：经B到C的候选距离为2+1=3，小于当前5，因此d(C)更新为3，前驱改为B。
6. 5 · 确定C并还原路径：最后确定C。沿前驱C←B←A反向追溯，再逆序得到A→B→C，总代价3。此算法的定点规则要求非负边权。

Floyd 的动态规划与适用范围 039 按允许使用的中间顶点逐步扩展，计算全部顶点对的距离。

递推关系与循环次序

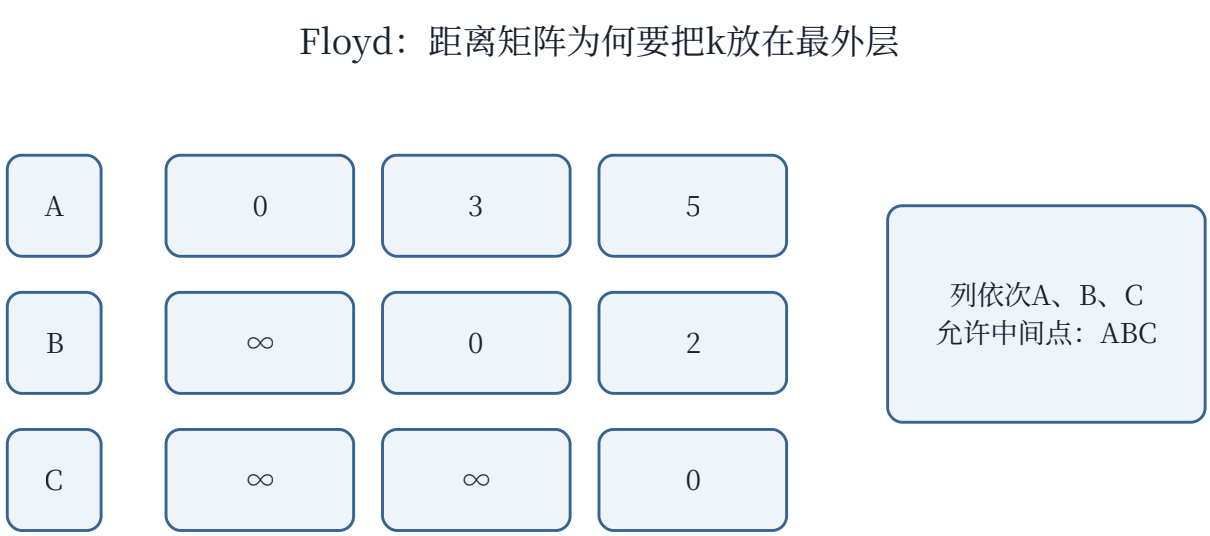
初始化 dist[i][i]=0，有边取对应权值，无边为 ∞。第 k 轮允许顶点 k 作为中间点，更新 dist[i][j]=min(dist[i][j],dist[i][k]+dist[k][j])。k 必须处于最外层，表示本轮扩展的中间顶点集合。

同时维护中转点或前驱／后继矩阵，可以恢复具体路径。三重循环时间 $O(|V|^3)$，距离和路径矩阵空间 $O(|V|^2)$。相加前应检查两段是否可达，避免用有限整数模拟 ∞ 时溢出。

负边与负环

Floyd 可以处理负权边，但通常最短路问题要求不存在可达的负权回路；负环会使受影响路径长度没有有限下界。运行后 dist[i][i]<0 可用于识别相关负环。无向图一条负边按往返游走解释便会引出负环问题。

图解与逐步推演：Floyd：距离矩阵为何要把k放在最外层



单独查看图片

Floyd：距离矩阵为何要把k放在最外层

查看完整步骤（也可离线阅读）

1. 初始距离：原有向边A→B=3、B→C=2、A→C=10。
2. 允许经过A：检查D[i][k]+D[k][j]，本轮没有更短路径。

3. 允许经过B：以B作中间点，A→C更新为min(10,3+2)=5。
4. 允许经过C：检查D[i][k]+D[k][j]，本轮没有更短路径。

DAG、AOV 网与拓扑排序 040 拓扑序表达先后约束，不保证任意两项之间都有直接关系。

活动在顶点上

有向无环图称 DAG。AOV 网把活动作为顶点，弧 u→v 表示 u 必须先于 v 完成。拓扑序要求每条有向边的起点都先于终点，每个顶点恰好出现一次；存在有向环时无法输出完整拓扑序。

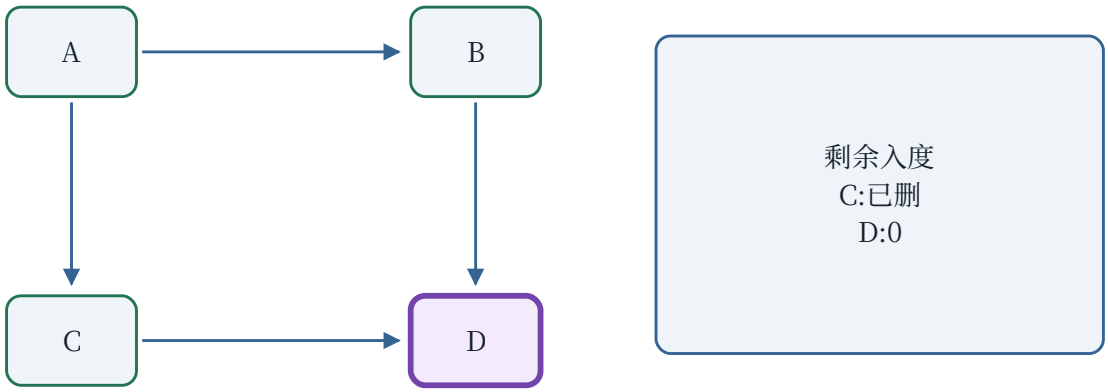
入度法

先计算所有顶点入度，把入度为 0 的顶点放入队列或栈。每取出一个顶点就输出，并对其所有出边终点入度减 1，减为 0 后加入候选集合。输出数不足 |V| 时，图中存在环。邻接表实现时间 O(|V|+|E|)。

同时有多个零入度顶点时，选择不同通常产生不同拓扑序。对当前 DAG 若每一步都只有一个可选零入度顶点，则拓扑序唯一。不要将拓扑序列中的相邻项都理解成原图存在直接边。

图解与逐步推演：拓扑排序：删除边怎样让入度变为0

拓扑排序：删除边怎样让入度变为0



单独查看图片

拓扑排序：删除边怎样让入度变为0

查看完整步骤（也可离线阅读）

1. 输出 A：选择零入度顶点并输出，删除它的出边，再把新出现的零入度顶点加入队列。
2. 输出 B：选择零入度顶点并输出，删除它的出边，再把新出现的零入度顶点加入队列。
3. 输出 C：选择零入度顶点并输出，删除它的出边，再把新出现的零入度顶点加入队列。

4. 输出 D：选择零入度顶点并输出，删除它的出边，再把新出现的零入度顶点加入队列。

AOE 网与关键路径的四组时间 041 工程最短完工时间由最长依赖路径决定。

事件、活动与约束

AOE 网用顶点表示事件，用有向边表示活动，边权为活动持续时间。事件发生后，其出边活动才可开始；所有入边活动完成后，该事件才可发生。通常设置唯一源点和汇点，多源多汇可引入虚拟事件统一处理。

源点到汇点的最长带权路径称关键路径，长度是满足依赖的最短工期。关键活动时间余量为 0；关键路径可能不止一条。

正向取最大、逆向取最小

ve 表示事件最早发生时间；vl 表示不延长总工期的最迟发生时间。活动最早开始取弧尾 ve，最迟开始取弧头 vl 再减自身持续时间。

$$ve(v)=\max_{(u,v)\in E}\{ve(u)+w(u,v)\}$$

源点 ve=0，按拓扑序向前计算。

$$vl(u)=\min_{(u,v)\in E}\{vl(v)-w(u,v)\}$$

汇点 vl=ve(汇点)，按逆拓扑序向后计算。

$$e(u,v)=ve(u),l(u,v)=vl(v)-w(u,v),d=l-e$$

d=0 的活动为关键活动；事件时间与活动开始时间不要混用。

资料的六事件八活动示例

活动为 $v_1\to v_2(3)$、$v_1\to v_3(2)$、$v_2\to v_5(3)$、$v_2\to v_4(2)$、$v_3\to v_4(4)$、$v_4\to v_6(2)$、$v_3\to v_6(3)$、$v_5\to v_6(1)$。正推得到 ve=[0,3,2,6,6,8]，逆推得到 vl=[0,4,2,6,7,8]。

按上述八条边顺序，活动余量为 [1,0,1,1,0,0,3,1]，关键路径为 $v_1\to v_3\to v_4\to v_6$，总工期 2+4+2=8。正推 v_4 时须等 v_2 和 v_3 两条入活动都完成，故取 max(3+2,2+4)=6。

缩短活动不必然缩短工期

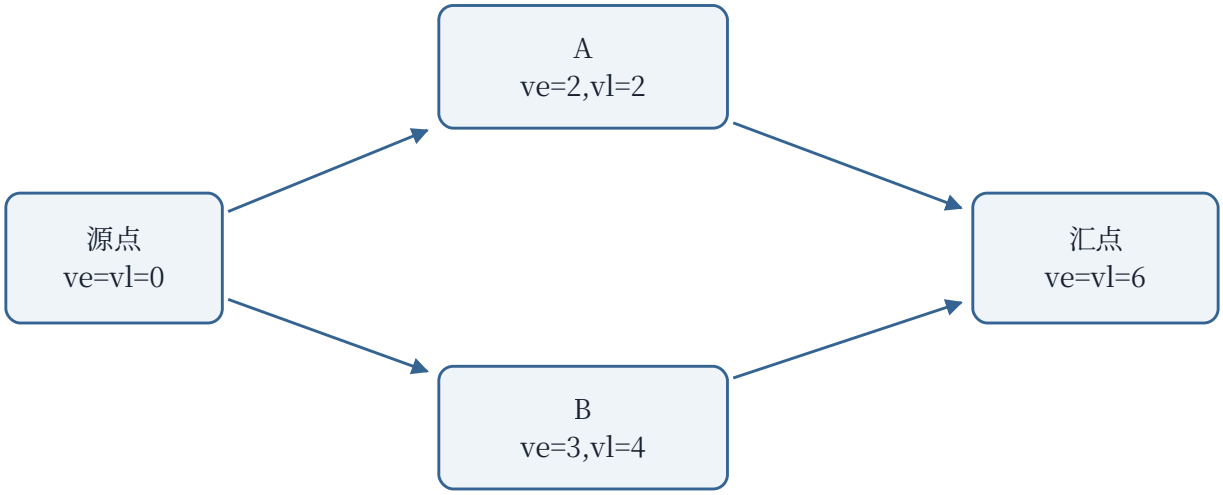
存在多条关键路径时，只缩短其中一条路径的独占活动，其他关键路径可能仍决定原工期。缩短共同关键活动或同时处理所有关键路径后才可能缩短总工期；继续缩短还可能使原非关键路径转为新的关键路径。

结构对照图：AOE网：最长工期与零余量活动

两条路径工期分别2+4=6、3+2=5，因此最短完成工期6，关键路径源→A→汇。

活动B→汇的最早开始e=ve(B)=3，最迟开始l=vl(汇)−2=4，余量1。事件ve/vl与活动e/l不能混用。

AOE网：最长工期与零余量活动



单独查看图片

查找

数据结构 · 4 条

查找表、关键字与平均查找长度 042 平均查找长度对比较次数取期望，概率条件不能省略。

静态、动态与关键字

查找是在集合中寻找满足条件的数据元素。主关键字可唯一标识记录，次关键字不一定唯一；讨论允许重复键时应明确规则。静态查找表主要做查找，动态查找表还频繁插入与删除，因此适合的数据结构可能不同。

ASL 的定义

成功与失败的平均查找长度分别计算。判定树中成功查找的比较次数通常等于目标结点层次；失败结点是空区间或空链接，不应自动额外计作一次关键字比较，须按题目约定计数。

$$ASL=\sum_{i=1}^{n} p_i c_i$$

p_i 为查找第 i 项的概率，c_i 为相应关键字比较次数；概率和为 1。

顺序查找与哨兵 043 不要求有序或随机访问，但扫描范围与结束条件要清楚。

普通扫描与哨兵

顺序查找从表一端依次比较关键字，匹配即成功，到达边界仍未匹配则失败。顺序表用下标推进，链表用 next 推进，两种存储均适用。

哨兵实现可把待查关键字临时存入保留单元，例如下标 0，从表尾向前比较，直到相等；返回 0 表示未在有效元素中找到。哨兵减少循环中的边界判断，并不减少渐近阶，且保留单元不能与有效记录冲突。

等概率的比较次数

n 个有效记录等概率查找成功时，比较次数为 1、2、…、n 的平均值，即 (n+1)/2。普通实现失败通常比较 n 个有效关键字；若把与哨兵的比较也计入，则失败为 n+1，计数口径必须统一。

有序表顺序查找可提前失败：升序扫描首次遇到大于 key 的值即可停止。若查找概率不等且不要求关键字有序，可把高概率记录排在更靠近查找起点的位置，降低成功 ASL。

折半查找的边界与判定树 044 适用于有序、支持随机访问的表，不能直接套在普通链表上。

查找区间逐步收缩

每次比较中间元素后，排除中间元素及不可能包含目标的一半；low>high 表示区间为空。若 mid 取下整或上整不同，判定树形态可能改变，但最坏时间仍 O(log n)。避免 low+high 溢出可用 low+(high−low)/2。

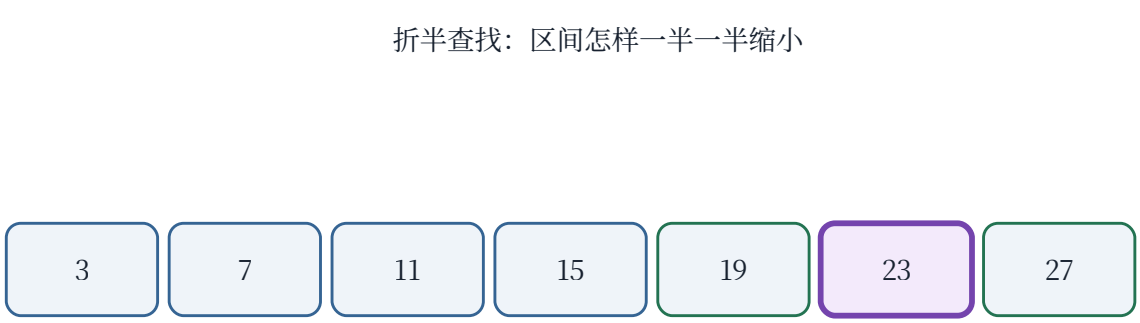
```
int binarySearch(const int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

成功与失败的判定树

n 个有效键对应 n 个内部成功结点，空区间对应 n+1 个外部失败位置。成功比较次数为根到目标的结点数，失败比较次数为到外部结点父亲的关键字比较数。普通折半判定树高度为 $\lfloor \log_2 n \rfloor + 1$（$n \geqslant 1$），不应笼统称为完全二叉树。

等概率成功 ASL 应按各层结点数加权求和，近似 O(log n)。仅在 $n = 2^h - 1$、各层全满等条件下，可直接使用按满树求和的特定公式。

图解与逐步推演：折半查找：区间怎样一半一半缩小



单独查看图片

折半查找：区间怎样一半一半缩小

查看完整步骤（也可离线阅读）

1. low=0，high=6，mid=3：查找23：比较a[3]=15。23较大，low移到mid+1；已排除位置不会重新进入。
2. low=4，high=6，mid=5：查找23：比较a[5]=23。命中，结束。

分块查找的两段代价 045 块间有序、块内可无序，先定位块再查块内。

索引表如何组织

将记录分成若干块，前块最大关键字小于后块的所有关键字。索引项保存块内最大关键字和块首地址，索引表按关键字有序。先在索引表找到目标可能所属块，再在块内顺序查找；块内无需排序。

平均查找长度分解

若均匀分为 b 块，每块 s 个元素、n=bs，索引和块内都采用顺序查找且成功等概率，则 ASL=(b+1)/2+(s+1)/2=(s^2+2s+n)/(2s)。当 s 约为 √n、b 也约为 √n 时，该表达式最小，约为 √n+1。

索引也可折半查找，则总 ASL=索引定位平均代价+块内顺序扫描平均代价。$\log_2(b+1)$ 的上取整只是索引最坏比较层数的一种表达，不能不加条件当作准确平均值。

多路查找树

数据结构 · 4 条

B 树的结点约束与高度 046 先固定“阶”和“叶结点”的定义，才能数关键字与层数。

m 阶 B 树

这里 m 表示一个结点最多有 m 个孩子，最多 m−1 个关键字。结点内关键字严格递增，k 个关键字分隔出 k+1 个子树区间；每棵子树内全部关键字落在相邻分隔键规定的范围。

非空根至少有 1 个关键字，若非终端则至少 2 个孩子；除根以外，每个实际结点至少有 t−1 个关键字，其中 t=⌈m/2⌉，非终端结点至少 t 个孩子。所有外部失败结点处于同层，它们不存实际关键字；最下层实际结点则是终端结点。

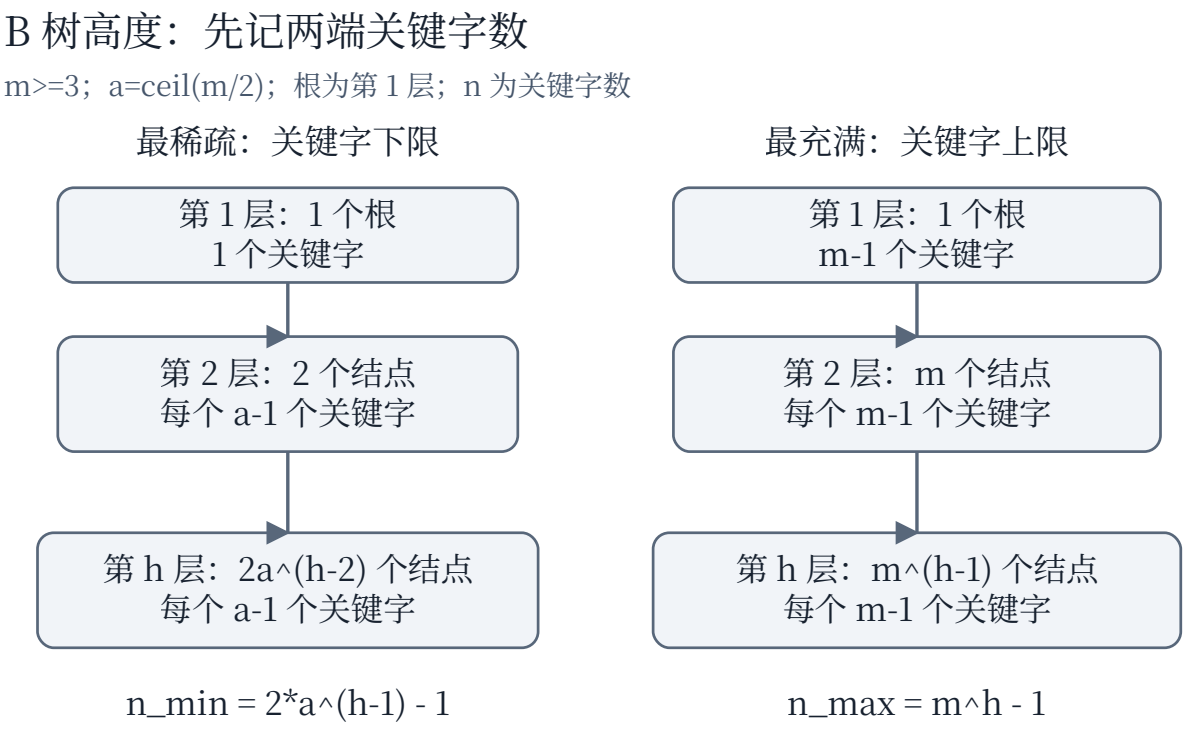
按实际关键字层数计算高度

最满时 $n \leqslant m^h-1$；最稀疏时根至少两个孩子，之后每层至少扩展 t 倍，可得 $n \geqslant 2t^{h-1}-1$。磁盘模型下访问结点的次数与树高相关，结点内部查键通常在已读入内存的数据块中完成。

$$\lceil \log_m (n+1) \rceil \leq h \leq 1+\left\lfloor \log_{\lceil m/2 \rceil} \frac{n+1}{2} \right\rfloor$$

非空 m 阶 B 树，h 不含外部失败结点层，n 为关键字总数，m⩾3。

机制图解



单独查看图片

B 树的查找、插入与分裂 047 新增关键字先落在最低实际层，超上限才向上分裂。

查找与插入定位

进入一个结点后，先在该结点的有序关键字中查找；相等则成功，否则选择包含目标区间的孩子指针，继续下降，到外部空位置则失败。插入时把新关键字加到查找失败所对应的终端结点中。

插入后关键字数不超过 m−1 则完成；达到 m 时，对结点分裂，取约定中间关键字上升到双亲，左、右部分形成两个孩子。双亲也超满时继续向上，根分裂则新增根，树高增加 1。

5 阶树的一次分裂

5 阶 B 树最多 4 个关键字。若终端结点已有 [1,2,6,7]，插入 11 后为 [1,2,6,7,11]，中间键 6 上升，左右结点为 [1,2] 与 [7,11]；若原结点就是根，则新根为 [6]。两个孩子各有 2 个关键字，满足 ⌈5/2⌉−1=2 的下限。

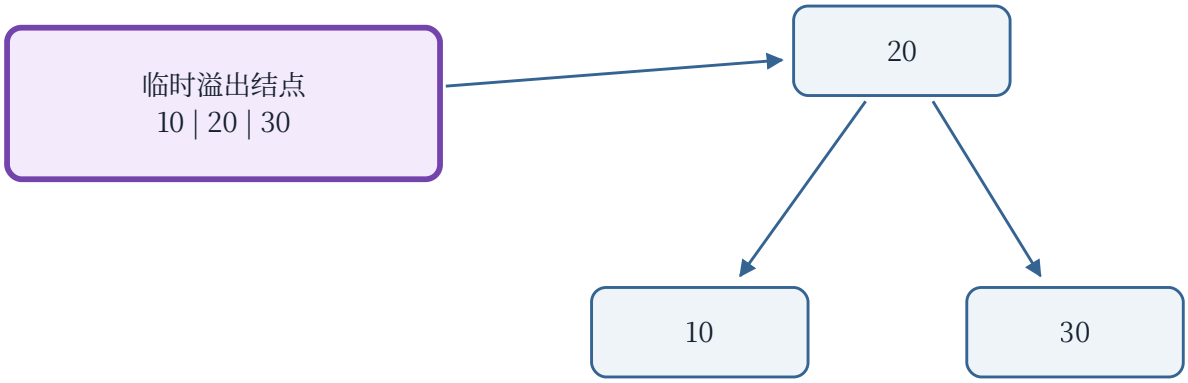
后续插入先按新根定位目标区间；一次插入可能沿路径触发多级分裂，但不会任意重排整棵树的其他分支。

结构对照图：3阶B树：溢出时中间关键字上升

3阶B树的一个结点最多2个关键字。图中是向原叶结点[10,20]插入30后的临时溢出状态。

把中间关键字20上提，10和30分别留在左右结点；若原结点为根，新建根，高度增加1。非根分裂时上提可能继续导致父结点溢出。

3阶B树：溢出时中间关键字上升



单独查看图片

B 树删除后的借位、合并与根收缩 048 真正需要检查的是删除后是否低于关键字下限。

先把内部键删除转移到终端层

若删除内部结点关键字，可以用中序前驱或后继替换，再在对应终端结点删除那个替代键。随后检查实际删除位置的关键字数。不能直接把带有子树的键及其整棵子树一并删掉。

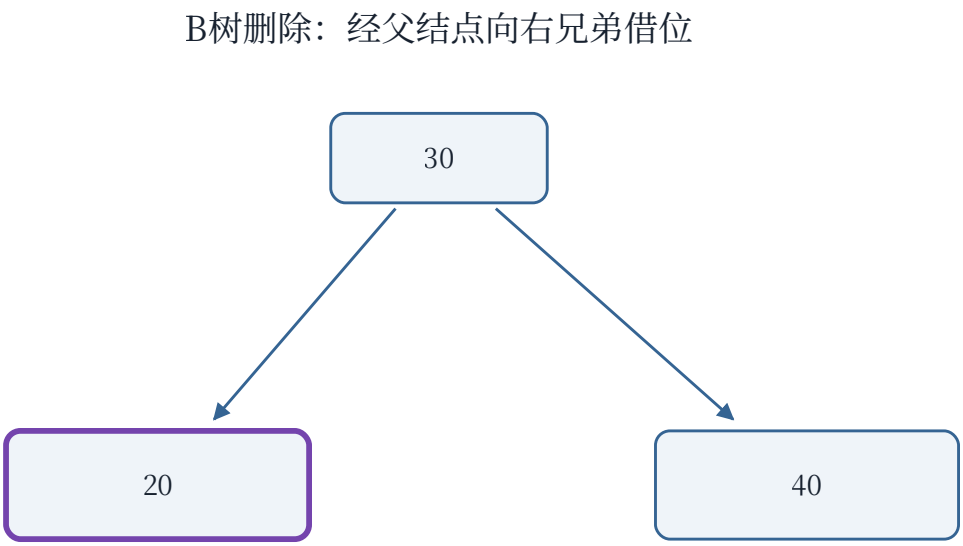
令 t=⌈m/2⌉。删除后若仍至少 t−1 个关键字，无需调整；若只剩 t−2 个，先检查相邻兄弟能否借。兄弟若至少有 t 个关键字，就通过双亲分隔键旋转：双亲键下降填空，兄弟边界键上升到双亲，相应子树指针一并转移。

不能借时合并并向上传播

若相邻兄弟都只有 t−1 个关键字，则将欠缺结点、双亲分隔键与一个兄弟合并，双亲关键字数减 1。双亲若也欠缺，继续向上借位或合并。

根不受普通结点下限约束；若根关键字减少为 0 且只有一个实际孩子，删除空根，让该孩子成为新根，树高减少 1。空树与只有根的边界应单独处理。不同阶数下“预先合并”与“删除后修复”的容量条件不同，不能不检查结点上限就直接拼接。

图解与逐步推演：B树删除：经父结点向右兄弟借位



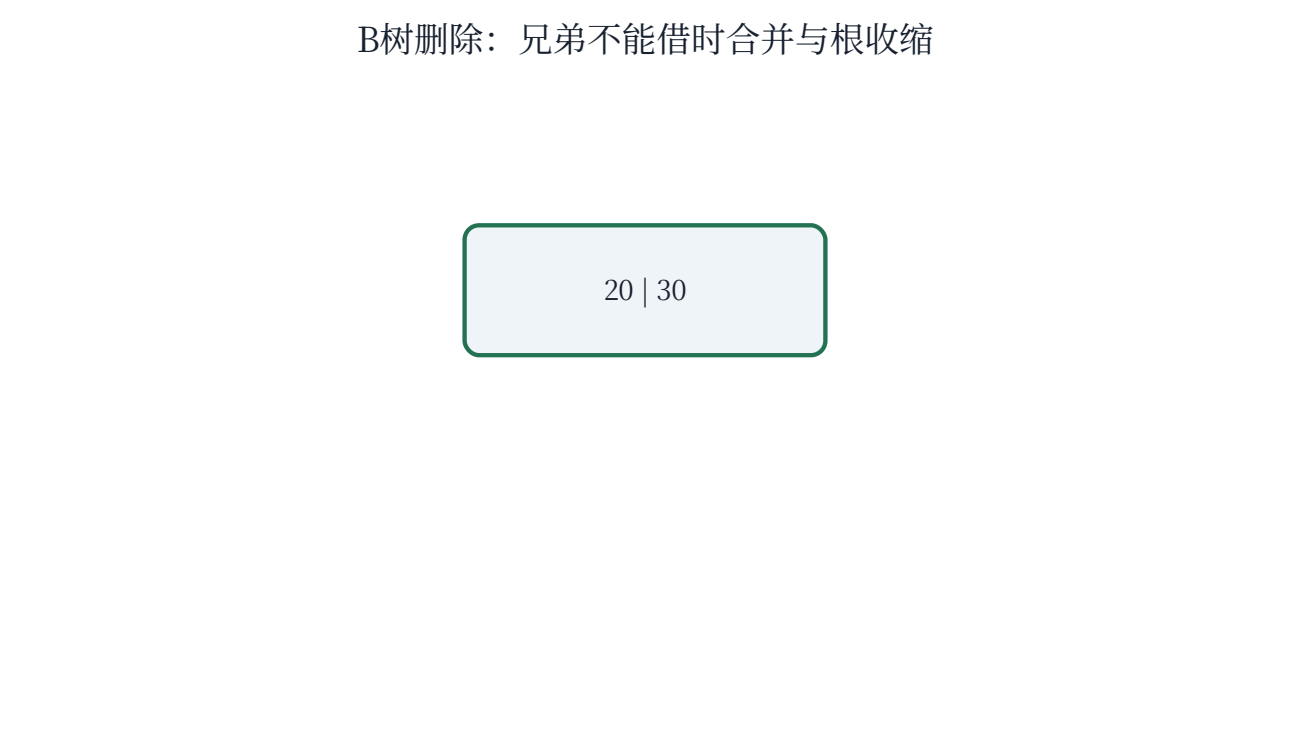
单独查看图片

B树删除：经父结点向右兄弟借位

查看完整步骤（也可离线阅读）

1. 删除前：3阶B树每个非根至少1个、至多2个关键字。根[20]，左右叶分别[10]、[30,40]。
2. 删10后左叶下溢：左叶变空，右兄弟有2个关键字，可借；不能把30直接跳过父结点搬到左叶。
3. 父键下移，兄弟最小键上移：20下移到左叶，右兄弟30上移替换父键，右叶留下40；范围顺序仍成立。

图解与逐步推演：B树删除：兄弟不能借时合并与根收缩



单独查看图片

B树删除：兄弟不能借时合并与根收缩

查看完整步骤（也可离线阅读）

1. 删除前：3阶B树根[20]，两个叶为[10]、[30]，兄弟都只有最低允许的1个关键字。
2. 删除10，不能再借：左叶变空，右叶只有30，不能再借；必须把父分隔键20与右叶合并。
3. 合并并收缩根：把空左叶、父键20、右叶30合成[20,30]。旧根变空且仅余一个孩子，因此该孩子提升为新根，高度减1。

B+ 树与 B 树的区别 049 B+ 树让内部结点只做索引，全部记录集中在叶层。

按本资料的 m 阶约定比较

性质	B 树	B+ 树
k 个关键字的子树数	通常为 k+1。	内部索引关键字数与子树数相等。
实际数据位置	各实际结点可保存记录信息。	全部记录／记录地址在叶结点，内部仅保存子树边界键及指针。
索引键是否重复出现在叶层	各结点实际关键字通常不重复。	内部索引键也在叶层数据中出现。
查找停止位置	可能在任意实际层成功。	从根查找均下降至叶层。
顺序访问	通常靠树遍历。	叶结点按关键字顺序链接，便于范围与顺序扫描。

两条入口与边界键

可同时保存根指针和最小叶结点指针，分别支持多路定位和从最小键开始的顺序访问。资料以内部分支的最大关键字作为索引边界；其他实现也可能使用最小键分隔，计算时必须沿用题设。

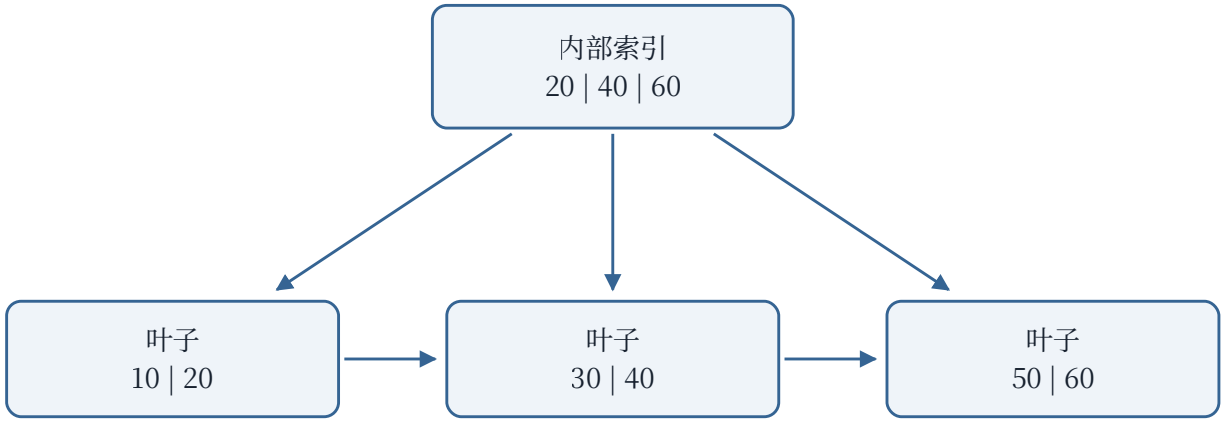
内部根至少两个孩子，其他内部结点孩子数下限通常为 $\lceil m/2 \rceil$，上限为 m；叶层的容量定义要与题目中“阶”的约定一致，不能直接照搬 B 树的 $m-1$ 个关键字上限。

结构对照图：B+树：内部索引与叶子顺序链

本图沿用正文资料的子树最大键约定：内部键20、40、60分别对应三棵子树，关键字数与孩子数相等。查找35先选最大键40对应的中间子树。

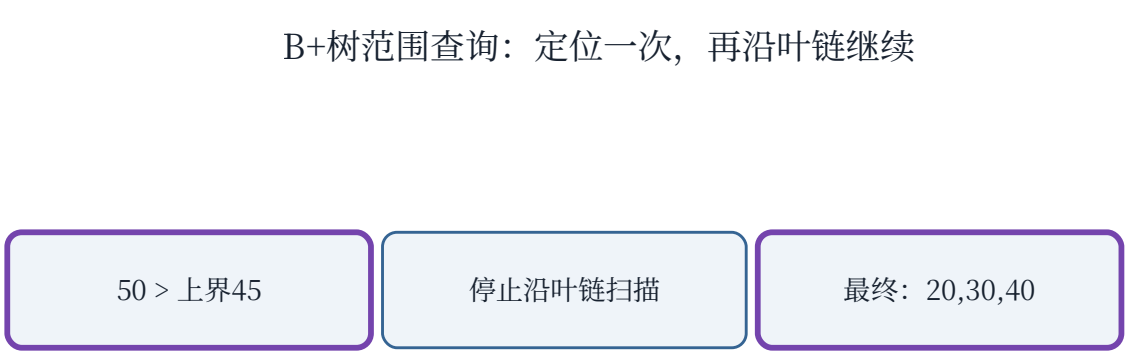
所有记录入口在叶子，内部索引键20、40、60也在叶子出现；叶子顺序链接方便扫描区间。内部命中索引键仍要下降到叶子；其他实现可能采用最小键分隔，作题须跟随题设。

B+树：内部索引与叶子顺序链



单独查看图片

图解与逐步推演：B+树范围查询：定位一次，再沿叶链继续



单独查看图片

B+树范围查询：定位一次，再沿叶链继续

查看完整步骤（也可离线阅读）

1. 查询[18,45]：沿用子树最大键约定，先从根选择上界20对应的第一个孩子，定位第一个≥18的记录。
2. 在第1个叶命中20：10小于18，不输出；20满足区间条件。
3. 顺序扫描第2个叶：进入后继叶无需每个记录都重新从根定位。
4. 见到50停止：叶层整体有序，后面所有键也不会再满足上界，因此可以停止。

散列查找

数据结构 · 2 条

散列函数、冲突与装填因子 050 散列给出初始地址，不保证不同关键字永远得到不同地址。

映射与同义词

散列函数 H(key) 把关键字映射为表中初始地址，两个不同关键字映射到同一地址称冲突，它们互为同义词。散列表的性能与散列函数分布、冲突处理方法及装填因子共同相关。装填因子 α=n/m，是记录数与表长之比。

构造方法

方法	构造与适用条件
直接定址	H(key)=key 或 a·key+b；在目标范围内保持单射时可无冲突，适合连续且范围不大的键，稀疏范围可能浪费空间。
除留余数	H(key)=key mod p，p≤表长，常选合适质数；函数只给初始位置，冲突后探查可能使用整个表长。
平方取中	取 key^2 的中间若干位，使地址尽可能综合原键各位信息，具体均匀性依赖键分布。
数字分析	在已知关键字集合中选分布较均匀的数位组合地址；键分布改变后可能需要重新选择。

查找长度看实际探查过程

成功 ASL 对各已有关键字从初始地址到实际位置的比较次数取平均；失败 ASL 按可能的初始地址与相应概率求探查到可判定不存在的代价。α 越大通常冲突和探查越多，但不能说 ASL 只由 n 决定，也不能忽略冲突策略。

开放定址的探查与链地址法 051 插入、查找与删除必须沿用一致的冲突处理规则。

开放定址的四种步长

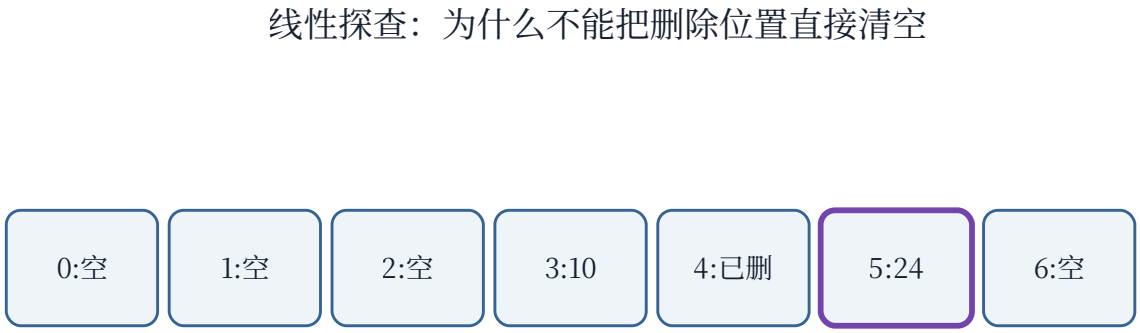
- 线性探查按 H、H+1、H+2...（模表长）寻找位置，容易形成连续聚集。
- 平方探查可按 $H\pm1^2$、$H\pm2^2$...（模表长）访问，能缓解一次聚集，但并非对任意表长都保证访问全部位置；覆盖性质依赖表长与探查公式。
- 双散列用第二个散列函数决定步长，例如 $(H_1(key)+iH_2(key)) \bmod m$；应让步长与 m 互素等，以保证探查覆盖。
- 伪随机探查使用可重复生成的伪随机步长序列；查找时必须重现相同序列。

删除标志与拉链法

开放定址删除一般不能直接改成“从未使用的空位”，否则可能截断其他键的探查链。可设置“已删除”标记，查找继续经过它，插入时按规则复用。

链地址法在每个散列桶保存同义词链入口，冲突键接入相应链表；查找在桶内比较，删除可改链并释放结点。装填因子可以大于 1，与开放定址必须保留空位的限制不同。

图解与逐步推演：线性探查：为什么不能把删除位置直接清空



单独查看图片

线性探查：为什么不能把删除位置直接清空

查看完整步骤（也可离线阅读）

1. 插入10：h(k)=k mod 7，从槽3开始线性探查[3]，放入槽3。
2. 插入17：h(k)=k mod 7，从槽3开始线性探查[3, 4]，放入槽4。
3. 插入24：h(k)=k mod 7，从槽3开始线性探查[3, 4, 5]，放入槽5。
4. 删除17只留墓碑：槽4标为已删，不能直接标成从未用过的空槽，否则查找24会错误提前停止。
5. 查找24越过墓碑：沿3→4→5：槽3是10，槽4已删仍继续，槽5命中24。只有真正未使用的空槽才可据此判失败。

树型查找

数据结构 · 5 条

二叉排序树的查找、插入与删除 052 局部改链接必须维护整个子树的关键字范围。

查找与建树

在不重复键约定下，左子树所有键小于根、右子树所有键大于根，左右子树继续满足同一性质；中序遍历得到递增序列。查找时 key 小于当前键走左，大于走右，相等成功，走到空链接失败。

插入落在失败空链接位置，新增结点起初为叶子。按输入次序从空树逐个插入即可建树；插入序列不同，树形可能不同，排序输入可能形成单支树。重复键的保留与插入方向应另按题设。

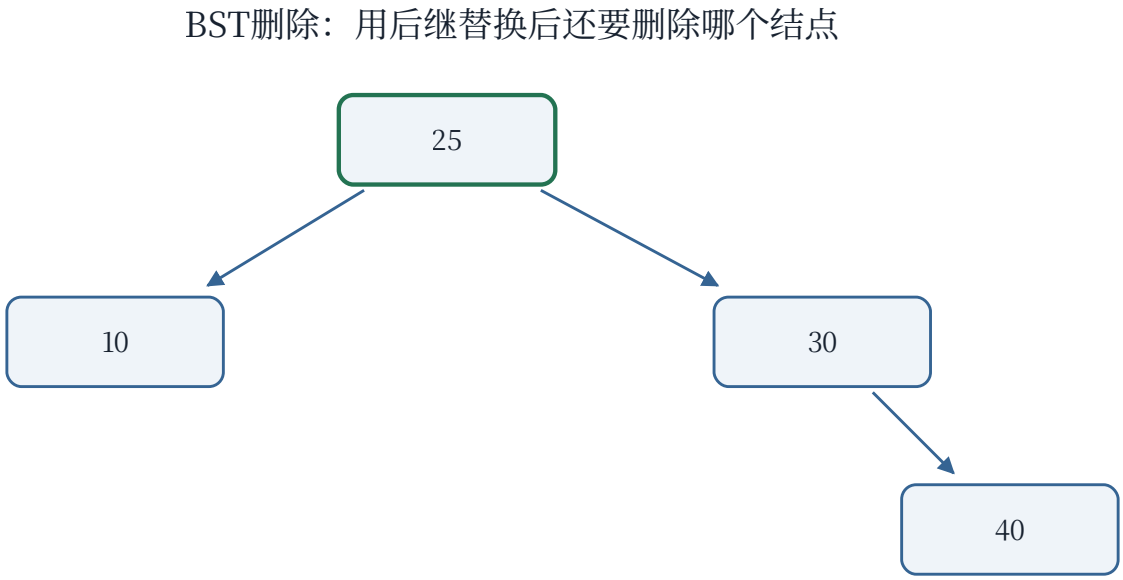
删除的三种情况

待删结点	处理
叶子	把双亲相应链接置空并释放。
只有一个孩子	让唯一孩子接到原结点双亲下，维护根删除的特殊入口。
有两个孩子	用中序前驱（左子树最右结点）或后继（右子树最左结点）替代，再删除该前驱／后继，转成前两类。

高度决定查找效率

查找沿一条根到叶路径，代价 O(h)。高度保持对数时约 O(log n)，退化单支时 O(n)；平均复杂度须说明随机输入等假设。与有序数组相比，BST 插删无需整体移动元素，但未平衡的 BST 不享有折半查找那样始终对数的最坏界。

图解与逐步推演：BST删除：用后继替换后还要删除哪个结点



单独查看图片

BST删除：用后继替换后还要删除哪个结点

查看完整步骤（也可离线阅读）

1. 删除有两个孩子的20：取右子树中最小关键字25作为中序后继；不是随便用右孩子30替换。
2. 后继关键字移到目标位置：复制或移植后继记录25到原20的位置；若用复制法，原后继结点暂时还在，需继续删除它。
3. 删除原后继位置：本例后继25是叶子，可直接摘除；一般后继无左孩子，但可能有右孩子，须接回其右子树。

AVL 的四种旋转与高度界 053 调整最小不平衡子树，并保持中序关键字次序不变。

平衡因子与旋转

AVL 是满足二叉排序性质的平衡树，每个结点平衡因子 BF=左高−右高，只允许 −1、0、1。插入后从新结点向根回溯，首先遇到 |BF|>1 的祖先是最小不平衡子树根 A。

四种插入失衡

类型	路径特征	调整
LL	插入到 A 左孩子的左子树。	对 A 右旋一次。
RR	插入到 A 右孩子的右子树。	对 A 左旋一次。
LR	插入到 A 左孩子的右子树。	先对左孩子左旋，再对 A 右旋。
RL	插入到 A 右孩子的左子树。	先对右孩子右旋，再对 A 左旋。

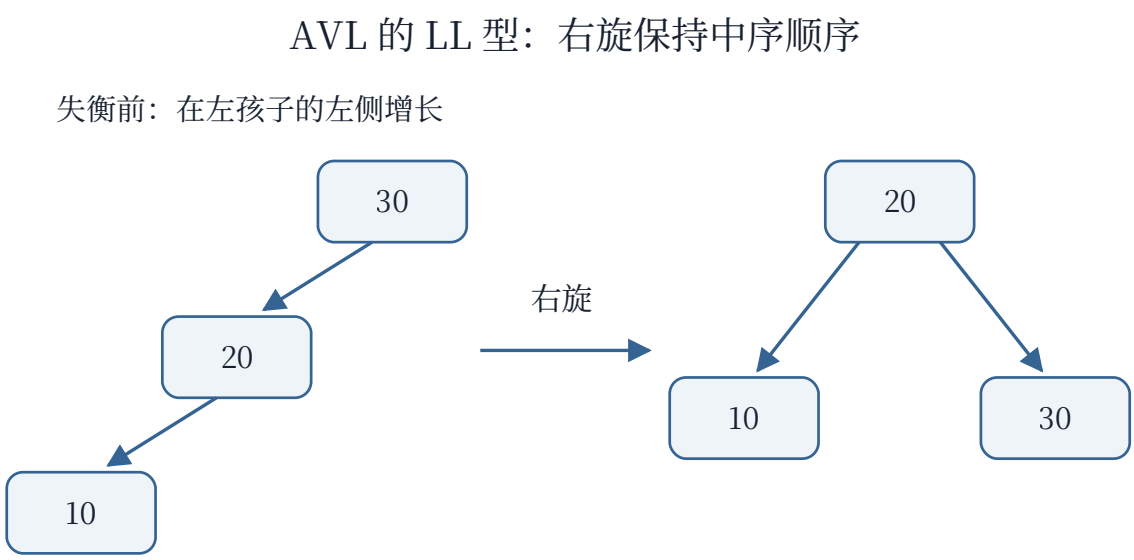
插入 15、3、7、10、9、8

插入 7 后，15 的左孩子 3 的右侧增长，发生 LR，局部根调整为 7；再插入 10、9 时，在 15 处发生 LL 右旋；插入 8 后，在 7 处发生 RL，最终根为 9，左侧为 7（孩子 3、8），右侧为 10（右孩子 15）。

删除与最少结点递推

删除可能使多级祖先失衡，因此需沿路径继续检查；删除叶子后重插不一定恢复原结构，删除分支后重插也不一定改变最终结构。高度 h 的 AVL 最少结点数 N(h)=N(h−1)+N(h−2)+1，取 N(0)=0、N(1)=1，得到高度 O(log n)，查找最坏 O(log n)。

机制与步骤图解



旋转不改变中序序列；LR 型须先对左孩子左旋，再对失衡点右旋。

单独查看图片

逐步操作与动画

例：依次插入30、20、10，处理LL失衡

查看完整步骤（也可离线阅读）

1. 0 · 空树插入30：30成为根；这里高度以结点层数计，空树高度0。
2. 1 · 插入20：20小于30，放入30的左侧。30左右子树高度差为1，仍平衡。
3. 2 · 插入10：10小于30且小于20，成为20的左孩子。30的平衡因子为2，最先失衡点是30，属于LL型。
4. 3 · 确定旋转关系：令z=30、y=20。右旋后y上升，z成为y的右孩子；y原来的右子树应接到z左侧，本例该子树为空。
5. 4 · 执行右旋：20移到根位置，30移到20右侧，10留在左侧。动画移动的是结点位置，键值不交换。
6. 5 · 更新高度并检查：先更新下降结点30的高度，再更新20。三者高度为1、2、1，平衡因子均为0；中序仍为10、20、30。

结构对照图：AVL四类失衡：先认插入方向再旋转

图以插入后沿路径找到的最低失衡祖先为根，LL/RR/LR/RL描述其重侧及下一段方向。

旋转保持中序序列不变。删除后的平衡因子和修复传播有额外情况，不能机械套用插入时“只修复一次”的结论。



单独查看图片

红黑树的约束与旋转含义 054 用较宽松的颜色约束保持对数高度，不能把它当作 AVL 的高度差限制。

红黑树的五项结构约束

- 它首先是一棵二叉排序树，每个实际结点颜色为红或黑。
- 根为黑色，外部空叶 NIL 视作黑色。

· 红结点的孩子必须为黑，因而不存在相邻红结点。

· 从任意结点到其所有后代 NIL 的路径包含相同数量黑结点，即黑高一致。

黑高与高度界

黑高通常按从当前结点向下到 NIL 的路径所包含黑结点数计，是否计当前结点需按教材约定统一。由于红结点不能连续，任意向下路径的最长长度不超过最短长度的两倍；有 n 个实际结点时，树高至多 $2\log_2(n+1)$。

一次左旋把右孩子提升为局部根，原根成为其左孩子，并把中间子树接回原根右侧；右旋完全对称。旋转保持中序次序，颜色修复保持黑高。不能将“局部旋转次数有界”误说成整个插删只需 O(1)：查找位置和向上调整仍为 O(log n)。

红黑树插入的父叔祖父分类 055 新结点先着红，避免直接改变所有经过它的路径黑高。

先检查父结点颜色

新结点若为根，染黑后结束；父为黑时无需修复；父为红则违反“不连续红”，需检查叔结点与祖父。以下以父是祖父左孩子为例，右侧情况完全镜像。

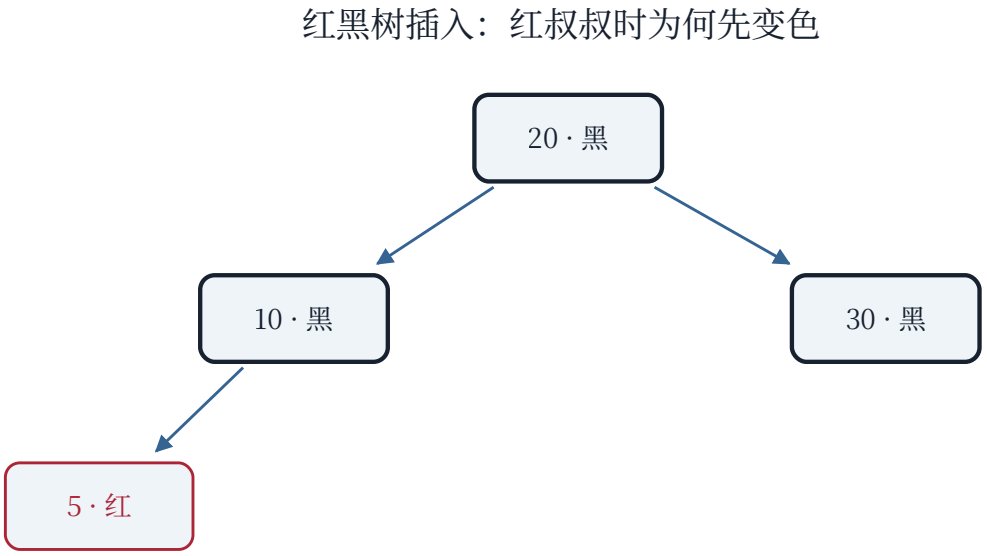
三种修复分支

条件	处理与后续
叔为红	父、叔染黑，祖父染红，把祖父当作当前结点继续向上检查；最终根仍染黑。
叔为黑，当前是父右孩子（折线 LR）	先对父左旋，把结构变为直线 LL，再按下一行处理。
叔为黑，当前是父左孩子（直线 LL）	父染黑、祖父染红，对祖父右旋，消除当前局部的连续红冲突。

NIL 叔结点

叔链接为空时，把叔当作黑色 NIL，不能省略该分支。插入修复不仅移动结点，也可能只重着色并把冲突上移。

图解与逐步推演：红黑树插入：红叔叔时为何先变色



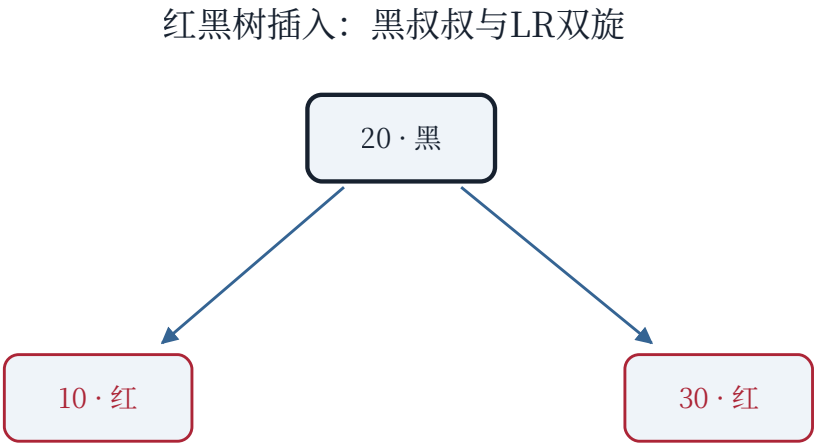
单独查看图片

红黑树插入：红叔叔时为何先变色

查看完整步骤（也可离线阅读）

1. 原树：省略的NIL叶子均为黑色；初始根20为黑、孩子10与30为红，满足红黑性质。
2. 插入5，出现红红相邻：按BST规则把5插在10左侧，新增结点先红。父10与叔30都红，不能只把5改黑。
3. 父叔变黑，祖父变红：把10、30改黑，把20改红，抵消子树黑高变化；红色冲突可能向祖先传播。
4. 根恢复黑色：20恰好是根，最后把根改黑。所有根到NIL路径具有相同黑高；本次不需旋转。

图解与逐步推演：红黑树插入：黑叔叔与LR双旋



单独查看图片

红黑树插入：黑叔叔与LR双旋

查看完整步骤（也可离线阅读）

1. 插入20后形成LR：原树根30黑、左孩子10红。插入20红成为10右孩子，父红，叔是黑色NIL。
2. 先对10左旋：先把折线变成直线：20代替10成为30的左孩子，10转为20的左孩子；中序仍为10、20、30。
3. 再右旋30并重新着色：20上升后改黑，30下沉改红，10仍红。根黑、无相邻红结点，黑高一致。

红黑树删除的黑高修复 056 先执行排序树删除，再根据实际移除结点的原颜色判断修复。

转成至多一个孩子的删除

待删结点有两个实际孩子时，用中序后继或前驱的键替代，实际移除该替代结点；它至多有一个实际孩子。移除红叶不会改变路径黑高。黑结点若只有一个实际孩子，该孩子必为红，提升该孩子并染黑即可；实际移除黑结点而替代位置仍为黑（可能为 NIL）时，需要补偿黑高。

以欠黑位置 x 是左孩子为例

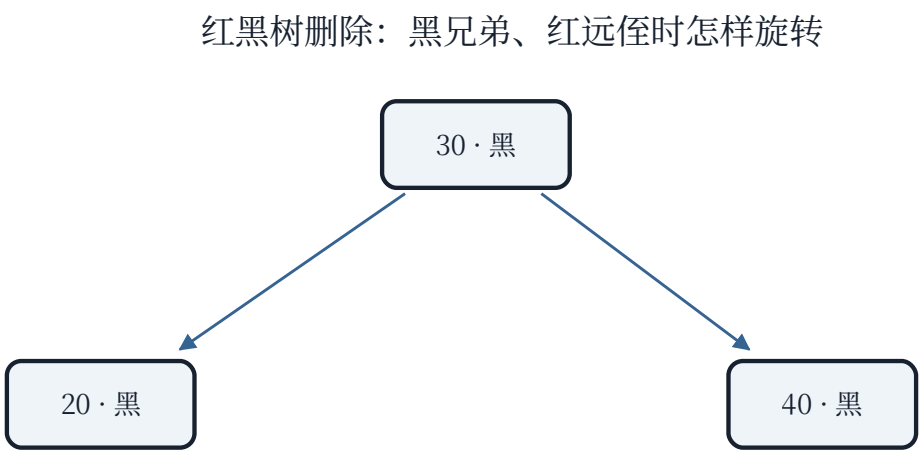
兄弟 w 的状态	修复动作
w 为红	w 染黑、父染红，对父左旋，改写成黑兄弟情形并重新取兄弟。
w 黑，两个孩子都黑	w 染红，把父视为新的欠黑位置继续向上；若父原红，可将父染黑结束。
w 黑，近侄（w 左）红、远侄（w 右）黑	近侄染黑、w 染红，对 w 右旋，转成远侄红情形。

兄弟 w 的状态	修复动作
w 黑，远侄（w 右）红	w 继承父颜色，父与远侄染黑，对父左旋，修复完成。

镜像与边界

x 是右孩子时，将左、右及近侄、远侄对应交换。欠黑只是分析修复时的状态，不是普通结点多出第三种永久颜色；根处可以吸收黑高减少。处理 NIL 时要依靠保存的父链接或哨兵，不能解引用空指针。

图解与逐步推演：红黑树删除：黑兄弟、红远侄时怎样旋转



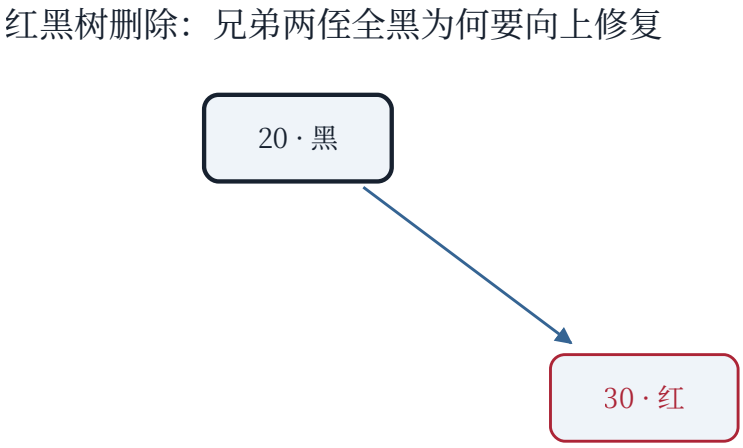
单独查看图片

红黑树删除：黑兄弟、红远侄时怎样旋转

查看完整步骤（也可离线阅读）

1. 删除10之前：根20黑，两个孩子10黑、30黑，30右孩子40红；省略的NIL全黑。这棵树初始黑高一致。
2. 删除黑叶，左侧少一重黑：删除10后用NIL代替；用“双黑”标记缺少的那一重黑，只是修复算法的记号，不是真实第三种颜色。
3. 黑兄弟的远侄为红：x在左，兄弟30在右，远侄是40。令兄弟继承父色、父20和远侄40变黑，再对20左旋。

图解与逐步推演：红黑树删除：兄弟两侄全黑为何要向上修复



单独查看图片

红黑树删除：兄弟两侄全黑为何要向上修复

查看完整步骤（也可离线阅读）

1. 原树三个黑结点：根20黑，左10黑，右30黑，未画出的NIL均黑。
2. 删去黑叶10：x由NIL替代，左侧路径少一重黑。兄弟30为黑，近侄与远侄都是黑NIL。
3. 兄弟改红，把缺黑上推：把30改红，使右侧也少一重黑；将修复位置上推到父20。父若非根，还需继续看新的父、兄弟、侄。
4. 到根后消去额外黑标记：20恰好是根，去掉额外黑标记即可结束；所有路径的黑高一起减少1，但仍彼此相同。

内部排序

数据结构 · 10 条

排序的稳定性与算法分类 057 相同关键字记录的先后关系，也是排序结果的一部分。

稳定性与内外排序

排序按关键字重排记录。若相等关键字记录在排序后仍保持输入时的相对次序，算法稳定；例如两个 49 分别标为 49_1、49_2，排序后仍需 49_1 在 49_2 前。仅有一个样例未打乱相等项，不能证明整个算法稳定。

全部记录能在内存处理称内部排序；大文件需反复读写外存，称外部排序。内部通常关注比较、移动和辅助空间，外部还必须重点计算块 I/O。

按主要思想分类

类别	主要动作	代表算法
插入	把当前记录插到已排好序的部分。	直接插入、折半插入、希尔。
交换	通过交换使记录靠近或到达最终位置。	冒泡、快速。
选择	每轮选出当前最小或最大记录。	简单选择、堆。
归并	合并两个或多个有序段。	二路归并、外部多路归并。
分配收集	按可分解关键字的各位分组。	基数排序。

直接插入排序的移动与稳定性 058 每轮扩大已排序前缀，插入时从后向前寻找空位。

算法与循环不变量

处理 a[i] 前，a[0...i−1] 已有序，后段仍待处理。先保存当前记录，向右搬移所有大于它的前缀元素，再填入空位。只移动严格大于的项，遇相等就停止，保证稳定。

```
void insertionSort(int a[], int n) {
    for (int i=1; i<n; ++i) {
        int x=a[i], j=i-1;
        while (j>=0 && a[j]>x) {
            a[j+1]=a[j];
            --j;
        }
        a[j+1]=x;
    }
}
```

49、38、65、97、76、13、27、49

依次插入后，有序前缀变化为：[38,49]；[38,49,65]；[38,49,65,97]；[38,49,65,76,97]；[13,38,49,65,76,97]；[13,27,38,49,65,76,97]；[13,27,38,49,49,65,76,97]。最后一个 49 插在原有 49 后，不跨越相等记录。

复杂度

已有序时，每轮只需常数次比较而不移动，最好 O(n)；逆序时第 i 次插入需移动 i 个记录，总量 n(n−1)/2，最坏与平均 $O(n^2)$。只使用常数临时变量，辅助空间 O(1)。适合规模较小或基本有序的序列。

图解与逐步推演：直接插入排序：有序区逐轮扩大



单独查看图片

直接插入排序：有序区逐轮扩大

查看完整步骤（也可离线阅读）

1. 初始数组：采用稳定的直接插入排序：每次把一个元素插入左侧有序前缀。
2. 插入 2：只把严格大于2的元素右移；相等元素不越过，所以稳定。
3. 插入 4：只把严格大于4的元素右移；相等元素不越过，所以稳定。
4. 插入 1：只把严格大于1的元素右移；相等元素不越过，所以稳定。

折半插入排序的比较与移动 059 改变的是定位方法，数组中腾出位置的移动仍然存在。

在有序前缀二分定位

处理第 i 个记录时，对前 i−1 个有序记录折半查找插入位置，然后把插入位置后的记录整体后移一位，再写入新记录。为了稳定，相等时应把新记录放在已有相等记录之后。

闭区间实现可令 low=0、high=i−1；若 x<a[mid] 则 high=mid−1，否则 low=mid+1，结束时 low 是插入位置。相等归右正是保持稳定的关键。

不能把查找代价当作全部代价

比较次数总计 O(nlog n)，与初始有序程度相对不敏感，但精确比较次数仍可能随定位路径不同而差一个常数；移动次数与直接插入同阶。通常最好整体为 O(nlog n)，平均、最坏仍为 $O(n^2)$，空间 O(1)。若增加“已在末尾”快速判断等优化，最好时间又可能变化，必须对应具体代码。

冒泡排序与简单选择排序 060 两者每轮都可让至少一个记录到最终位置，但稳定性与交换次数不同。

冒泡：相邻逆序交换

从左向右比较相邻元素，前者大于后者就交换，一轮把最大值推到未排序段末尾；下一轮缩小范围。也可反向扫描把最小值向前移动，但必须沿用同一方向分析每轮结果。

设置本轮是否交换的标志，若一轮无交换则整体已有序，可提前结束。相等不交换时稳定；最好 $O(n)$，平均和最坏 $O(n^2)$，空间 $O(1)$。无提前终止标志的实现，最好比较次数也可能仍为 $O(n^2)$。

简单选择：先找最小，再一次交换

在未排序段扫描出最小元素下标，与该段首元素交换，下一轮从后一个位置继续。关键字比较总数为 $n(n-1)/2$，与初始次序无关，最好、平均、最坏均 $O(n^2)$；交换次数至多 $n-1$，空间 $O(1)$。

远距离交换可能把相等记录的顺序打乱，因此简单选择通常不稳定。例如 $[2_1,2_2,1]$ 第一轮把 1 与 2_1 交换后成为 $[1,2_2,2_1]$。当记录移动代价很大、规模又不大时，较少交换可能有意义。

希尔排序与增量子序列 061 增量控制跨距离移动，最后一轮必须回到增量 1。

分组不是切成连续小段

对增量 gap，把下标相差 gap 的记录分成一组，在每组内做插入排序；再逐步缩小 gap，最后 gap=1 对全体做直接插入。前面的大步调整减少远距离逆序，最后一轮利用基本有序性。

各增量应满足最终覆盖到 1，合适增量序列影响性能；“所有增量必须互质”不是一般定义要求，不能当作判断希尔排序是否合法的必要条件。

稳定性与复杂度依赖增量

跨组跳跃可能越过相等记录，所以希尔排序不稳定，辅助空间通常 $O(1)$。复杂度随增量序列而变，不能统一断言平均必为 $O(n\log n)$。简单折半增量最坏可达 $O(n^2)$，某些精心设计序列有更好界；分析具体题目应使用给定增量和实现。

图解与逐步推演：希尔排序：增量4、2、1怎样改变数组

希尔排序：增量4、2、1怎样改变数组



单独查看图片

希尔排序：增量4、2、1怎样改变数组

查看完整步骤（也可离线阅读）

1. 初始数组：增量依次4、2、1。每趟对相距gap的元素子序列做插入排序。
2. 完成增量4的一趟：本趟结果：5,6,3,1,9,8,4,7。当gap>1时，跨组元素仍可能整体无序。
3. 完成增量2的一趟：本趟结果：3,1,4,6,5,7,9,8。当gap>1时，跨组元素仍可能整体无序。
4. 完成增量1的一趟：本趟结果：1,3,4,5,6,7,8,9。当gap>1时，跨组元素仍可能整体无序。

快速排序的划分与递归边界 062 一次划分使枢轴到最终位置，不代表左右子段都已经有序。

双指针挖坑划分示例

保存枢轴后，右侧先找小值填左空位，左侧再找大值填右空位，直到两指针相遇，最后放回枢轴。每次划分O(n)，递归只处理枢轴两侧，不再把枢轴放回子问题。不同划分方案的等号和指针推进不同，代码应成套使用。

```
int partition(int a[], int low, int high) {
    int pivot=a[low];
    while (low<high) {
        while (low<high && a[high]>=pivot) --high;
        a[low]=a[high];
        while (low<high && a[low]<=pivot) ++low;
        a[high]=a[low];
    }
    a[low]=pivot;
```

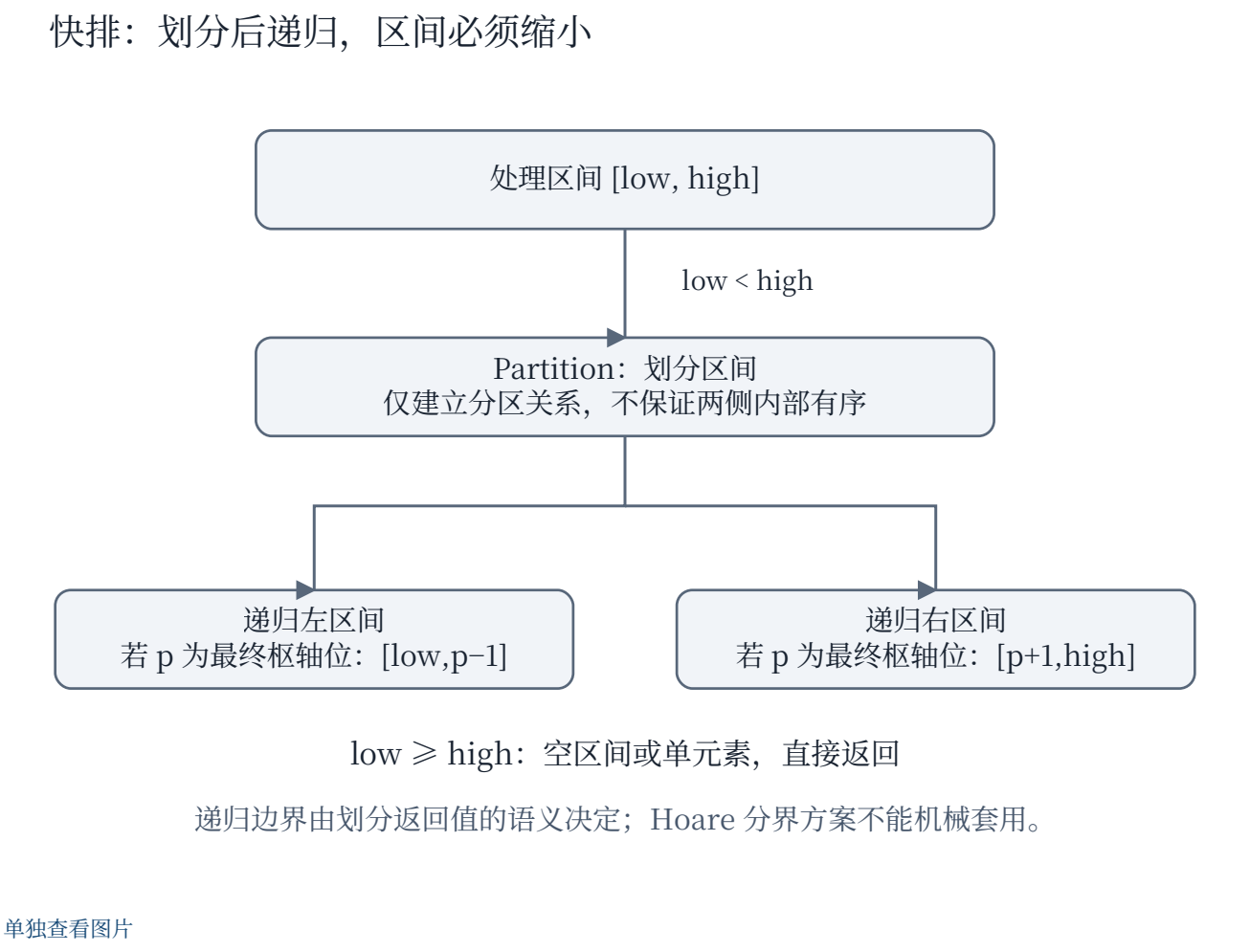
```
    return low;
}
void quickSort(int a[], int low, int high) {
    if (low>=high) return;
    int p=partition(a,low,high);
    quickSort(a,low,p-1);
    quickSort(a,p+1,high);
}
```

平衡程度决定效率

划分近乎均衡时递归深度 O(log n)，总时间 O(nlog n)；每次只排定一个端点时退化为 $O(n^2)$，栈空间最坏 O(n)。平均情况下时间 O(nlog n)，栈空间通常 O(log n)，算法不稳定。

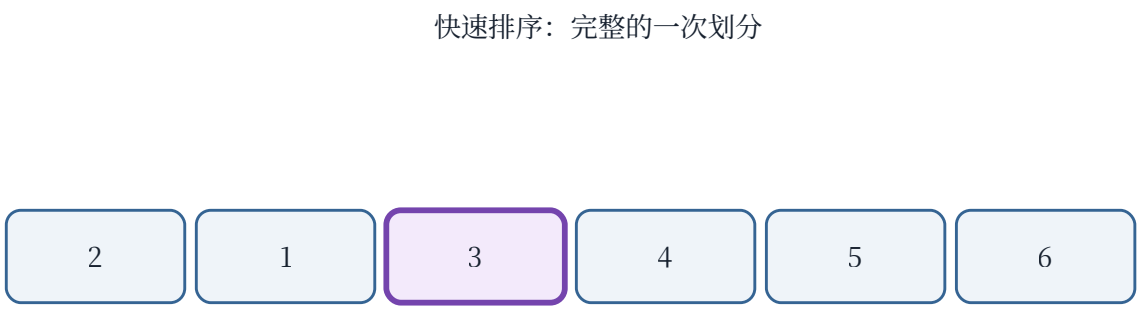
以首元素为枢轴的朴素实现遇有序序列容易退化；随机枢轴、三数取中等可降低特定输入风险。因此“不适用于有序序列”应限定为对应的朴素策略，不能套用到所有快排实现。

机制图解



单独查看图片

图解与逐步推演：快速排序：完整的一次划分



单独查看图片

快速排序：完整的一次划分

查看完整步骤（也可离线阅读）

1. 选择枢轴3：采用Lomuto划分示例，枢轴固定为末尾元素3；不同教材划分实现的中间数组可能不同。
2. 扫描下标0：检查4，维护左侧[0,0)中的元素均小于3；本步只完成划分扫描，不代表全部有序。
3. 扫描下标1：检查4，维护左侧[0,1)中的元素均小于3；本步只完成划分扫描，不代表全部有序。
4. 扫描下标2：检查6，维护左侧[0,1)中的元素均小于3；本步只完成划分扫描，不代表全部有序。
5. 扫描下标3：检查4，维护左侧[0,2)中的元素均小于3；本步只完成划分扫描，不代表全部有序。
6. 扫描下标4：检查5，维护左侧[0,2)中的元素均小于3；本步只完成划分扫描，不代表全部有序。
7. 枢轴归位：3落在最终位置2；左侧均小于3，右侧均大于3；两侧仍需递归排序。

堆的结构、筛选与线性建堆 063 堆兼有完全二叉树形状和父子大小约束。

大根堆与小根堆

大根堆每个父结点不小于孩子，根是全堆最大；小根堆反之。堆不要求左右子树相互有序，也不等于二叉排序树。用 1 起始数组时，i 的孩子为 2i、2i+1，最后一个非叶结点为 ⌊n/2⌋。

向下筛选与建堆

大根堆向下筛选先挑较大孩子，再与当前保存值比较；小根堆则选较小孩子。child≤n 判断有没有孩子，child<n 判断有没有右兄弟，不可混用。由最后一个非叶结点倒序建堆，保证处理父时其孩子子树已经是堆。

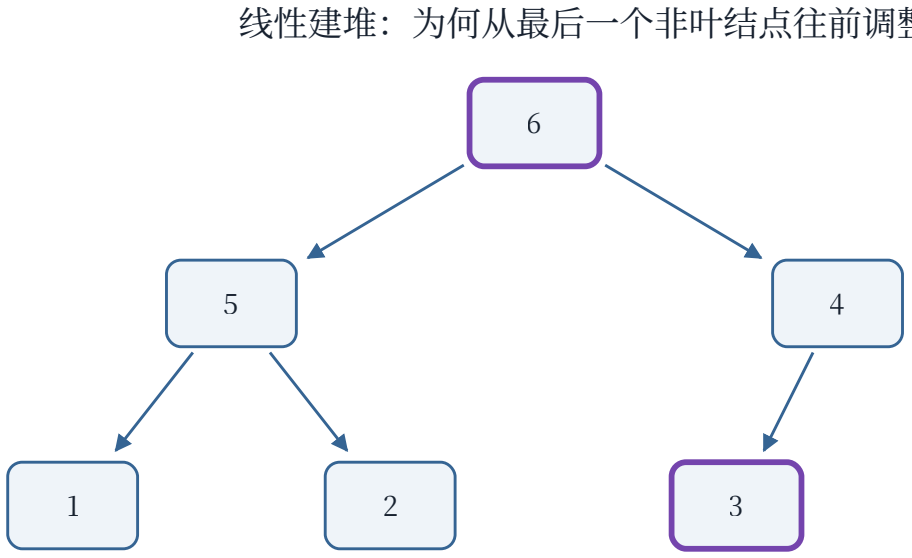
单次筛选最坏 O(log n)，但建堆各层需要下沉的高度不同，累计为 O(n)，不能把 n 个结点都机械乘上 log n。

```
void siftDown(int a[], int root, int n) {
    int x=a[root];
    for (int child=2*root; child<=n; child=2*root) {
        if (child<n && a[child]<a[child+1]) ++child;
        if (x>=a[child]) break;
        a[root]=a[child];
        root=child;
    }
    a[root]=x;
}
void buildMaxHeap(int a[], int n) {
    for (int i=n/2; i>=1; --i) siftDown(a,i,n);
}
```

小根堆构造

对 [23,17,72,60,25,8,68,71,52]，按 60、72、17、23 的根位置依次筛选：60 与较小孩子 52 交换；72 与较小孩子 8 交换；17 已小于 52 与 25；23 先与 8 交换，再与 68、72 比较停下。最终可得 [8,17,23,52,25,72,68,71,60]。

图解与逐步推演：线性建堆：为何从最后一个非叶结点往前调整



单独查看图片

线性建堆：为何从最后一个非叶结点往前调整

查看完整步骤（也可离线阅读）

1. 从下标2向下筛选：0起数组，从最后非叶结点floor(n/2)−1=2倒序处理，先保证孩子子树是堆，再修复父结点。
2. 从下标1向下筛选：0起数组，从最后非叶结点floor(n/2)−1=2倒序处理，先保证孩子子树是堆，再修复父结点。
3. 从下标0向下筛选：0起数组，从最后非叶结点floor(n/2)−1=2倒序处理，先保证孩子子树是堆，再修复父结点。

堆排序与堆的插入删除 064 排序缩小有效堆区，优先队列操作则维持全堆有效。

堆排序流程

升序排序先建大根堆，把根与当前末元素交换，最大值落到最终位置；有效堆长减 1，再从根向下筛选，重复直到剩一个元素。降序可采用小根堆对应处理。

堆排序各常见情况下为 O(nlog n)，常数辅助变量使空间为 O(1)。长距离交换与筛选可能颠倒相等记录，因此不稳定；与快排不同，它有 O(nlog n) 的最坏保证。

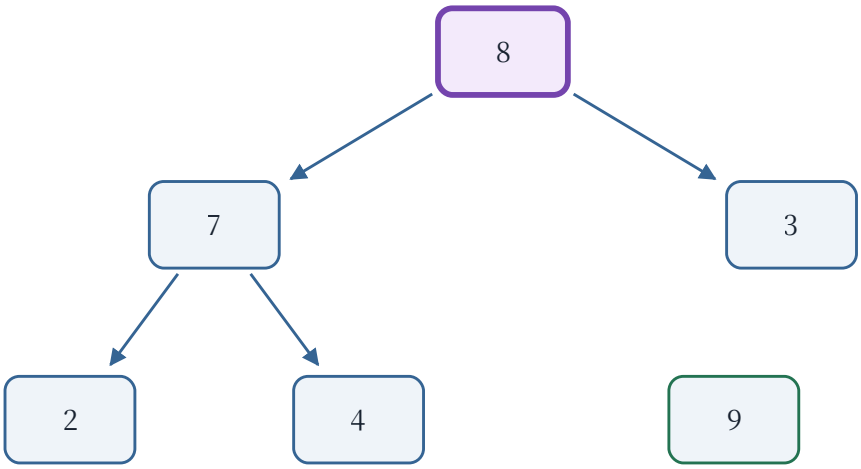
插入与删除的调整方向

插入先把新记录放在完全二叉树末尾位置，保持形状，再与双亲比较向上筛选。删除堆顶时，用最后一个结点填根位置，移除末位置，再向下筛选。

若删除任意内部位置，末元素填入后可能需向上，也可能需向下，应先与双亲比较决定；不能把所有任意结点删除都只写成向下调整。每次插入或删除通常 O(log n)。

图解与逐步推演：最大堆取出堆顶与向下筛选

最大堆取出堆顶与向下筛选



单独查看图片

最大堆取出堆顶与向下筛选

查看完整步骤（也可离线阅读）

1. 最大堆：数组表示完全二叉树，父结点关键字不小于孩子；孩子下标为2i+1、2i+2。
2. 根与末尾交换：把9移到末尾有序区；有效堆规模从6减为5。
3. 向下筛选：3与较大的孩子8交换，得到规模5的最大堆；9不再参与调整。

二路归并排序的合并与稳定性 065 两边已经有序，是线性合并成立的前提。

双指针合并

将待排序列看成长度 1 的有序段，两两归并成长度 2、4、8…的有序段，直到整表有序。递归实现先排左半，再排右半，最后合并；迭代实现按段长逐轮扩大。

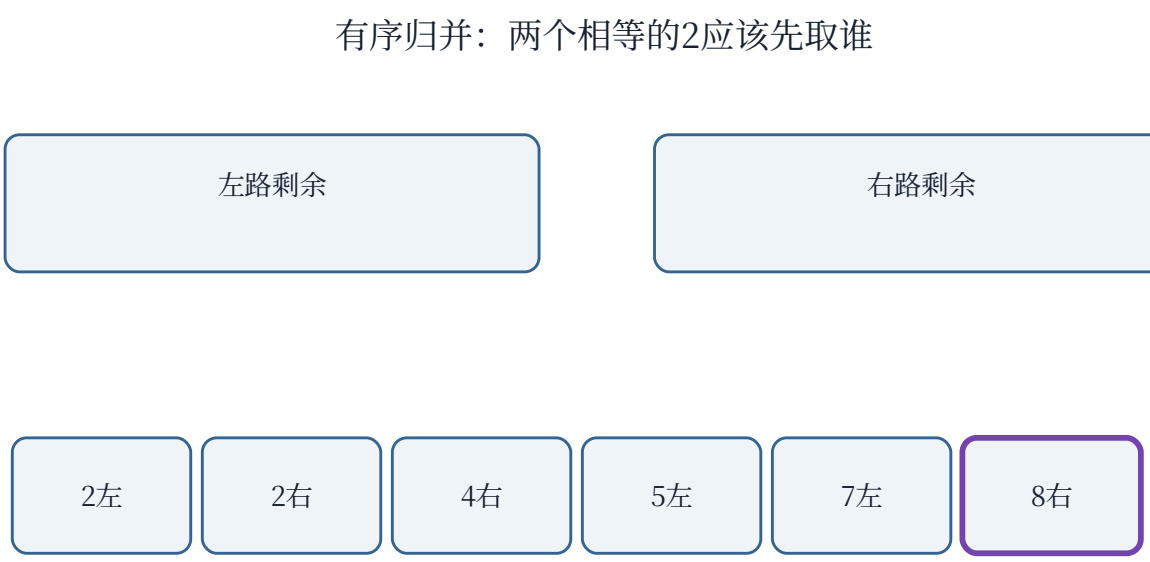
合并时把两段复制到辅助空间，用 i、j 分别指向两段首项，比较后取较小者写回；一段用尽后，另一段剩余项可以直接整体追加。遇相等键时先取左段记录，才保持原有相对次序。

时间与空间

一轮合并处理全体 n 个记录，为 O(n)；迭代二路归并约有 $\lceil \log_2 n \rceil$ 轮，总时间各常见情况 O(nlog n)。普通数组归并需要 O(n) 辅助数组，递归还有 O(log n) 调用栈，整体额外空间仍为 O(n)。

链表归并可通过改链接实现，不必复制所有记录，因此空间和移动代价不能不区分存储方式。归并适合稳定排序和外部排序，但普通数组实现不属于 O(1) 额外空间的原地排序。

图解与逐步推演：有序归并：两个相等的2应该先取谁



单独查看图片

有序归并：两个相等的2应该先取谁

查看完整步骤（也可离线阅读）

1. 输出 2（左路）：比较两路当前最小值；相等时先取左路，保持同关键字记录的原始先后次序。
2. 输出 2（右路）：比较两路当前最小值；相等时先取左路，保持同关键字记录的原始先后次序。
3. 输出 4（右路）：比较两路当前最小值；相等时先取左路，保持同关键字记录的原始先后次序。
4. 输出 5（左路）：比较两路当前最小值；相等时先取左路，保持同关键字记录的原始先后次序。
5. 输出 7（左路）：比较两路当前最小值；相等时先取左路，保持同关键字记录的原始先后次序。

6. 输出 8（右路）：比较两路当前最小值；相等时先取左路，保持同关键字记录的原始先后次序。

基数排序的 MSD、LSD 与稳定分配 066 它依赖关键字可按位分解，不能用比较排序下界直接限制。

多关键字的两种方向

把一个逻辑关键字分成 d 位，每位有 r 种取值。MSD 从最高位分组，再在各组内处理次高位；LSD 从最低位开始，逐轮稳定地按更高位分配、收集。LSD 每轮必须保持同桶记录相对顺序，否则会破坏已完成的低位次序。

分配与收集

使用 r 个队列或桶，扫描记录把它按当前位放入相应桶尾，然后按位值从小到大顺次收集各桶。每轮扫描 n 个记录并访问 r 个桶，时间 O(n+r)，共 d 轮，为 O(d(n+r))，与初始排列通常无关。

空间取决于表示：链式队列可利用原记录链接并配 r 组首尾指针，额外桶指针 O(r)；数组分配一般还要 O(n+r) 辅助空间。不能把 d 轮各自可复用的桶空间再乘 d，也不能忽略数组实现实际保存记录的位置。

适用前提

数字、定长编码等可直接分位的关键字适合；负数、可变长串及混合字符必须先约定编码、位值顺序和补齐规则。算法稳定性依赖每一轮分配收集稳定。

图解与逐步推演：LSD基数排序：个位排完为何还不算有序



单独查看图片

LSD基数排序：个位排完为何还不算有序

查看完整步骤（也可离线阅读）

1. 输入序列：采用十进制LSD基数排序，三位关键字；先个位再十位再百位，每趟分配与收集都必须稳定。
2. 按个位稳定分配并收集：本趟收集结果：720,355,436,457,657,329,839。相同当前位的元素在桶内保留上一趟的相对次序。
3. 按十位稳定分配并收集：本趟收集结果：720,329,436,839,355,457,657。相同当前位的元素在桶内保留上一趟的相对次序。
4. 按百位稳定分配并收集：本趟收集结果：329,355,436,457,657,720,839。相同当前位的元素在桶内保留上一趟的相对次序。

外部排序

数据结构 · 3 条

k 路平衡归并与磁盘 I/O 067 归并路数决定趟数，缓冲区决定一次能够同时处理多少段。

从初始有序段到完整文件

文件大于可用内存时，先分批读入、在内存排序并写出，得到若干初始归并段，再进行外部多路归并。一次最多把 k 个有序段归为一个，每轮段数由 r 变为 ⌈r/k⌉。

趟数和块读写

提高 k 或减少初始段数 r 可减少趟数。k 路归并至少要给 k 个输入段准备输入缓冲，还需要输出缓冲；若只有总共 M 个缓冲块，常见单缓冲模型要求 k≤M−1。实际双缓冲会增加需求。

增大 k 会增加缓冲空间并改变内存选最小值的成本，不能只看趟数。逐次线性比较 k 个段首每输出一条需 O(k) 比较，败者树可降到 O(log k)。

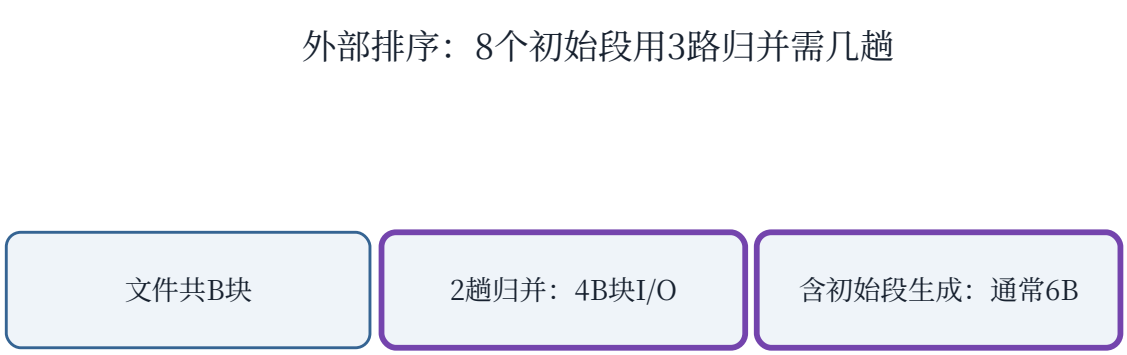
$S = \lceil \log_{k} r \rceil$

r 个初始归并段做平衡 k 路归并需要的合并趟数；r=1 时无需合并。

$I/O_{merge} \approx 2BS$

若文件共 B 个磁盘块，每轮完整读取并写回一次；初始段生成另计约 2B。

图解与逐步推演：外部排序：8个初始段用3路归并需几趟



单独查看图片

外部排序：8个初始段用3路归并需几趟

查看完整步骤（也可离线阅读）

1. 初始段已形成：假设每个缓冲一块，归并路数3。这里只数形成初始段之后的归并趟数。
2. 第1趟：8个段按最多3路合并，得到ceil(8/3)=3个更长有序段。
3. 第2趟：ceil($\log_3 8$)=2；每趟读完并写出全部记录。
4. 计算I/O：按无额外缓存、每趟全文件读写模型，初始段生成另算一次读和写。只有题目要求完整外排时才加上该部分。

置换选择与初始归并段长度 068 新记录比刚输出值小时先冻结，留给下一段。

活动集合与冻结集合

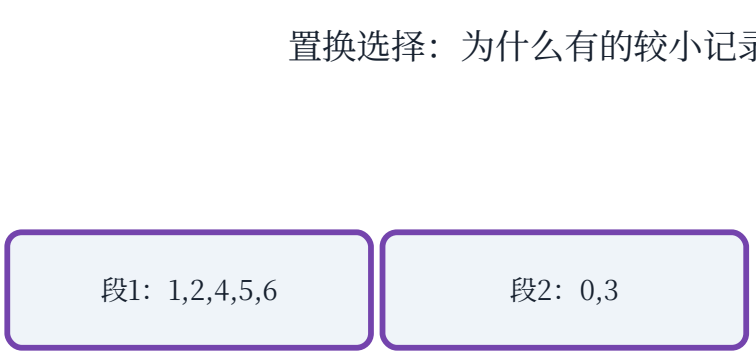
将工作区填满后，在当前可输出记录中取最小值写入当前有序段，再读入下一记录填空。新值若不小于刚输出值，可继续参与当前段；若更小，标记为下一段冻结记录。

当前活动集合耗尽时，本段结束，解除冻结开始下一段，直到输入与工作区都处理完。初始段可能长于内存可容纳记录数，且各段长度可以不同；随机输入下常见平均长度约两倍工作区大小，但具体输入并不保证这一长度。

小工作区的冻结示意

工作区最多 3 条，初始为 4、6、8，先输出 4；若补入 3，因为 3<4，冻结 3；再输出 6，补入 7，7 可留在本段；随后输出 7、8 等。冻结的 3 不应为了继续取全局最小而插入当前段，否则会破坏当前段升序。

图解与逐步推演：置换选择：为什么有的较小记录必须冻结



单独查看图片

置换选择：为什么有的较小记录必须冻结

查看完整步骤（也可离线阅读）

1. 输出1，读入2：新记录不小于刚输出值，继续参与当前段。 活动区为空才开始新段。
2. 输出2，读入6：新记录不小于刚输出值，继续参与当前段。 活动区为空才开始新段。
3. 输出4，读入3：新记录更小，冻结到下一段。 活动区为空才开始新段。
4. 输出5，读入0：新记录更小，冻结到下一段。 活动区为空才开始新段。
5. 输出6，输入已尽：不再补入新值，继续排空活动区。 活动区为空才开始新段。
6. 输出0，输入已尽：不再补入新值，继续排空活动区。 活动区为空才开始新段。
7. 输出3，输入已尽：不再补入新值，继续排空活动区。 活动区为空才开始新段。
8. 得到两个初始段：最终初始段为[1,2,4,5,6]和[0,3]。段可长于内存容量，但长度取决于输入，不保证每段恰为2M。

最佳归并树与败者树 069 一个优化归并顺序，一个优化每次从段首选择记录，职责不同。

最佳归并树

当初始段长度不同时，不同合并顺序造成记录被反复读写的次数不同。把段长作为叶权，每次选 k 个最短段合并，构造 k 叉哈夫曼式最佳归并树；必要时补 0 权虚拟叶，使叶数满足满 k 叉树条件。

每个原始段被合并的次数等于其叶深度，因此总读写量为 2×WPL（权重单位为记录或块时，结果单位也应一致）。这优化的是加权 I/O 总量，不等同于单纯让所有叶子高度相同。

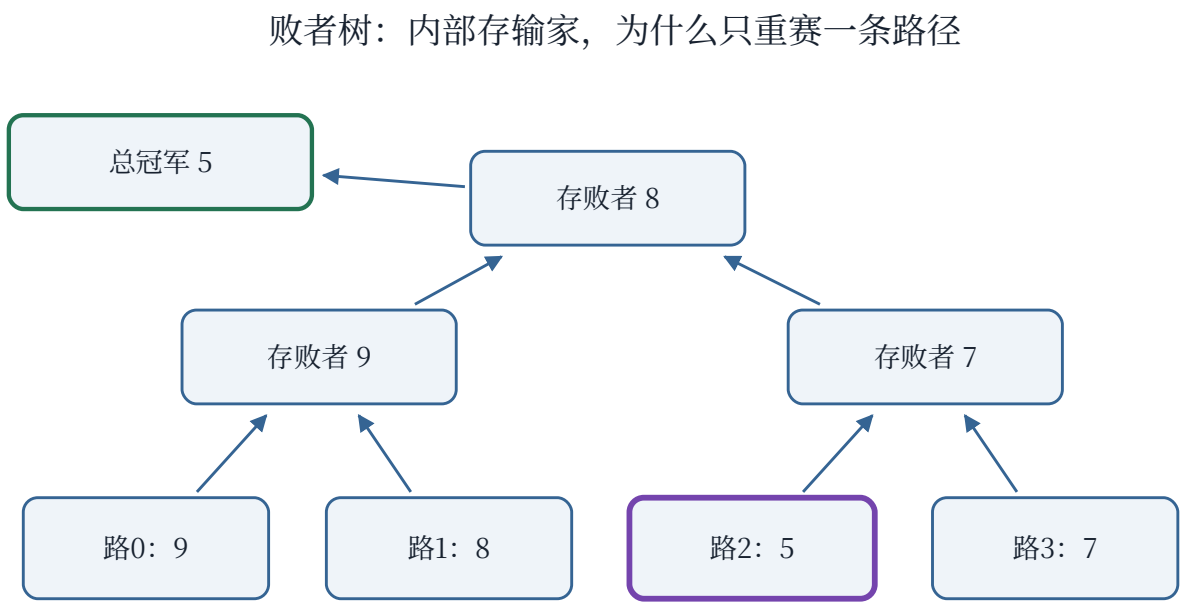
败者树保存局部比较结果

k 个叶子保存 k 个归并段当前段首记录；内部结点记录比较中的失败者，让胜者继续向上。最终总胜者的位置通常另设冠军单元保存，不能把所有内部结点都说成保存胜者。

输出最小记录后，只替换其所属段的叶子，沿该叶到根的路径与已保存对手比较，更新约 O(log k) 层。段耗尽时可用 ∞ 哨兵避免继续输出它。树本身需要 O(k) 状态，另外有输入输出缓冲，不是整体 O(1) 空间。

普通逐个插入式建树可 O(klog k)，专门自底向上构造可 O(k)；应按实际初始化方法计数。败者树减少内部比较次数，归并路数是否能扩大仍受内存缓冲约束。

图解与逐步推演：败者树：内部存输家，为什么只重赛一条路径



单独查看图片

败者树：内部存输家，为什么只重赛一条路径

查看完整步骤（也可离线阅读）

1. 建立比赛：叶子为四路段首。比较3与8，败者8留左内部结点；比较5与7，败者7留右内部结点；再比3与5，根内部留败者5，总冠军3另存。
2. 只重赛冠军路径：输出冠军3后，只从其所属第0路补9。9先与缓存败者8比，8胜、9留原内部点；8再与缓存败者5比，5胜、8留根。其他分支不用重赛。

排序的比较与选择

数据结构 · 1 条

复杂度、稳定性与使用场景总对照 070 统一实现前提后再比较，不能把某一算法的最坏界写成平均界。

常用内部排序对照

算法	最好	平均	最坏	典型额外空间	稳定性
直接插入	O(n)	$O(n^2)$	$O(n^2)$	O(1)	稳定
折半插入	O(nlog n)	$O(n^2)$	$O(n^2)$	O(1)	相等插在后方时稳定
希尔	依增量序列	依增量序列	依增量序列，简单方案可 $O(n^2)$	O(1)	不稳定
冒泡（可提前结束）	O(n)	$O(n^2)$	$O(n^2)$	O(1)	相等不交换时稳定
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	不稳定
快速	O(nlog n)	O(nlog n)	$O(n^2)$	平均 O(log n)、最坏 O(n)	不稳定
堆	O(nlog n)	O(nlog n)	O(nlog n)	O(1)	不稳定
二路归并（普通数组）	O(nlog n)	O(nlog n)	O(nlog n)	O(n)	稳定
LSD 基数	O(d(n+r))	O(d(n+r))	O(d(n+r))	链式桶 O(r)，数组通常 O(n+r)	稳定分配时稳定

按约束选择算法

- 记录少或基本有序，可优先考虑直接插入；移动昂贵且 n 较小时，简单选择交换少可能有优势。
- n 大且要求稳定、最坏 O(nlog n)，常用归并；要求 O(1) 额外空间及 O(nlog n) 最坏界，可考虑堆。
- 快排常有良好的平均实际性能，但需关注枢轴策略、递归栈和最坏情况，不能脱离实现声称它永远最快。
- 键位数少且可分解时可考虑基数；记录本体很大时，可间接排序指针或索引以减少移动。
- 若采用链式存储，要检查算法是否依赖随机访问；快排常见数组划分和堆的下标寻址不能原样照搬链表。

比较次数、趟数与下界

简单选择的比较次数不依赖初始序列；折半插入和标准归并的比较量相对稳定，但精确次数仍可能取决于比较结果与区间边界。冒泡是否提前结束、快排划分层数、堆筛选的实际比较数会受输入影响；标准堆排序主循环次数仍固定为 n−1。

交换类（冒泡、快速）及选择类（简单选择、堆）通常每轮或每次划分至少确认一个最终位置；插入中已排序前缀的记录仍可能被后续插入后移，不能把“当前有序”误认为“最终位置不变”。

对 n 个互异关键字，仅基于比较的排序要区分 n! 种输入次序，决策树最坏高度至少 $\log_2(n!)$，所以最坏比较下界为 Ω(nlog n)。基数排序利用键结构，不受这个纯比较模型下界直接限制。

计算机系统与性能

计算机组成原理 · 5 条

硬件结构：从运算器中心到存储器中心 071 先看信息存在哪里，再看部件如何协同工作。

系统的两部分

计算机系统由硬件和软件共同组成。硬件包含 CPU、存储器和外部设备；软件包括程序及其相关数据、文档。硬件提供执行环境，软件决定完成的任务。

存储程序与五大部件

- 基本功能部件是运算器、控制器、存储器、输入设备和输出设备。
- 指令与数据以二进制形式存入存储器，按地址访问；指令由操作码与地址码等字段组成。
- 操作码说明完成什么操作，地址信息用于确定操作数或下一条指令；程序按顺序存放，控制转移指令可改变执行顺序。
- 早期结构以运算器为中心，输入输出的信息经运算器传递；现代结构以存储器为中心，使输入输出尽量直接与存储器交换信息，提高整体效率。

主机、CPU 与外设

运算器与控制器组成 CPU；CPU 与主存组成主机。辅助存储器及其他主机外部设备属于外设。辅存中的程序与数据通常先调入主存，再由 CPU 处理；不能把辅存容量当成主存容量。

功能部件与寄存器：地址和数据各走哪条路 072 把寄存器的用途、位宽和数据流联系起来记。

运算器与控制器

- 运算器执行算术与逻辑运算，由 ALU、通用寄存器及暂存操作数、中间结果的寄存器等构成；ACC、MQ、变址寄存器、基址寄存器等名称以具体机器结构为准。
- 控制器统一指挥部件工作，常见组成包括程序计数器 PC、指令寄存器 IR 与控制单元 CU。PC 保存待执行指令地址；IR 保存当前取出的指令。

MAR 和 MDR

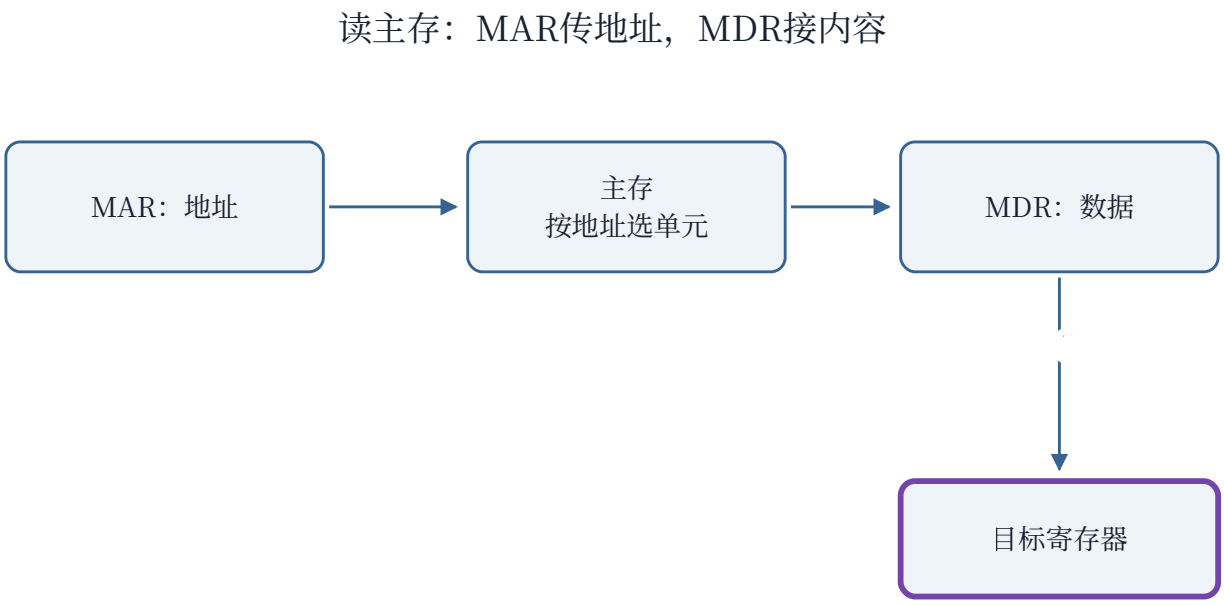
部件	保存内容	位宽关联
MAR	待访问存储单元的地址	若按编址单位寻址，n 位地址可区分 2^n 个单元
MDR	将读出或写入存储器的数据	通常对应存储字长
PC	下一条待取指令地址	应能覆盖指令地址空间
IR	当前指令	应能容纳机器相应的指令信息

取指与取数的区别

取指时，把 PC 中的地址送 MAR，由主存取出的内容经 MDR 送 IR；指令的操作码交 CU 译码。取操作数时，按照指令地址信息形成有效地址，再由主存经 MDR 送执行部件。

在寄存器传送语句中，(PC)+1→PC 的括号不能随意省去；这里增加的是 PC 内容，且增量应按实际指令长度和编址方式确定，不能机械认为所有机器都加 1。

图解与逐步推演：读主存：MAR传地址，MDR接内容



单独查看图片

读主存：MAR传地址，MDR接内容

查看完整步骤（也可离线阅读）

1. 发出地址：CPU把有效地址送入MAR，地址线用于指出本次访存位置。
2. 存储器返回数据：在读控制与时序条件满足时，主存把所选单元内容送入MDR。
3. 送入目标寄存器：把MDR中的操作数转送到目标通用寄存器；取指时则可转送到IR。

软件层次、软硬件等效与程序生成过程 073 高级语言程序经过转换后才成为机器能直接执行的指令。

软件分类与功能实现

系统软件管理资源并为其他软件提供服务，如操作系统、语言处理系统、数据库管理系统和服务程序；应用软件针对计算、设计、统计处理等具体任务。

同一逻辑功能可以由软件或硬件实现，称为软硬件在逻辑功能上等效。硬件往往执行效率高，软件往往易修改、扩展；“等效”不表示成本、速度、结构完全相同。

从源文件到可执行文件

阶段	处理内容	常见产物
预处理	处理宏定义、头文件及条件编译等指令	.i
编译	把高级语言转换为汇编形式	.s
汇编	转换成机器指令组成的可重定位目标文件	.o
链接	合并目标文件与所需库，完成符号解析和重定位	可执行目标文件

机器语言、汇编语言和高级语言

机器语言是硬件直接识别的指令编码；汇编语言以助记符表示机器指令，需要汇编程序转换。高级语言更接近问题表达，需要编译或解释等处理。编译通常先产生目标程序；解释方式通常在执行期间逐条或逐段翻译、执行，不能把两者简单等同。

字长、存储容量与数据通路带宽 074 不同“长度”描述不同对象，先明确单位。

四种宽度

指标	含义	注意
机器字长	CPU 一次整数运算通常能够处理的二进制位数	通常与整数 ALU 和通用寄存器宽度相关
指令字长	一条指令编码的位数	可能等于、短于或长于机器字长
存储字长	一个存储字包含的位数	通常与 MDR 位宽相联系
数据通路带宽	相应路径一次并行传送的数据位数	看题目指定的是内部通路还是外部数据总线

主存容量的计算

主存容量等于可编址单元数×每单元位数。按字节编址时，地址数对应字节数；按字编址时，再乘每字字节数。不要把地址线条数直接当作存储容量的位数。

取指时间不能只凭一个字长断定

当指令字长等于存储字长且一次存储访问能取出该指令时，一次访问即可完成指令读取；整个取指阶段还包含地址传送、控制等过程。机器周期和时钟周期关系取决于具体实现，须服从题设。

主频、CPI 与执行时间：先统一计算口径 075 性能由指令数、平均 CPI 和时钟频率共同决定。

时间与吞吐量

响应时间从请求发出到结果返回，可能包含 CPU 执行、I/O、排队等等待。吞吐量是单位时间完成的任务或请求数量，受 CPU、存储系统、I/O 及并发程度共同影响；提高吞吐量不必然等比例缩短单个任务的响应时间。

CPU 时间的核心关系

$T_{CPU} = N_{inst} \times CPI \times T_{clk} = \frac{N_{inst} \times CPI}{f}$

Ninst 为执行的指令条数，CPI 为该程序的平均每条指令时钟周期数，f 为主频。

$CPI_{avg} = \sum_i p_i CPI_i$

以各类指令在动态执行指令中的比例加权，不能只把各类 CPI 直接求平均。

MIPS 与浮点性能

MIPS 表示每秒执行的百万条指令，MIPS=f/(CPI×10⁶)。不同指令集、不同程序的单条指令工作量不同，不能仅用 MIPS 比较所有计算机。MFLOPS 等浮点指标计的是浮点操作数，应分清一次指令是否含多个浮点操作。主频更高也不自动代表程序执行更快。

图解与逐步推演：平均CPI：先按指令数加权，再换成执行时间



单独查看图片

平均CPI：先按指令数加权，再换成执行时间

查看完整步骤（也可离线阅读）

1. 两类指令：比例是动态执行的指令条数比例，不是执行时间比例。

2. 按条数加权：不能把1与3直接算术平均为2，除非两类条数相等。
3. 主频2GHz：T=指令数×平均CPI/主频；这里只计算CPU执行时间，是否含额外等待按题意。
4. 优化B类到CPI=2：优化只影响B类指令，整体加速不等于把局部3→2的1.5倍直接当成整体加速。

数制、编码与运算

计算机组成原理 · 13 条

进位计数制与相互转换 076 把基数、位权和有限精度联系起来。

基数与位权

基数 r 表示一个数位可使用的不同数码个数；第 i 位的位权为 r^i。一个数的真值由各位数码与位权相乘后求和得到。二进制便于由双稳态器件表示，并与逻辑真、假对应，编码和运算电路较简单。

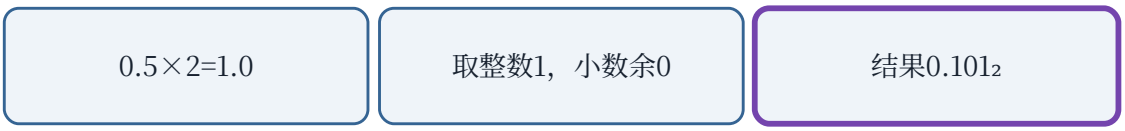
整数与小数分别转换

- 十进制整数转 r 进制：反复除以 r 取余，余数从后向前排列；最先得到的是最低位。
- 十进制小数转 r 进制：反复乘以 r 取整数部分，整数按产生顺序排列；最先得到的是最高小数位。
- 达到精度要求或小数部分变为 0 时停止；某一步乘积恰为 1.0 时，取出整数位 1 后剩余小数为 0，也应停止；不能要求所有转换都必须达到同一个乘积。
- 二进制与八进制按 3 位分组、与十六进制按 4 位分组；以小数点为界，整数左端和小数右端可补 0。

有限表示与误差

任意有限位二进制小数都能写成有限十进制小数；有限十进制小数未必能由有限位二进制准确表示。因此机器中的有限位小数是离散的，转换和运算可能需要舍入。

图解与逐步推演：0.625转二进制：乘2后取出的整数放哪里



单独查看图片

0.625转二进制：乘2后取出的整数放哪里

查看完整步骤（也可离线阅读）

1. 初始小数0.625：小数转二进制用反复乘2取整数，取出的位按出现顺序从左到右写。
2. 第一次乘2：整数1是小数点后的第一位，权值2^{-1}。
3. 第二次乘2：第二位权值2^{-2}，不能把取出的位倒过来排。
4. 第三次乘2：检验1/2+0/4+1/8=0.625。某些十进制小数不会在二进制中有限结束，需按精度要求截取或舍入。

原码、反码、补码与移码的统一辨析 077 先规定总位数及小数点位置，再谈范围与运算。

编码的分工

原码用符号位和数值位直接表达符号与绝对值；反码中负数由对应正数逐位取反形成；补码中负数可由相同位宽的正数按位取反再加 1得到。正数的原、反、补码形式相同，不能把“取反加一”无条件用于正数。

常见模型用补码表示定点整数，用原码形式表示浮点数尾数，用移码表示阶码；具体浮点标准还包含隐含位、特殊指数等约定。

n 位编码的范围与零

编码	n 位整数范围	零
原码、反码	$-(2^{n-1}-1)$ 至 $+(2^{n-1}-1)$	各有正零与负零
补码	-2^{n-1} 至 $2^{n-1}-1$	唯一的全 0

编码	n 位整数范围	零
偏置 2^{n-1} 的移码	-2^{n-1} 至 $2^{n-1}-1$	100…0

移码与补码的关系

在同一位宽且偏置为 2^{n-1} 时，移码可由补码仅翻转最高位得到。移码按无符号编码比较即可保持真值次序。IEEE 754 单精度指数偏置是 127，不能照搬“8 位移码偏置 128”；移码的具体偏置须以定义为准。

图解与逐步推演：4bit的−3：原码、反码、补码如何对应



单独查看图片

4bit的−3：原码、反码、补码如何对应

查看完整步骤（也可离线阅读）

1. 以+3为起点：本例讨论整数编码，不把小数点位置另作移动。
2. 负数原码：原码最高位表示符号，其余表示绝对值。
3. 负数反码：对于本例负数，等价于把+3的整个位串逐位取反。
4. 负数补码：补码可按模16理解；原码、反码有正负零，补码只有一个零且多表示一个最小负数。
5. 偏置8的移码：这里显式采用4bit偏置8；IEEE浮点阶码的偏置另有规定，不能机械套用同一个数值。

算术、逻辑和循环移位 078 移位规则取决于数的编码，等效乘除还取决于溢出与精度。

算术移位的补位规则

对象	左移补入	右移补入
正数原码、反码、补码	0	0
负数原码	0	0
负数反码	1	1
负数补码	0	1

逻辑与循环移位

逻辑移位把操作数当作无符号位串，左右移均补 0；高低位移出的部分被舍弃。循环移位把移出的一端重新送回另一端，可分为带进位标志 CF 与不带 CF 两种。循环移位适合字节交换、位循环处理，但不等价于普通乘除。

有限位数下的失真

算术左移可能超出表示范围：正数移出有效的 1、负数原码移出 1、负数反码或补码破坏必要的符号扩展位，都可能导致错误。右移丢失低位一般影响精度；补码算术右移相当于除以 2 后向负无穷方向取整，遇负奇数时与向 0 截断不同。

图解与逐步推演：同一位串右移：逻辑、算术与循环为何不同



单独查看图片

同一位串右移：逻辑、算术与循环为何不同

查看完整步骤（也可离线阅读）

1. 初始8bit位串：先固定8bit宽度；图中每种移位都从原位串独立进行一位右移。
2. 逻辑右移1位：最低位移出丢弃；逻辑右移适用于按无符号解释的除2取整。
3. 补码算术右移1位：对本例偶数恰好除2；一般负奇数算术右移结果常向负无穷取整，不能等同所有语言的整数除法规则。
4. 循环右移1位：循环规则是移出的位回填，不是固定补0；若原最低位为1，两种结果就不同。

加法器、进位链与 ALU 079 区分求和的并行性和进位传播的并行性。

半加器与全加器

半加器接收两个本位输入，给出本位和与进位；全加器还接收来自低位的进位，因此是三输入、两输出。全加器可由半加器与附加逻辑组合实现。

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

三个输入中 1 的个数为奇数时，本位和为 1。

$$C_i = A_i B_i + (A_i \oplus B_i) C_{i-1}$$

至少两个输入为 1 时产生进位。

串行和并行结构

串行加法器用一个全加器逐位处理，操作数由移位寄存器从低位依次送入，并用触发器保存上一步进位；器件少、成本低，但速度慢。并行加法器使用多个全加器同时处理各位，本位和仍可能等待进位，故“并行加法器”不等于“并行进位”。

串行进位链逐级传递进位；先行进位用进位产生函数 $G_i=A_iB_i$和进位传递函数 $P_i=A_i\oplus B_i$展开关系，使高位进位不必逐级等待。

ALU 与逻辑电路

ALU 根据控制信号完成算术、逻辑和移位等操作。组合逻辑电路的输出只取决于当前输入，本身不记忆状态；时序逻辑电路含存储元件，输出还与原状态有关。一个触发器保存一位，多位暂存器由多个存储元件组成。正逻辑与负逻辑对高低电平的 0、1 定义相反，同一物理电路在两种约定下可对应不同逻辑函数。

先行进位展开示例

$$C_1 = G_1 + P_1 C_0$$

本位产生进位或允许低位进位传递。

$$C_2 = G_2 + P_2 G_1 + P_2 P_1 C_0$$

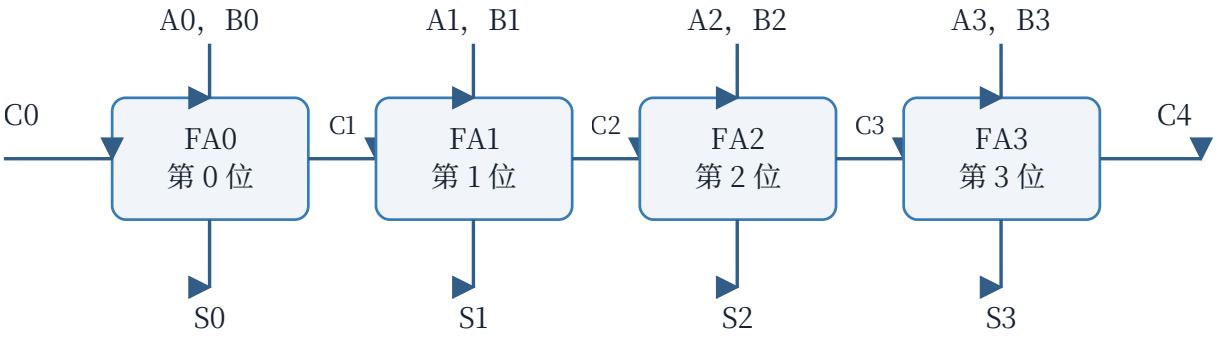
把前一级进位展开后，C_2 可直接由各位输入及初始进位形成，减少逐级等待。

图解：全加器与四位进位链

图中采用从0开始编号：C_i输入第i位，C_{i+1}从第i位输出。四个本位输入可以同时到达，但高位和仍可能等待低位进位。图上的加号表示逻辑或，乘积表示逻辑与。

四位串行进位加法器

低位在左，高位在右；蓝框为全加器，横向箭头为进位



$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_iB_i + (A_i \oplus B_i)C_i$$

例：0111 + 0001 = 1000，进位从低位逐级传到高位

图中下标：C_i 是第 i 位的输入进位；与正文按位编号的写法等价

单独查看图片

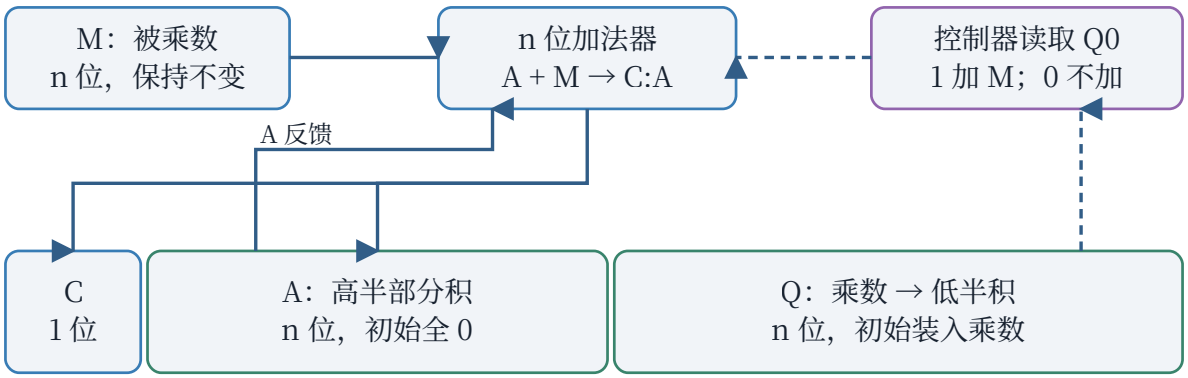
图解：无符号乘法器与寄存器变化

这是用一个加法器反复工作的移位相加模型。M保留被乘数，A保留高半部分积，Q最初装乘数并逐渐成为低半积，C暂存加法进位。每轮先看当前Q0，必要时相加，再联合右移；相加产生的进位不能提前丢弃。

例：3位无符号数3×5，M=011，初始C:A:Q=0:000:101。下表每一行给出完成联合右移后的状态。

轮次	本轮Q0与动作	右移后 C:A:Q
初始	A=0，Q=101	0:000:101
1	Q0=1，000+011=011	0:001:110
2	Q0=0，不加	0:000:111
3	Q0=1，000+011=011	0:001:111
结果	A:Q=001111_2	15=3×5

无符号乘法器：判位、相加、联合右移



每轮：读 Q0 → 必要时相加 → C:A:Q 联合逻辑右移

右移方向：C → A 的最高位；A 的最低位 → Q 的最高位

完整执行 n 轮，最终 A:Q 是 2n 位乘积

此图是无符号算法；补码 Booth 的判位与算术右移规则不同。

单独查看图片

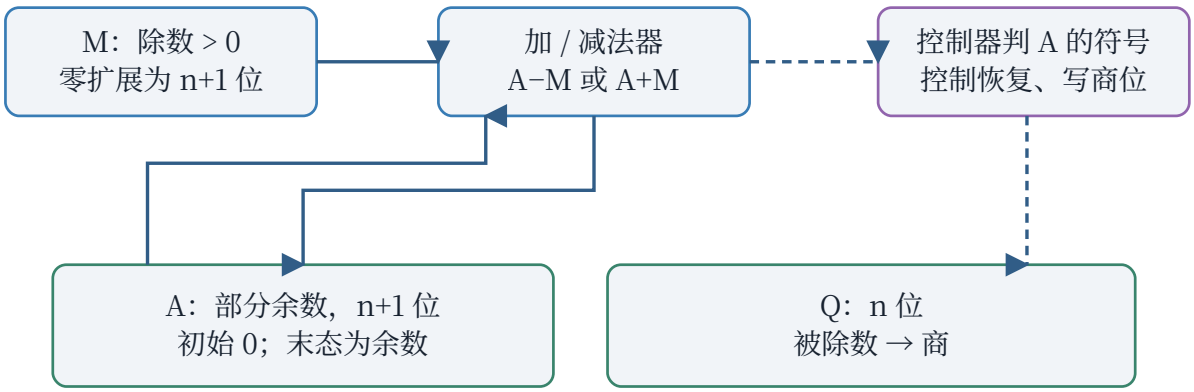
图解：恢复余数除法器

此图限定无符号n位被除数、非零n位除数、n位商。A使用n+1位保留试减的符号，M零扩展到同宽。每轮A:Q先联合左移，再试减；若试减为负，就加M恢复并写商位0，否则保留试减值并写商位1。

例：13÷3，n=4，初始A=00000、Q=1101、M=00011。表中试减按整数真值列出，便于看出何时需要恢复。

轮次	左移后的A	A−M与处理	轮末A / Q
1	1	−2：恢复，商位0	00001 / 1010
2	3	0：保留，商位1	00000 / 0101
3	0	−3：恢复，商位0	00000 / 1010
4	1	−2：恢复，商位0	00001 / 0100
核对	商4，余数1	13=3×4+1	0⩽1<3

恢复余数除法器：左移、试减、恢复



每轮先把 A:Q 联合左移，再计算 A ← A−M



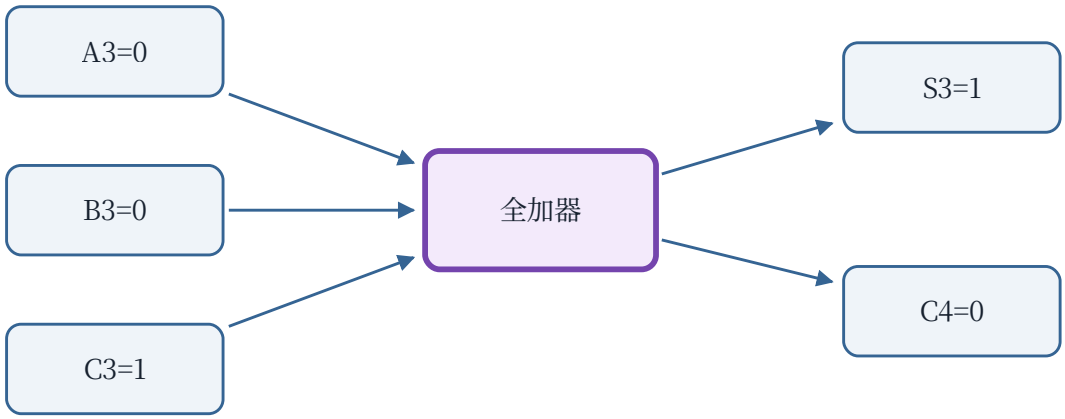
执行 n 轮：Q 为商，A 为余数；0 ⩽ A < M

边界：除数为 0 必须先处理；此模型限定 n 位无符号被除数。

单独查看图片

图解与逐步推演：逐位看进位：0111 + 0011

逐位看进位：0111 + 0011



单独查看图片

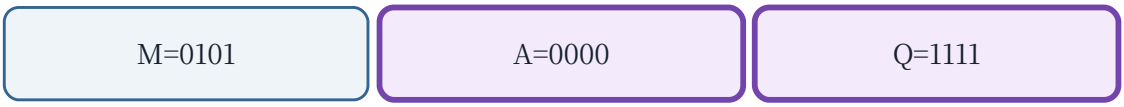
逐位看进位：0111 + 0011

查看完整步骤（也可离线阅读）

1. 第0位相加：1+1+0=2，本位和0，向高位进位1。
2. 第1位相加：1+1+1=3，本位和1，向高位进位1。
3. 第2位相加：1+0+1=2，本位和0，向高位进位1。
4. 第3位相加：0+0+1=1，本位和1，向高位进位0。

图解与逐步推演：寄存器逐轮演示：5×3的无符号乘法

寄存器逐轮演示：5×3的无符号乘法



单独查看图片

寄存器逐轮演示：5×3的无符号乘法

查看完整步骤（也可离线阅读）

1. 初始化：无符号4位乘法，M=0101(5)，Q=0011(3)，A=0000；使用5位部分积保留加法进位。
2. 轮1：检查Q_0=1：Q_0为1，加M到部分积A。
3. 轮1：整体右移：把进位、部分积和乘数寄存器连接后逻辑右移；左端补0。
4. 轮2：检查Q_0=1：Q_0为1，加M到部分积A。
5. 轮2：整体右移：把进位、部分积和乘数寄存器连接后逻辑右移；左端补0。
6. 轮3：检查Q_0=0：Q_0为0，本轮不加M。
7. 轮3：整体右移：把进位、部分积和乘数寄存器连接后逻辑右移；左端补0。
8. 轮4：检查Q_0=0：Q_0为0，本轮不加M。
9. 轮4：整体右移：把进位、部分积和乘数寄存器连接后逻辑右移；左端补0。

图解与逐步推演：长除法逐位试减：13÷3=4余1

长除法逐位试减：13÷3=4余1



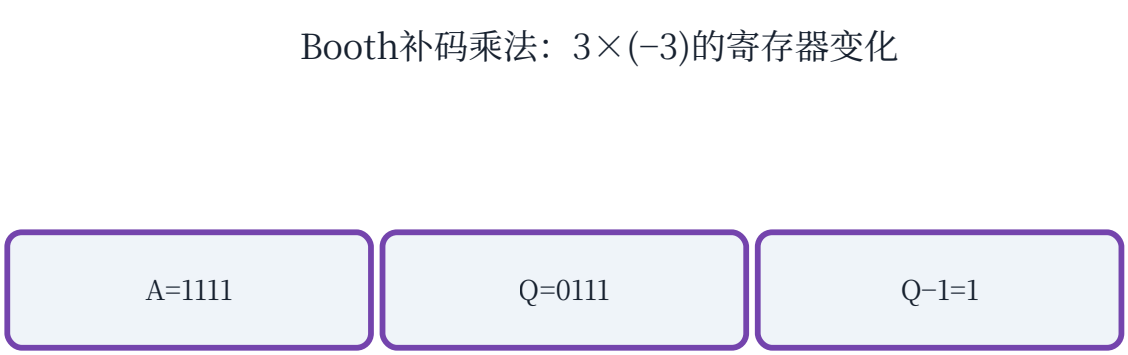
单独查看图片

长除法逐位试减：13÷3=4余1

查看完整步骤（也可离线阅读）

1. 处理第1位：1：余数左移并移入该位：1。不够减3，置商位0。
2. 处理第2位：1：余数左移并移入该位：3。够减3，减后置商位1。
3. 处理第3位：0：余数左移并移入该位：0。不够减3，置商位0。
4. 处理第4位：1：余数左移并移入该位：1。不够减3，置商位0。

图解与逐步推演：Booth补码乘法：3×(−3)的寄存器变化



单独查看图片

Booth补码乘法：3×(−3)的寄存器变化

查看完整步骤（也可离线阅读）

1. 初始化：4bit补码，M=0011(+3)，Q=1101(−3)，A=0000，附加位Q−1=0。目标3×(−3)=−9。
2. 轮1，Q₀Q−1=10：A=A−M；Booth规则：01加M、10减M、00/11不变，然后整体算术右移。
3. 轮1整体算术右移：把A、Q和附加位连接后右移1位，最高位复制原A符号位。
4. 轮2，Q₀Q−1=01：A=A+M；Booth规则：01加M、10减M、00/11不变，然后整体算术右移。
5. 轮2整体算术右移：把A、Q和附加位连接后右移1位，最高位复制原A符号位。
6. 轮3，Q₀Q−1=10：A=A−M；Booth规则：01加M、10减M、00/11不变，然后整体算术右移。
7. 轮3整体算术右移：把A、Q和附加位连接后右移1位，最高位复制原A符号位。
8. 轮4，Q₀Q−1=11：不加减；Booth规则：01加M、10减M、00/11不变，然后整体算术右移。
9. 轮4整体算术右移：把A、Q和附加位连接后右移1位，最高位复制原A符号位。 四轮后A|Q=11110111，8bit补码为−9；附加位不属于乘积。

补码加减与溢出：符号、进位和双符号位 080 运算器处理同一位串，是否溢出须按解释方式判断。

加减运算规则

补码加法连符号位一起相加，超出固定位宽的进位舍去；减法转为加被减数的相反数补码。舍弃最高进位不等于发生有符号溢出。

$$[A-B]_{comp}=[A]_{comp}+[-B]_{comp}\ (\mathrm{mod}\ 2^n)$$

固定 n 位补码按模 2^n 运算；最小负数的相反数不在同位宽正数范围内，须注意边界。

三种检测思路

- 同号相加得到异号结果，发生有符号溢出；异号相加不会溢出。减法须检查“被减数与减数异号，结果符号又与被减数不同”，不能把加法同号规则原样套用。
- 最高数值位向符号位的进位与符号位向外的进位异或，结果为 1 时有符号溢出。
- 双符号位结果为 00 或 11 时未溢出；01 为正溢出，10 为负溢出，最高位保留结果本应有的符号。双符号位连数值位一起参加运算，最外层进位舍弃。

定点小数与乘法边界

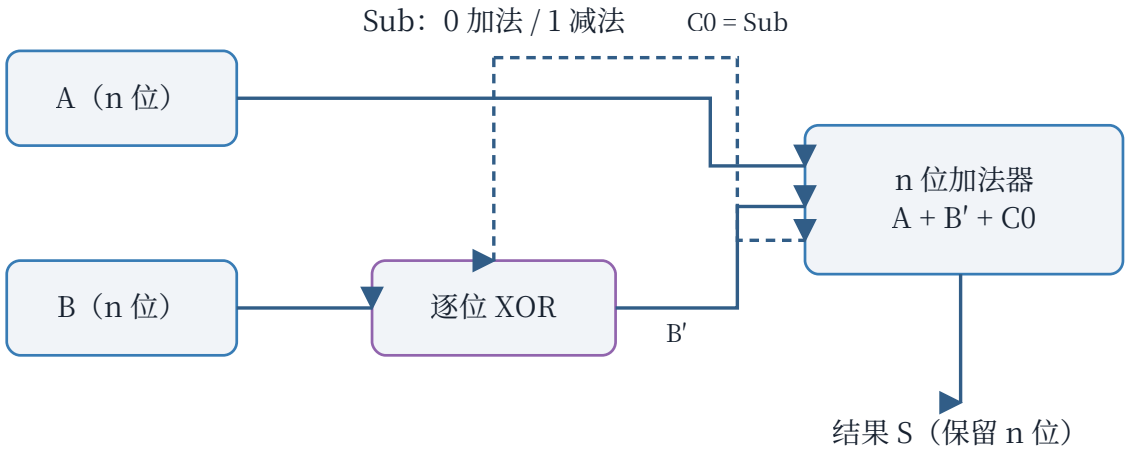
补码定点小数的范围包含 −1 而不包含 +1，因此结果≥1 为正溢出，结果<−1 为负溢出。n 位乘 n 位用完整 2n 位保存结果可容纳乘积；若只保留低 n 位，无符号乘积需高 n 位全 0，有符号乘积需被舍弃的高位全部是保留结果符号位的正确扩展，等价于乘积前 n+1 位全 0 或全 1。

图解：加减法共用一套加法器

Sub同时连接每一位B输入前的异或门以及最低位进位。Sub=1时，B按位取反且最低进位为1，完成补码减法；Sub=0时B不变且最低进位为0。

例：4位运算0101−0011，先得~B=1100，再算0101+1100+1=1 0010，保留0010，即2。最外层进位被舍弃，这一次没有有符号溢出。

同一个加法器怎样完成减法



Sub=0：A+B；Sub=1：A+~B+1 = A−B（模 2^n）

虚线：控制信号；实线：数据通路。最高进位与有符号溢出分别判断。

单独查看图片

图解与逐步推演：补码最小负数：按位取反加1为何仍是自己



单独查看图片

补码最小负数：按位取反加1为何仍是自己

查看完整步骤（也可离线阅读）

1. 4bit补码最小值：取相反数应为+8，但+8不在4bit有符号范围内。
2. 按位取反：电路按低4位执行取反加1，结果位串回到1000。
3. 识别有符号溢出：这是固定宽度补码取负的边界溢出，位串结果不代表数学真值已正确保存。
4. 先扩宽再取负：扩大到能表示+8的位宽后再做取负，便可得到正确结果。实际C最小整数取负须遵循类型范围与溢出规则。

ZF、SF、OF、CF：四个标志的含义 081 同一次加减会同时产生标志，但比较时选择的标志不同。

四类状态位

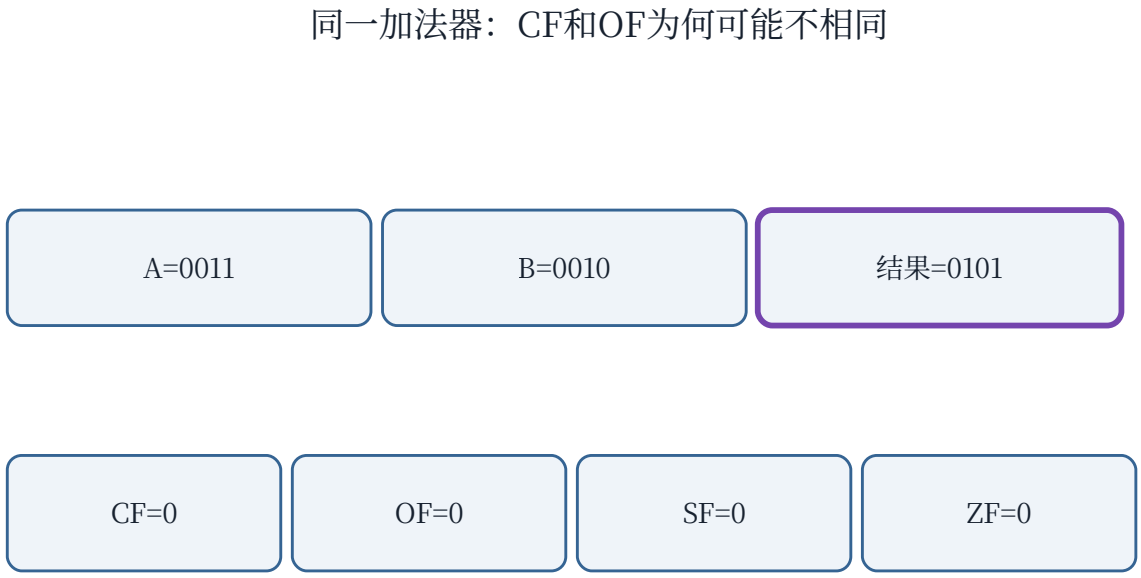
标志	为 1 的含义	用于判断
ZF	运算结果各位全 0	是否相等、是否为零
SF	运算结果最高位为 1	结果符号；需结合 OF 判断有符号比较
OF	有符号结果超出范围	有符号溢出
CF	加法有最高进位，或减法需要借位（本模型）	无符号溢出、无符号比较

硬件生成与解释

SF 直接取结果最高位，ZF 由全部结果位是否为 0 得到，OF 可用最高两级进位异或得到。对采用“减法借位置 1”约定的加减运算器，CF=Sub⊕Cout：加法时取最高进位，减法时取其反。

OF 与 SF 的有符号解释不能直接用于无符号大小比较；CF 不负责判断有符号运算是否超出范围。具体 ISA 对 CF、借位的定义可能不同，判断题设电路和指令时先确认约定。

图解与逐步推演：同一加法器：CF和OF为何可能不相同



单独查看图片

同一加法器：CF和OF为何可能不相同

查看完整步骤（也可离线阅读）

1. 0111+0001：无符号解释：7+1=8；4位有符号解释：7+1=8。CF看最高位进位，OF看有符号真值是否超出[−8,7]。
2. 1111+0001：无符号解释：15+1=16；4位有符号解释：-1+1=0。CF看最高位进位，OF看有符号真值是否超出[−8,7]。
3. 1000+1000：无符号解释：8+8=16；4位有符号解释：-8+-8=-16。CF看最高位进位，OF看有符号真值是否超出[−8,7]。
4. 0011+0010：无符号解释：3+2=5；4位有符号解释：3+2=5。CF看最高位进位，OF看有符号真值是否超出[−8,7]。

整数类型转换：位串不变、截断与扩展 082 先判断位宽，再判断有符号或无符号解释。

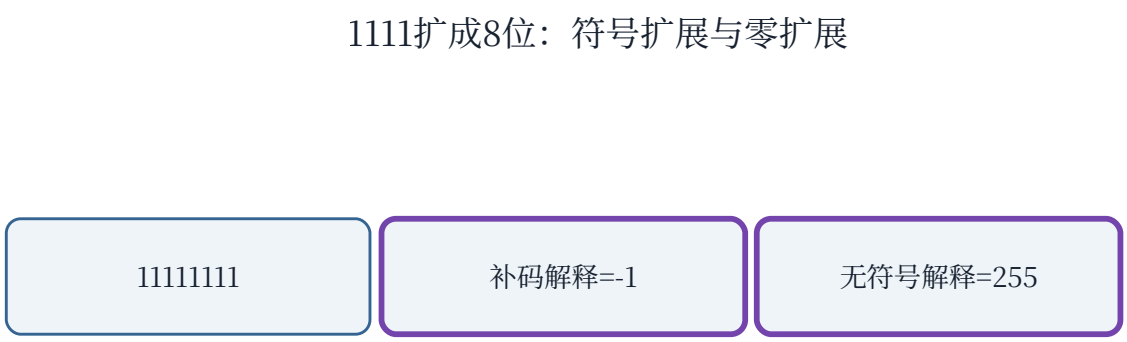
相同位宽的重新解释

在题目采用补码整数模型时，相同位宽的有符号与无符号转换通常保留低 n 位位串，改变解释。最高位为 1 的位串，解释为无符号数时比其有符号真值大 2^n。例如 8 位 11111111，无符号为 255，补码为 −1。

位宽变化

- 长整数转短整数：保留低位、截去高位，结果可能改变真值。
- 无符号数扩展位宽：高位补 0；补码有符号数扩展：重复原最高符号位。
- 原码扩展不能直接照搬补码符号扩展；讨论时应明确机器实际采用的编码。
- C 语言类型位宽由实现和数据模型决定，不能无条件认定 long 总是 32 位或 64 位、char 总为有符号。混合表达式的比较应先进行整数提升和通常算术转换，再决定按何种类型比较。

图解与逐步推演：1111扩成8位：符号扩展与零扩展



单独查看图片

1111扩成8位：符号扩展与零扩展

查看完整步骤（也可离线阅读）

1. 原始4bit：同一位串在4bit补码与无符号解释下真值不同。
2. 有符号扩宽：要保持有符号真值，扩宽时在高位复制符号位。
3. 无符号扩宽：要保持无符号真值，扩宽时高位补0。
4. 同宽重新解释：同宽改变解释不改位串；实际C表达式还要先按整数提升与类型转换规则确定位宽。

浮点表示与 IEEE 754 编码 083 数值范围主要看阶码，有效精度主要看尾数。

一般浮点模型

浮点数写成 $N=M\times r^{E}$，r 为约定的基数，M 为尾数，E 为阶码真值。阶码决定小数点位置和可表示范围，尾数决定有效数字精度，尾数符号决定整个数的正负。一般教材模型的阶码和尾数编码须看题设，不能与 IEEE 编码混用。

IEEE 常用格式

格式	符号位	指数位 / 偏置	尾数字段 / 规格化有效位
binary32（通常为 float）	1	8 / 127	23 / 24
binary64（通常为 double）	1	11 / 1023	52 / 53

规格化数、零与特殊值

- 指数全 0、尾数字段全 0 表示带符号零；IEEE 存在 +0 和 −0。
- 指数全 1、尾数字段全 0 表示正负无穷；尾数字段非 0 表示 NaN。
- binary32 最小正规格化值是 2^{-126}，最大有限值是 $(2-2^{-23})\times 2^{127}$；非规格化数还可表示更接近 0 的数。
- 一般机器零处理与 IEEE 渐进下溢不同，应使用题设标准，不能说“阶码全 0 就必为零”。

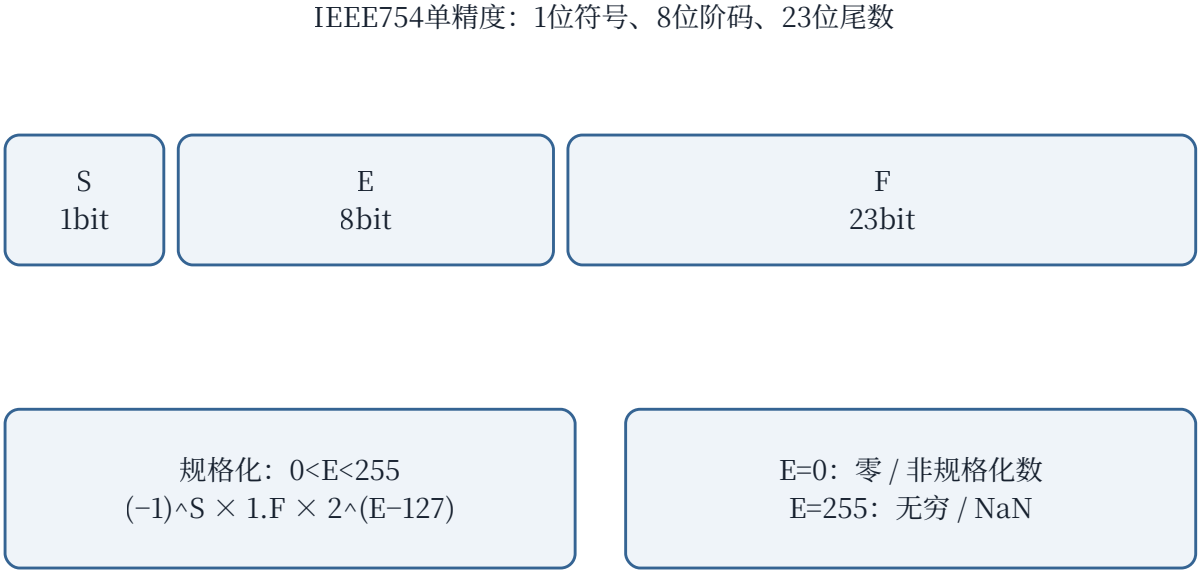
$x=(-1)^{s}(1.f)2^{E-bias}$
指数编码既非全 0 又非全 1 时，前导 1 隐含，不实际存入尾数字段。

$x=(-1)^{s}(0.f)2^{1-bias}$
指数编码全 0 且尾数字段非 0 时为非规格化数，前导位是 0。

结构对照图：IEEE754单精度：1位符号、8位阶码、23位尾数

1.5的S=0，E=127（01111111），F为100…0，十六进制编码0x3FC00000。

隐含的首位1只适用于规格化数；E=0时指数采用1−127，尾数为0.F。E=255且F=0表示无穷，F非0表示NaN。



单独查看图片

尾数规格化与浮点加减的五个步骤 084 先对阶，再运算；尾数溢出可以通过右规解决。

规格化的判断

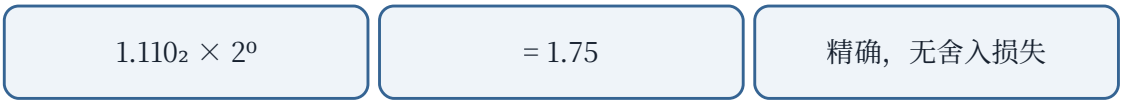
基数为 2、原码小数尾数模型中，非零规格化尾数满足 $1/2 \leqslant |M| < 1$，即第一数值位为 1。补码小数尾数的规格化特征是符号位与第一数值位不同，负数的边界与原码不同；−1 可作为补码规格化尾数，而 −1/2 的补码不是规格化形式。IEEE 的 1.f 是另一种等价表达约定，不要混写小数点位置。

浮点加减流程

- 对阶：小阶向大阶靠齐，小阶尾数右移，阶码增加；保留必要的附加位以便舍入。大阶尾数左移可能丢失高位，不能作为通常的对阶方法。
- 尾数求和或求差：对齐小数点后，按题设编码完成定点加减。
- 规格化：尾数绝对值过小则左规、阶码减小；双符号位出现 01 或 10 等尾数溢出形态时右规、阶码增加。一次加减后的右规通常至多一次，左规次数由抵消程度决定。
- 舍入：按规定处理被舍弃位。“0 舍 1 入”“末位置 1”是教材中的特定舍入模型；IEEE 常用舍入到最近且偶数优先，必须服从题设。舍入可能再次引起尾数进位，需要重新右规。
- 检查阶码：规格化和舍入后，阶码超出上限才是浮点上溢；阶码过小按题设作下溢、机器零或非规格化处理。尾数临时溢出本身不直接判整个浮点运算溢出。

图解与逐步推演：浮点加法：1.5 + 0.25如何对阶

浮点加法：1.5 + 0.25如何对阶



单独查看图片

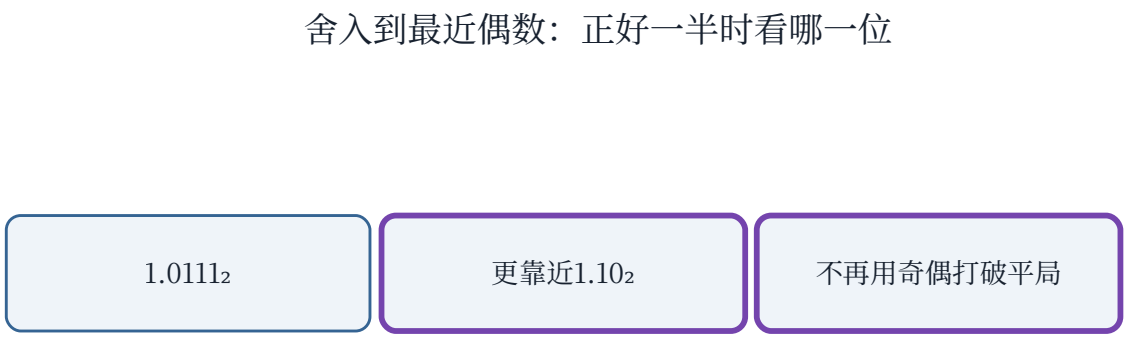
浮点加法：1.5 + 0.25如何对阶

查看完整步骤（也可离线阅读）

1. 原数与阶码：用可精确表示的二进制数说明流程；尚未考虑IEEE字段编码。

2. 小阶向大阶对齐：第二个尾数右移2位，阶码加2；不能把大阶向小阶硬移。
3. 尾数相加：相同阶码下尾数相加，得1.110_2。
4. 规格化与舍入：结果已规格化；一般还需按保留位、保护位和舍入规则处理并检查阶码范围。

图解与逐步推演：舍入到最近偶数：正好一半时看哪一位



单独查看图片

舍入到最近偶数：正好一半时看哪一位

查看完整步骤（也可离线阅读）

1. 只保留3位有效尾数：1.011_2位于1.01_2与1.10_2正中间；ULP是本例相邻可表示数间隔。
2. 看保留末位奇偶：最近偶数中的偶数指保留尾数整数编码的最低位为0，不是要求数值本身为偶整数。
3. 另一半值1.001_2：这次正好一半时不进位，仍选择最低保留位0的候选。
4. 不是恰好一半时：保护位与粘滞信息用于判断大于、等于还是小于半ULP，不能把所有丢弃首位1都机械当作同一种情况。

浮点精度与 C 类型转换 085 范围够大不代表每一个整数都能精确保存。

精度与表示范围分开判断

同总位宽下，浮点数把部分位分给阶码，通常能覆盖比定点数更大的数量级，但有效数字位较少。规格化后才判断浮点阶码是否越界；固定点结果则直接与其固定表示范围比较。

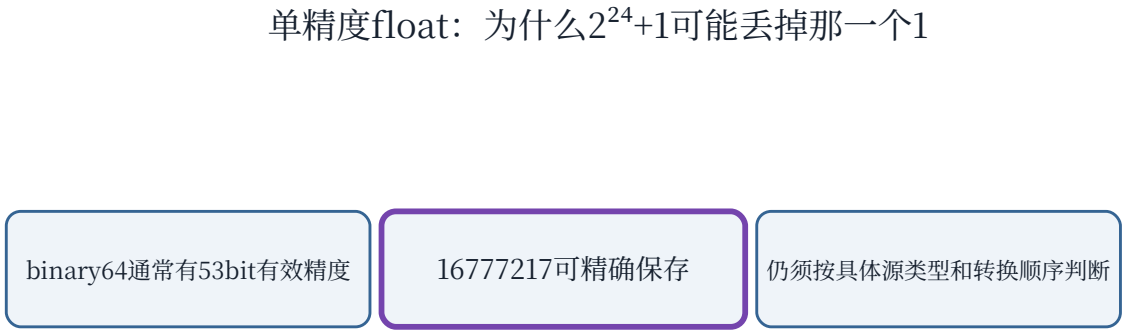
常见转换的损失

转换	主要检查	在常见 32 位 int、IEEE float/double 模型下
int→float	有效二进制位能否装入 24 位有效尾数	可能丢失低位精度，但范围足够
int→double	有效二进制位能否装入 53 位有效尾数	全部 32 位 int 可精确表示
float→double	更宽精度和范围	可精确保持原 float 值
float/double→int	是否有小数、是否越界	小数部分向 0 截断；越界不能简单当作安全回绕
double→float	范围与有效位同时检查	可能舍入，也可能上溢或下溢

不能把类型名当作固定精度

long、long double 的宽度和格式与平台有关；固定长度的例子应理解为题设数据模型。判断整数是否能精确转浮点，应检查实际有效位及尾随零，而不是仅按类型最大位数认定每次转换都丢失。

图解与逐步推演：单精度float：为什么$2^{24}+1$可能丢掉那一个1



单独查看图片

单精度float：为什么$2^{24}+1$可能丢掉那一个1

查看完整步骤（也可离线阅读）

1. IEEE754单精度规格化数：这里假设常规binary32、舍入到最近偶数。
2. 紧邻的大整数：数值范围还远未达到无穷，但精度已不能覆盖这个区间的每个整数。
3. 转换正好落在中点：损失发生在整数转float的舍入，后续转回整数无法找回已丢失的信息。

4. double对照：不能把“浮点范围更大”当作“所有整数都精确”的保证。

检错纠错编码：码距、奇偶校验与海明码 086 冗余位不是重复数据，而是按规则建立监督关系。

码距与能力

在信息位之外加入依赖于信息内容的冗余位，使合法码字符合特定关系。两个码字不同的位置数为汉明距离；整个编码的最小码距决定最坏情况下的检错纠错能力。

$d_{min} \geq t+1$
保证检测 t 位错误所需的最小码距。

$d_{min} \geq 2t+1$
保证纠正 t 位错误所需的最小码距。

奇偶校验与海明码

一位奇偶校验使整个码字的 1 的数量满足奇数或偶数约定；能发现奇数个位翻转，不能保证发现偶数个位翻转，也不能定位错误。海明码把校验位置于 1、2、4、8…等位置，按位号二进制展开决定各校验组。

$2^r \geq k+r+1$
k 个信息位配 r 个校验位，要区分 k+r 个单比特错误位置及无错状态。

编码与校验流程

- 先取满足不等式的最小 r，把校验位放在 2 的幂次位置，其余位置放信息位。位号从左还是从右编号须先统一。
- 每个校验位监督所有位号在该二进制位为 1 的位置，按奇偶约定求其值。
- 接收时重复各组校验得到校验综合征；假定至多一位出错时，综合征二进制数可指向出错位。
- 普通最小码距 3 的海明码可纠正单比特错误；若还要可靠区分双比特错误与单比特错误，通常增加整体奇偶校验形成 SECDED，不能把普通综合征直接当作任何错误模式的定位。

6 位信息 101101 的海明编码

采用偶校验，位号从右到左为 1～10。k=6，取最小 r=4，使 $2^4 \geqslant 6+4+1$。把 P_1、P_2、P_3、P_4 放在 1、2、4、8 号位；按高位到低位把 $D_6 \ldots D_1=101101$ 放在 10、9、7、6、5、3 号位。

- P_1 监督 1、3、5、7、9 号位，因此 $P_1=D_1 \oplus D_2 \oplus D_4 \oplus D_5=1 \oplus 0 \oplus 1 \oplus 0=0$。
- P_2 监督 2、3、6、7、10 号位，因此 $P_2=D_1 \oplus D_3 \oplus D_4 \oplus D_6=1 \oplus 1 \oplus 1 \oplus 1=0$。
- P_3 监督 4、5、6、7 号位，因此 $P_3=D_2 \oplus D_3 \oplus D_4=0 \oplus 1 \oplus 1=0$；$P_4$ 监督 8、9、10 号位，$P_4=D_5 \oplus D_6=0 \oplus 1=1$。
- 按位号 10→1 写出码字 1011100100。若单比特翻转后校验综合征 $P_4P_3P_2P_1$ 为 0101，则定位为第 5 位；必须先约定综合征各位的排列方向。

图解与逐步推演：海明(7,4)码：校验组与综合征怎样定位错位

海明(7,4)码：校验组与综合征怎样定位错位



单独查看图片

海明(7,4)码：校验组与综合征怎样定位错位

查看完整步骤（也可离线阅读）

1. 计算校验位p1：偶校验覆盖位置[1, 3, 5, 7]，计算p1=0，使该组1的总数为偶数。
2. 计算校验位p2：偶校验覆盖位置[2, 3, 6, 7]，计算p2=1，使该组1的总数为偶数。
3. 计算校验位p4：偶校验覆盖位置[4, 5, 6, 7]，计算p4=0，使该组1的总数为偶数。
4. 传输中第6位翻转：接收端按相同覆盖组检查，得到s1=0、s2=1、s4=1；综合征$s4s2s1=110_2=6$，定位到第6位。
5. 翻转第6位恢复：把第6位从0改回1，重新检验各组均为偶校验，得到码字0110011。

CRC：补零、模 2 除与接收检验 087 余数非零能说明有错，余数为零不等于绝对无错。

编码和检验

- 选定 r 次生成多项式 G(x)，其二进制表示有 r+1 位，最高项和常数项通常为 1。
- 在信息位后附 r 个 0，再用 G(x) 对应位串做模 2 除法；减法采用异或，不借位。
- 将所得余数补足 r 位放在原信息位之后，组成发送码字；前导 0 也是校验位的一部分，不能省略。
- 接收端用同一生成多项式除收到的码字：余数非 0 判为出错；余数为 0 只说明未检出错误。CRC 通常配合重传，不应一般性地由余数自行纠正某一位。

完整示例

信息串 1011001，生成位串 11001（$G(x)=x^4+x^3+1$）。先构造 10110010000，模 2 除以 11001 得到余数 1010，发送码字为 10110011010。接收时对完整码字作相同除法，可检查余数是否为 0。

检错能力的条件

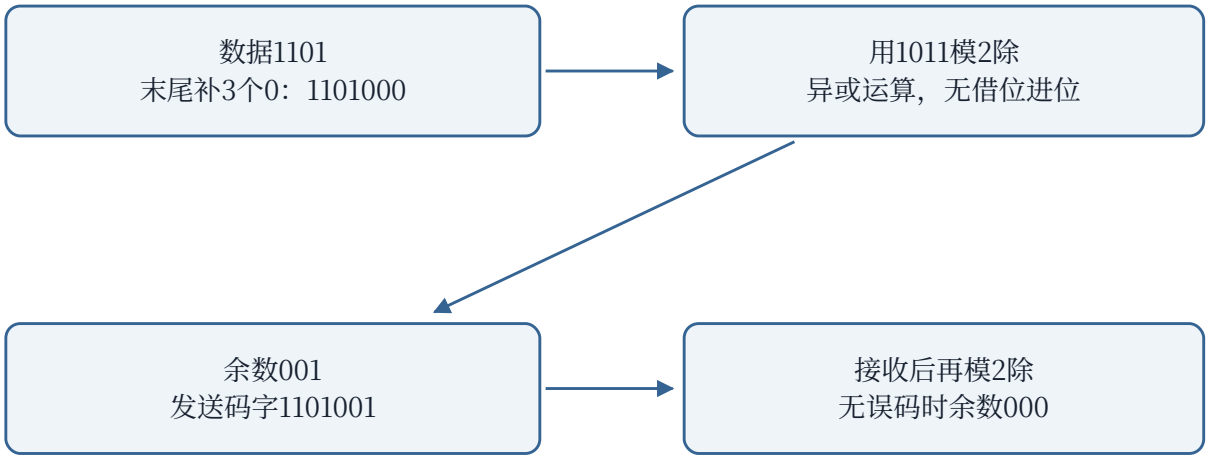
生成多项式选得适当可保证检测长度不超过 r 的突发错误；更长错误存在漏检可能，概率分析还依赖错误模型与生成多项式。不要把“余数某个值”无条件解释为某一位翻转，也不要宣称 CRC 能发现所有错误。

结构对照图：CRC示例：数据1101，生成多项式1011

生成多项式最高次数r=3，所以补3个0，余数也按3bit写为001。1101000 ÷1011的模2余数为001。

发送码字为原数据拼余数1101001；接收端用同一除数得到000可通过检验，但余数为0并不能保证绝对没有任何未检出的差错。

CRC示例：数据1101，生成多项式1011



单独查看图片

浮点减法完整例子：−5/256 减 59/1024 117 用明确的位宽模型串起对阶、尾数运算和规格化。

题设与初始表示

设阶符 2 位、阶码数值部分 3 位，尾符 2 位、尾数数值部分 9 位，均按题设双符号位补码模型。
$x=-5/256=2^{-5}\times(-0.101000000)$，$y=59/1024=2^{-4}\times0.111011000$。

初始阶码 $[E_x]$补=11011，$[E_y]$补=11100；尾数 $[M_x]$补=11.011000000，$[M_y]$补=00.111011000。

分步计算

- 对阶：$E_x-E_y=-1$，让 x 的尾数算术右移 1 位，阶码增至 −4。得到 $[M_x]$补=11.101100000；y 不变。
- 求差：把 y 的尾数取负，得 $[-M_y]$补=11.000101000；与 11.101100000 相加，得到 10.110001000（保留题设尾数宽度）。
- 规格化：尾数双符号位 10 表示临时负溢出，算术右移 1 位成为 11.011000100，同时阶码从 −4 增至 −3。

- 舍入：本例被舍去位为 0，按给定规则不需增加尾数。阶码 −3 的补码为 11101，双符号位 11 表示未发生阶码溢出。
- 解释结果：尾数真值为 -0.100111100_2，乘 2^{-3} 得 −79/1024，与直接有理数计算 x−y 一致。

检查点

尾数临时溢出后右规只是恢复规格化，不应立即判整个结果溢出；对负尾数右移应补符号位，不能按无符号右移补 0。

存储器层次结构

计算机组成原理 · 15 条

存储器分类与层次：速度、容量和成本折中 088 Cache 解决速度矛盾，虚拟存储主要解决容量与地址空间管理。

按不同标准分类

分类维度	类别	识别点
作用层次	Cache、主存、辅存	越靠近 CPU 通常越快、越贵、容量越小
介质	半导体、磁表面、光存储等	物理存储方式不同
访问方式	随机、顺序、直接存取	随机时间近似与位置无关；顺序需依次经过；直接存取兼有定位与顺序访问
保持信息	易失性、非易失性	是否需要持续供电维持
读出影响	破坏性、非破坏性	读后是否必须恢复原信息

层次之间的信息交换

CPU 可访问寄存器、Cache 和主存；辅存内容通常先调入主存。Cache—主存之间以块交换，主要由硬件管理，对一般程序员透明；主存—辅存之间的调入调出由操作系统和硬件协同完成。

局部性使少量快速存储器可以保存近期常用内容，形成接近高速存储器的访问性能和接近低层存储器的容量、成本。

“副本”关系不能忽略写策略

常用教学层次把高层内容看作低层部分数据的副本，但写回 Cache 的脏块可能暂时比主存更新，包含性也依具体体系而定。判断命中和一致性时，应结合写策略，而不把“每一时刻各层完全相同”当作普遍结论。

RAM 与 ROM 家族

类别	含义	保持和修改方式
RAM	随机读写存储器	通常易失；SRAM 和 DRAM 是常见形式
MROM	掩膜只读存储器	制造时确定内容
PROM	可编程只读存储器	用户可一次性编程
EPROM	可擦除可编程只读存储器	典型以紫外线整片擦除后重编程
EEPROM	电可擦除可编程只读存储器	采用电方式擦写，粒度以器件为准
Flash	闪存	电擦写、非易失，典型按块擦除

存储芯片结构与读写时序 089 地址线选单元，数据线传内容，控制线规定何时有效。

芯片接口

半导体存储芯片一般含存储矩阵、译码驱动电路、读写电路和控制逻辑。地址线通常单向输入；数据线读出与写入时方向不同；片选 CS/CE、读写 WE、输出使能 OE 等控制线须看高低电平有效标记。单译码直接选取存储行，双译码用行列共同定位单元。

存取时间与存取周期

存取时间是从启动一次存储操作到该操作完成的时间；存取周期是连续两次独立存储操作启动之间必须间隔的最小时间，可能包含恢复时间，通常不小于存取时间。不能把它们当作同一个参数。

读写控制条件

读周期中要在地址和片选有效后等待数据稳定，并满足保持时间。写周期除地址有效外，还应使片选与写允许满足规定的脉冲宽度，数据也要满足建立、保持时间。只有电平极性与时序均满足要求，才能保证读写正确；不要只看地址是否相同就判断数据已可用。

SRAM、DRAM 与刷新 090 保持供电不意味着 DRAM 不会自行丢失电荷。

两类 RAM 对照

比较项	SRAM	DRAM
存储信息	双稳态电路	电容电荷
刷新	不需周期刷新	需要周期刷新
读出	通常非破坏性	需读后再生
速度、成本	速度快、位成本高	速度较慢、位成本低
集成度与典型用途	密度较低，常用于 Cache	密度较高，常用于主存

比较项	SRAM	DRAM
断电保持	易失	易失
地址传送	一般直接送地址	常行列地址复用，分时送入

三种刷新组织

- 集中刷新：在一个刷新周期中安排一段集中刷新所有行，期间形成连续死区。
- 分散刷新：每个存储周期包含正常读写和一行刷新，拉长存储周期；没有集中成片的死区，但刷新时间并未消失。
- 异步刷新：在最大刷新间隔内均匀安排各行刷新。例如 2 ms 内刷新 128 行，则约每 15.625 μs 刷新一行。
- 刷新按行进行，同一行的各列共同刷新；刷新只需行地址，由刷新控制电路产生，不要求 CPU 逐行给地址。

刷新与再生

再生针对一次破坏性读出后恢复所读内容；刷新是为防止电荷泄漏而按计划定期恢复整行信息。刷新周期指同一行必须再次刷新的最大间隔，不是一次读写周期，也不是刷新一行所需时间。

集中刷新死时间的计算

若存储周期为 0.5 μs，要求 2 ms 内刷新 16 行，每行刷新占一个周期，则集中刷新耗时 16×0.5=8 μs。2 ms 中共 4000 个存储周期，死时间率为 16/4000=0.4%。这里 16 是行数，不能误用总存储单元数；全部芯片若并行按同一行号刷新，也不能再乘芯片数量。

图解与逐步推演：DRAM刷新：周期、行数与死区怎样联系



单独查看图片

DRAM刷新：周期、行数与死区怎样联系

查看完整步骤（也可离线阅读）

1. 刷新要求：假设逐行刷新，所有行都必须在2ms保持期内刷新至少一次。
2. 集中刷新：把各行刷新集中执行，这一段内普通访存受到阻塞。
3. 异步分散安排：均匀分散时平均每15.625μs启动一次行刷新；不是每个存储周期都必须刷新。
4. 区分三个时间：三者含义不同。实际DRAM内部行数、刷新命令粒度和并行行为由器件规定，此为考试简化模型。

Flash 与 SSD：页写、块擦除和磨损均衡 091 闪存没有机械寻道，但写入不是无条件覆盖旧数据。

Flash 特性

Flash 从 EEPROM 技术发展而来，是可电擦写的非易失存储器，断电仍可保持信息。典型 NAND Flash 的读取、编程以页组织，擦除以更大的块组织；不同类型闪存访问粒度不同，不能统称“一律按块读取”。

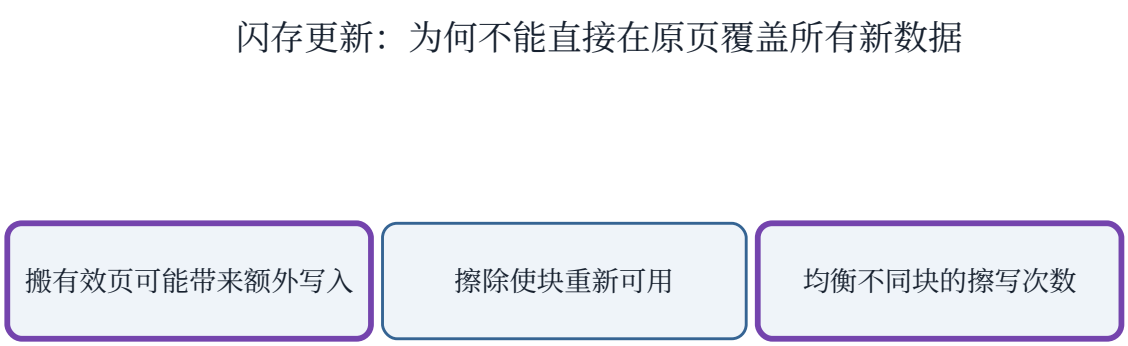
SSD 的组成与特点

SSD 由闪存芯片和控制器等组成，闪存转换层 FTL 把主机逻辑地址映射到闪存物理位置，承担地址映射、垃圾回收、磨损均衡等工作。无机械运动使随机访问延迟、噪声、抗震性和能耗等优于传统磁盘。

写放大与寿命

Flash 写入前通常需要擦除，且擦除不能无限次进行。FTL 尽量把擦除分散到不同块，以避免某些块过早磨损；逻辑覆盖可能触发搬移、回收等操作，实际内部写入量可大于主机写入量。SSD 不需要磁盘的寻道和旋转等待，但仍有控制器、闪存读写和回收的延迟。

图解与逐步推演：闪存更新：为何不能直接在原页覆盖所有新数据



单独查看图片

闪存更新：为何不能直接在原页覆盖所有新数据

查看完整步骤（也可离线阅读）

1. 原逻辑页L映射到物理页P：简化为页编程、块擦除的闪存管理模型；实际器件约束由规格决定。
2. 更新采用异地写：逻辑地址不变，物理位置改变；需要维护映射及掉电一致性。
3. 旧页P标为无效：垃圾回收选择擦除块，将其中仍有效的页搬走后，才能整块擦除。
4. 回收与磨损均衡：这解释写放大与磨损均衡的必要性；没有机械寻道不等于写入零开销。

双端口 RAM、单体多字与多体交叉存储 092 并行访问是否成功，关键看地址落在哪个体及访问时序。

双端口与单体多字

双端口 RAM 提供两套独立端口，访问不同单元，或同时读取同一单元，一般可并行进行；同时写同一单元、或一读一写同一单元存在冲突，应按仲裁与忙信号规则延后一个端口。

单体多字存储器用更宽的单元和总线一次读出多个连续字，提高单次传输量；这些字需位于同一宽存储单元，不能把任意离散地址同时读取。

高位交叉与低位交叉

组织	连续地址分布	访问特征
高位交叉	同一个体内连续，高位区分体号	方便扩充；顺序读取同一区域时通常仍受单体周期限制

组织	连续地址分布	访问特征
低位交叉	相邻字分到相邻体，低位区分体号	可轮流启动各体，以重叠周期提高连续访问带宽

轮流启动的条件与时间

- 若访问步长使请求反复落在同一体，低位交叉也可能冲突，不能仍照套理想流水公式。
- 高位交叉不是任何情况下都不能并行：不同请求源访问不同体可并行；只是单个请求源连续地址通常不能充分交错。
- “总线传输周期”“体存储周期”“首字访问时间”是不同参数，计算 Cache 块填充时间前先画启动与返回时序。

$m\tau \geq T$

m 个体、体存储周期 T、相邻启动间隔 τ；再次访问同一体前需满足其恢复时间。

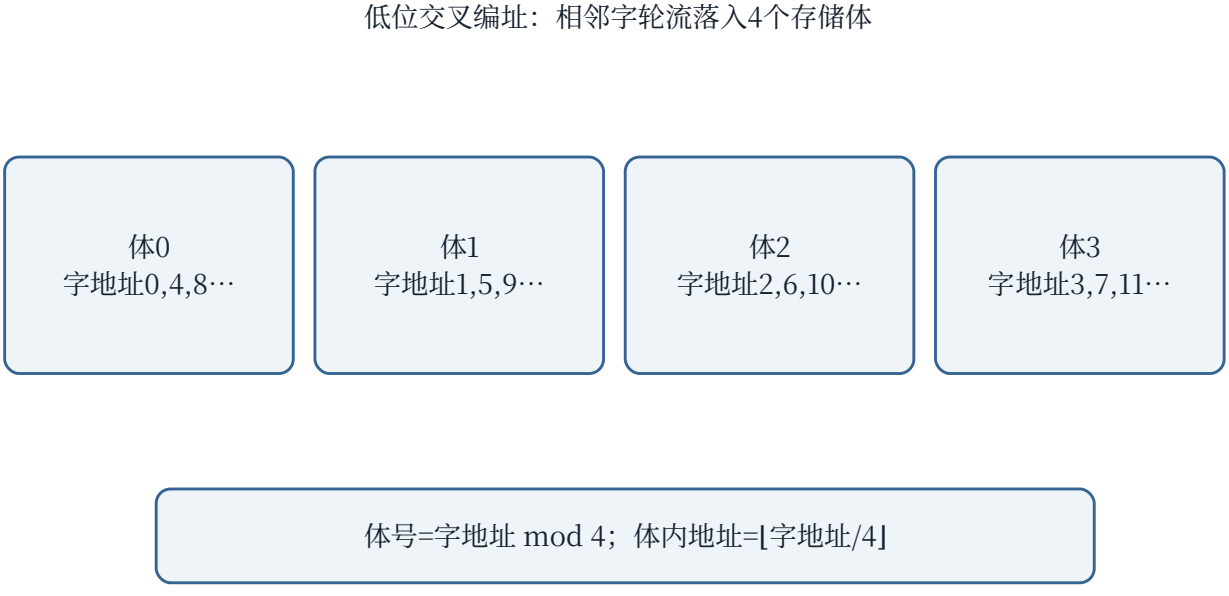
$t = T + (n-1)\tau$

题设允许 n 次连续访问按间隔 τ 流水启动、首字延迟取 T 时使用；实际首字读出时间以题设为准。

结构对照图：低位交叉编址：相邻字轮流落入4个存储体

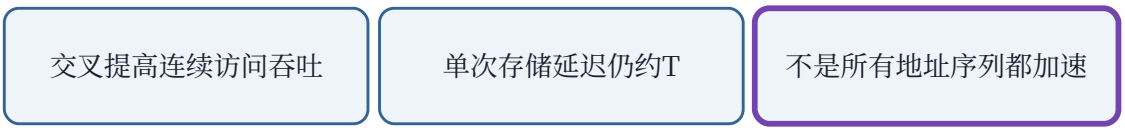
低2位选择存储体，连续字可按启动间隔τ轮流启动。若存储周期T≤4τ，理想连续访问可维持每τ一个字的启动节拍。

传送n个字的常见理想模型为T+(n−1)τ；同一体冲突、总线占用、未满足轮流启动条件都会改变结果。



单独查看图片

图解与逐步推演：低位交叉：连续字与跨4字访问为何不同



单独查看图片

低位交叉：连续字与跨4字访问为何不同

查看完整步骤（也可离线阅读）

1. 4个存储体：每体启动一次后，至少隔4τ才能再次启动同体请求。
2. 连续字0,1,2,3,4：连续访问轮流分散到不同体，可按τ间隔启动。
3. 跨4字访问0,4,8,12：地址跨度恰为体数，低位体号不变，必须等待该体恢复，不能继续每τ启动。
4. 区分吞吐与首字延迟：给出T、τ、体数以及访问地址序列后才能判断可否轮流启动。

主存容量扩展与 CPU 连接 093 先满足位宽，再满足字数，最后检查片选地址空间。

位扩展、字扩展与字位扩展

- 位扩展增加一个存储字的位数：如两片 1K×4 合成 1K×8。地址线和片选并接，各芯片分别连接不同的数据位，同一时刻共同被选中。
- 字扩展增加可寻址单元数：如两片 1K×8 合成 2K×8。低位地址和数据线共享，用额外高位地址区分芯片，同一时刻仅选相应组。
- 字位同时扩展：如八片 1K×4 合成 4K×8，先两片构成一组 1K×8，再用四组扩大字数；需 12 位地址，其中低 10 位到组内，高 2 位译码选组。

连接步骤与片选

依次核对地址线、数据线、读写控制、片选译码和芯片规格。系统程序区可用 ROM，读写数据区使用 RAM，具体区域按题设分配。片选还要与 CPU 的访存请求信号协调，避免非存储访问误选芯片。

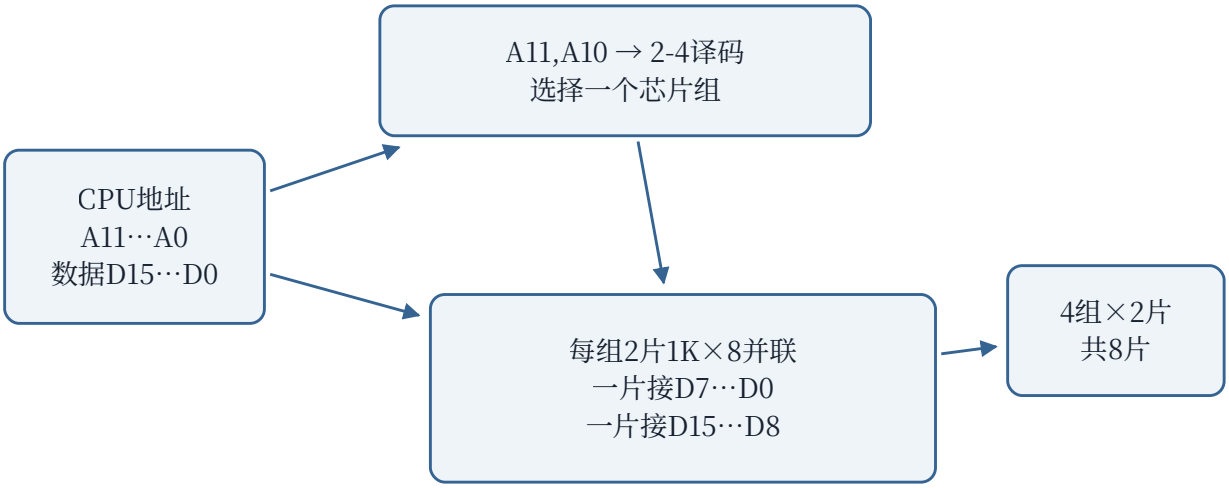
片选选择芯片或芯片组，字选选择已选芯片内的具体单元。线选法简单但地址利用与别名问题需检查；译码片选可用 n 位产生最多 2^n 路选择信号。总芯片数可由总位容量比求得，但接线仍须分别判断字数和位数。

结构对照图：8片1K×8芯片：组成4K×16存储器

A9…A0接每片芯片内部地址，共1024个字位置；A11、A10参与片选，扩展成4组。

数据位并联使字宽从8bit扩为16bit；总容量4K×16bit=8KiB。此图按16bit字编址说明，若CPU按字节编址还要处理字节选择和地址接法。

8片1K×8芯片：组成4K×16存储器



单独查看图片

磁盘结构、地址与访问时间 094 柱面定位、旋转等待和传输是三段不同的时间。

盘面、磁道、柱面和扇区

一个记录面对应一个磁头，盘面分为同心磁道，各磁道进一步划分扇区。不同记录面相同半径的磁道合成柱面。教学模型以柱面号、盘面号、扇区号定位扇区；扇区是基本读写单位，操作系统逻辑块可能由若干扇区组成。

容量与访问时间

$$C = N_{surface} \quad N_{track} \quad N_{sector} \quad B_{sector}$$

各面同样组织时，容量为盘面数×每面磁道数×每道扇区数×每扇区字节数。

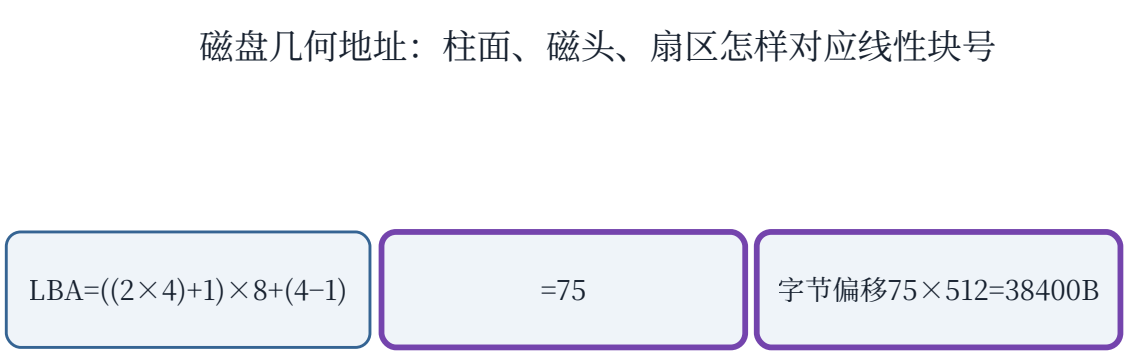
$$T_{access} = T_{seek} + T_{rotation} + T_{transfer}$$

平均旋转等待通常取一圈时间的一半；实际题设还可能另计控制器开销。

地址字段划分

若有 4 个驱动器，每个盘组 256 个柱面、16 个记录面，每道 16 个扇区，则驱动器号、柱面号、盘面号、扇区号分别需 2、8、4、4 位，共 18 位。要确认“每盘面磁道数”和“每磁道扇区数”，不能因资料口述简略而把两者互换。

图解与逐步推演：磁盘几何地址：柱面、磁头、扇区怎样对应线性块号



单独查看图片

磁盘几何地址：柱面、磁头、扇区怎样对应线性块号

查看完整步骤（也可离线阅读）

1. 理想几何模型：柱面号与磁头号从0起，扇区号按本例从1起。真实硬盘的逻辑块映射不一定暴露这种固定几何布局。
2. 给定C=2,H=1,S=4：先算这个位置前面一共有多少个扇区。
3. 求0起线性扇区号：LBA75是第76个逻辑扇区；不要把扇区号的1起约定直接用于0起LBA。

RAID：条带、镜像和校验的分工 095 并行提高带宽，冗余提高容错；两者不是同一件事。

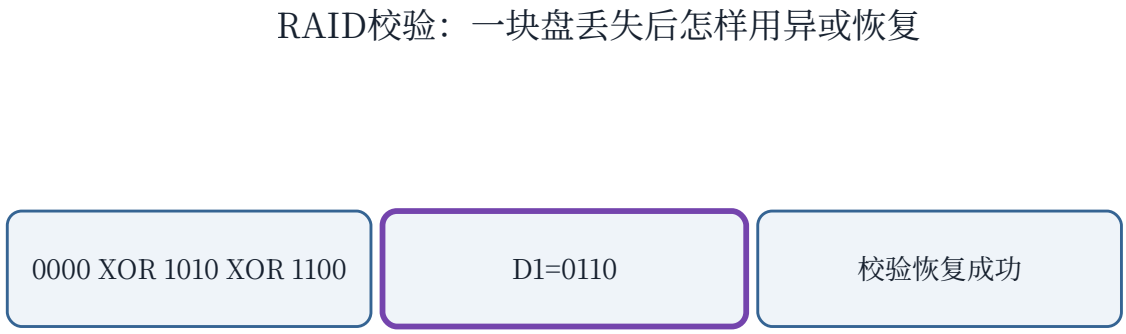
磁盘阵列

RAID 把多个物理盘组织成一个逻辑存储体，利用条带化并行传输，利用镜像或校验等冗余方式提高可靠性。并非每一级 RAID 都同时具有性能提升和容错能力。

各级组织对照

级别	组织方式	关键区别
RAID 0	条带化，无镜像或校验	可并行但无容错能力
RAID 1	镜像	典型双副本可用容量约为总容量一半
RAID 2	以海明码组织纠错冗余	采用位级纠错思想
RAID 3	字节或细粒度条带、专用校验盘	适合连续大块并行传输的教学模型
RAID 4	块级条带、专用校验盘	校验盘可能成为写瓶颈
RAID 5	块级条带、分布式校验	没有独立固定校验盘；典型可容忍一个盘故障

图解与逐步推演：RAID校验：一块盘丢失后怎样用异或恢复



单独查看图片

RAID校验：一块盘丢失后怎样用异或恢复

查看完整步骤（也可离线阅读）

1. 一个条带：以三个数据块和一个校验块说明按位异或，图只画一个条带。
2. 计算校验块：异或满足交换律与a XOR a=0。RAID5的校验位置会在条带之间轮转，本条带只是某一位置的示例。
3. D1所在盘失效：相同数据异或两次抵消，留下缺失块。
4. 恢复D1：本例P=0000。RAID5通常只能容忍一个成员盘失效；它不是文件误删或所有灾难情形的备份替代品。

Cache 工作原理、局部性与平均访问时间 096 先判断是否命中，再按题目定义的失效代价计算时间。

块与局部性

Cache 是主存部分活跃内容的高速副本，容量远小于主存，不能增加程序可使用的逻辑存储容量。时间局部性指近期使用的信息可能再次使用；空间局部性指其相邻地址的信息可能很快被访问，常见于循环、顺序指令和连续数组。

Cache 与主存按相同大小的块交换，Cache 块又称行；CPU 访问的对象可为字节、字或更宽操作数，不应把 CPU—Cache 访问粒度无条件固定为一个机器字。命中检查同时涉及标记与有效位。

平均时间的两种写法

$H=$

命中率为命中次数占全部相关访问次数的比例。

$AMAT=T_{hit}+(1-H)P_{miss}$

Pmiss 定义为命中检查之外的额外失效开销时使用。

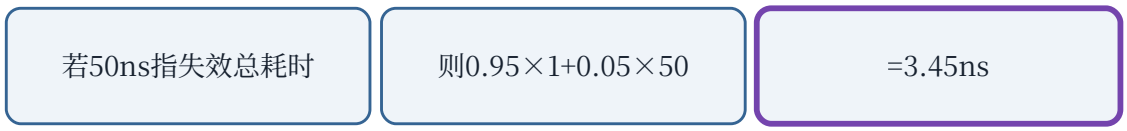
$T_{avg}=HT_{hit}+(1-H)T_{miss,total}$

若 Tmiss,total 已包含一次失效的全部访问过程，不能再重复加命中检查时间。

命中率由多因素决定

Cache 容量、块大小、相联度、替换算法、访问序列和程序局部性都会影响命中率。块变大可利用空间局部性，但会减少行数并增加填充代价，命中率并非一定单调提高。多级 Cache 的局部命中率和全局命中率也要区分。

图解与逐步推演：平均访存时间：失效代价是否已经包含查Cache



单独查看图片

平均访存时间：失效代价是否已经包含查Cache

查看完整步骤（也可离线阅读）

1. 给定模型：失效额外代价是查Cache之后增加的时间，因此一次失效总时间为1+50=51ns。
2. 按额外代价公式：所有访问先花1ns，只有5%再增加50ns。
3. 用总耗时求期望核对：若题目直接给失效总耗时51ns，应这样加权，不能再额外重复加一次Cache查询。
4. 概念口径改变时：公式差别来自“50ns”定义不同。先写命中与失效各自完整路径，再套期望最稳妥。

Cache 三种映射与地址划分 097 先算块内偏移，再算组数，剩余才是标记。

映射方式对照

方式	允许的位置	地址字段与替换
直接映射	主存块号 mod Cache 总行数，唯一固定行	标记 / 行号 / 块内偏移；无替换对象选择
全相联	任意 Cache 行	标记为完整主存块号 / 块内偏移；满时选择替换行
R 路组相联	主存块号 mod 组数指定组，组内任意 R 行	标记 / 组号 / 块内偏移；组满时在组内替换

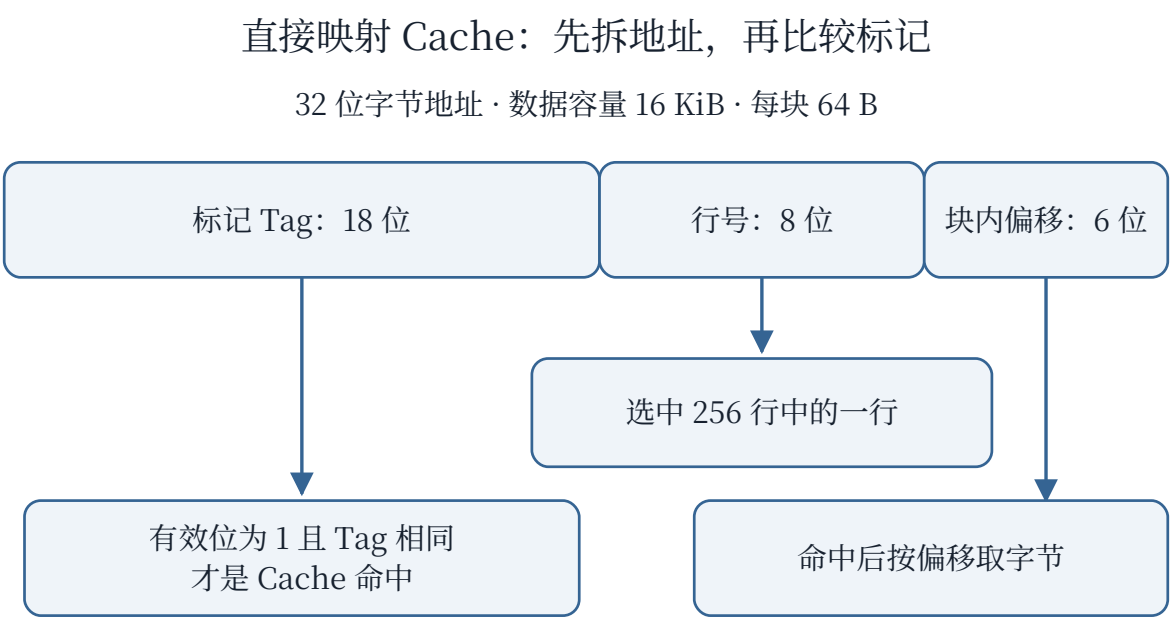
地址划分与比较

- 直接映射用行号定位一行，只比较该行 tag；全相联比较所有有效行；组相联先定位组，再并行比较组内各路。
- 标记相同但有效位为 0 仍然不命中；偏移仅用于命中后选出块内目标数据，不参与确定是哪一个主存块。
- 组相联只有一组时为全相联，每组只有一行时为直接映射。
- 总 Cache 存储开销还包括标记、有效位、脏位和替换状态，题目给“数据区容量”时不能把这些位混入行数计算。

$N_{line} = \frac{C}{B}, N_{set} = \frac{N_{line}}{R}$
C 是数据区容量，B 是块字节数，R 是相联路数；按字节编址且各量为 2 的幂时便于按位划分。

$b = \log_2 B, s = \log_2 N_{set}, t = a - s - b$
a 位地址分成 t 位标记、s 位组号、b 位块内字节偏移。

机制与步骤图解



单独查看图片

逐步操作与动画

例：访问地址 0x12345678，随后访问同一块

查看完整步骤（也可离线阅读）

1. 0 · 明确配置：32位字节地址，直接映射Cache数据容量16KiB，块大小64B，共256行。先不含标记等额外开销。
2. 1 · 求块内偏移：64=2^6，所以低6位是偏移。0x12345678 mod 64 = 56，即块内第56号字节（从0开始）。
3. 2 · 求Cache行号：先除64去掉偏移，再对256取余：(地址 >> 6) mod 256 = 89。只能检查第89行。
4. 3 · 比较标记和有效位：Tag=地址 >> 14=0x48D1。假设初始第89行无效，因此本次不命中；即使数据位碰巧相同，也不能算命中。

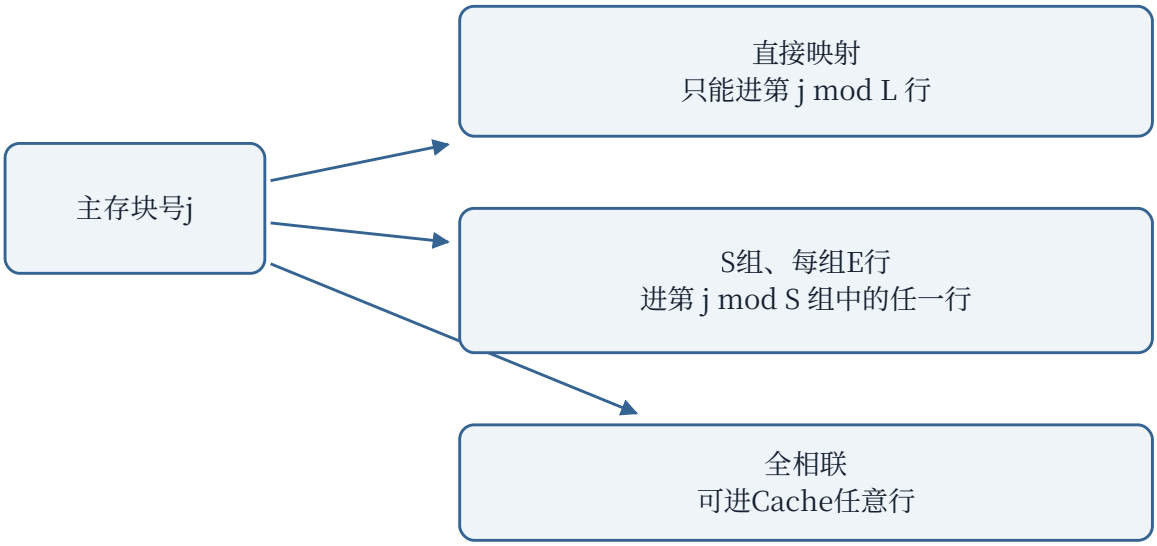
5. 4 · 装入整个块：读入主存区间0x12345640～0x1234567F，填写第89行的Tag并置有效位为1，再取偏移56处字节。
6. 5 · 再访问0x1234567C：假定期间没有替换或失效。行号仍为89、Tag仍为0x48D1，偏移变为60，此次命中；相邻地址可能共享一个块。

结构对照图：直接映射、组相联、全相联：候选位置有多少

块内偏移由块大小决定；直接映射的行号位数为$\log_2 L$，组相联的组号位数为$\log_2 S$，全相联没有组/行索引位。

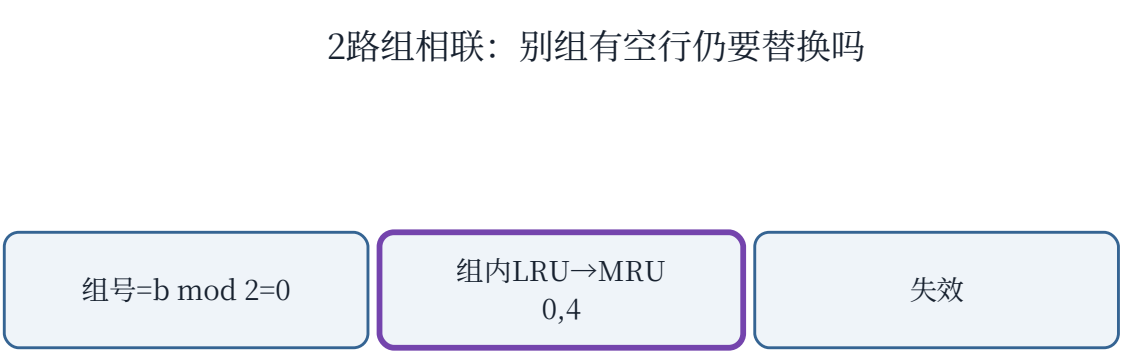
同一组内比较候选标记并按替换策略选行；全相联灵活但比较开销大。容量计算要说明是否含标记、有效位和脏位。

直接映射、组相联、全相联：候选位置有多少



单独查看图片

图解与逐步推演：2路组相联：别组有空行仍要替换吗



单独查看图片

2路组相联：别组有空行仍要替换吗

查看完整步骤（也可离线阅读）

1. 访问主存块0：失效，装入空行。
2. 访问主存块2：失效，装入空行。
3. 访问主存块0：命中，刷新该块为最近使用。
4. 访问主存块4：失效，同组已满，淘汰最久未用块2。

替换算法：FIFO、LRU、LFU 和随机 098 LRU 看最近一次，LFU 看累计次数。

什么时候需要替换

直接映射中目标行固定，发生冲突就覆盖该行，不需要选择替换算法。组相联只在指定组内选择，全相联可在全部行中选择；有无效空行时通常优先使用空行。

四种规则

算法	选择依据	使用时注意
FIFO	最早进入 Cache 的块	命中不改变进入顺序
LRU	最长时间没有访问的块	命中也更新最近使用次序
LFU	一段统计范围内访问次数最少的块	次数和老化规则须看题设
随机	随机选取候选行	硬件简单，不依赖精确访问历史

LRU 顺序更新

可把组内各行按新近程度排队：命中的行移到最近端，其余相对次序不变；新块装入最近端，满时淘汰最久端。若用计数器表达次序，应只调整受影响的次序值，不能每次把所有行无条件加 1 而导致相同次序。进行访问序列模拟时，把组号、命中/失效和每组的使用顺序分列记录。

写策略：写命中与写失效分开选择 099 写回对应脏位，写分配对应是否调块，两者不是同一维度。

两个独立决策

情形	策略	动作
写命中	全写 / 写直达	同时更新 Cache 和下级存储
写命中	写回	仅改 Cache，置脏位；脏行被替换时写回
写失效	写分配	把相应块调入 Cache，再修改
写失效	非写分配	直接写下级，不为此分配 Cache 行

常见组合及成本

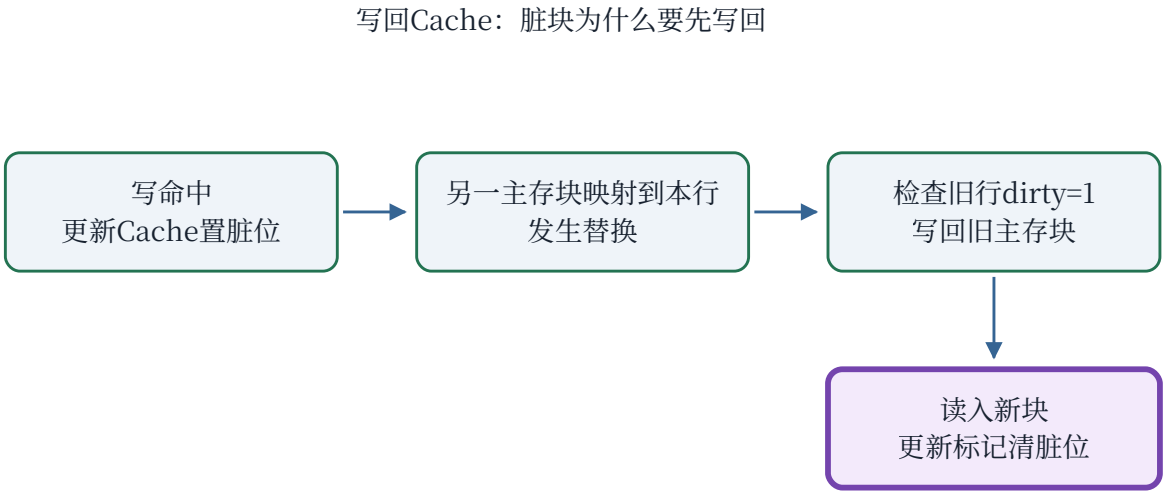
写回常与写分配组合，以利用随后对同块的访问；全写常与非写分配组合。它们是常见搭配而非硬性规定，多级 Cache 也不能仅凭层级推断唯一策略。

全写增加下级写流量，可用写缓冲隐藏部分等待；写回需记录脏位，替换脏块时增加写回代价。全写的写命中时间不一定机械等于完整主存周期，写缓冲与总线协议会影响 CPU 实际等待时间。

一致性与有效位

有效位说明该行是否包含可使用的标记数据；脏位说明当前 Cache 内容是否比下级更新。脏位为 0 不代表无效，有效位为 0 也不能凭残留标记判定命中。计算写失效代价时分别计调入、写修改、必要的旧脏块写回。

图解与逐步推演：写回Cache：脏块为什么要先写回



单独查看图片

写回Cache：脏块为什么要先写回

查看完整步骤（也可离线阅读）

1. 写命中 · 更新Cache置脏位：本例采用写回策略，写命中先只更新Cache，置dirty=1。
2. 另一主存块映射到本行 · 发生替换：以直接映射为例，冲突失效要求替换旧块。
3. 检查旧行dirty=1 · 写回旧主存块：必须按旧标记定位旧主存块后写回，不能用新块地址写回旧数据。
4. 读入新块 · 更新标记清脏位：本例新请求是读失效；新块装入后初始dirty=0。写失效是否装入另由写分配策略决定。

分页虚拟存储：页表字段与地址变换 100 页表项是否有效，决定先访问数据还是先处理缺页。

虚地址与实地址

虚拟地址是程序使用的逻辑地址，物理地址指向实际主存单元。虚拟存储依靠局部性，在主存与辅存间按需调入内容，以时间开销换取较大的逻辑地址空间，并提供隔离、保护等能力；“具有主存速度和辅存容量”是目标近似，不是每次访问都没有额外代价。

分页模型

虚地址划为虚页号与页内偏移，物理地址划为页框号与相同页内偏移；页大小等于页框大小。页表给出虚页与页框的映射，常包含存在/有效位、脏位、访问位、权限和外存定位信息。页表由操作系统维护，基址寄存器指出其内存位置；不能理解为整张页表都直接塞在 PCB 中。

$$A_{PTE} = PTBR + VPN \times S_{PTE}$$

按字节编址时必须乘页表项字节数，不能把页号直接当作字节位移。

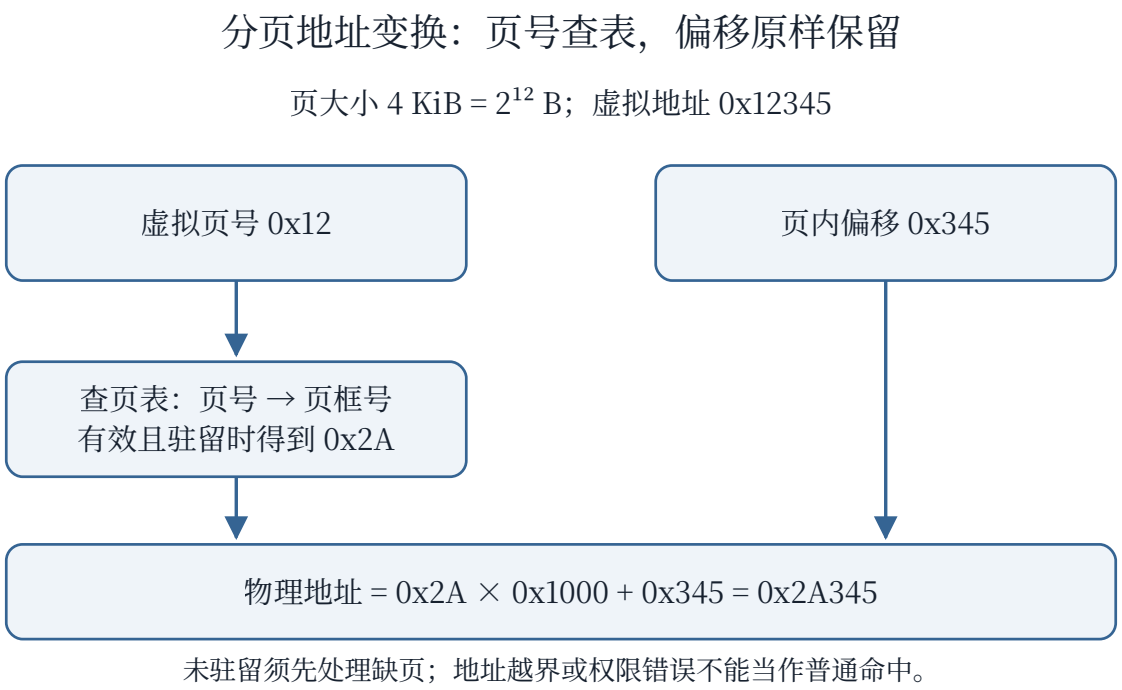
$$PA = PFN \times P + offset$$

P 为页大小；以 2 的幂分块时等价于页框号与页内偏移拼接。

缺页与内部碎片

存在位为 1 且权限允许时可形成物理地址；存在位为 0 引起缺页异常，由操作系统调入页面后重试。分页便于管理固定大小单元，但进程最后一页通常存在内部碎片，页本身未必对应程序逻辑模块。

机制与步骤图解



单独查看图片

分段与段页式：逻辑保护和物理分配 101 段内地址要检查界限，不能只做基址相加。

分段模型

程序按代码、数据等逻辑模块划分为长度可变的段，虚地址由段号、段内位移组成。段表项记录基址、长度和保护等信息；先检查段号与位移是否合法，再由段基址加位移得到物理地址。段便于按逻辑单位共享、保护与独立管理，但长度不一可能产生外部碎片。

段页式的查表链

程序先按逻辑分段，每段再划分固定大小的页，主存仍按页框分配。地址由段号、段内页号、页内偏移组成：段号定位段表项得到该段页表基址，段内页号定位页表项得到页框号，再与页内偏移组成物理地址。

查表地址均须考虑表项大小；在未命中地址转换缓存的简化模型中，段页式需要查段表和页表，再访问数据。逻辑段长度不必恰好等于整页倍数，物理分配占用整页所留下的尾部才是内部碎片。

三种方式对照

方式	划分依据	主要碎片
分页	系统规定的固定大小	内部碎片
分段	程序逻辑，长度可变	外部碎片
段页式	逻辑分段、段内分页	末页内部碎片；避免按整段连续分配的外部碎片问题

TLB、页表与 Cache：三种“命中”不能混为一谈 102 TLB 缓存地址转换，Cache 缓存指令和数据。

缺失分别意味着什么

部件	保存内容	未命中后的动作
TLB	近期页表映射及权限信息	查页表；可能仅补充 TLB，也可能发现缺页
页表存在位	页面当前是否驻留主存及映射	页面不在内存时由操作系统处理缺页
Cache	物理主存块中的数据或指令副本	一般由硬件从下级存储调入

一次访存的逻辑流程

在采用物理 Cache 的简化串行模型中，先由虚地址查 TLB；TLB 未命中则遍历页表，驻留时补充 TLB，不驻留时处理缺页。获得物理地址后再查 Cache，失效再访问主存并调块。真实机器可并行查询部分字段，不能把该逻辑流程当作所有硬件的固定时序。

TLB 未命中不等于缺页；Cache 未命中也不等于缺页。在正常有效映射下，TLB 命中说明已有可使用的驻留映射，Cache 命中说明所需数据有高速副本；Cache 命中不能反推 TLB 必命中。

时间题的边界

计算访问次数时明确是否含取指、页表多级遍历、TLB 查询时间、Cache 查询时间和缺页处理开销。最好情况下 TLB 与 Cache 都命中，无须访问主存；但不能说整个访存完全没有任何硬件访问。关联存储器按内容比较，适用于快表等结构，不应与“随机按地址访问”混淆。

图画推演与逐步动画

例：一次TLB未命中与下一次命中

查看完整步骤（也可离线阅读）

1. 0 · 拆分虚拟地址：页大小4KiB，虚拟地址0x12345分成页号0x12、偏移0x345。假定访问权限合法，页表项驻留。
2. 1 · 查询TLB未命中：TLB没有页号0x12的缓存翻译，因此需查询页表。TLB未命中本身不等于缺页。
3. 2 · 页表给出页框：假定页表项有效且驻留，页框号为0x2A。这里没有调入磁盘页面，不产生本例意义下的缺页处理。
4. 3 · 填入TLB并拼地址：缓存VPN到PFN的映射，物理地址为0x2A000+0x345=0x2A345。页内偏移不参与重映射。

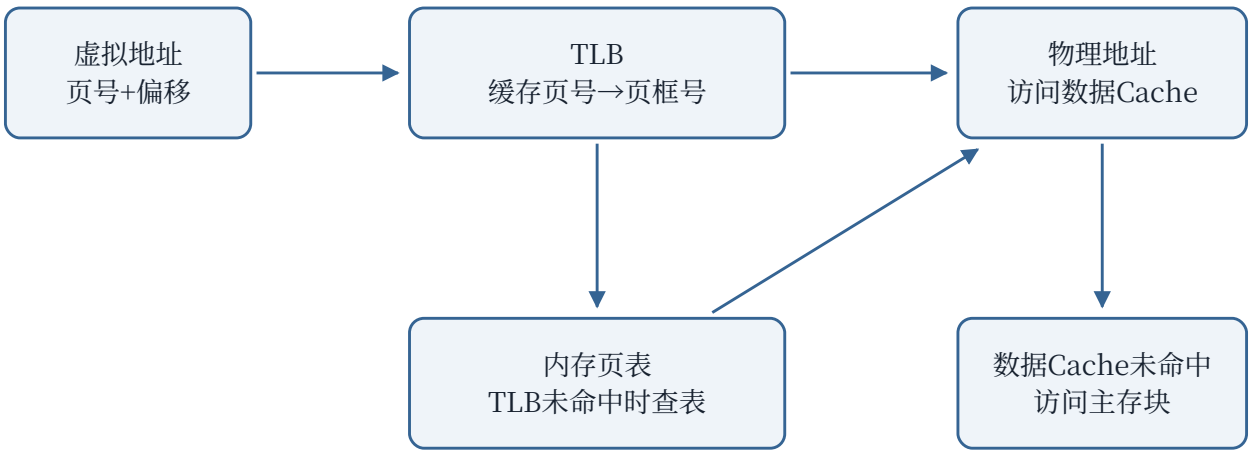
5. 4 · 同页地址再次访问：再访问0x1237C，页号仍0x12，偏移为0x37C。假定TLB映射未替换且仍有效，这次可命中TLB。
6. 5 · 翻译命中仍需检查数据：生成PA=0x2A37C。TLB缓存地址翻译，Cache缓存数据块，TLB命中不能保证数据Cache命中。

结构对照图：地址变换与数据缓存：查到映射还要取数据

图采用物理地址访问Cache的简化模型；TLB命中只说明地址映射较快得到，不能推断数据Cache也命中。

TLB未命中但页表项驻留有效，可以查表后继续，不一定发生缺页。真实处理器可能并行访问或使用VIPT等设计，计时要看题设。

地址变换与数据缓存：查到映射还要取数据



单独查看图片

指令系统与机器级程序

计算机组成原理 · 9 条

指令格式、地址数与扩展操作码 103 显式地址字段少，不表示参与操作的信息一定少。

指令系统是软硬件接口

一条指令规定机器完成的基本功能，指令系统是全部机器指令的集合。操作码指定操作性质，地址字段可给出操作数、操作数地址或地址形成方法；结果位置和下一条指令地址也可能由约定隐含给出。指令字长描述编码占用空间，与机器字长、存储字长分别讨论。

按显式地址数分类

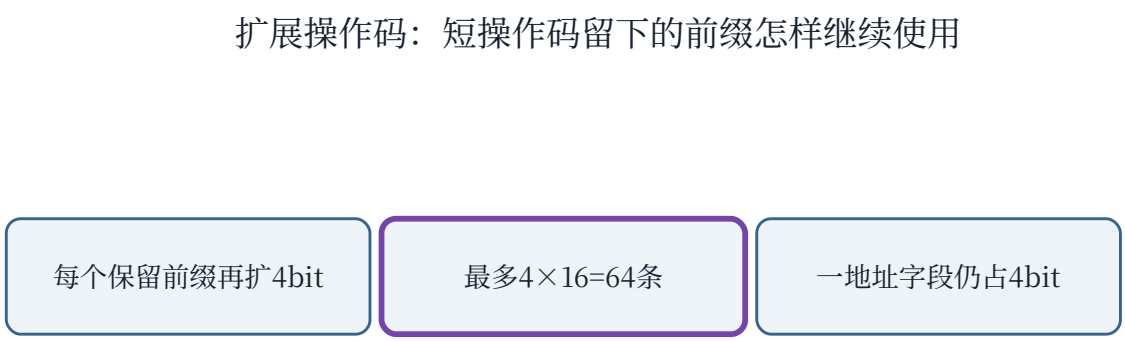
地址数	典型形式	隐含信息
零地址	仅操作码；或栈式运算	操作数可能隐含在栈顶和次栈顶
一地址	对 A 单目操作；或 ACC 与 A 运算	ACC 可隐含为另一源和目的
二地址	两个源之一兼作目的	源目的顺序由指令语法规定
三地址	两个源、一个独立目的	下一条地址通常由 PC 顺序形成
四地址	两个源、结果、下一条地址	显式给出后继指令地址

操作码长度与访存次数

定长 n 位操作码最多区分 2^n 种编码。扩展操作码在地址字段较少时利用腾出的位扩展操作码，要求短操作码不能成为长操作码的前缀；若上一层保留 m 个扩展入口，下一级增加 n 位就可产生 $m\times2^n$ 种编码。

指令可按数据传送、算术逻辑、移位、程序控制和 I/O 等分类。访存次数取决于指令长度、操作数位置和间接层数；例如单次取指且三地址均在主存的运算，需取指一次、读两个源、写一个结果共四次，不能仅凭“三地址”无条件断定所有机器都四次访存。

图解与逐步推演：扩展操作码：短操作码留下的前缀怎样继续使用



单独查看图片

扩展操作码：短操作码留下的前缀怎样继续使用

查看完整步骤（也可离线阅读）

1. 16bit指令字：假设地址字段数与宽度按此模型固定，不引入其他保留字段。

2. 定义15条三地址指令：保留的1111前缀让后面4bit也成为操作码，用来扩展两地址指令。
3. 定义12条两地址指令：两地址还占8bit，操作码可扩为8bit；保留4个8bit前缀继续扩展。
4. 扩为一地址指令：一地址时操作码宽12bit。前缀不能同时解释为已使用的短指令，否则译码不唯一。

指令寻址与 PC 增量 104 PC 的顺序增量等于指令占用的编址单位数。

顺序与跳跃

顺序寻址由当前指令后继位置形成下一条地址；跳跃寻址由转移指令改变 PC。若按字节编址，32 位定长指令通常顺序 PC+4；按字编址且一字一条指令则 PC+1。变长指令须先识别实际长度，不能固定增加机器字长。

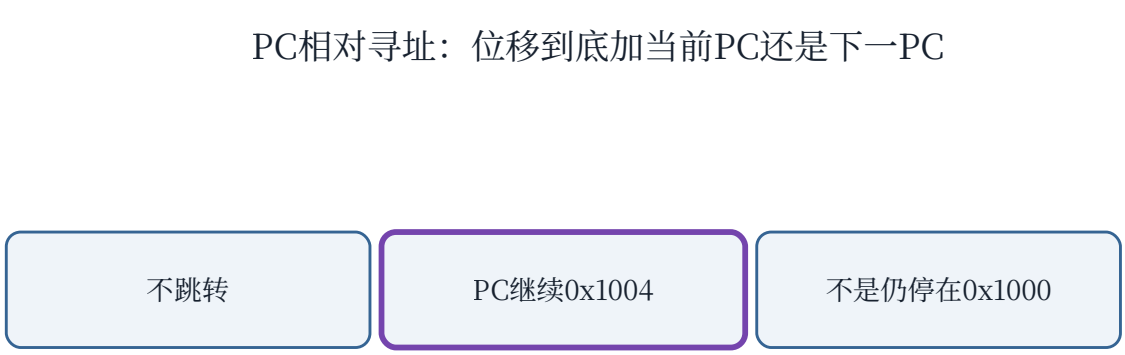
相对寻址的基准

相对地址通常 EA=(PC)+有符号位移 A，但 PC 是当前指令地址还是已递增的后继地址，要以 ISA 与题设为准。位移可能以字节为单位，也可能先左移若干位表示按指令或字对齐的距离。相对寻址利于程序在一定范围内浮动，常用于条件分支。

计算前先列四项

计算分支目标时依次明确：基准 PC、位移位宽与补码真值、位移单位、目标地址位宽。例如位移 −10 若以字节计就减 10；若以 4 字节指令计则减 40，不能只看到 −10 就直接代入。

图解与逐步推演：PC相对寻址：位移到底加当前PC还是下一PC



单独查看图片

PC相对寻址：位移到底加当前PC还是下一PC

查看完整步骤（也可离线阅读）

1. 本例定长4B指令：明确约定分支相对基准是下一条指令地址；具体ISA必须按其定义。

2. 按下一PC加位移：这里位移已经是字节，不再额外乘4。
3. 若题目改给3条指令的位移：位移字段有时隐含低位0或以字为单位，只有题设如此规定时才缩放。
4. 条件不满足时：条件判断、位移解释和PC更新时点必须配套分析。

立即、直接、间接与寄存器寻址 105 把形式地址 A、有效地址 EA 和最终操作数分开写。

六类基本形式

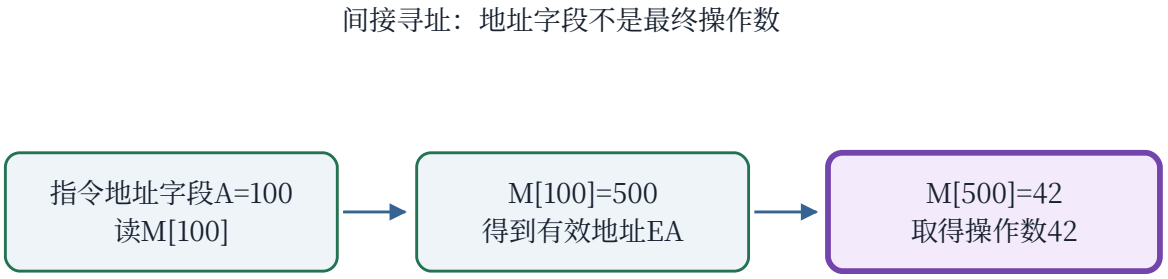
方式	操作数或 EA	执行阶段额外主存读取（简化模型）
立即	操作数就在指令字段 A 中	0
直接	EA=A，操作数 M[A]	1
一次间接	EA=M[A]，操作数 M[EA]	2
寄存器	操作数 R_i	0
寄存器间接	EA=(R_i)，操作数 M[EA]	1
隐含	由操作码或约定寄存器指定	取决于隐含位置

优点、限制与计数口径

立即数不需再读取操作数，但其范围受字段长度限制；直接寻址简单，范围受地址字段限制；间接寻址可用存储字中的完整地址扩大范围并支持指针、子程序等，但增加访存。寄存器寻址编码短、速度快，但寄存器数量有限；寄存器间接便于指针与循环。

表中的次数不含取指，也不含结果写回、多级间接或跨界访问；若题目问总访存，应把这些加回。寄存器寻址“地址字段是寄存器编号”，不能把 R_i 中内容再误当成内存地址。

图解与逐步推演：间接寻址：地址字段不是最终操作数



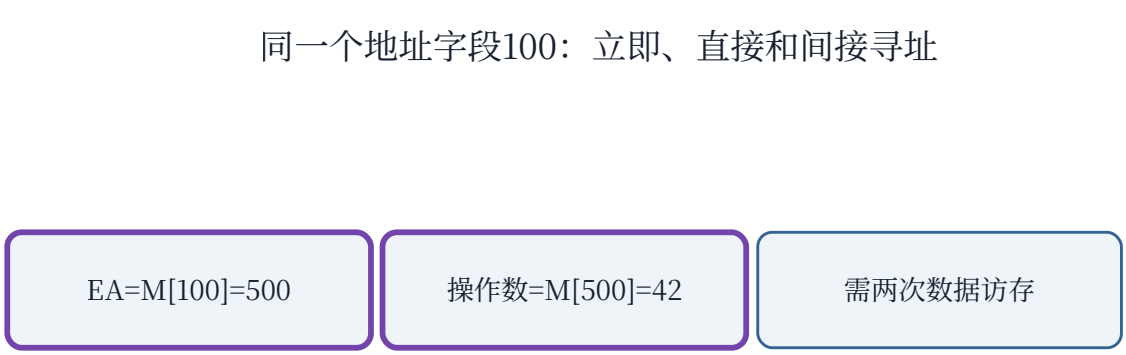
单独查看图片

间接寻址：地址字段不是最终操作数

查看完整步骤（也可离线阅读）

1. 指令地址字段A=100 · 读M[100]：采用一次存储器间接寻址，指令给出指针所在地址100。
2. M[100]=500 · 得到有效地址EA：第一次数据访存取出指针500，有效地址是500。
3. M[500]=42 · 取得操作数42：第二次数据访存得到操作数42；这里不计取指本身和Cache影响。

图解与逐步推演：同一个地址字段100：立即、直接和间接寻址



单独查看图片

同一个地址字段100：立即、直接和间接寻址

查看完整步骤（也可离线阅读）

1. 内存内容固定：以下只改变寻址方式，不改变这两个内存单元内容。
2. 立即寻址：这里只是不为获取这个立即数额外访存，取指本身仍需从指令存储层次获得。
3. 直接寻址：地址字段直接给出操作数位置。
4. 一次存储器间接寻址：第一次取指针，第二次取数据；忽略Cache命中与取指开销。

基址、变址与堆栈寻址 106 计算公式可能相同，寄存器的语义和使用方式不同。

基址与变址

基址寻址 EA=(BR)+A，常以程序或区域基地址加偏移，便于重定位；变址寻址 EA=A+(IX)，常把 A 作为数组起点、IX 作为随循环变化的偏移。二者硬件都是相加，但用途和寄存器更新约定不同。教材的操作系统基址模型与通用寄存器基址模式应分开理解，不能绝对断言任何基址寄存器用户都不能改。

复合寻址的先后顺序

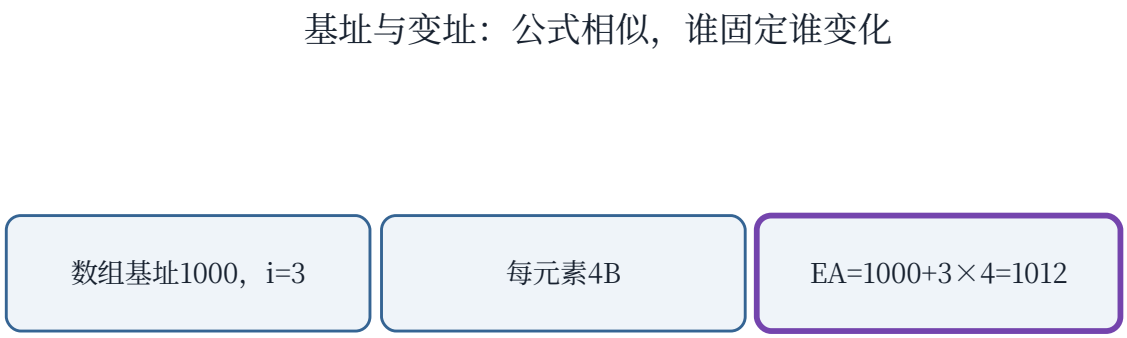
先间接再变址是 EA=M[A]+(IX)，先变址再间接是 EA=M[A+(IX)]。括号改变访问哪一个内存单元，应画出每一步取值，不能因加法形式相似就交换次序。

堆栈与 SP

堆栈按后进先出组织，可由寄存器构成硬栈，也可用内存区域形成软栈。栈顶由 SP 指示；向低地址增长还是向高地址增长，以及 SP 指向当前顶元素还是下一空位置，都以题设为准。

在向低地址增长、先减后存的栈中，入栈先减 SP 再写，出栈先读再加 SP；增减量等于一个栈元素占用的编址单元数。例如按字节编址、元素 16 位，则每次为 2，而不是无条件 1。

图解与逐步推演：基址与变址：公式相似，谁固定谁变化



单独查看图片

基址与变址：公式相似，谁固定谁变化

查看完整步骤（也可离线阅读）

1. 统一数值示例：这里只比有效地址形成，实际ISA还可能有比例因子、符号扩展及多寄存器组合。
2. 基址寻址：在传统教材模型中基址寄存器保存某区域起始基址，位移A定位该区域内对象。
3. 变址寻址：相同数值可得相同EA，但用途与谁随循环变化的解释不同，不由加法左右位置区分。
4. 数组元素带比例因子：比例变址模型把元素下标转成字节偏移；不能把下标3直接当成3B偏移。

边界对齐与大小端存储 107 对齐影响访存次数，端序影响字节排列。

边界对齐

在常见 32 位字、按字节编址的自然对齐模型中，半字地址为 2 的倍数，字地址为 4 的倍数。结构体成员前和结尾可能加入填充，以满足各字段及整个结构的对齐要求。对齐可避免一个数据横跨多个存储字，提高访问效率，但会浪费部分空间；具体对齐和非对齐支持由 ABI、ISA 与硬件决定。

同一数值的两种端序

地址	0x4000	0x4001	0x4002	0x4003
大端存 0x12345678	0x12	0x34	0x56	0x78

地址	0x4000	0x4001	0x4002	0x4003
小端存 0x12345678	0x78	0x56	0x34	0x12

高低地址与高低有效字节

大端把最高有效字节放在最低地址，小端把最低有效字节放在最低地址。只有一个字节时不存在字节端序差别。端序不改变一个字节内部二进制位的书写顺序，也不改变数值本身。

图解与逐步推演：同一个32位数的大端与小端布局

同一个32位数的大端与小端布局



单独查看图片

同一个32位数的大端与小端布局

查看完整步骤（也可离线阅读）

1. 大端：32位数0x12345678，大端把最高有效字节0x12放在最低地址。
2. 小端：小端把最低有效字节0x78放在最低地址。字节内部bit顺序没有因此颠倒。

CISC 与 RISC：比较设计取向 108 典型特征不是对所有现代处理器的绝对约束。

主要差异

维度	CISC 典型设计	RISC 典型设计
指令与寻址	种类多、功能复杂、格式可能变长	常用简单指令为主，格式较规整
访存	多类指令可直接操作内存	主要由 Load/Store 访存，运算在寄存器间完成

维度	CISC 典型设计	RISC 典型设计
寄存器	典型早期设计较少	通常提供较多通用寄存器
控制实现	常用微程序便于复杂指令	常用硬布线，便于流水实现
编译优化	复杂功能不一定都频繁使用	依赖编译器组合简单指令完成复杂工作

一周期的正确理解

RISC 常见的“多数指令一周期”指设计目标或理想吞吐特征，不代表一条指令从取指到完成只经历一个流水段。现代 CISC/RISC 都可能采用流水、乱序和内部微操作，不能只靠名称判定实际微结构。

x86 Intel 语法：数据、算术和逻辑指令 109 以下示例以 32 位 x86 模型为边界，目的操作数在前。

寄存器与传送

EAX、EBX、ECX、EDX 为 32 位通用寄存器，AX 等表示其低 16 位，AH/AL 等表示相应高、低 8 位；ESI/EDI 常用于索引，EBP/ESP 常用于栈帧与栈顶。mov dst,src 复制源到目的，普通 mov 不能两端同时为内存。

运算形式

指令	基本含义	结果
add / sub	目的加源 / 目的减源	写回目的
inc / dec	操作数加 1 / 减 1	原位置写回；标志影响以 ISA 为准
and / or / xor	按位与、或、异或	写回目的
not / neg	逐位取反 / 取相反数	两者不是同一操作
shl / shr	逻辑左移 / 逻辑右移	空位补 0；有符号右移另看 sar
imul	有符号乘法	两操作数或三操作数形式均写目的
idiv	有符号除法	32 位形式以 EDX:EAX 为被除数，商 EAX、余数 EDX

不能把编译器与硬件标志混为一谈

乘法是否截断、哪些标志有效由具体 imul 形式定义；OF/CF 由 CPU 执行指令时产生，并非编译器在运行时直接设置。置溢出标志不等于必然自动调用异常处理程序。除法需检查除数和商范围，不能把高位寄存器残留值忽略。

条件分支、选择与循环的机器级实现 110 先看比较产生的标志，再看按有符号还是无符号解释。

cmp、test 与跳转

cmp dst,src 按减法 dst−src 更新标志而不保存差；test 按位与更新相关标志但不保存与结果。jmp 无条件改变指令指针，条件跳转依据状态字选择后继地址。

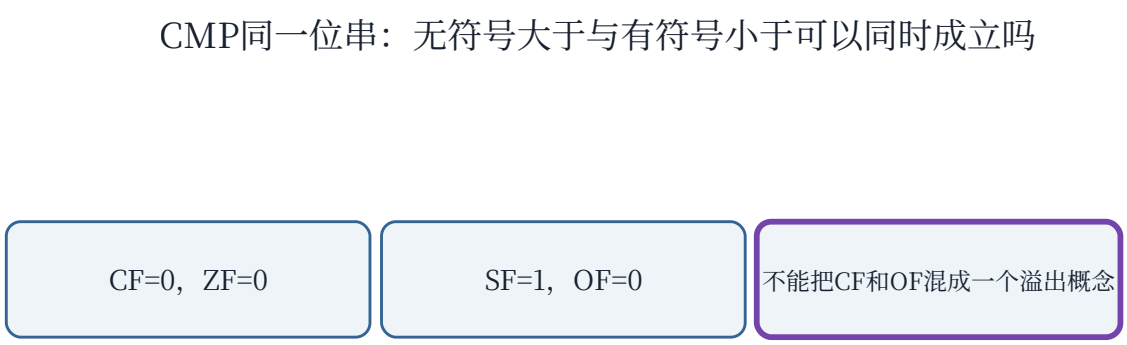
比较条件

解释	小于	等于	大于
无符号 a−b	CF=1	ZF=1	CF=0 且 ZF=0
有符号 a−b	SF≠OF	ZF=1	SF=OF 且 ZF=0

选择和循环

选择结构先比较，再按条件跳向某分支，另一个分支可通过无条件跳转避开。循环可在入口先检查退出条件，主体末尾更新变量并再次判断是否回跳；do-while 则通常先执行主体再判断。计数循环可使用专门指令或计数寄存器自减配分支，关键是检查时点和执行次数。

图解与逐步推演：CMP同一位串：无符号大于与有符号小于可以同时成立吗



单独查看图片

CMP同一位串：无符号大于与有符号小于可以同时成立吗

查看完整步骤（也可离线阅读）

1. 比较8bit 0xFF与0x01：以常见x86的cmp减法标志约定为例。

2. 无符号解释：比较时选择无符号条件，不能只看差的最高位。
3. 有符号解释：同一次运算的标志不变，改变的是按哪种类型解释原位串以及采用哪组跳转条件。
4. 四标志汇总：本例两种判断并不矛盾：255>1与−1<1描述的是不同数值解释。

函数调用、栈帧与返回地址 111 call 保存返回位置，栈帧保存本次调用所需的现场。

32 位向下增长栈的示例

以常见 32 位 x86 约定为例，push 先使 ESP 减 4 再写入，pop 先读出再使 ESP 加 4。call 保存调用之后下一条指令的地址并跳转，ret 从栈恢复该返回地址；不能把返回地址写成 call 指令自身的起始位置。

建立与撤销栈帧

- 调用者按约定传递参数；call 放入返回地址。被调者保存旧帧指针，建立新帧，再为局部变量和保存的寄存器分配空间。
- EBP 指向当前帧的固定参考位置，ESP 指向当前栈顶；旧 EBP 形成调用链，返回地址指向上层函数应继续执行的位置。
- 返回时先恢复局部空间及保存的寄存器，再恢复旧 EBP，最后由 ret 恢复指令指针；参数由谁清理由调用约定决定。
- 递归或嵌套调用每次都有独立栈帧，因此不同调用的局部变量、返回地址不能混用。

```
; 示意：具体参数传递与清栈责任由调用约定规定
push ebp
mov ebp, esp
sub esp, local_size
; 使用参数与局部变量，必要时保存寄存器
mov esp, ebp
pop ebp
ret
```

结构对照图：32位x86栈帧：EBP固定参照，ESP随栈变化

示例为常见32位x86、向低地址增长、使用push ebp; mov ebp,esp建立帧的约定，不代表所有ABI都这样传参。

call保存其后下一条指令的返回地址；递归每次建立独立栈帧。使用帧指针省略优化时不应照搬图中的EBP布局。

32位x86栈帧：EBP固定参照，ESP随栈变化



单独查看图片

中央处理器与控制

计算机组成原理 · 7 条

CPU 功能、寄存器与可见性 112 控制单元决定怎么做，运算部件执行，寄存器保存阶段状态。

五类功能

CPU 负责指令控制、操作控制、时间控制、数据加工及中断异常处理。逻辑上可分控制单元、运算单元和内部暂存/寄存器等部分；指令的程序顺序语义必须保持，但实际微结构可以重叠甚至乱序执行，不能由教学描述推出所有指令物理执行都完全串行。

可见与透明

类别	典型对象	意义
程序可见的体系结构状态	通用寄存器、PC、部分状态字	程序或特权程序通过指令约定访问
内部实现寄存器	IR、MAR、MDR、暂存 Y/Z 等	通常对应用程序员透明
状态字	条件码、控制状态等	不同位的访问权限和更新方式不同

常见数据通路部件

三态门使一个输出可处于高阻态，避免共享总线多个源同时驱动；多路选择器根据选择信号从多路输入选一路输出；译码器把输入编码转换为相应选通信号；加法器可专用于 PC 递增或地址计算，ALU 承担更一般运算。单总线 ALU 往往配 Y、Z 暂存器，以满足两个输入同时有效并保存组合逻辑输出。

寄存器位宽、地址宽度和实际实现地址空间须分别讨论。64 位寄存器不自动意味着实现全部 2^{64} 字节主存，也不能靠基址寻址凭空突破物理地址线可表示的空间。

指令周期、机器周期、时钟周期与四类阶段 113

阶段按需出现，不是每条指令都有间址和中断。

三个时间层次

指令周期是取出并完成一条指令的过程；机器周期是实现中用于组织基本阶段的时间单位；时钟周期是主时钟的一次周期。一个指令可含若干机器周期，一个机器周期可含若干时钟周期；具体对应数量并不固定，一个节拍可完成多个互不冲突的微操作。

阶段与数据流

阶段	主要目的	典型动作
取指	把当前指令送 IR	PC→MAR，存储读，MDR→IR，形成后继 PC
间址（需要时）	取得间接地址中的有效地址	指令地址字段→MAR，存储读，MDR 保存有效地址
执行	完成该条指令规定功能	随指令而异，可运算、访存、转移或控制
中断响应（满足条件时）	保存断点并转入服务程序	保存 PC/状态，形成入口，改变处理状态

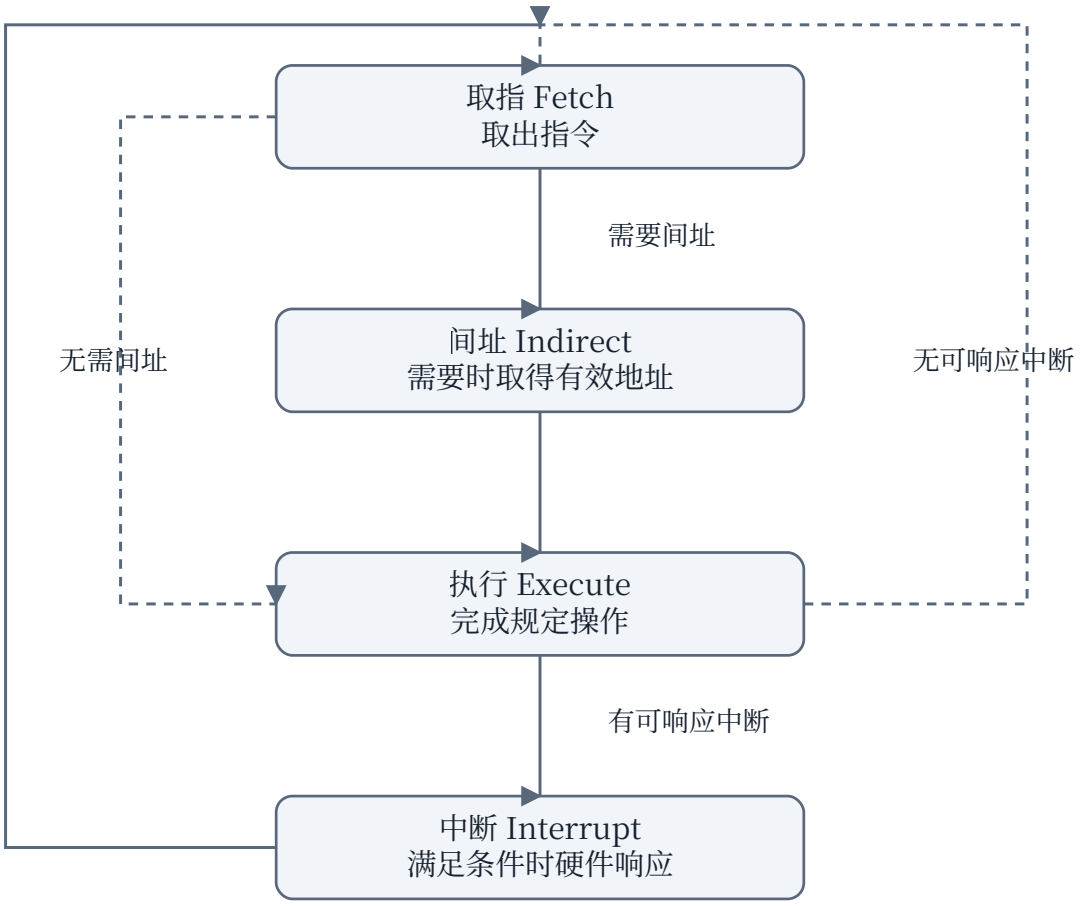
不同指令的执行差别

CLA 可把累加器清零，不需执行阶段读主存；ADD A 需读 A 指定操作数后与累加器相加；STA A 应把累加器内容写到指定地址；JMP A 把目标地址送 PC；NOP 不改变程序规定的数据状态，但仍经历取指等过程。若是间接存储，须先取得真实目标地址，不能把直接存储的动作当成间接寻址。

访存与中断的条件

执行阶段不一定访存，间址也可能来自寄存器而不是主存；中断保存现场可用栈或专用寄存器，取决于体系。响应通常在指令边界检查，但异常、流水线与精确中断还需按具体模型判断，不能笼统称“四个阶段必然各访问一次主存”。

机制图解

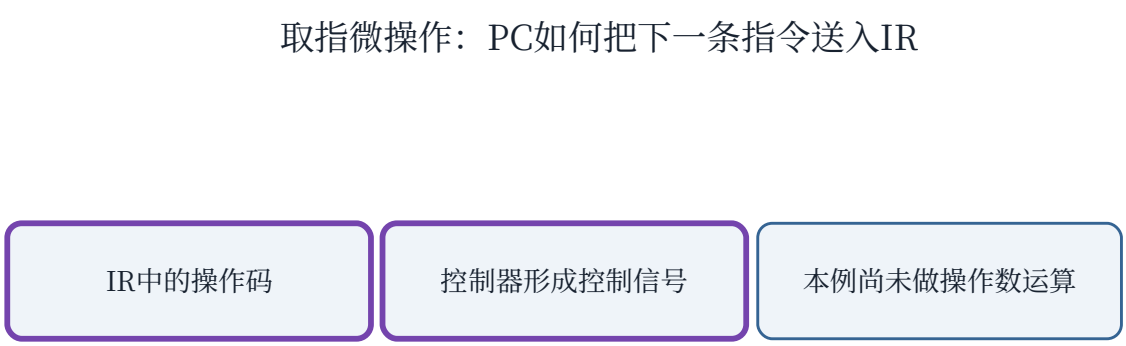


响应后转入服务程序，仍逐条取指执行

图示为教材指令周期模型；并非每条指令都经历间址和中断阶段。

单独查看图片

图解与逐步推演：取指微操作：PC如何把下一条指令送入IR



单独查看图片

取指微操作：PC如何把下一条指令送入IR

查看完整步骤（也可离线阅读）

1. PC送地址：以按字节编址、定长4B指令的简化CPU为例；这里只演示取指阶段。
2. 读出指令：MAR提供地址，MDR暂存返回数据；实际访存可能跨多个时钟周期。
3. 进入指令寄存器：把指令送入IR并按定长指令长度更新PC；两项能否同拍由数据通路决定。
4. 译码后执行：取指完成不代表整条指令完成。变长指令、分支与不同架构的PC更新规则另按题设。

数据通路与微操作：单总线怎样完成加法 114 操作可以同拍，前提是无资源冲突且数据已就绪。

三种结构

数据通路是信息在寄存器、运算器、存储接口等部件之间流动的路径。单总线结构简单但共享冲突明显；多总线结构能同时传送更多数据；专用通路按常见数据流设置连接，性能高但硬件量较大。控制信号决定哪个源输出、哪个目的接收以及 ALU 做何种操作。

寄存器传送与取指

PC→MAR 可使 PCout 与 MARin 同时有效；取指还须发存储读命令，等数据进入 MDR 后，再使 MDRout 与 IRin 有效。不能在存储数据尚未到达前就把 MDR 中旧值当新指令。

R1+R2→R3 的单总线示意

· 同一条共享总线在同一时刻只能由一个源驱动，但可让多个允许的目的同时接收同一数据。

· ALU 是组合逻辑，两个输入须同时有效，结果应由寄存器保存；不能认为 ALU 自身记住上一次输入。

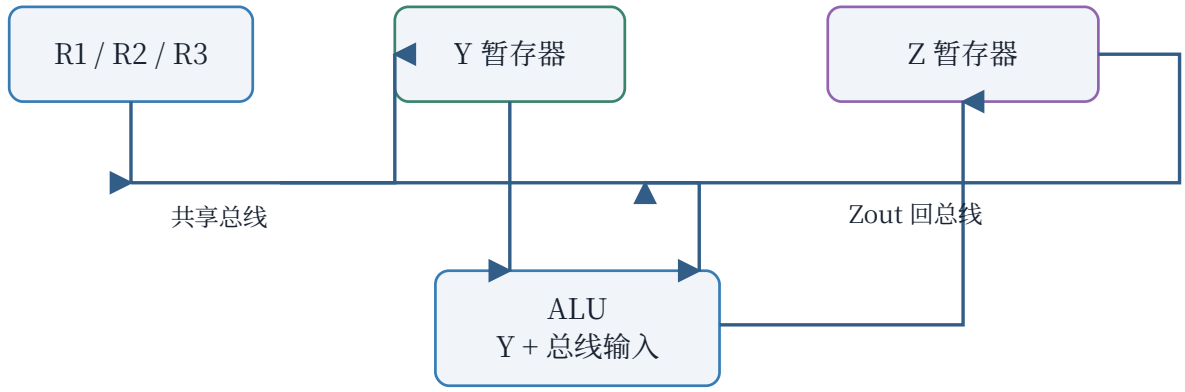
· 带自增 PC、独立地址加法器或双读端口寄存器组的结构可改变微操作合并方式，不能脱离图纸固定拍数。

```
第 1 拍: R1out, Yin                ; R1 经总线存入 Y
第 2 拍: R2out, ALU=ADD, Zin        ; Y 与总线的 R2 相加，结果锁存 Z
第 3 拍: Zout, R3in                ; Z 经总线写回 R3
```

图解：Y、ALU、Z与共享总线

图中寄存器组R1/R2/R3为简化画法，Y和Z分别保存第一个输入和运算结果。箭头表示可用传送方向，不代表所有源能同时驱动总线；控制器在各拍只使相应输出门有效。

单总线数据通路：为什么需要 Y 和 Z



① R1out，Yin：第一个操作数先存入 Y

② R2out，ADD，Zin：相加并把结果存入 Z

③ Zout，R3in：结果经总线写回 R3

单独查看图片

硬布线控制：操作码、节拍与状态共同决定信号 115 控制信号不是仅由操作码决定。

工作依据

硬布线控制器用组合逻辑及状态存储元件形成控制信号。输入包括指令译码结果、当前执行阶段、节拍和条件标志等；同一指令在不同阶段需要不同微操作，条件指令还受当前状态影响。

设计和时序

一般先列各指令的微操作序列，安排可同时完成的相容微操作，再形成各控制信号的逻辑表达式。时钟与节拍组织要给数据传播和锁存留足时间，机器周期通常按基本操作要求安排，不能仅凭指令数推算。

优点与代价

硬布线响应快，适合规整指令与高性能实现；指令和控制流程复杂时，逻辑设计、修改与扩充较困难。RISC 常采用硬布线是典型关联，不是硬性等价关系。

微程序控制：微命令、微指令与控制存储器 116 机器指令由微程序解释执行，主存和控存各存不同内容。

概念层次

微命令是控制部件发出的基本控制信号，微操作是相应硬件执行的动作；微指令把一个节拍允许发出的微命令及后继地址控制信息组织成一条控制字；有序微指令序列形成微程序，用以实现机器指令。微指令周期是取出并执行一条微指令的时间。

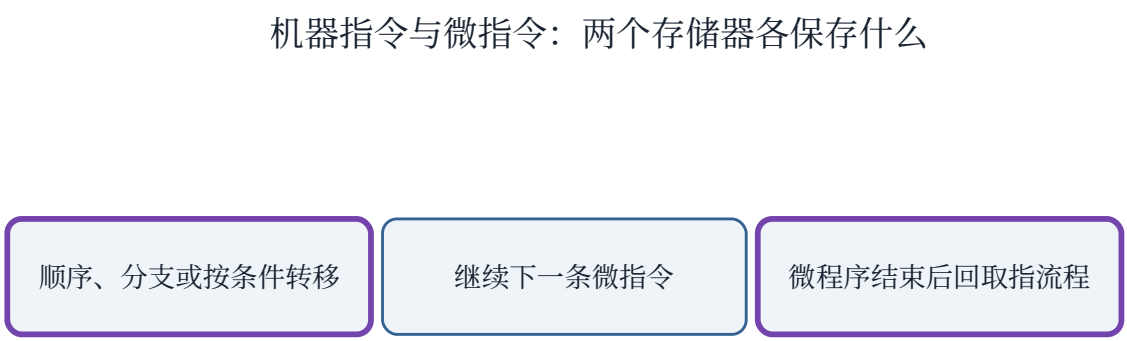
控制存储器与相容性

控制存储器 CM 位于控制器内部，保存微程序；主存保存用户程序与数据。相容微操作可在资源和依赖允许时同拍执行，相斥微操作不能同时发出。不能把机器指令操作码当成微指令完整编码，也不能认为一条机器指令恰好只对应一条微指令。

动态与多级设计

固定微程序通常保存在只读或固化控制存储中；可修改控制存储支持动态微程序设计。毫微程序设计进一步把微指令分解，用第二级控制存储实现，属于用存储层次折中控制字长度与实现复杂度的思路。

图解与逐步推演：机器指令与微指令：两个存储器各保存什么



单独查看图片

机器指令与微指令：两个存储器各保存什么

查看完整步骤（也可离线阅读）

1. 执行机器指令ADD：机器程序中的ADD是ISA可见指令，不等于一条微指令。
2. 入口地址形成：微程序控制器内部用控制存储器保存微程序，和程序主存的用途不同。
3. 微指令发控制信号：一条微指令可包含多个相容微命令，能否并行取决于数据通路与资源冲突。

4. 后继微地址形成：微指令地址变化属于控制器内部流程，不等于每一步都改写机器程序PC。

微指令编码与后继地址形成 118 控制字段决定做什么，顺序字段决定接着读哪条微指令。

执行链与组成

微程序控制器包含控制存储器 CM、微指令寄存器 CMDR、微地址寄存器 CMAR、微地址形成部件和转移判别逻辑。取指微程序通常可共享，再按机器指令操作码转入相应执行微程序；“n 条机器指令必至少 n+1 个独立微程序”依教材把共享取指和执行段分开计数的约定，不是不可共享执行片段的普遍定理。

控制字段编码

编码	组织	取舍
直接编码	一位直接控制一个微命令	译码少、并行性好，微指令长
字段直接编码	互斥微命令编入同字段，不同字段可并行	字长缩短，但需译码；通常保留一种编码表示无操作
字段间接编码	某字段的解释受另一字段影响	进一步压缩，控制更复杂、并行度受限
混合编码	部分直接、部分字段编码	按高频控制和资源关系折中

后继微地址与水平/垂直

后继地址可由下地址字段直接给出，按机器操作码映射得到入口，或由 CMAR/微程序计数器加 1 顺序产生；分支时再依据条件选择转移地址。多路分支还可结合测试字段和状态，在若干候选地址间选择。

水平型可同条微指令定义多种并行操作，微指令较长、微程序较短；垂直型以微操作码表达较少操作，微指令较短、微程序较长。混合型在紧凑编码上保留部分并行性。设计需同时权衡控制字宽度、控存容量、微程序长度、执行速度和修改便利性。

图解与逐步推演：字段编码：9个控制选择为何可以用6bit表示



单独查看图片

字段编码：9个控制选择为何可以用6bit表示

查看完整步骤（也可离线阅读）

1. 抽象三组互斥控制：同一组内不允许同时选多个；不同组是否可同时执行由本例数据通路保证。
2. 直接编码：直接编码的控制字较长，能否实际并行仍由硬件资源相容性决定。
3. 每组保留无操作编码：一组3个有效选择加1个无操作，共4种状态。
4. 三组合并：字段编码需要译码硬件，不代表整条微指令总宽度只有6bit；控制字段与顺序控制字段应分开计数。

异常、中断与返回位置 119 异常通常与当前指令相关，外部中断通常异步到达。

异常的三类

类别	发生与返回特点	示例
故障 Fault	常在指令完成前检测，处理后可重试该指令	缺页、部分非法操作或除错情形
自陷 Trap	通常在指令完成后报告，返回后继指令	断点、显式陷入
终止 Abort	严重故障，通常无法可靠恢复原执行	部分硬件错误

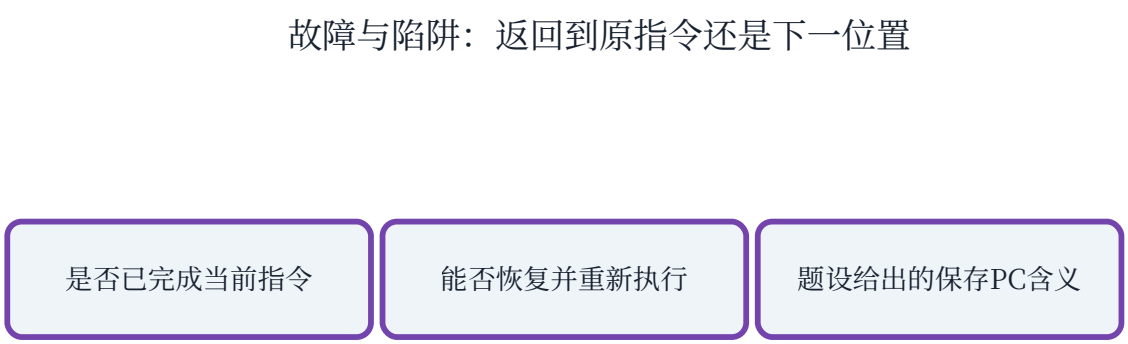
中断与检测

CPU 内部按指令执行状态检测异常；外部设备通过中断请求等机制通知 CPU。外部中断分可屏蔽和不可屏蔽，是否响应还受当前优先级、使能及处理状态约束。中断与异常都需转入相应处理入口，但不同类别保存的 PC 含义可能不同，不能把所有异常都说成返回当前指令。

响应与进程切换分开

硬件通常完成保存必要断点与状态、改变处理状态并形成入口等响应动作；服务程序再保存所需通用寄存器、处理事件、恢复状态并返回。是否需要调度其他进程由操作系统及事件决定，响应中断本身并不必然切换进程。

图解与逐步推演：故障与陷阱：返回到原指令还是下一位置



单独查看图片

故障与陷阱：返回到原指令还是下一位置

查看完整步骤（也可离线阅读）

1. 故障类示例：缺页：这是可恢复缺页故障的模型，不能把非法访问也一律当作调入页面就继续。
2. 陷阱类示例：具体返回地址由异常种类和ISA定义；断点、单步等不能只看“异常”二字统一处理。
3. 外部中断示例：不同体系结构对精确异常、流水线清空和状态保存有具体规则。
4. 检查题目关键词：别把处理程序入口地址、故障指令地址、下一条指令地址三个值混用。

指令流水线与并行体系

计算机组成原理 · 6 条

流水线的分段、锁存器与吞吐 120 流水让不同指令占据不同阶段，主要改善整段程序吞吐。

时间重叠与五级模型

将一个任务拆成多个有先后关系的子任务，用不同功能部件重叠执行不同任务的子阶段。典型五级为 IF 取指、ID 译码读寄存器、EX 运算或地址计算、MEM 数据访问、WB 寄存器写回。每级之间的流水寄存器保存数据和后续仍需使用的控制信息。

不同指令使用哪些阶段

指令	EX	MEM	WB
算术运算	ALU 运算	通常无数据访存动作	写运算结果
Load	基址加位移得有效地址	读取内存/数据 Cache	写读得数据
Store	计算地址，并传递待存数据	写内存/数据 Cache	通常不写寄存器
分支/跳转	比较或计算目标（具体阶段依题设）	可能完成 PC 更新或为空	一般为空

实现条件与边界

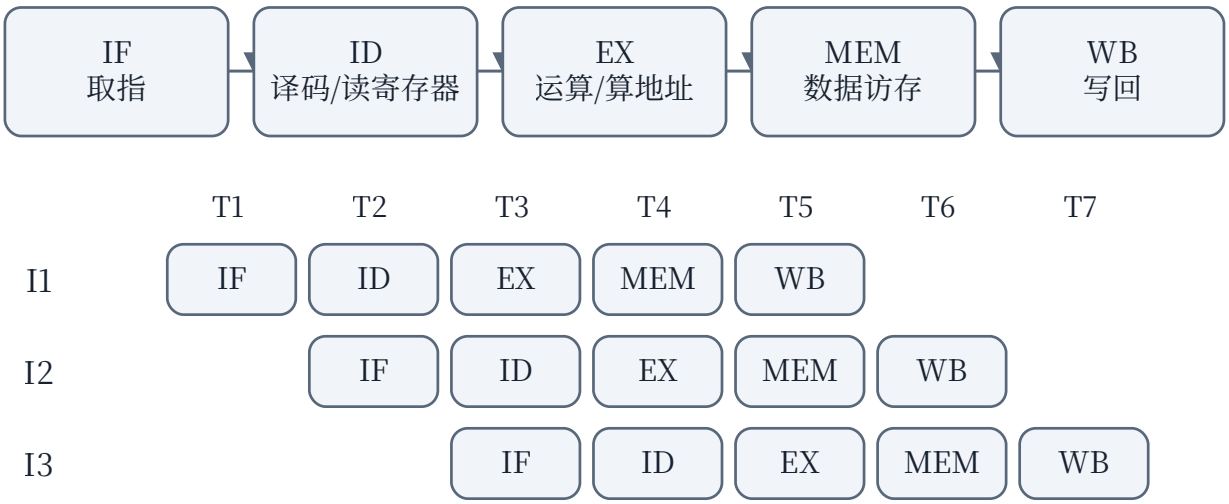
规整的指令长度和寄存器字段位置、Load/Store 设计以及对齐访问，有利于流水段安排，但不是流水线能存在的唯一条件。各段时间应尽量均衡，否则最慢段限制时钟或形成等待；连续任务才能摊薄装入、排空成本。

流水不保证单条指令延迟缩短，额外锁存器甚至可能增加延迟。并发是任务在一段时间内交错推进，并行强调时间上真正重叠；指令级并行主要由硬件开发，任务/进程级并行还依赖软件组织。

机制图解

五级流水：阶段名称与时间重叠

经典五级教学模型：下表按无停顿、每级 1 拍示意



同一拍各指令可处于不同阶段；并非各级都实际访存或写寄存器。

load：EX 算地址 → MEM 读数据 → WB 写回；store 通常不写回寄存器。

单独查看图片

图解与逐步推演：五段流水线：三条指令怎样重叠执行

五段流水线：三条指令怎样重叠执行



单独查看图片

五段流水线：三条指令怎样重叠执行

查看完整步骤（也可离线阅读）

1. 第1个时钟周期：理想五段、每段1周期、无冒险：第i条指令第s段在i+s−1周期进行；同一时刻不同指令占用不同段。
2. 第2个时钟周期：理想五段、每段1周期、无冒险：第i条指令第s段在i+s−1周期进行；同一时刻不同指令占用不同段。
3. 第3个时钟周期：理想五段、每段1周期、无冒险：第i条指令第s段在i+s−1周期进行；同一时刻不同指令占用不同段。
4. 第4个时钟周期：理想五段、每段1周期、无冒险：第i条指令第s段在i+s−1周期进行；同一时刻不同指令占用不同段。
5. 第5个时钟周期：理想五段、每段1周期、无冒险：第i条指令第s段在i+s−1周期进行；同一时刻不同指令占用不同段。
6. 第6个时钟周期：理想五段、每段1周期、无冒险：第i条指令第s段在i+s−1周期进行；同一时刻不同指令占用不同段。
7. 第7个时钟周期：理想五段、每段1周期、无冒险：第i条指令第s段在i+s−1周期进行；同一时刻不同指令占用不同段。

结构、数据和控制冒险 121 先找冲突资源或依赖，再按结果可用时点补停顿。

三类冒险

类别	原因	常见处理
结构冒险	同一时刻争用同一不可并行资源	等待、复制资源、分离指令与数据存储
数据冒险	后续读写与前序结果存在时序依赖	停顿、旁路、编译调度，复杂流水可重命名
控制冒险	下一条正确指令地址尚未确定	等待、预测、提前判定、错误路径冲刷

RAW、WAR、WAW

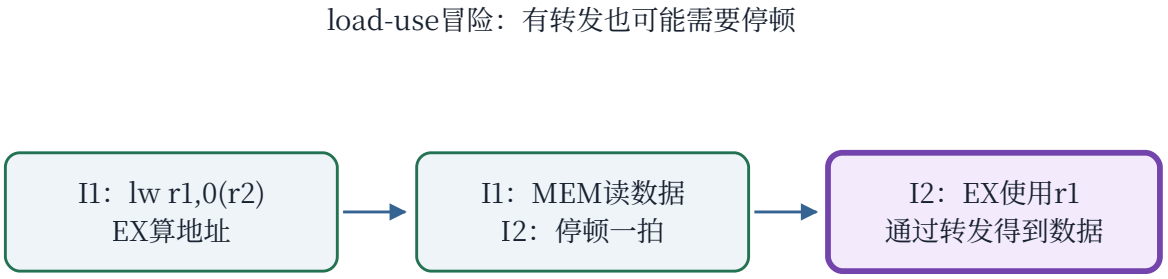
RAW（写后读）要求消费者获得前面生产者的新值；WAR（读后写）要求后写不能破坏前面尚未读取的旧值；WAW（写后写）要求最终值来自程序顺序靠后的写。经典顺序五级、固定阶段读写的模型主要面对 RAW，WAR/WAW 是否可能出现需看是否乱序、多周期和读写时点。

旁路把执行阶段已经产生的结果直接送给后续使用者，无须等寄存器写回；Load 的数据通常到 MEM 后才可用，因此即使有旁路，也可能有 Load-use 停顿。不能固定认为“只检查后面三条”适用于所有流水线。

分支停顿不是固定数字

分支在第几段确定方向、何时更新 PC、是否先预测继续取指，决定需要停顿或冲刷多少拍。停 2 或 3 拍仅适用于相应的特定分支完成阶段；未发生跳转也可能产生已经等待的控制代价，不能笼统说不跳转就永远没有控制冒险。

图解与逐步推演：load-use冒险：有转发也可能需要停顿



单独查看图片

load-use冒险：有转发也可能需要停顿

查看完整步骤（也可离线阅读）

1. I1：lw r1,0(r2) · EX算地址：假设经典五段流水线、完整转发，I2为add r3,r1,r4，紧接I1；I1在EX阶段还没有读到内存值。
2. I1：MEM读数据 · I2：停顿一拍：I1在MEM结束才产生加载数据；I2原本同周期EX需要它，来不及，所以冻结相关前段并向EX插入气泡。
3. I2：EX使用r1 · 通过转发得到数据：下一周期把MEM/WB中的值转发到I2的EX输入；具体停顿数取决于题设流水线和存储时序。

图解与逐步推演：相邻ALU指令：转发如何消除RAW等待

相邻ALU指令：转发如何消除RAW等待



单独查看图片

相邻ALU指令：转发如何消除RAW等待

查看完整步骤（也可离线阅读）

1. 两条指令：假设经典五段流水线，寄存器值在I1的EX结束时形成。
2. I1在EX，I2在ID：仅靠ID阶段读到的寄存器值不一定满足数据相关。
3. 下一周期I2进入EX：选择正确旁路输入，使I2使用新值；在此模型中ALU→ALU相关通常无需停顿。
4. 与load-use区分：是否停顿由数据何时产生与何时需要决定，不是见到RAW就固定停一拍。

流水性能计算：吞吐率、加速比和效率 122 先画时空图，公式只适用于相应的等时段理想模型。

等时 k 段、n 个任务

$T_{pipe} = (k+n-1)\tau$

各段均为 τ、无冒险停顿且每拍启动一个任务。

$TP = \frac{n}{(k+n-1)\tau}, TP_{max} = \frac{1}{\tau}$

实际吞吐率计有限任务的装入排空开销；最大吞吐率是任务充分多时的极限。

$S = \frac{nk}{k+n-1}, E = \frac{n}{k+n-1}$

非流水执行时间取 nkτ 时；理想加速比极限为 k，效率等于有效工作面积占全部段时面积。

带停顿的计算

五级等时流水执行 8 条指令，无停顿需 (5+8−1)=12 拍；若依赖和分支共引入 3 个未重叠停顿拍，则需 15 拍。计算加速比应使用同一批指令的非流水时间，而不是单条指令时间除以整批流水时间。

不等长阶段与多发射

段时间不等时，时钟需至少覆盖最慢组合逻辑段及锁存开销；不能直接把全部阶段时长相加作为每拍。若题设允许多个任务同拍发射、不同功能单元异步完成，须按实际资源画图，不能继续无条件使用单发射公式。

超标量、超流水与超长指令字 123 增加同拍并发数与细化阶段是两条不同路线。

三种技术对照

技术	并行方式	主要组织者
超标量	配置多个执行部件，同一时钟可发射多条独立指令	硬件动态选择、调度；也受编译安排影响
超流水	把原阶段进一步细分，提高流水时钟、缩短启动间隔	硬件分段，依赖与分支代价也随之变化
VLIW 超长指令字	一条长指令含多个可并行操作字段	编译器静态挖掘并安排并行操作

不要混淆发射和完成

超标量可以有顺序或乱序实现，不能定义为“一定不能调整指令顺序”；超流水也不意味着每个旧时钟周期都能无条件完成多条依赖指令。并行度仍受数据依赖、资源数和控制流限制，多部件并不自动带来线性加速。

SISD、SIMD、MIMD 与向量处理器 124 分类看指令流和数据流，不是简单数 CPU 芯片数量。

四类结构

类别	指令流 / 数据流	特点
SISD	单 / 单	单一指令流处理标量数据，内部仍可流水
SIMD	单 / 多	同一控制操作作用于多组数据
MISD	多 / 单	常规通用体系中少见，不宜绝对说逻辑上不存在
MIMD	多 / 多	多个处理器独立执行不同指令流、处理不同数据

适用任务与存储组织

SIMD 适合对数组各元素实施相同运算，可把多对数据并行送 ALU；若不同数据走不同控制分支，利用率会下降。向量处理器可直接处理向量，将数据送入向量寄存器后流水完成批量操作，适用于数值计算等规则工作负载。

MIMD 可采用消息传递的多计算机，每节点拥有独立地址空间，通过通信交换数据；也可采用共享存储多处理器，多个处理器通过共同物理地址空间中的变量通信。

硬件多线程、多核与共享存储 125 硬件线程保存执行状态，核提供实际执行资源。

三种硬件多线程

为多个线程保留各自寄存器等体系结构状态，可降低软件保存恢复开销。细粒度多线程频繁轮换线程，通常每拍选择不同线程；粗粒度多线程在长延迟阻塞时切换，流水清空与重装代价更明显；SMT 可在同一拍从不同线程发射多条指令，共享执行部件。

多核与 UMA/NUMA

多核在同一芯片集成多个处理核心，各核可独立执行线程，Cache 可私有或共享；硬件线程与核不能按数量直接画等号，也不必假定所有现代核心均对称。

共享存储体系使用单一物理地址空间，但各进程仍可有独立虚地址空间。UMA 中各处理器访问各存储位置的时间大致相同；NUMA 中延迟取决于处理器与数据所在节点的关系，可分无一致性 Cache 或具有一致性 Cache 的组织。

共享不等于自动同步

共享变量可能被多个处理器同时访问，需锁、原子操作或其他同步方式防止竞争。Cache 一致性解决多个副本的可见性，不自动保证复合操作原子性，也不代替程序同步。

总线与输入输出

计算机组成原理 · 7 条

总线分类、特性与组织结构 126 共享线路一次由谁驱动、可否并发，须看具体总线段。

共享与特性

总线是多个部件共享的信息传输介质，同一共享数据通路同一时刻通常只允许一个源驱动，但可有多个接收者。机械特性规定尺寸与引脚；电气特性规定电平和方向；功能特性规定各线用途；时间特性规定信号先后与保持关系。

分类与系统线组

分类	对象	要点
片内总线	CPU 内寄存器、ALU 等	实现内部数据传送

分类	对象	要点
系统总线	CPU、主存、I/O 接口等	包括地址、数据和控制信号
通信总线	系统间或系统与其他设备	适合相应通信距离和协议
串行 / 并行	按位 / 多位同时传输	串行信号少；并行需考虑偏斜、串扰和距离

单、双、三总线模型

单总线结构连接简单、易扩充，但部件争用大，难以同时开展多次传输。双总线可用存储器总线和 I/O 总线经通道等连接，把低速外设流量分离；三总线教学模型再增加 DMA 等专用通路。

不同总线段可能并行，但若都争用同一个单端口主存，仍需仲裁。总线条数不能直接证明整个系统每时刻只能用一种，也不能由“共享地址线”推出必然内存映射 I/O；编址方式和控制信号还须单独看。

总线性能、复用与事务四阶段 127 时钟频率、传输率和事务频率使用不同口径。

关键性能关系

$$BW = f_{transfer} \times$$

W 为每次数据传输的有效位数，ftransfer 为每秒有效传输次数，带宽单位 B/s。

$$f_{transfer} = f_{clock} \times q$$

只有题设每个时钟进行 q 次有效传输时可这样换算；协议空拍、地址和等待还会降低实际带宽。

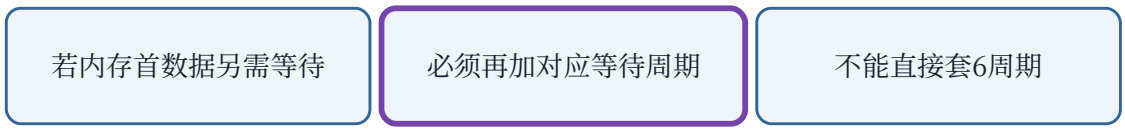
周期与复用

总线周期是完成一次规定总线操作所需时间，可能包含多个时钟周期；总线时钟周期是同步节拍，两者不一定相同。数据宽度指同时传输的数据位数，信号总线数还包含地址与控制线。地址/数据复用在不同时段共用同一组引脚，减少引脚数量，但需分时传送和锁存。

事务的四段

申请分配阶段提出请求并仲裁；寻址阶段由主模块给出从模块地址和读写等命令；传输阶段完成源到目的的数据交换；结束阶段撤除信号、释放总线。读事务、写事务、突发事务可能有不同细节，计算时不能把整个事务周期都当成在发送有效数据。

图解与逐步推演：突发总线传送：地址阶段不能每个字重复算



单独查看图片

突发总线传送：地址阶段不能每个字重复算

查看完整步骤（也可离线阅读）

1. 题设模型：假设一次突发仅发一次地址，后续每周期传8B，忽略额外等待和结束开销。
2. 数据拍数：位宽、字节与总线周期必须统一；这里数据拍已经按8B计量。
3. 总线周期10ns：32B/60ns≈5.33×10^8B/s，按十进制MB/s写约533；每拍峰值带宽为800MB/s，含地址开销的平均值更低。
4. 题设增加等待时：主存访问延迟与总线传输时间是否重叠，应从题给时序确认。

同步、异步、半同步与分离事务 128 存储器总线不必然拥有统一公共时钟。

定时方式

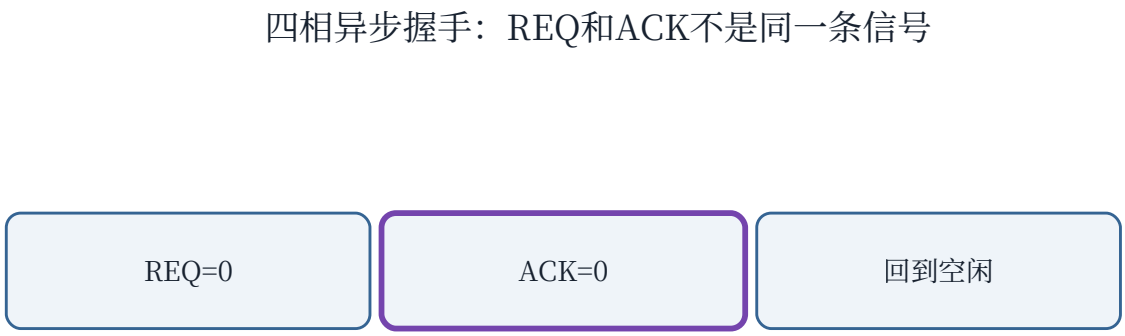
方式	协调机制	适用与代价
同步	统一时钟规定传送时刻	控制较简，适于距离短、速度相近；慢设备可能需等待
异步	请求—回答握手，无统一公共时钟作为完成标准	适应速度差异，控制和握手开销较大
半同步	统一时钟基础上结合等待信号	按设备响应插入等待周期
分离事务	请求和返回数据阶段分离，中间释放总线	利用从模块准备时间服务其他请求

异步互锁的三类

不互锁方式下，请求和回答撤销不都等待对方确认；半互锁中，请求端等待回答后撤请求，但回答端可按约定延时撤销回答；全互锁中，发请求、发回答、撤请求、撤回答依次由对方状态确认。全互锁完成关系明确，但仍须考虑信号传播及控制实现。

异步不是完全没有任何局部时钟，而是不依赖双方统一公共时钟规定数据完成时刻。半同步的等待一般按时钟边界延长事务，不应随意解释为任意改变系统时钟频率。

图解与逐步推演：四相异步握手：REQ和ACK不是同一条信号



单独查看图片

四相异步握手：REQ和ACK不是同一条信号

查看完整步骤（也可离线阅读）

1. 空闲：主从双方无未完成握手。
2. 主方提出请求：主方准备好本次信息后置REQ=1，并保持规定信号稳定。
3. 从方完成响应：从方准备完成后置ACK=1；主方看到ACK再接受结果或确认完成。
4. 主方撤销请求：主方处理确认后撤销REQ。
5. 从方撤销确认：从方看到REQ撤销后撤销ACK，四相互锁握手完成；不是所有异步总线都采用此种握手。

I/O 接口、端口与编址 129 接口是一组控制电路，端口是可访问的寄存器位置。

接口承担的功能

I/O 接口在主机与外设之间完成设备选择、时序联络、数据缓冲、信号格式转换和命令状态传送。常含数据缓冲寄存器、状态寄存器、控制寄存器、地址译码及控制逻辑；接口连接系统总线的一侧与连接设备的一侧信号并不相同。

端口与两种编址

数据端口用于数据读写，状态端口报告设备状态，控制端口接受命令；具体读写副作用和是否可回读由设备规范决定。统一编址（内存映射 I/O）把端口放入存储地址空间，用普通访存指令访问；独立编址提供独立 I/O 地址空间，使用专门 I/O 指令。

方式	优点	代价
统一编址	指令使用较灵活，不需另设一套 I/O 指令	占用部分物理地址空间，需设备映射约定
独立编址	I/O 与内存地址分离，专用指令清晰	需识别 I/O 与存储访问，指令灵活性可能较少

其他分类

按传送方式可分串行接口与并行接口；按 CPU 控制方式可分程序查询、中断和 DMA 接口；按功能选择是否可配置又分可编程与不可编程。并行/串行描述数据传送，查询/中断/DMA 描述组织控制，两者不能互相替代分类。

程序查询与中断 I/O 的时间计算 130 用缓冲区容量先求服务频率，再求 CPU 占用。

查询与中断的组织

程序查询反复读设备状态，准备好后由 CPU 搬数据，简单但忙等会占用执行资源；中断方式让 CPU 在设备准备期间继续执行其他指令，设备就绪再请求服务，数据搬运仍由 CPU 执行。阻塞式系统调用与硬件忙轮询不是同一概念，不能断言 scanf 必然以 while 循环持续占 CPU。

统一计算框架

$f_{service} \geq$

设备连续输入速率为 R 字节/秒、每次可服务缓冲量为 B 字节，需及时取走；具体安全裕量看题设。

$U_{CPU} = f_{service} \times T_{service}$

服务时间取查询或中断的实际 CPU 开销；若已用指令数表示，先除主频/CPI 换成秒。

中断额外开销

中断服务的时间通常应包含硬件响应与服务程序执行，以及题设要求的现场保存恢复；不能只计读取一个字的指令。缓冲区填满的间隔 B/R 要与可及时开始并完成取数的时点比较，防止新数据覆盖旧数据。双缓冲、批量中断等组织会改变每次可用容量和服务次数，不能总按一字一中断。

中断响应、向量入口与嵌套屏蔽 131 硬件响应和软件服务两个阶段共同保护现场。

响应与服务步骤

满足请求、允许与优先级条件后，CPU 通过硬件响应动作保存必要断点和状态、确定入口并改变中断状态；所谓中断隐指令是一系列硬件操作，不是程序中可见的一条普通指令。随后服务程序保存需使用的寄存器和屏蔽字，处理设备，恢复现场并执行中断返回。

服务入口可由软件按状态查询确定，也可由中断类型号索引向量表得到。向量表项的大小、是否包含额外状态和入口格式依体系规定，不能把中断号直接当字节地址。

响应优先级、处理优先级与嵌套

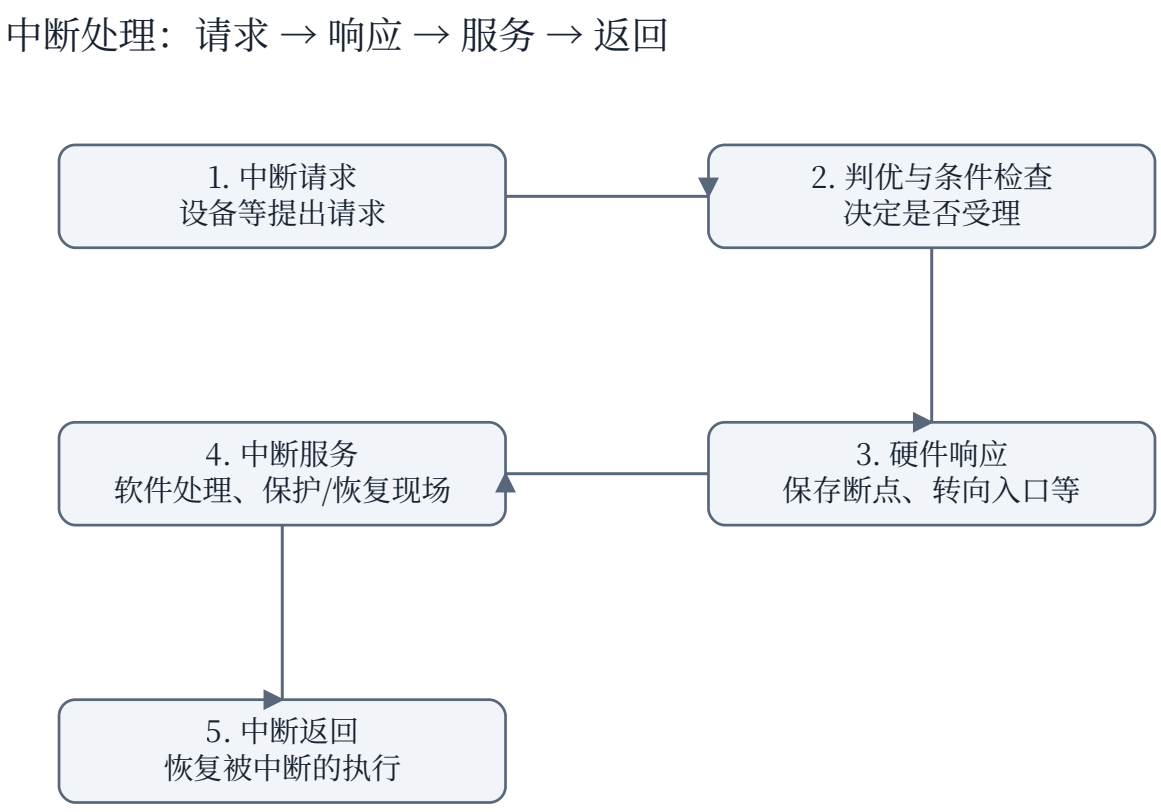
响应优先级决定多个请求同时到达时先受理哪个；处理优先级决定服务过程中哪些请求能够打断当前服务，可由屏蔽策略调整。支持嵌套需在安全保存现场后开放相应更高优先级请求；退出前也需保证恢复动作不被破坏。

屏蔽字每一位对应一个中断源，0/1 表示屏蔽还是允许必须按题设。若按“1 表示屏蔽”且顺序为严格全序，自身和所有较低优先级应屏蔽，较高者允许；不能脱离具体矩阵只数 1 的数量推断所有优先级。

优先级不是普遍固定清单

不可屏蔽请求、异常、DMA 总线请求和设备中断属不同机制，其响应关系由处理器与系统设计确定。DMA 传输请求通常不是“DMA 中断”，不要把 DMA 争总线与块完成后的中断通知混为同一事件。

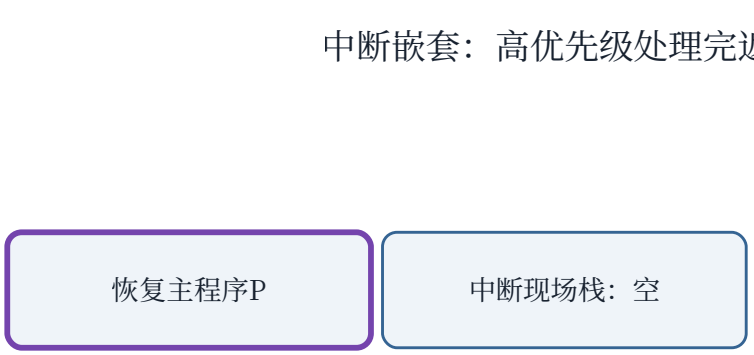
机制图解



请求到达 ≠ 已响应；响应 ≠ 完整服务程序；硬件/软件分工以体系结构为准。

单独查看图片

图解与逐步推演：中断嵌套：高优先级处理完返回哪里



单独查看图片

中断嵌套：高优先级处理完返回哪里

查看完整步骤（也可离线阅读）

1. 执行主程序：假设允许嵌套，H的处理优先级高于L，均未被屏蔽。
2. 进入低优先级L：响应L时保存P的必要现场；L处理程序按体系结构要求保存额外寄存器。
3. L中允许更高优先级中断：仅在允许嵌套并满足屏蔽/优先级条件时，H才可打断L；不是所有处理中断时都开放全部中断。
4. H先返回L：H完成后弹出L的现场，先继续L，不直接跳回P。
5. L再返回P：L完成再恢复P的断点与状态；优先级与进程调度是不同概念。

DMA：控制器、传送方式和完整过程 132 CPU 参与前后处理，数据搬运阶段由 DMA 硬件直接控制。

直接通道与控制器

DMA 在外设接口与主存之间组织直接数据传送，数据无须逐字经 CPU 通用寄存器。控制器通常保存主存地址、传送计数、方向/控制状态和暂存数据，并具有总线请求与握手逻辑；每传一个单位后相应更新地址和计数。主存中需准备合适缓冲区。

三种总线占用

方式	动作	影响
停止 CPU 访存 / 成组传送	取得总线后连续传一块，再归还	吞吐较高，CPU 访存长时间受阻

方式	动作	影响
周期挪用	请求时在安全边界借用一个或数个存储周期	CPU 可穿插执行，发生争用时让出相应周期
DMA 与 CPU 交替访存	预先安排交错时隙	需 CPU 周期与存储器时序允许，不能凭空同时访问单端口主存

预处理—搬运—后处理

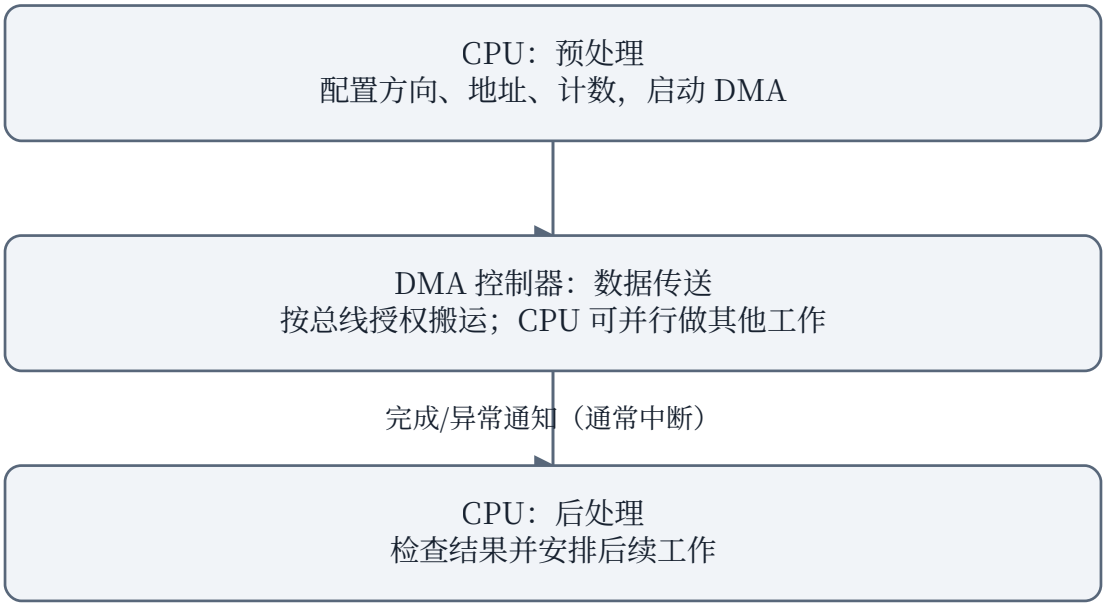
- 预处理由 CPU 设置缓冲区地址、数量、方向与模式，启动设备和控制器，然后可继续执行原程序。
- 设备就绪后向 DMA 控制器请求，控制器再申请总线；获得使用权后直接在接口与主存间搬运，更新地址与计数。CPU 当前不可中断的存储事务须先完成，不能随时破坏一个正在进行的存储周期。
- 整块完成或出现错误后，DMA 通常通过中断通知 CPU，由软件检查结果、处理状态及后续工作。
- DMA 管理上以一个数据块为任务，实际总线传输可逐字节、逐字或突发进行；设备侧传送粒度也依接口实现，并非所有 DMA 都一律按一个字。

并行的限度

CPU 与设备可以重叠工作，但若共享单总线或单端口主存，真正的存储访问仍可能互斥。周期挪用不是不耗 CPU 时间，而是减少 CPU 直接搬运的指令开销，同时占用部分可用总线周期。

机制图解

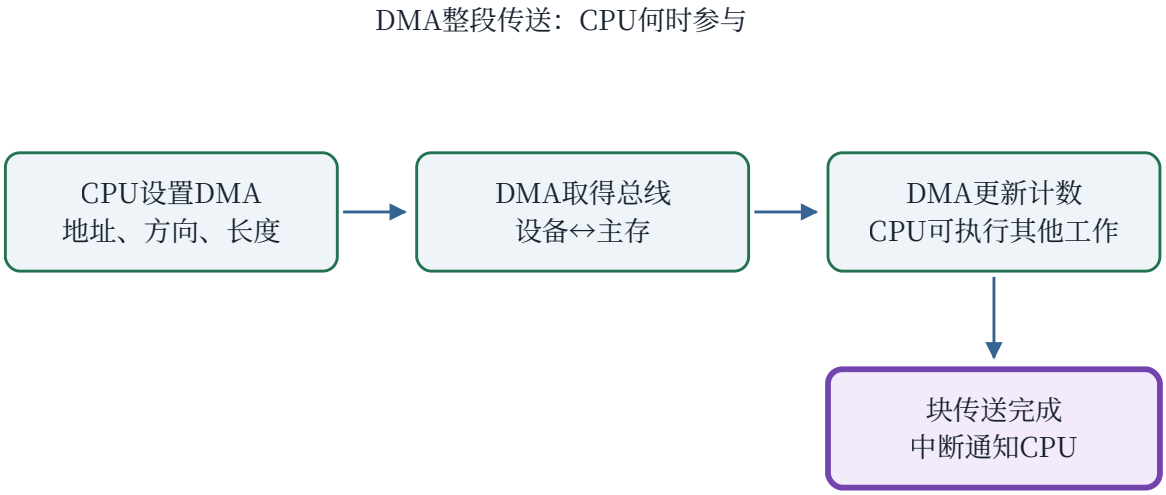
DMA：CPU 负责前后，控制器负责搬运



周期挪用、成组传送、透明 DMA 是传送方式，不是这三个先后阶段。

单独查看图片

图解与逐步推演：DMA整段传送：CPU何时参与



单独查看图片

DMA整段传送：CPU何时参与

查看完整步骤（也可离线阅读）

1. CPU设置DMA · 地址、方向、长度：CPU或驱动初始化DMA控制器。
2. DMA取得总线 · 设备↔主存：控制器按请求方式取得总线/内存访问权，数据不必逐字经CPU寄存器中转。
3. DMA更新计数 · CPU可执行其他工作：每次传送更新地址与计数；CPU与DMA可能竞争内存，不能理解成永不互相影响。
4. 块传送完成 · 中断通知CPU：控制器报告完成或错误；CPU在中断处理程序中收尾；DMA仍需要CPU初始化和结束处理。

操作系统基础与运行机制

操作系统 · 8 条

操作系统的地位、目标与资源管理 133 从硬件之上的系统软件，理解操作系统向下管理、向上服务的两面。

地位与目标

操作系统是管理和控制计算机硬件与软件资源、组织计算机工作并为用户提供服务的系统软件。它是硬件之上的基础软件，使裸机成为便于使用的扩充机器。

方便性强调用户使用；有效性强调资源利用率与吞吐量；可扩充性强调适应功能和硬件发展；开放性强调与其他系统协同。系统软件还包括语言处理系统等，不能把全部系统软件等同于操作系统。

四类资源管理

管理对象	主要工作	理解线索
处理机	进程控制、同步、通信、调度、死锁处理	决定谁运行、如何合作与竞争
存储器	分配与回收、地址映射、保护与共享、逻辑扩充	让程序装得进、访问对、互不破坏
文件	存储空间、目录、读写和访问保护	按名存取，定位数据并限制非法访问
设备	设备分配、驱动、缓冲、I/O处理、虚拟设备	协调CPU与外设，屏蔽硬件细节

资源管理与服务接口

管理资源是内部职责，提供接口是让用户和程序使用这些职责；同一个文件读请求会同时涉及文件管理、存储管理和设备管理，四类功能并非互相隔绝。

并发、共享、虚拟与异步 134 先建立四个特征的关系，再区分容易混用的词。

并发与共享

并发指多个事件在同一时间间隔内推进，单核上通常表现为交替执行；并行指同一时刻实际同时执行，需相应硬件支持。多核既可并行，也仍存在进程之间的并发。

共享是多个并发进程共同使用系统资源。互斥共享要求某时段内只有一个进程访问临界资源；同时访问允许多个进程在宏观时间段内共同使用，例如多个文件访问请求，底层仍可能交替服务。并发和共享相互依存：并发产生共享需求，有序共享又支持并发。

虚拟与异步

虚拟把一个物理实体表现为若干逻辑对应物。时分复用利用不同时段，例如把一个处理器虚拟成多个逻辑处理器；空分复用利用不同空间，例如虚拟存储等技术对存储空间的组织。

异步指进程受资源竞争和调度影响，以不可预知的速度走走停停。异步不等于随机得不到正确结果；正确的同步互斥机制应保证结果满足程序要求。

几个判断边界

- 单核可以并发，不能在同一时刻执行多条不同进程的指令；多核不保证所有进程始终并行。
- 临界资源一次只允许一个执行者使用；“共享资源”并不全部要求互斥。
- 虚拟内存的具体机制还包括分页、请求调页等；不能仅凭“空间扩大”就忽略时间和空间局部性。

从手工操作到批处理、分时与实时 135 按要解决的问题理解系统发展，保留各类系统的适用条件。

系统类型对照

类型	工作方式	优势与局限
手工操作	用户独占机器，人工装入和启动	人机速度矛盾突出，CPU大量等待
单道批处理	监控程序自动依次处理作业，内存一次一项	减少人工干预，但I/O时CPU仍可能空闲
多道批处理	多项作业驻留内存，共享资源并交替执行	提高利用率、吞吐量；缺乏交互，周转可能较长
分时系统	以较短时间片轮流服务多个终端用户	多路性、独立性、及时性和交互性
实时系统	在规定期限内响应并完成事件处理	及时性与可靠性优先；硬实时期限严格，软实时容忍有限违约

不要把典型模型当成一切实现

多道批处理的宏观并发与单处理器上的微观交替是一组描述；当系统有多个处理器时，可以有真正并行。分时强调交互响应，实时强调期限保证；不能将“运行很快”直接等同于实时。

网络操作系统强调联网计算机之间的通信和资源共享；分布式操作系统进一步追求把分散资源组织成统一、透明的整体。个人计算机操作系统是按使用场景分类，与是否支持多道、分时等特性并不互斥。

用户接口、系统调用与库函数 136 把用户如何提出请求和内核如何完成请求连成一条路径。

两类接口

命令接口供用户直接操作：联机命令接口适合交互，一次命令得到反馈；脱机命令接口把一组命令预先写好，由系统依次执行。图形界面也是面向用户的交互方式。

程序接口由系统调用组成。应用程序通过系统调用请求进程控制、文件与设备操作、内存管理、通信等内核服务，内核统一检查和管理资源。

调用链

典型过程：应用准备参数与调用号 → 执行陷入/系统调用指令 → 硬件转入内核入口并保存必要现场 → 内核检查参数与权限、分派服务 → 完成后恢复现场并返回用户态。参数准备通常发生在用户态，真正的内核服务在内核态执行。

普通函数调用通常不改变CPU特权级；系统调用涉及受控的特权级转换。库函数是语言或运行库提供的接口：一部分在用户态即可完成，一部分封装一个或多个系统调用。不能把“库函数”和“系统调用”看作同义词。

文件读写的分层

格式化输出函数可能先在用户缓冲区中处理数据，缓冲区需要提交时才调用内核写服务。因此一次库函数调用不必恰好对应一次系统调用；内核能在入口处统一检查文件描述符、缓冲地址和访问权限。

CPU运行模式与特权级转换 137 状态切换回答“以什么权限运行”，进程切换回答“由谁运行”。

内核态与用户态

CPU通过程序状态寄存器中的模式位等硬件机制区分运行权限，具体位值由体系结构规定。内核态可以执行特权指令和非特权指令；用户态只能执行允许的非特权指令。

修改关键控制寄存器、执行某些I/O控制或开关中断等操作属于典型特权操作。访管/陷入指令必须允许用户程序执行，否则用户程序便无法请求内核服务；服务请求本身不赋予用户任意内核权限。

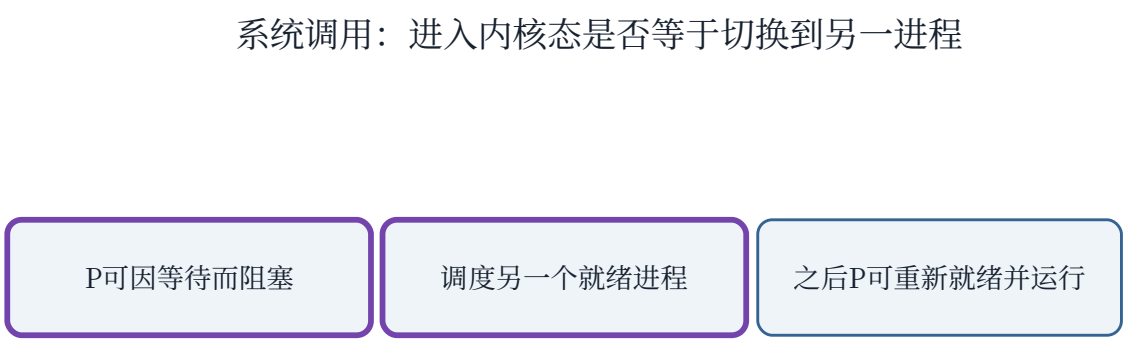
转换方向与触发条件

- 用户态 → 内核态：系统调用、异常、外部中断等受控事件触发，硬件按入口规则切换。
- 内核态 → 用户态：内核完成服务或调度后，通过受控返回恢复用户程序状态。
- 一次系统调用、中断或异常可以只完成同一进程的模式切换，也可能引发调度；是否切换进程取决于阻塞、时间片、优先级等条件。

时钟管理

时钟中断可更新系统时间、记录进程已用CPU时间和剩余时间片，并为超时、定时唤醒及调度提供机会。一次时钟中断不等于必然发生一次进程切换；当前进程仍可被继续选中。

图解与逐步推演：系统调用：进入内核态是否等于切换到另一进程



单独查看图片

系统调用：进入内核态是否等于切换到另一进程

查看完整步骤（也可离线阅读）

1. P在用户态执行：同一进程的执行可在用户态与内核态之间转换。
2. 发出系统调用：硬件和内核按体系结构约定切换权限与入口；一般不能让用户程序自行跳到任意内核位置获得特权。
3. 请求立即完成：这里没有切换进程，只发生执行模式转换。
4. 请求需等待I/O时：阻塞可能引发进程切换；是否切换要看服务行为和调度条件，不能仅由“进入内核态”判断。

中断、异常与处理流程 138 按事件来源区分，再分清硬件响应和软件处理。

分类与例子

外部中断通常来自CPU外部设备，与当前指令不直接对应，例如I/O完成和时钟中断；异常在执行当前指令时产生，例如缺页、越界、非法指令、除法错误以及受控陷入。

陷阱是程序有意触发的受控事件，如系统调用；故障可能在修复后重新执行原指令，如缺页；终止类异常表示难以恢复的严重错误。具体异常归类、返回地址和处理方式依体系结构而定，不应把“除零一定属于终止类”当通用结论。

典型处理顺序

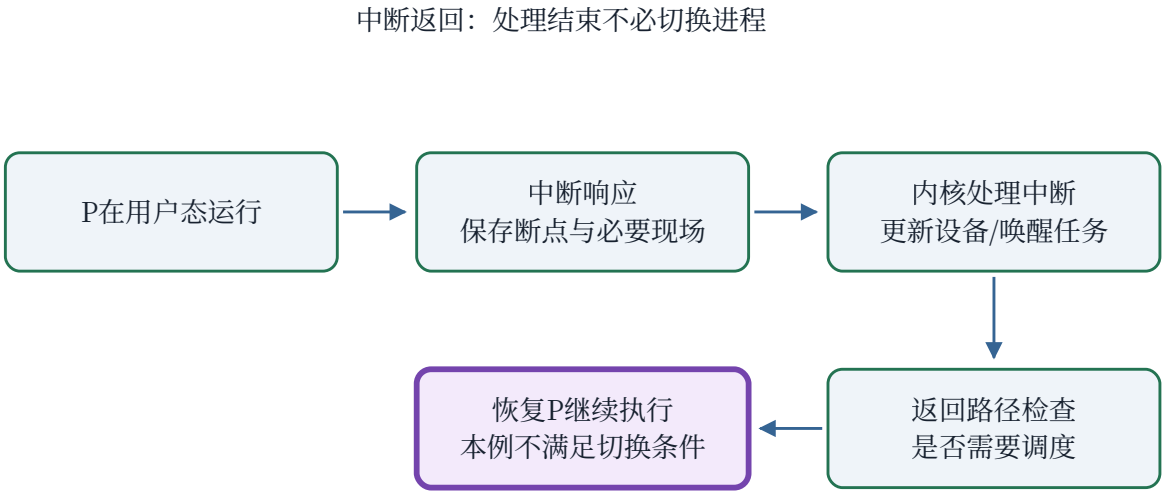
- 硬件检测并仲裁中断/异常，保存必要的断点和处理器状态，转入规定内核入口；断点是PC还是PC加状态应按题设术语。
- 软件保存尚需保护的通用寄存器、处理中断请求；若系统支持中断嵌套，需在现场可恢复且屏蔽规则允许后再开放相应中断。
- 完成服务后，在必要的受保护区间恢复现场与原屏蔽状态，通过中断返回继续原执行流，或在满足调度条件时转到其他进程。

中断嵌套与调度

单重中断处理期间不响应其他可屏蔽中断；多重中断允许更高优先级中断嵌套。中断响应优先级决定同时请求先响应谁，中断处理优先级由屏蔽等规则决定能否打断当前服务。

中断处理程序通常称中断服务例程。进入服务例程不等于创建新进程；中断栈/内核栈上的控制流与被调度进程的状态需要分别分析。

图解与逐步推演：中断返回：处理结束不必切换进程



单独查看图片

中断返回：处理结束不必切换进程

查看完整步骤（也可离线阅读）

1. P在用户态运行：本例P正在执行普通指令。
2. 中断响应 · 保存断点与必要现场：硬件按体系结构约定进入内核入口，保存返回所需状态；软件可能继续保存通用寄存器。
3. 内核处理中断 · 更新设备/唤醒任务：处理中断与进程调度是不同动作，唤醒任务也不等于必然立即抢占。
4. 返回路径检查 · 是否需要调度：若调度策略和可抢占条件要求切换，可在适当返回点运行调度器。
5. 恢复P继续执行 · 本例不满足切换条件：本例继续P，说明中断不必导致进程切换；发生特权级转换也不等于发生进程切换。

内核结构、层次结构与启动 139 把功能如何组织和系统如何开始运行放在基础机制之后。

内核与体系结构

内核保留与硬件紧密相关、执行频繁和需特权保护的核心功能，如中断、时钟和原语；进程、内存、文件和设备管理在不同体系结构中可安排在不同层次。

大内核将较多管理功能放在内核态，内部调用高效但耦合和故障影响范围较大；微内核只保留基本机制，将较多服务作为用户态服务器，通过消息通信合作，便于扩展和隔离，但通信与模式转换有开销。

分层、模块化与启动

分层结构使上层使用下层服务、接口清晰；层次过多会增加穿越成本，合理划分并不容易。模块化通过明确接口组织功能，允许相对独立扩展，但模块边界和依赖仍需设计。

开机时固件完成基本检查并寻找引导设备，装入引导程序，再把内核装入内存；内核建立基本数据结构、初始化设备与内存管理，启动后续系统进程，最后提供用户环境。引导、内核初始化和普通用户进程执行属于不同阶段。

模块独立性、机制与策略 140 结构设计既要清楚职责，也要控制模块通信与特权转换成本。

内聚与耦合

内聚性衡量模块内部各部分联系的紧密程度，耦合度衡量模块之间相互依赖的程度；高内聚、低耦合有利于模块独立。划分过小会使接口和跨模块调用过多，过大会增加内部复杂度，因此不是模块越多越好。

可加载模块允许按需要向内核增加驱动或文件系统等功能，不必每次重新编译整个内核；模块仍运行在内核权限时，其错误仍可能影响系统，模块化不等于天然的用户态故障隔离。

机制与策略分离

机制说明怎样完成基本操作，如切换线程、传递消息或建立地址映射；策略决定在什么条件下对谁执行这些操作，如选择哪个线程、给多少页框。微内核可把基础机制留在内核，把更多策略交给服务器，从而便于更换策略。

微内核的低级存储管理提供基本映射和保护，较高层虚拟存储策略可在外部服务器；中断入口完成前期处理并转换为消息交给相关服务。收益包括扩展、移植、维护和分布式通信组织，代价包括频繁消息和模式转换。

可靠性不是绝对免疫

用户态服务器故障通常比内核故障更易隔离，但其他服务若依赖它仍可能受影响；不能把“不会影响任何其他服务”作为无条件保证。分层也不应与模块化对立：一个系统可以同时采用分层、模块接口和可加载组件。

进程、线程与进程间通信

操作系统 · 9 条

进程的概念、组成与特征 141 以动态执行过程区别静态程序，以PCB识别进程。

程序、进程与进程实体

程序是描述计算步骤的静态指令和数据集合；进程是程序的一次执行，是资源分配和保护的基本单位（引入线程后调度单位可转为线程）。同一程序可以形成多个进程，进程运行时也可更换所执行的程序映像。

进程实体由程序段、数据段和进程控制块PCB组成。PCB保存标识、处理器现场、状态、调度与资源信息，是系统感知和管理进程存在的依据。进程创建时建立PCB，撤销时回收。

进程的特征

- 动态性：进程有创建、运行、阻塞和终止的生命周期；这是与静态程序最直接的区别。
- 并发性：多个进程可在同一时间段推进。
- 独立性：通常独立获得资源、独立调度，拥有受保护的地址空间；不是完全不与别人通信。

· 异步性：推进速度受竞争与调度影响，合作时要建立同步关系。
· 结构性：系统用PCB、程序和数据等组成的实体表示和管理执行活动。

PCB中的标识、现场、调度与资源信息 142 按使用场景理解字段，而不只背一个名词清单。

字段分类

字段类别	典型内容	何时使用
标识与关系	PID、用户标识、父子关系及外部名称	定位进程、授权及管理进程家族
处理器状态	PC、PSW、通用寄存器、栈指针等	暂停与恢复运行现场
调度信息	当前状态、优先级、已用CPU时间、等待事件	决定是否可运行、由谁先运行
资源与控制	地址空间指针、已分配/所需资源、打开文件信息、通信队列与链接指针	访问资源、同步通信及队列组织

各指针指向什么

程序计数器指出将继续执行的位置，程序状态字记录条件码、模式和中断等状态；用户栈/内核栈指针用于恢复相应调用现场，不是栈中全部数据。PCB保存页表等结构的指针，并不表示把程序代码、全部页表和文件数据都直接存入PCB。

运行现场暂停后要保存到内核管理的上下文结构，恢复时再装回寄存器；具体字段可在PCB、TCB或关联内核栈中分布，教材以PCB概括是抽象模型。

进程状态与状态转换 143 用“是否有CPU、是否缺少CPU以外条件”判断状态。

基本状态

运行态表示正在CPU上执行；就绪态表示除CPU外所需条件已满足；阻塞态表示等待某事件或资源，即使有空闲CPU也不能继续。创建态完成必要建立工作，终止态做退出清理。

转换条件

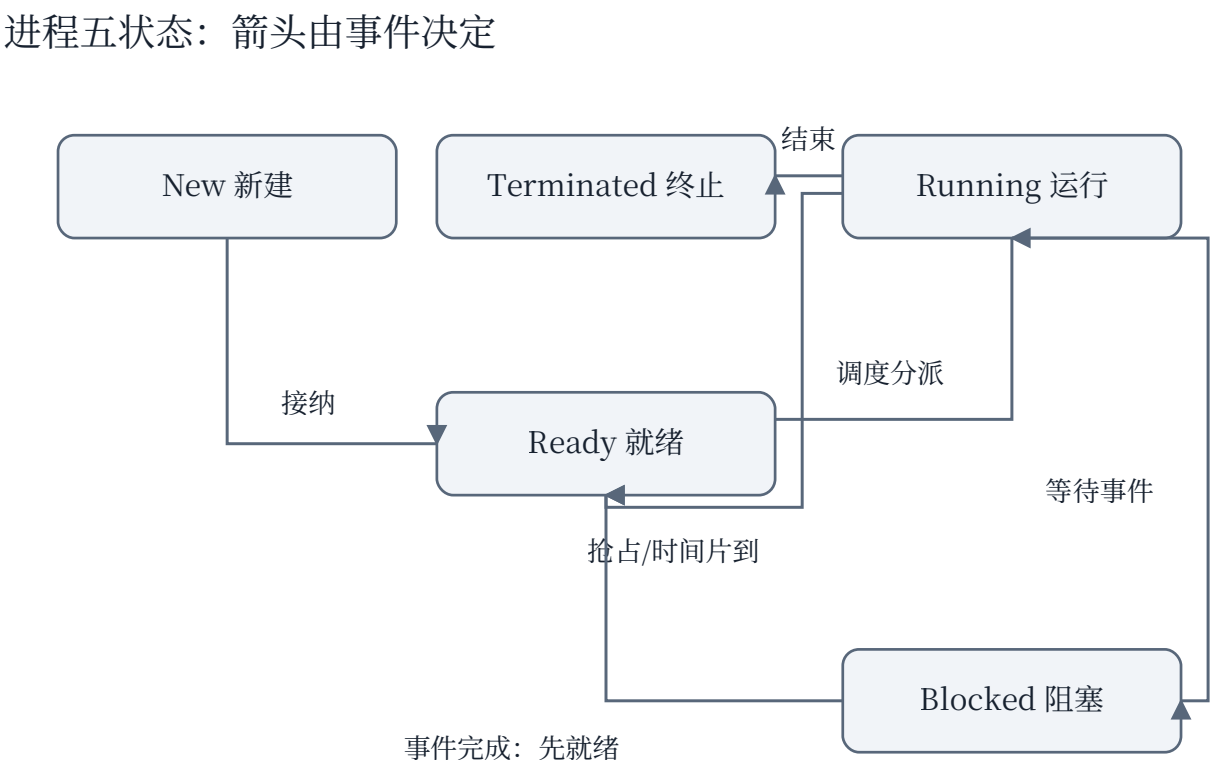
转换	触发原因	关键区别
就绪 → 运行	调度程序分配CPU	不是I/O完成直接进入运行
运行 → 就绪	时间片用完、被更高优先级进程抢占	执行条件仍满足，仅失去CPU
运行 → 阻塞	等待I/O、信号量、消息等	主动等待某事件，通常由执行相应内核操作引起
阻塞 → 就绪	等待事件到达或资源可用	唤醒后仍要参加调度

挂起与阻塞

挂起关注进程是否暂时被调出内存或被停止参与正常调度；阻塞关注是否等待事件。引入挂起后可区分就绪挂起、阻塞挂起。阻塞事件完成后，阻塞挂起进程通常先变为就绪挂起，是否调入还要由中级调度决定。

PCB可按状态链接成队列，或由索引表组织。状态是进程属性，队列是系统组织方式，不能把某种数据结构当作状态定义。

机制图解

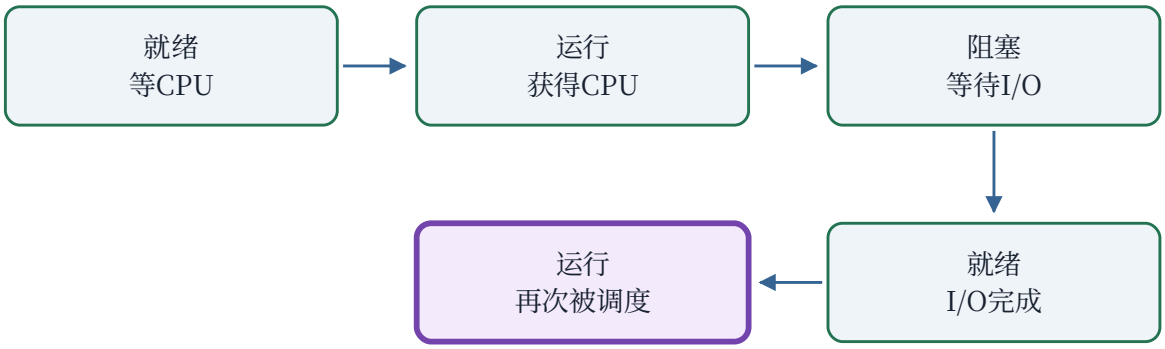


基本五状态模型：就绪只缺 CPU；阻塞还缺事件，不存在“唤醒即运行”。

单独查看图片

图解与逐步推演：运行、就绪、阻塞：转换由什么事件触发

运行、就绪、阻塞：转换由什么事件触发



单独查看图片

运行、就绪、阻塞：转换由什么事件触发

查看完整步骤（也可离线阅读）

1. 就绪 · 等CPU：进程具备执行条件，只缺处理机。
2. 运行 · 获得CPU：调度程序分派CPU，恢复上下文。
3. 阻塞 · 等待I/O：发起需等待的I/O，主动阻塞，让出CPU。
4. 就绪 · I/O完成：I/O完成使进程具备运行条件，通常先进入就绪队列。
5. 运行 · 再次被调度：被再次选中才运行；运行进程时间片耗尽通常转就绪，阻塞进程不能直接因时间片耗尽而运行。

进程控制与原语 144 创建、撤销、阻塞和唤醒必须保持状态与队列一致。

创建与终止

- 创建：申请空白PCB与唯一标识 → 分配地址空间及必要资源 → 初始化寄存器/调度信息 → 插入就绪队列。创建可能由用户请求、作业调度或其他进程派生触发。
- 终止：找到PCB并停止其执行 → 按规则处理子进程与未完成请求 → 释放或交还资源 → 从各队列中撤出并回收PCB。正常退出、错误或外部干预均可能触发。

阻塞与唤醒

阻塞时保存运行现场，修改PCB状态并挂入对应等待队列，然后调度其他就绪进程。唤醒时把等待条件已经满足的进程从等待队列摘下，改为就绪并入就绪队列。二者按同一事件配对，不能出现状态已改但队列未改的半完成状态。

原语要求执行期间不被与之冲突的控制流破坏，体现原子性。单处理器上可借助关中断保护短小内核临界区；多处理器上仅关闭本CPU中断不能阻止另一CPU访问共享结构，仍需原子指令或锁。

切换与模式转换

进程切换需保存旧进程现场并装入新进程现场，可能切换地址空间等；用户态和内核态转换可以发生在同一进程内部。系统调用进入内核后若没有阻塞或抢占，返回的仍可为原进程。

进程映像、fork与exec 145 区分地址空间复制、资源共享和程序替换。

进程地址空间

进程的程序映像通常包含代码、已初始化与未初始化数据、堆和栈，以及操作系统记录的相关信息。代码区可按实现只读共享；堆用于动态分配，栈保存调用帧、局部数据和返回信息，二者并非PCB。

父子进程是不同进程：具有不同标识与独立控制信息；哪些资源继承、共享或复制由创建语义决定。使用写时复制时可暂时共享物理页，写入时才复制，逻辑地址空间仍相互独立。

fork、exec与等待

fork创建子进程，父子从规定返回位置分别继续，返回值用于区分角色；不能只按源代码行数计执行次数，要画派生树。exec以新程序映像替换当前进程映像，不等于创建新进程。父进程可通过等待接口回收退出状态；子进程退出与父进程收取状态是不同事件。

两次派生的计数思路

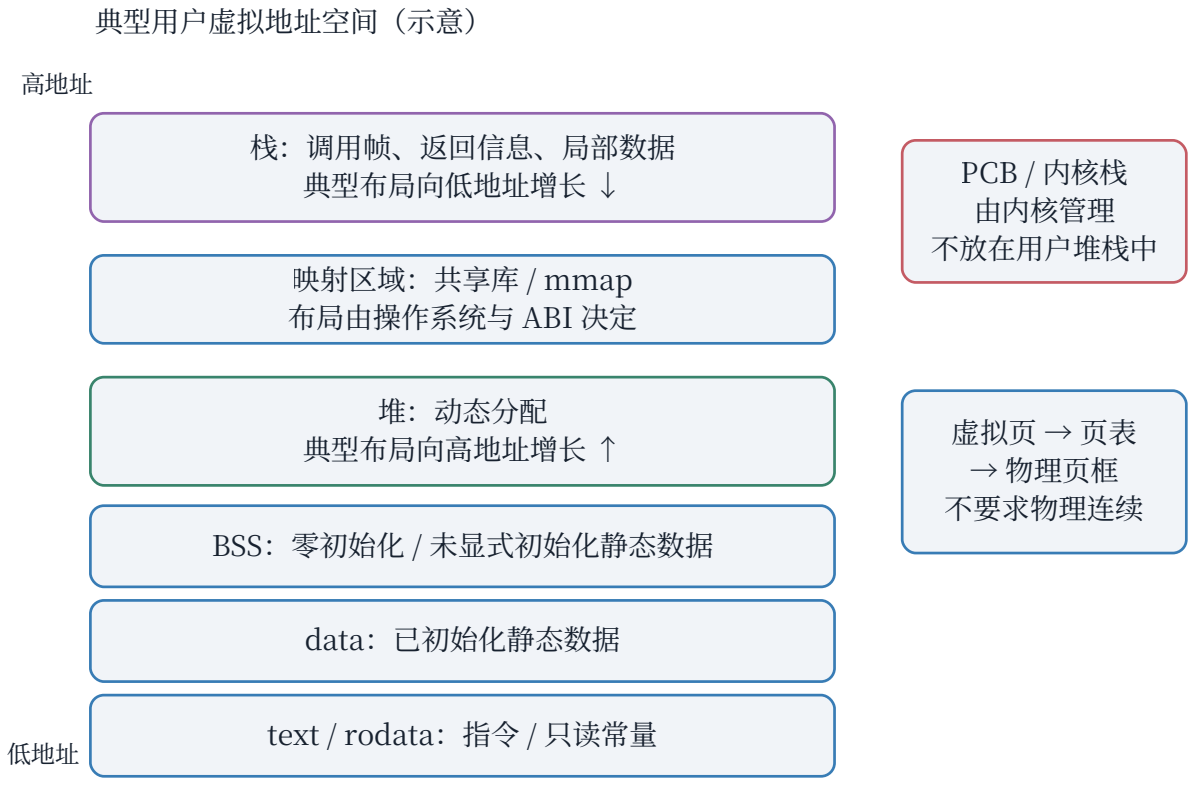
每个成功执行到fork的进程都会派生一个子进程。若两次fork都无条件执行且均成功，进程数从1变2再变4；若fork在条件、循环或短路表达式中，必须逐条标出各进程是否走到调用点。

图解：进程用户地址空间与内核控制信息

图以常见布局帮助记忆，堆、栈的增长方向是典型约定，不能据此认定所有平台地址布局一致。多个线程常共享进程代码、数据和堆，但各线程有自己的调用栈；PCB是内核中的进程控制信息，不等于用户空间某一数据段。

代码区对应指令，静态数据随进程映像生存；栈中的自动局部变量随调用帧建立和撤销，动态分配对象的生存期由分配与释放决定。虚拟地址相邻也不要求对应物理页框相邻。

进程内存映像：各区域存什么



示意不代表固定地址；ASLR、线程、平台与实现会改变实际布局。

BSS 通常不在可执行文件中存放等量零字节，装载时建立零初始化区域。

单独查看图片

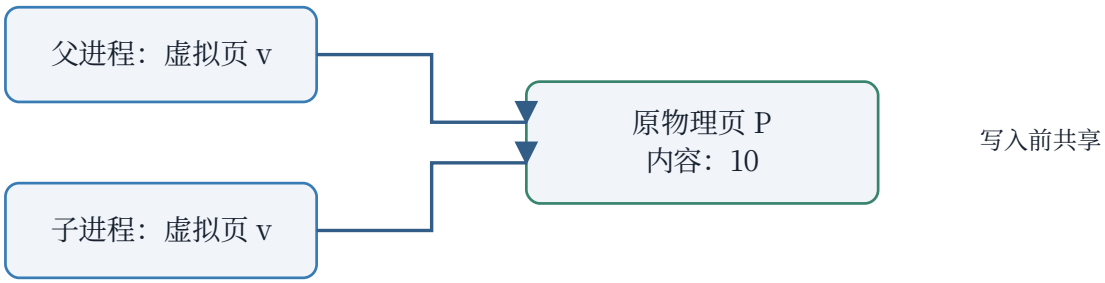
图解：fork之后什么时候真正复制页面

以采用写时复制的私有匿名页为例，fork后父子可通过不同页表映射同一个物理页；写保护异常由内核识别并处理。需要分离时分配新页并复制内容，随后恢复写入。实际系统还可能在页面已独占时直接恢复可写权限。

这与共享内存映射不同：显式共享映射的写入可以按其共享语义被其他进程看到。exec替换当前进程映像，通常保持当前PID，不能在图上再数成一次fork。

fork 写时复制：独立地址空间，共享物理页

以一个可写私有匿名页为例；页表保护位暂时禁止直接写入



子进程尝试把 10 写成 20：保护异常 → 分配新页 → 复制 → 改映射

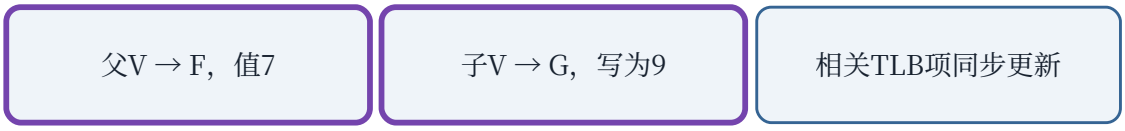


父子 PID 与页表独立；共享的是物理页，不是同一个逻辑地址空间。

单独查看图片

图解与逐步推演：写时复制：fork之后谁真正复制物理页

写时复制：fork之后谁真正复制物理页



单独查看图片

写时复制：fork之后谁真正复制物理页

查看完整步骤（也可离线阅读）

1. fork之前：为简化只跟踪一个私有可写数据页，不讨论共享映射。
2. fork采用COW：内核复制必要管理信息，数据页暂不立即复制；父子逻辑地址空间仍独立。
3. 子进程写值9：不是所有保护异常都允许复制后继续，必须确认该映射确为COW且有写权限语义。
4. 修改子页表并重试：子重新执行写操作后改G，不改变父的F。页引用计数减少后，原页可能恢复独占写权限。

共享存储、消息传递与管道 146 从传递方式、访问权限和同步责任理解进程通信。

主要方式对照

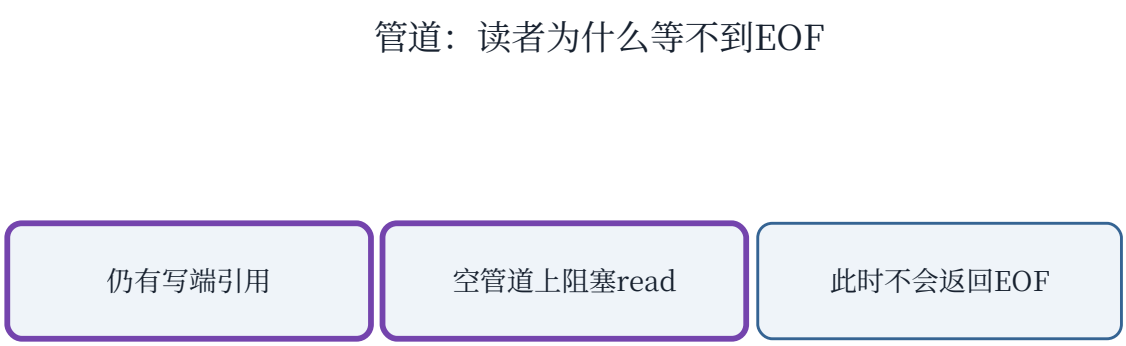
方式	信息怎样交换	同步与限制
共享存储	把同一物理存储区域映射给通信进程	数据量大、拷贝少；访问一致性需额外同步
消息传递	发送/接收带格式的消息	可直接指明目标，也可经信箱/端口间接传递；有缓冲与阻塞语义
管道	通过内核维护的缓冲区传递字节流	教材匿名管道常为半双工；读空、写满时的行为与接口规则相关

管道条件与读写行为

教材模型中匿名管道一般用于有亲缘关系的进程；双向通信通常需两条单向管道。命名管道扩展了通信对象范围，不能把匿名管道条件套给一切管道实现。

读空管道且仍有写者时，阻塞读会等待数据；全部写端关闭后，可读到文件结束。无读者仍写入会触发相应错误或信号。写满与一次读写是否原子，应按管道容量及操作系统约定分析，不能把“缓冲区有数据”理解为所有数据必须一次读完。

图解与逐步推演：管道：读者为什么等不到EOF



单独查看图片

管道：读者为什么等不到EOF

查看完整步骤（也可离线阅读）

1. 创建pipe后fork：常见Unix匿名管道场景，设父写子读。
2. 关闭不用的端点：fork继承的不用端点应关闭；特别是读进程自己也持有写端时会影响EOF。
3. 父写入ABC：一次read可能读少于请求长度；管道是字节流，不保留应用消息边界。
4. 父关闭最后写端：只有全部写端关闭且已有数据读完后，读端才见EOF。
5. 若子忘记关闭继承的w：即使父已经退出，子自己的写端引用仍阻止EOF；非阻塞模式下则按规则报告暂不可读。

信号：异步事件通知与响应 147 信号用于通知事件，信号量用于同步计数，两者不是同一机制。

产生、未决、递送与处理

信号是向进程通知某事件的机制，可以由异常、终端输入、定时器或其他进程通过系统接口产生。产生后可处于未决状态；若被屏蔽则暂不递送，解除屏蔽后再按规则处理。异步通知并不表示处理函数可以任意执行非安全操作。

收到信号后，进程按处置方式执行默认动作、忽略或调用自定义处理函数。处理函数完成后通常可恢复原执行流，但若默认动作或处理程序终止进程，就不会继续原代码。

常见信号与不可改变的处置

信号	典型来源/含义	边界
SIGINT	终端中断，如Ctrl+C	可按接口规则捕捉、忽略或默认终止
SIGILL	非法指令	不等同于一般外设中断
SIGFPE	算术异常，例如整数除零等	名称含浮点不表示仅来自浮点运算
SIGSEGV	无效内存访问	与访问保护相关
SIGALRM	定时器到期	默认动作与自定义处置要区分
SIGCHLD	子进程状态变化	通知父进程；状态回收仍需相应接口
SIGKILL / SIGSTOP	强制终止 / 停止	不能被捕捉、忽略或屏蔽
SIGABRT	abort请求异常终止	标准名称为SIGABRT

信号与其他IPC

普通信号适合简单控制与事件通知，不承担大量数据传输；实时信号等扩展可附带有限信息。共享内存、管道和消息队列承担数据交换，信号量用于同步互斥；管道和消息传递也可通过阻塞规则起到同步效果，不能绝对说“毫无同步能力”。

线程的资源、调度与实现方式 148 把共享部分和各线程私有现场分开，才能判断阻塞与并行。

线程与进程

引入线程后，线程是CPU调度和执行的基本单位，进程仍是资源分配的基本单位。同一进程的线程共享地址空间、打开文件等进程资源，但各有线程标识、寄存器现场、程序计数器和栈。共享堆并不意味着各线程不需要同步。

进程间切换通常需要切换地址空间等更多内容，同进程线程切换通常较轻；不同进程的内核线程仍可能涉及进程地址空间切换。

实现与多线程模型

方式/模型	调度可见性	优点与限制
用户级线程	内核只看到进程，由用户线程库调度	切换开销小；某线程阻塞的系统调用可能阻塞整个进程，通常不能直接在多核并行
内核级线程	内核看到并调度各线程	一线程阻塞不必阻塞同进程其他线程，可并行；管理切换开销较大
多对一	多个用户线程映射一个内核线程	用户态管理方便，受单一内核执行实体限制

方式/模型	调度可见性	优点与限制
一对一	每个用户线程对应内核线程	并发和并行能力好，内核资源消耗较多
多对多	多个用户线程映射到一组内核线程	兼顾灵活性与并行，映射和调度较复杂

线程独立性边界

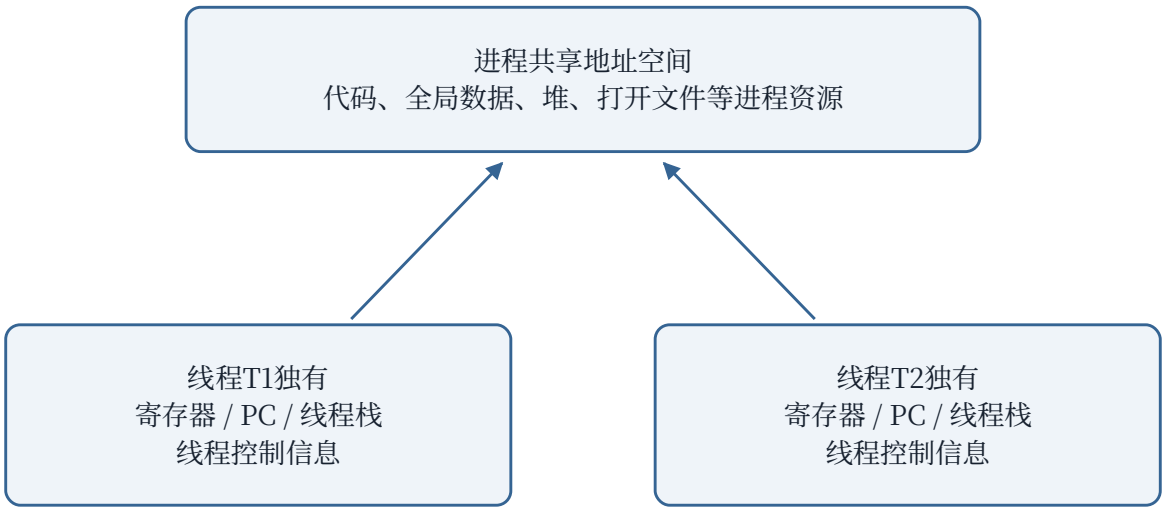
线程栈和寄存器是逻辑私有现场，但同进程线程在缺乏保护的共享地址空间中可能通过指针访问其他线程存储区；“私有”不是进程级隔离保护的同义词。线程终止也不应无条件等同整个进程终止。

结构对照图：同一进程的线程：共享什么，各自保存什么

线程共享进程资源，但每个执行流需要自己的寄存器现场和栈。某线程的栈在共享地址空间中，其他线程技术上可能访问其地址，不能把“独有使用”误解为天然硬件隔离。

用户级线程是否由内核直接调度，取决于线程映射模型；多对一模型下一个阻塞系统调用可能阻塞整个进程。

同一进程的线程：共享什么，各自保存什么



单独查看图片

线程控制块、创建、退出与栈 149 线程生命结束和其资源被其他线程回收是两步。

TCB与创建

TCB可记录线程标识、运行状态、优先级、程序计数器、状态寄存器和通用寄存器等现场，以及栈和局部存储管理信息。创建函数通常指定入口与参数，并可配置栈大小、优先级等，成功后返回线程标识。

初始线程可创建后续线程，同进程线程共享全局变量和地址空间；每个线程保留自己的调用栈，用来保存返回地址、调用关系和局部数据。切换线程至少需要保存/恢复寄存器和栈指针。

退出、join与detach

线程退出后，退出状态等资源在可连接模型中可能保留，供其他线程join收取；分离线程按规则自动回收。具体接口并非所有系统都相同，已经终止的线程不能通过一次普通函数调用“复活”，要继续工作通常需创建新线程或采用挂起/唤醒模型。

不要把线程退出与进程退出混淆；也不要把“栈属于该线程的执行现场”理解成硬件保证同进程其他线程绝不能通过指针碰到它。

处理机调度与多处理器

操作系统 · 9 条

三级调度、调度时机与调度方式 150 先判断从哪个队列选，再讨论选谁和是否抢占。

三级调度

层次	迁移方向	对象和频率
高级调度/作业调度	后备队列 → 内存就绪环境	选择接纳作业，发生相对较少
中级调度/内存调度	挂起状态 ↔ 相应驻留状态	改善内存利用和多道程序度
低级调度/进程调度	就绪 → 运行	从就绪实体中分配CPU，发生频繁

可调度与不可立即调度的情形

当前进程完成、阻塞，或发生允许抢占的时间片到期与更高优先级进程就绪等事件时，可能需要调度。处理中断的关键区间、持有相关内核锁或处于不可抢占区间时，通常需延迟到安全点再调度。

非抢占式让运行实体继续，直到完成或主动放弃；抢占式允许满足规则的更紧迫实体夺取CPU。调度只决定选谁，分派/切换还包括现场保存、恢复和控制权移交。

图解与逐步推演：三级调度：从外存作业到就绪队列再到CPU



单独查看图片

三级调度：从外存作业到就绪队列再到CPU

查看完整步骤（也可离线阅读）

1. 高级调度选入作业：高级/作业调度频率通常较低；某些系统不设独立的长期作业调度机制。
2. 中级调度处理挂起：挂起与等待I/O造成的阻塞不是同一个维度；挂起进程可处于就绪挂起或阻塞挂起状态。
3. 低级调度选CPU使用者：低级/处理机调度更频繁；一次调度决策与实际上下文切换也应区分。
4. 串起主线：三个调度回答不同问题，不能把任意I/O完成都叫作高级调度。

调度评价指标与时间计算 151 时间轴先画事件，再算等待、响应与周转。

指标定义

- CPU利用率＝CPU忙碌时间/总观察时间；吞吐量＝单位时间完成的作业数。
- 周转时间＝完成时间−提交/到达时间，包含等待、执行及I/O等全过程。
- 带权周转时间＝周转时间/实际服务时间，反映相对延误；平均值是各作业相应指标的算术平均。
- 等待时间是处于就绪队列等待CPU的累计时间；阻塞等待I/O的时间不应再算作就绪等待。
- 响应时间通常是从提交请求到首次响应或首次获得CPU的时间，具体以题目定义为准；不是完成时间。

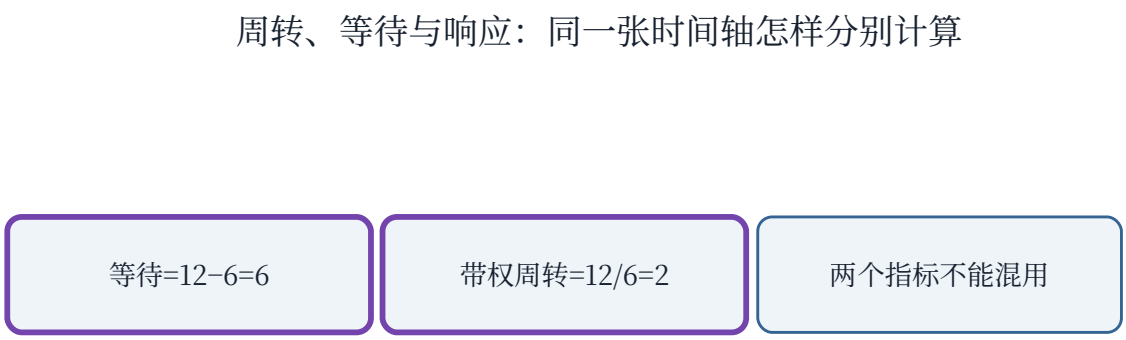
分步计算

若作业0时到达，CPU总需求4，完成于时刻10，周转时间为10、带权周转时间为2.5。若期间阻塞3且其余时间除运行外均在就绪队列，等待时间为10−4−3＝3。若首次执行始于2，首次CPU响应时间为2。

常见误差

多个CPU或CPU与设备可并行时，不能把所有资源忙碌时间相加当总历时；同一时刻结束、到达、唤醒和时间片用完的入队顺序若影响答案，需要明确采用的题设规则。

图解与逐步推演：周转、等待与响应：同一张时间轴怎样分别计算



单独查看图片

周转、等待与响应：同一张时间轴怎样分别计算

查看完整步骤（也可离线阅读）

1. 给定一次执行记录：假设只有就绪等待与CPU执行，没有I/O阻塞等其他时间。
2. 响应时间：响应时间关注从提交到第一次得到执行的等待，不等于完成整个任务所需时间。
3. 周转时间：周转包含该任务从到达到完成的全部经历。
4. 等待与带权周转：只有在本例无I/O阻塞条件下，等待时间才等于周转减CPU服务量；有阻塞时还要减去阻塞时间。

FCFS、SJF、SRTN与HRRN 152 从服务时间和等待时间两条线比较批处理调度。

算法比较

算法	选择规则	特点与代价
先来先服务FCFS	按到达就绪/后备队列的先后选择	实现简单，利于长作业；短作业可能被长作业拖延
短作业/短进程优先SJF/SPF	就绪者中服务时间短者优先	可降低典型平均等待，需估计执行时间；长作业可能饥饿

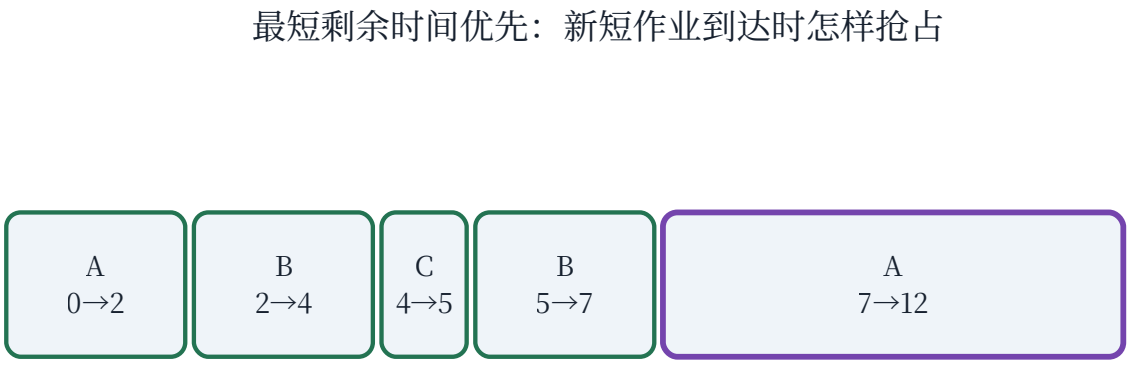
算法	选择规则	特点与代价
最短剩余时间优先SRTN	剩余CPU时间最短者优先，可抢占	新短任务到达可能打断长任务，要重新比较剩余量
高响应比优先HRRN	比较(等待+服务)/服务，较大者优先	非抢占典型；兼顾等待增长与短作业优势

执行步骤

每次出现调度点时，只在已到达且就绪的实体中比较。SJF比较本次预测服务时间，SRTN比较剩余时间；HRRN用当前时刻更新等待时间，已在运行者通常不因新作业到达立即被抢占。

高响应比中的“响应比”是调度优先值，不是“响应时间”。服务时间相同可按先来先服务打破平局；平局处理若题目另有规定则服从题设。

图解与逐步推演：最短剩余时间优先：新短作业到达时怎样抢占



单独查看图片

最短剩余时间优先：新短作业到达时怎样抢占

查看完整步骤（也可离线阅读）

1. 推进到t=2：在就绪进程中选择剩余CPU时间最短者；每次有新到达或当前进程完成都重新比较。
2. 推进到t=4：在就绪进程中选择剩余CPU时间最短者；每次有新到达或当前进程完成都重新比较。
3. 推进到t=5：在就绪进程中选择剩余CPU时间最短者；每次有新到达或当前进程完成都重新比较。
4. 推进到t=7：在就绪进程中选择剩余CPU时间最短者；每次有新到达或当前进程完成都重新比较。
5. 推进到t=12：在就绪进程中选择剩余CPU时间最短者；每次有新到达或当前进程完成都重新比较。

优先级、时间片轮转与多级反馈队列 153 交互性、紧迫性与公平性通过不同规则实现。

优先级与轮转

优先级调度可为抢占式或非抢占式；静态优先级建立后基本不变，动态优先级可依据等待、执行行为等调整。合理提高久等任务的优先级有助于防饥饿；具体优先数值大或小代表更高由题设决定。

时间片轮转RR按就绪队列循环服务，未结束且不阻塞者片末入队尾；阻塞者离开就绪队列，唤醒后按规则重新入队。片过大会近似FCFS、响应变差，过小会增大切换开销。

多级队列与多级反馈队列

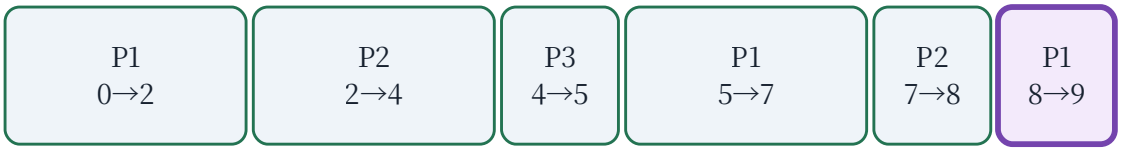
- 多级队列按任务类型设置多个队列，队列之间有优先关系或时间分配，各队列内部可用不同算法。
- 多级反馈允许进程在队列之间移动，通常新进程进入最高优先级队列；在本级时间片耗尽未完成则降到下一级。
- 高优先队列可抢占低优先队列；越低级时间片通常越长。短任务迅速完成，长任务逐步得到较长连续运行区间。
- I/O后重新入队位置、是否保持剩余时间片、是否提升优先级等属于具体算法规则；不能凭“多级反馈”四个字补出题目未给的细节。

画时间轴的顺序

记录队列级别、队首次序、剩余CPU量和剩余时间片；推进到最近的到达、完成、阻塞或片末事件；按规则更新队列后再选择下一运行者。把设备结束带来的唤醒也加入事件表，避免只盯CPU队列。

图解与逐步推演：时间片轮转：完整队列与甘特图

时间片轮转：完整队列与甘特图



单独查看图片

时间片轮转：完整队列与甘特图

查看完整步骤（也可离线阅读）

1. 全部在t=0到达：单CPU、时间片q=2、忽略切换开销，初始就绪队列P1、P2、P3。
2. P1执行 [0,2)：P1本次用2个时间单位，剩余3，排回队尾。
3. P2执行 [2,4)：P2本次用2个时间单位，剩余1，排回队尾。
4. P3执行 [4,5)：P3本次用1个时间单位，完成并退出队列。
5. P1执行 [5,7)：P1本次用2个时间单位，剩余1，排回队尾。
6. P2执行 [7,8)：P2本次用1个时间单位，完成并退出队列。
7. P1执行 [8,9)：P1本次用1个时间单位，完成并退出队列。

闲逛进程与两级线程调度 154 没有就绪工作时CPU仍有明确的受控执行路径。

idle的作用

没有普通就绪进程时可运行最低优先级的闲逛/空闲任务，它不等待普通资源来完成用户工作，负责在安全状态下等待中断、降低能耗或检查调度条件。有新工作就绪后，调度程序选择它们替代idle。

教材把闲逛进程描述为不阻塞、最低优先级，是为了保证随时有可运行实体；具体低功耗等待指令和实现结构不必都是所谓“0地址指令”。

用户与内核层调度

多对一用户线程模型中，内核先调度进程，再由进程内线程库决定哪个用户线程运行；用户线程之间切换可仅保存少量现场，不涉及系统调用。内核线程模型中，内核直接选择可运行内核线程，可能来自同一或不同进程。

同进程内核线程切换未必改变页表映像，不同进程切换才通常需要地址空间切换；缓存和TLB是否刷新取决于架构与标记机制。不能笼统认为“每次内核线程切换必使全部缓存失效”。

保证调度与公平分享 155 进程公平和用户公平是不同的分配目标。

保证调度

若n个同类进程共享处理机，理想情况下各得到约1/n的CPU份额。系统跟踪进程已获得的CPU时间，再与按存活时间及共享对象数计算的应得时间比较，优先补偿实际/应得比率较小的进程。

例如A、B、C比率分别0.5、0.8、1.2，A相对欠账最多，可先运行A直至其比率追上相邻候选。动态进程集合、权重等情况需按实际算法重新计算应得份额。

公平分享按用户而非仅按进程

如果用户甲有4个进程，乙有1个进程，给每进程相同时间会使用户分别得到80%与20%；这实现的是进程公平，不是用户公平。用户公平先确定用户份额，再在各自进程中分配。

调度序列

等长片下，甲的A/B/C/D与乙的E按“A E B E C E D E”循环，两个用户各占一半；按“A B E C D E”循环，甲占2/3、乙占1/3。此类序列仅是体现比例的一种构造，不是唯一实现。

实时任务、EDF与最低松弛度优先 156 截止时间与剩余执行量共同决定实时任务的紧迫程度。

实时调度前提

硬实时任务要求在期限内完成，软实时任务可容忍有限超期。非抢占式任务需等待当前任务释放CPU；抢占式调度可在时钟点或事件触发后更快响应，但仍受不可抢占临界区约束。

EDF选择绝对截止时间最早的已就绪任务，既可有抢占实现也可有非抢占实现。最低松弛度优先LLF比较“截止时间−当前时间−剩余执行时间”，松弛度越小越紧迫；0表示不能再等待，负值表示在当前条件下已无法按期完成。

EDF的逐段验证

A周期20ms、每次需10ms，B周期50ms、每次需25ms，截止时间等于周期末且均在0释放：0–10执行A_1；10–20执行B_1；20–30执行A_2；30–45继续B_1（40时新A_3期限60晚于B_1期限50）；45–55执行A_3；55后处理已在50释放的B_2。

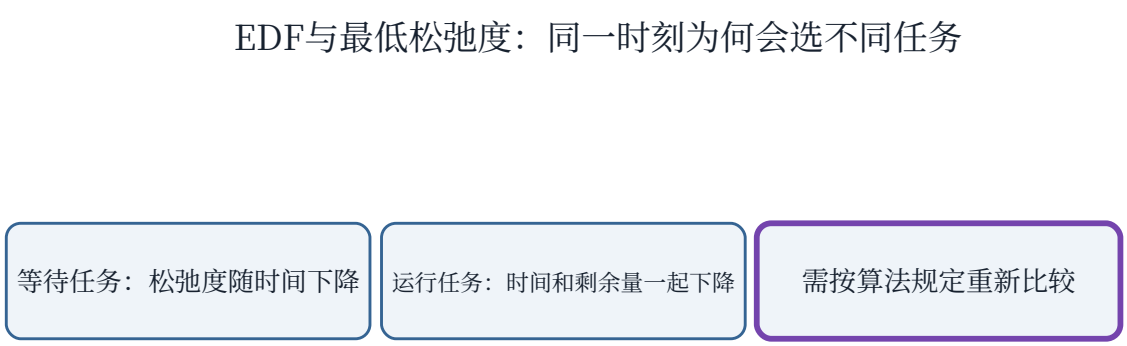
固定让A永远优先时，B_1在40时仍剩5ms，却被A_3占用40–50，导致B_1错过50期限；固定让B永远优先时，A_1会错过20期限。由此说明固定优先规则可能无法满足这一任务集，而EDF可根据期限变化选择。

LLF计算条件

只比较已经释放且就绪的任务；剩余运行量随执行减少，等待任务的松弛度随时间下降。若当前任务连续运行，其“当前时间增加”与“剩余量减少”抵消，松弛度在这一阶段保持不变。多个任务松弛度相等时应明确平局规则，连续时间LLF可能频繁切换。

例如deadline=200ms、当前时刻50ms、剩余100ms，松弛度为50ms；不能不写当前时刻就把“200−100”固定为任务全程的松弛度。

图解与逐步推演：EDF与最低松弛度：同一时刻为何会选不同任务



单独查看图片

EDF与最低松弛度：同一时刻为何会选不同任务

查看完整步骤（也可离线阅读）

1. 当前时刻t=2：截止时间均为绝对时间，忽略调度与切换开销。
2. EDF比较截止时间：最早截止时间优先只按当前待执行任务的截止时间排序，不先比较剩余时间。
3. 计算松弛度：松弛度=截止时间−当前时刻−剩余执行量；数值小者更迫切。
4. 松弛度随执行变化：在单位速率、无额外开销模型下，运行任务的松弛度可暂保持不变；LLF可能产生频繁抢占。

多处理器类型与SMP调度 157 多个CPU使分配、共享队列同步与负载平衡同时成为问题。

处理器组织

紧密耦合系统通常通过共享存储等高速互连合作；松散耦合系统的各结点具有更独立的存储器、I/O和操作系统，通过通信线路交换信息。通信代价取决于具体系统，不把某一组纳秒、微秒数当永久标准。

对称多处理SMP中处理器地位通常相等，各自可执行调度和系统代码；非对称多处理由主处理器承担较集中控制，其他处理器完成分派工作。非对称并不必然意味着CPU指令集或硬件型号不同，关键是控制职责的非对称。

就绪队列、分配与负载平衡

方案	工作方式	权衡
公共就绪队列	空闲CPU从同一队列取任务	负载自然较均衡，但共享队列访问需同步，可能形成争用
私有就绪队列	各CPU维护本地候选	本地操作高效、亲和性较好，但可能忙闲不均
静态/绑定式分配	任务较固定地留在指定CPU	迁移少，需考虑负载不均
动态分配/迁移	按运行需求在处理器间重新分配	改善均衡，增加迁移和缓存工作集重建代价

推迁移、拉迁移和亲和性

推迁移由负载检测机制把任务从繁忙处理器移出；拉迁移由空闲处理器主动获取任务。软亲和性表示尽量继续在原处理器运行，硬亲和性通过限制集合约束可运行CPU。

迁移后的新CPU可能缺少已热的缓存和TLB工作集，需重新建立局部性；不应绝对理解为旧CPU全部缓存必然被硬件标无效。负载均衡和亲和性需要权衡，共享调度队列也仍有NUMA、锁和缓存局部性等问题。

多处理器上的自调度、成组与动态分配 158 合作线程能否同时运行，影响阻塞、切换和处理器利用率。

三类调度

方式	基本做法	主要权衡
自调度	空闲处理器从公共就绪队列选线程/进程	沿用FCFS、优先级等规则，利用率高；队列竞争和迁移开销可能突出
成组调度	把一组合作线程同时分配到一组CPU	减少相互等待与上下文切换；可能为凑齐执行组而空闲部分CPU
动态处理器分配	按作业并行度和空闲情况增减分配CPU	适应线程数变化，提高利用率，但决策与重分配更复杂

按应用与按线程分配的差异

4台处理器上，应用A有4线程、B有1线程。若让两个应用各占整组处理器一半时段，B运行时只有1台忙、3台空闲；若按总5线程分配时间，A可占4/5、B占1/5，可减少这一模型下的闲置。是否允许B的空闲CPU再分配，取决于调度模型。

动态调度的事件

有空闲处理器时分配给请求者；新作业若需获得初始处理器，可从已占多个CPU的作业中回收，体现新作业优先的特定策略；无法满足的请求进入等待。CPU释放时扫描请求队列，按规定优先级和先来先服务分配。这里是补充资料描述的一种动态策略，不应推成所有系统都必须“新作业绝对优先”。

同步互斥与信号量模型

操作系统 · 11 条

同步、互斥、临界资源与临界区 159 先识别制约关系，再决定要保护数据还是规定先后。

直接与间接制约

同步是合作进程为完成共同任务而建立的执行次序关系，例如产品放入后才能取走；互斥是竞争同一临界资源导致的间接制约，例如同一时刻只能有一个执行者修改共享队列。

临界资源是一次只允许一个执行者使用的资源；临界区是访问该资源的代码段。进入区负责检查与申请，退出区负责释放，剩余区完成无关工作。不同临界资源可以分别保护，不必把所有并发操作串成一把大锁。

互斥方案的检查原则

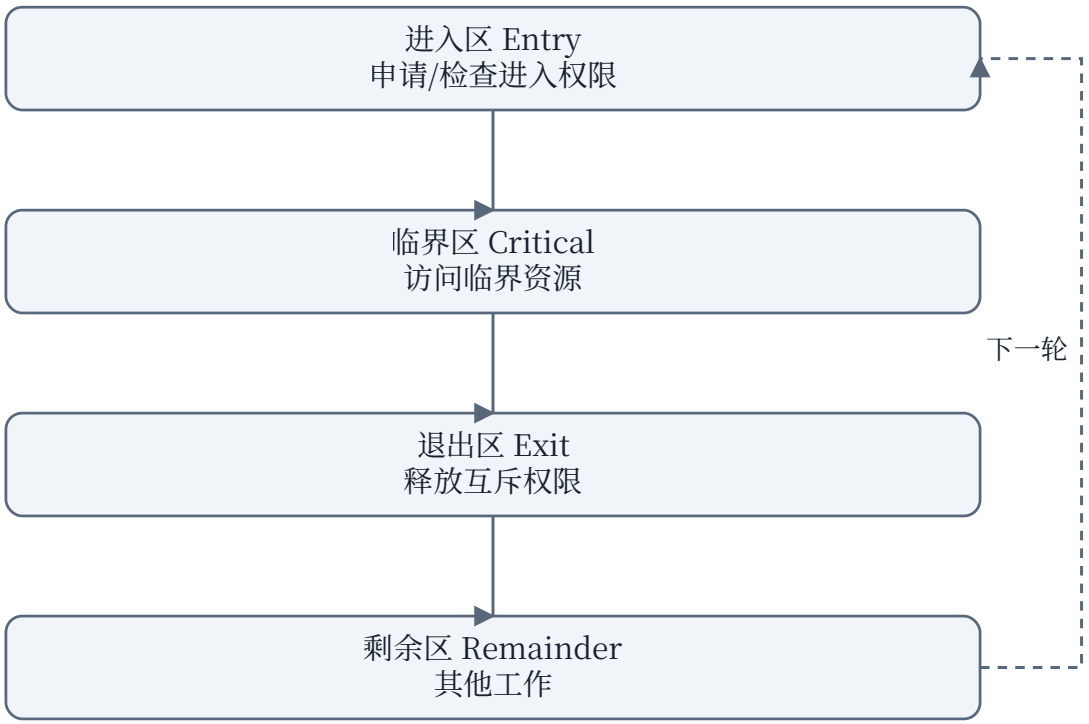
- 空闲让进：临界区空闲且有人请求时，应允许合适的请求者进入。
- 忙则等待：已有执行者访问时，其他竞争者不能同时进入。
- 有限等待：请求者不应无限期得不到进入机会。
- 让权等待：不能进入时释放CPU，避免无效忙等。前三项描述常见互斥正确性要求，让权等待是等待方式的进一步效率要求。

用户临界区与内核临界区

执行用户临界区时仍可被抢占或发生中断；只要锁状态保持正确，别的竞争者就不能进入同一临界资源。内核临界区能否被抢占取决于内核模型和锁规则，不等于系统中任何临界区都“一律不能调度”。

机制图解

互斥程序四段：申请 → 访问 → 释放 → 其他



临界区是代码段，临界资源是资源；退出区不表示进程终止。

单独查看图片

软件互斥：单标志、双标志与Peterson 160 逐条考虑交错执行，不能把检查与置位想成天然原子。

三种错误或受限方案

方案	进入逻辑	问题
单标志轮流	turn指定轮到谁，退出时转交权限	强制交替；对方不再进入时，本方也可能被挡住，违反空闲让进
双标志先检查	先看对方意愿，再设置自己意愿	两个进程都先看到对方false，可能同时进入
双标志后检查	先设置自己意愿，再看对方	双方同时置true后都等待，不能保证进展

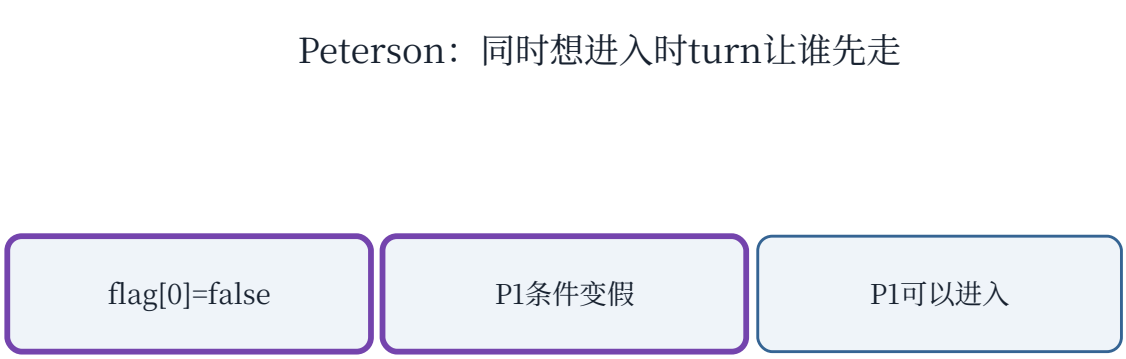
Peterson两进程模型

本进程先表明意愿，再把优先权让给对方；若双方都想进入，最后一次写turn使其中一个等待。该教材算法在对共享读写原子、可见且按预期顺序执行的模型下，满足互斥、进展和有限等待，但采用自旋，不满足让权等待。实际语言与处理器重排序环境需内存模型支持。

```
bool flag[2] = {false, false};
int turn = 0;

// 进程 i，另一进程 j=1-i
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j) { }
    // 临界区
    flag[i] = false;
    // 剩余区
}
```

图解与逐步推演：Peterson：同时想进入时turn让谁先走



单独查看图片

Peterson：同时想进入时turn让谁先走

查看完整步骤（也可离线阅读）

1. 双方表达意愿：假设普通教材的顺序一致内存模型，读写原子且执行顺序不被重排。
2. P0让P1先行：P0完成自己的入口赋值；此时若P0立即检查，会暂时等待P1。
3. P1最后写turn=0：P1把优先进入权让给P0；最后一次turn写入决定本次竞争结果。
4. P0通过，P1等待：只有P0可进入；P1自旋等候。

5. P0退出：P0退出时清自己的意愿，P1继续；现代语言/硬件实现须使用适当原子与内存顺序，不能直接照抄普通变量代码。

硬件原子指令、自旋锁与阻塞锁 161 “原子地取锁”解决竞争，“怎样等待”决定CPU开销。

关中断与原子交换

单处理器内核可在短小关键区间关可屏蔽中断，防止相关中断与调度打断；用户程序不能任意执行特权开关中断，多处理器上关闭本地中断也挡不住其他CPU。关中断过久会损害中断响应。

TestAndSet原子地返回旧值并写入true；Swap/XCHG原子交换锁值与局部值。原子性由硬件与内存系统保证，不能把普通的“读后再写”两条操作当成等价。

```
bool TestAndSet(bool *lock) { // 作为一个原子操作
    bool old = *lock;
    *lock = true;
    return old;
}

while (TestAndSet(&lock)) { } // 自旋等待
// 临界区
lock = false;                 // 释放锁
```

自旋与阻塞

自旋锁失败后反复检查，未释放CPU；临界区很短、持锁者可在另一CPU继续运行时，可避免睡眠与唤醒开销。单CPU上持锁者不运行时，等待者自旋难以等到释放。

阻塞锁把等待者挂起、释放CPU，解锁时唤醒；适合等待较长的情况。互斥锁不必然等于自旋锁，具体acquire/release实现决定是否忙等。简单原子取锁不自动保证公平，等待者仍可能饥饿。

记录型信号量的P、V与负值含义 162 原语是不可分割的，测试条件要跟所采用的定义一致。

整数型与记录型

信号量初始化后，通过P/wait与V/signal原语操作。整数型的传统等待循环会忙等；记录型增加等待队列，资源不足时阻塞，实现让权等待。资源计数初值r表示可用r个单位；同步事件初值常为0，互斥初值通常为1。

记录型原语

P先减1，结果小于0才阻塞；V先加1，结果不大于0说明仍有已等待者需要获得这次释放。按此定义，value>0表示可立即分配单位数，value<0时其绝对值表示等待者数量。0既不等于“队列一定有等待者”，也不能跳过P直接访问资源。

```
struct semaphore { int value; queue waiters; };

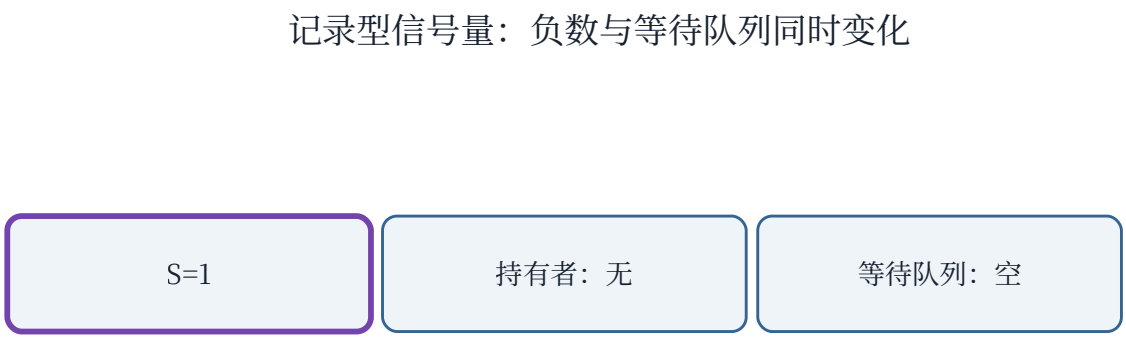
P(S) { // 整个更新与排队动作必须原子完成
    S.value--;
    if (S.value < 0) {
```

```
    把当前进程加入 S.waiters;
        block();
    }
}
V(S) {
    S.value++;
    if (S.value <= 0) {
    从 S.waiters 取一个等待进程;
        wakeup(该进程);
    }
}
```

用数值检查条件

初值1，A执行P后为0并进入；B执行P后为−1并阻塞；A执行V后为0，需唤醒B。若把V的条件误写成<0，则这一步无法唤醒B。实现使用非负计数配合另一个队列规则时，不能生搬这里的负数语义。

图解与逐步推演：记录型信号量：负数与等待队列同时变化



单独查看图片

记录型信号量：负数与等待队列同时变化

查看完整步骤（也可离线阅读）

1. 初始：按408常见记录型定义：P先减1，若<0则入队阻塞；V先加1，若≤0则唤醒一个等待者。
2. P0执行P(S)：P0把1减为0，仍非负，获得资源。
3. P1执行P(S)：P1减到−1而阻塞；负值绝对值对应本模型中等待者数。
4. P2执行P(S)：P2也阻塞；图按FIFO唤醒约定演示，真实队列策略可不同。
5. P0执行V(S)：加到−1仍≤0，唤醒P1。P1转为可运行状态，何时真正执行还取决于调度。
6. P1完成后V(S)：P1获得CPU并完成临界区后释放，唤醒P2；被唤醒者从原P操作之后继续。

7. P2完成后V(S)：所有等待都完成，资源重新空闲。

用信号量表达互斥、同步与前驱图 163 一前一后的事件关系通常用“前V后P”，资源互斥用成对申请释放。

三类建模

互斥：为共享临界资源设置mutex=1，进入前P(mutex)，退出后V(mutex)。不同资源应按需要分别设置锁；共享计数器的读改写也属于需要保护的临界操作。

同步：为“事件A完成后事件B才能执行”设置s=0，在A后V(s)、B前P(s)。V发生在P之前时，信号量保留许可，不会像未记录的通知那样天然丢失。

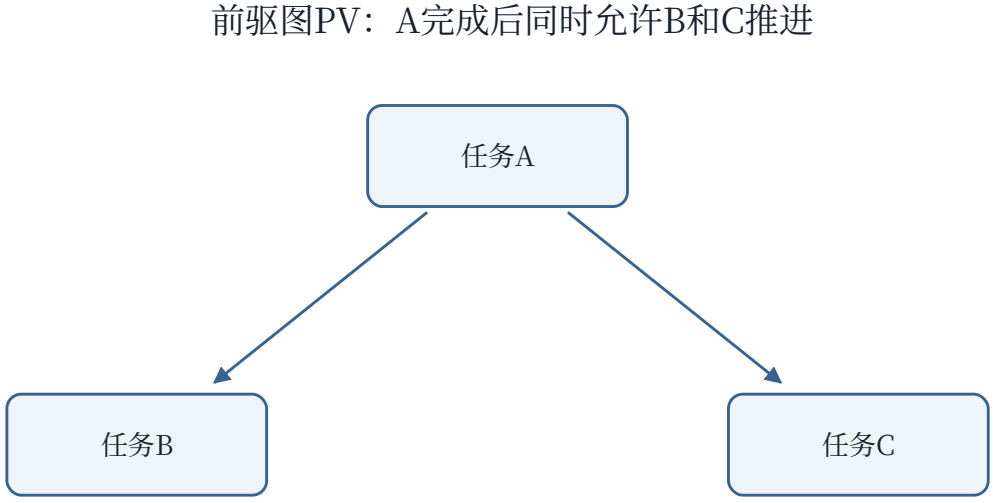
前驱图：可对每条前驱边设置初值0的信号量。一个结点必须在所有入边相应P成功后开始，完成后对所有出边执行V；分叉要分别通知各后继，汇合要等待所有必要前驱。

一分二再汇合

AB与AC的两次通知不能只用一次V替代，否则B、C只有一个能获得许可。D前两个P只是在等待独立前驱，可按规则排列；若同时持有互斥锁再等待其他进程发信号，则需检查能否产生循环等待。

```
semaphore ab=0, ac=0, bd=0, cd=0;
A: 执行A; V(ab); V(ac);
B: P(ab); 执行B; V(bd);
C: P(ac); 执行C; V(cd);
D: P(bd); P(cd); 执行D;
```

图解与逐步推演：前驱图PV：A完成后同时允许B和C推进



单独查看图片

前驱图PV：A完成后同时允许B和C推进

查看完整步骤（也可离线阅读）

1. 建立前驱约束：要求A完成后，B和C才能开始；B、C之间无先后约束。不能用一个信号量V一次同时放行两个P。
2. B和C先到时阻塞：B在自身任务前执行P(sAB)，C在自身任务前执行P(sAC)，都因对应初值0等待A。
3. A完成分别通知：A结束后依次V(sAB)、V(sAC)，分别使B和C满足各自前驱条件。
4. B与C可分别推进：两条通知彼此独立。被唤醒与真正获得CPU是两回事，但两者不再需要相互等待。

有界缓冲区的生产者—消费者 164 完整保留容量条件、互斥条件和两类资源计数。

资源与约束

有n个缓冲单元，初始全空：empty=n表示空位，full=0表示产品；mutex=1保护共享缓冲结构。生产者必须先取得空位才放入，消费者先取得产品才取走。生产产品和使用产品通常放在锁外，缩短临界区。

完整PV流程

```
semaphore empty=n, full=0, mutex=1;

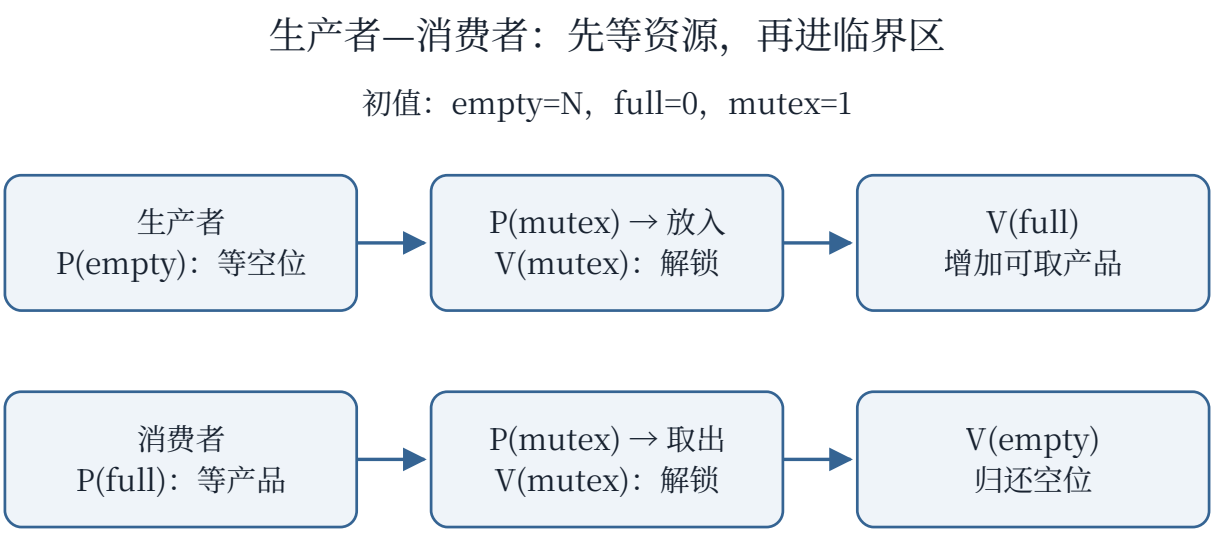
producer() {
    while (true) {
        item = produce();
        P(empty);
        P(mutex);
        insert(item);
        V(mutex);
        V(full);
    }
}
consumer() {
    while (true) {
        P(full);
        P(mutex);
        item = remove();
        V(mutex);
        V(empty);
        consume(item);
    }
}
```

为什么P顺序重要

若生产者先P(mutex)再P(empty)，缓冲已满时会持有mutex等待空位，消费者又需mutex才能取走产品，形成死锁；消费者先互斥后等待full同理。故此模型中等待资源的P在前，访问缓冲结构的P在后。

两个相邻V不因等待而直接阻塞，在不改变资源正确计数和共享结构可见性的前提下常可交换，但把通知放在释放锁前可能使被唤醒者再次等待锁。多个full表示多个可消费单位，不要求一次消费全部，也不要求全部缓冲区读完后才允许再次写入。

机制与步骤图解



单独查看图片

逐步操作与动画

例：容量2，一个产品从放入到取出

查看完整步骤（也可离线阅读）

1. 0 · 初始化：缓冲区容量2：empty=2、full=0、mutex=1。只演示一个生产者和一个消费者的合法执行次序。
2. 1 · 生产者P(empty)：先取得一个空位许可，empty由2减到1。此刻尚未向缓冲区写入产品，因此full仍为0。
3. 2 · 生产者P(mutex)并放入：mutex从1变0，生产者进入临界区并放入产品A。写入之后尚未发布full许可，full仍为0。
4. 3 · V(mutex)，再V(full)：先解锁使mutex=1，再发布产品许可使full=1。此时消费者可以获得产品。
5. 4 · 消费者P(full)：full由1减为0，表示产品许可已被消费者预留。A暂时仍在缓冲区中，因此不能始终用full直接等同物理产品数量。
6. 5 · P(mutex)并取出A：消费者获得互斥锁，mutex=0，在临界区更新缓冲区和读指针。取出后还未归还空位许可。
7. 6 · V(mutex)，再V(empty)：先解锁，再把empty增到2，回到初始计数。P(full)/P(empty)不能放在持有mutex之后等待，否则可能阻塞使条件成立的另一方。

多类产品与多生产者—多消费者 165 按“空位”和“某类产品就绪”划分事件，不按人员机械增加信号量。

单盘模型

一个只能放一个水果的盘子，父亲放苹果、母亲放橘子、女儿取苹果、儿子取橘子。盘子变空由任一消费者产生，放水果由任一生产者执行，所以共用plate=1；apple=orange=0分别通知对应消费者。

带显式互斥的通用写法

```
semaphore plate=1, apple=0, orange=0, mutex=1;
father:  while(true) { 准备苹果; P(plate); P(mutex); 放苹果; V(mutex); V(apple); }
mother:  while(true) { 准备橘子; P(plate); P(mutex); 放橘子; V(mutex); V(orange); }
daughter:while(true) { P(apple); P(mutex); 取苹果; V(mutex); V(plate); 吃苹果; }
son:     while(true) { P(orange); P(mutex); 取橘子; V(mutex); V(plate); 吃橘子; }
```

容量为1的特殊情况

若严格保证单槽且每次只放/取一个，plate及类别信号量已约束同一时刻只有一方访问，mutex可能可以省略。容量增加、操作拆分或共享管理信息改变后，不能照搬省锁结论。

建模顺序是列参与者 → 找临界资源与事件 → 确定初值 → 按每方动作放置P/V → 在空、满和竞争同时到达的边界检查。两个生产者不需要各自一份“盘子空”信号，否则可能重复占用同一个空盘。

读者—写者：读者优先及其饥饿问题 166 读者可成组共享，写者必须排除所有读者和其他写者。

共享与互斥

允许多个读者同时读，任一写者工作时不允许其他读者和写者访问。readCount表示正在访问的读者数，用rmutex保护计数更新，用rw保护读者组和写者之间的排他关系。第一个读者申请rw，最后一个读者释放rw。

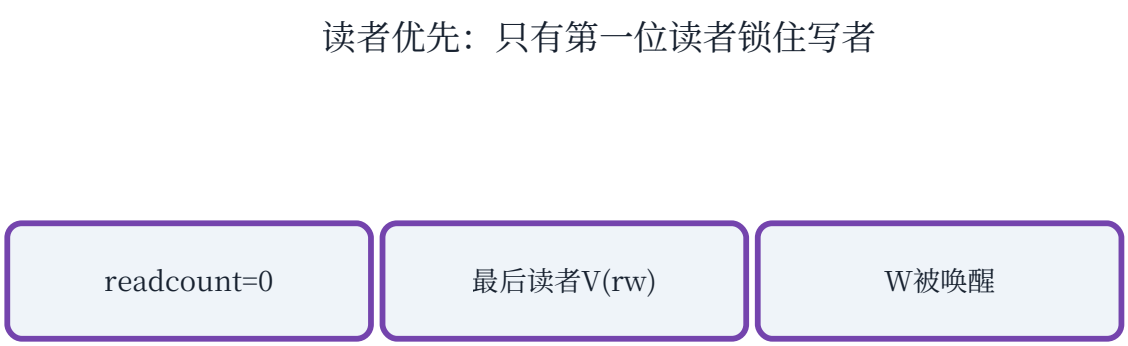
读者优先伪代码

```
int readCount=0;
semaphore rmutex=1, rw=1;
reader() {
    P(rmutex);
    if (readCount==0) P(rw);
    readCount++;
    V(rmutex);
  读共享文件;
    P(rmutex);
    readCount--;
    if (readCount==0) V(rw);
    V(rmutex);
}
writer() { P(rw); 写共享文件; V(rw); }
```

计数保护与写者饥饿

不能先在锁外判断readCount==0再进锁更新，否则两个读者都可能把自己当第一个，或退出时错误释放rw。若不断有新读者加入当前读者组，readCount始终不为0，等待写者可能饥饿；互斥正确并不自动意味着公平。

图解与逐步推演：读者优先：只有第一位读者锁住写者



单独查看图片

读者优先：只有第一位读者锁住写者

查看完整步骤（也可离线阅读）

1. 初始：rw控制读者组与写者互斥，mutex保护readcount；图省略每次计数临界区外的mutex恢复过程。
2. 第一位读者R1进入：R1在mutex保护下把计数从0加到1，第一位读者执行P(rw)。
3. R2进入并发读：后续读者只增加计数，不再P(rw)。计数修改受mutex保护，读文件本身可并发。
4. 写者W到达：W等待整组读者离开；按记录型实现rw.value可变为−1，此处rw=0的前两步只表示无等待时资源占用。
5. R1退出：非最后一位读者不释放rw。
6. R2退出：计数归零时释放写锁。若读者持续到达使计数永不归零，写者可能饥饿；入口闸门可用于改进。

读者—写者：增加入口闸门 167 阻止新读者不断插队，同时保留已经进入的读者并发。

入口闸门版本

```
int readCount=0;
semaphore gate=1, rmutex=1, rw=1;
reader() {
    P(gate);
    P(rmutex);
    if (readCount==0) P(rw);
    readCount++;
    V(rmutex);
    V(gate);
  读共享文件;
    P(rmutex);
    readCount--;
```

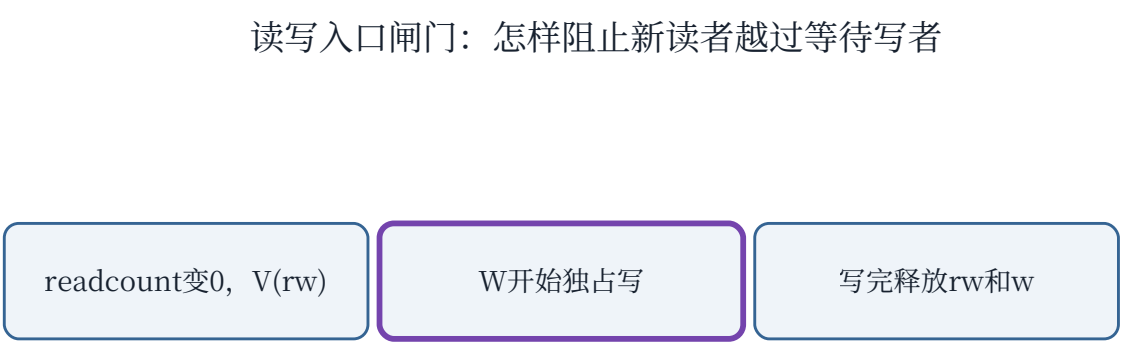
```
    if (readCount==0) V(rw);
    V(rmutex);
}
writer() {
    P(gate);
    P(rw);
  写共享文件;
    V(rw);
    V(gate);
}
```

它改变了什么

写者一旦取得gate，就不再允许后到读者加入，已有读者继续执行并在最后离开时释放rw；写者获得rw后写入，再开放gate。gate不应一直包围普通读操作，否则所有读者会被不必要地串行化。

此方案通常称读写公平闸门或教材中的写优先改进。严格无饥饿仍依赖信号量排队公平性；不能只因增加一把锁便声称任何调度下都满足有限等待。

图解与逐步推演：读写入口闸门：怎样阻止新读者越过等待写者



单独查看图片

读写入口闸门：怎样阻止新读者越过等待写者

查看完整步骤（也可离线阅读）

1. 已有读者在读：采用正文中写者先申请w再申请rw的入口闸门模型；读者进组前也要通过w。
2. 写者到达先占闸门：等待文件写权限期间先关住入口，防止后来读者继续加入读者组。
3. 新读者R2到达：R2尚未成为已入组读者；已在读的R1退出不需要重新申请入口闸门。
4. 最后读者退出唤醒W：W完成后再放开入口。严格公平性仍依赖信号量队列与调度，不凭一个闸门就保证任意实现都绝不饥饿。

哲学家进餐：多资源申请的死锁与解决 168 两个资源分别互斥仍不够，还要控制取得顺序。

死锁来源

5名哲学家各需左右两支筷子才能进餐。若所有人都先取同一侧再等另一侧，可能各占一支并循环等待。每支筷子设初值1的信号量只保证单支互斥，并不消除多资源死锁。

三种常用破坏方法

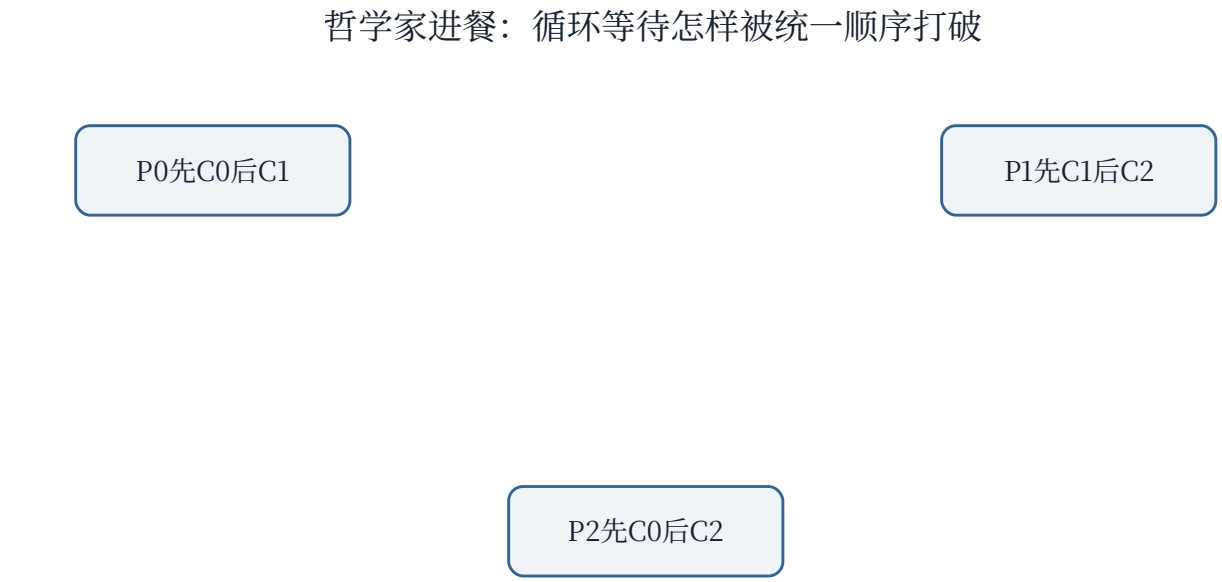
- 限制最多4人同时尝试取筷，可至少留下一个能取得两支的机会，避免全员各持一支闭环。
- 规定不同哲学家采用不同左右顺序，或统一按筷子编号从小到大申请，破坏循环等待。
- 用额外互斥量串行化“取齐两支”的过程，防止多人各持一支；取齐后释放此互斥量，让不冲突者并行进餐。

串行化取筷过程

take只保护取齐动作，不包住思考与整个进餐阶段；释放筷子不需要take，因此持有take等待另一支筷子者不会阻止现有食者释放。无死锁与无饥饿仍须区分。

```
semaphore chopstick[5]={1,1,1,1,1};
semaphore take=1;
philosopher(i) {
    while(true) {
   思考;
        P(take);
        P(chopstick[i]);
        P(chopstick[(i+1)%5]);
        V(take);
   进餐;
        V(chopstick[i]);
        V(chopstick[(i+1)%5]);
    }
}
```

图解与逐步推演：哲学家进餐：循环等待怎样被统一顺序打破



单独查看图片

哲学家进餐：循环等待怎样被统一顺序打破

查看完整步骤（也可离线阅读）

1. 各拿左筷：以3位哲学家构成的简化环为例，每人已获得一根筷子；若所有人都这样做，仍可能等待下一根。
2. 再请求右筷：P0等C1、P1等C2、P2等C0，形成循环等待。此处箭头表示等待持有者。
3. 按筷子编号递增申请：统一先申请编号小的筷子，再申请编号大的筷子。原来的持有分配不会同时形成，消除循环等待。

吸烟者模型：按材料组合通知 169 作为事件同步的拓展，重点是组合就绪与完成反馈。

参与者与信号

三个吸烟者分别长期持有烟草、纸、胶水的一种；供应者每次放另外两种材料。只有持有缺少第三种材料的人能取走组合。设offer[3]=0，分别通知匹配吸烟者，done=1表示允许供应下一轮。

配对流程

一次只允许一组材料在桌上，供应者等待完成反馈后才进入下一轮，给出隐含的单槽互斥。若允许多组材料并存，需要重新定义数量和临界区保护，不能继续把这一省锁版本当作通用解法。

```
semaphore offer[3]={0,0,0}, done=1;
provider() {
    while(true) {
        P(done);
        k = 选择本轮缺少的材料对应吸烟者;
    放置另外两种材料;
```

```
        V(offer[k]);
    }
}
smoker(k) {
    while(true) {
        P(offer[k]);
    取材料并卷烟、吸烟;
        V(done);
    }
}
```

死锁预防、避免与检测

操作系统 · 4 条

死锁、饥饿与死循环 170 先看互相等待，再判断是哪一种无法推进。

概念与区别

死锁指一组进程因资源或事件依赖而互相等待，若无外界干预，这组进程无法继续；饥饿是某进程长期得不到需要的服务，其他进程仍可推进；死循环是控制流一直重复，既可能是逻辑错误，也可能是有意循环。

教材资源模型下死锁参与者通常阻塞；自旋锁场景也可能表现为忙等死锁，因此不要把“死锁必然所有线程都处于阻塞队列”当作一切实现的定义。系统并不一定所有进程都死锁。

四个必要条件

- 互斥：某些资源一次只能被一个进程占用。
- 不可剥夺：已获资源只能由持有者主动释放，不能强行收回。
- 请求和保持：一边保留已得资源，一边等待其他资源。
- 循环等待：存在闭环，每个进程等待下一个进程持有的资源。

必要不等于充分

死锁意味着存在循环等待，但某类资源有多个实例时，仅看图中有环不一定足以认定死锁；每类资源单实例的相应模型中，有环才可成为充分判据。错误P/V顺序同样可通过持锁等待形成死锁。

死锁预防：逐一破坏必要条件 171 预防在资源使用规则中施加限制，代价是利用率或灵活性。

四种方向

破坏条件	做法	适用性与代价
互斥	可共享化、假脱机等把独占访问转成排队服务	不是所有资源都能共享化；物理设备服务仍可串行

破坏条件	做法	适用性与代价
不可剥夺	请求失败时释放已持资源，或在可恢复资源上强行剥夺	需保存恢复状态，可能工作回退、开销与饥饿
请求和保持	运行前一次申请所需全部资源，满足后才运行	简单但资源可能长期闲置，需预知需求
循环等待	统一给资源编号，按递增顺序申请，同类按规则一次申请	不易扩充，可能限制编程并造成资源闲置

静态预防和动态避免

一次性分配消除的是边持有边继续申请的机会，不是简单“使用后早点释放”；有序分配必须覆盖所有相关资源和代码路径，不能一处按升序另一处逆序。预防改变请求规则，避免则在每次请求时分析是否仍安全。

安全状态与银行家算法 172 试分配后是否仍存在完成顺序，是“能给”与“应给”的分界。

安全与不安全

若存在一个进程序列，使每个进程按声明的最大剩余需求都能从当前可用资源及前面完成者释放资源中得到满足，则状态安全。序列可能不唯一；安全意味着在假设成立并按合适策略分配时可避免死锁。

不安全不等于已经死锁；它表示无法保证所有声明最大需求都能完成。实际进程可能少于最大需求就结束，因此不安全状态也可能继续运行。

数据结构与请求检查

Available[m]是可用向量；Max[n][m]是最大需求；Allocation是当前已分配；Need=Max−Allocation；Request是本次请求。各向量比较均按分量进行。

请求处理：①若Request>Need，超过声明需拒绝/报错；②若Request>Available，资源不足需等待；③否则试减Available、增加Allocation、减少Need；④执行安全性检验，安全才正式分配，不安全则回滚并等待。

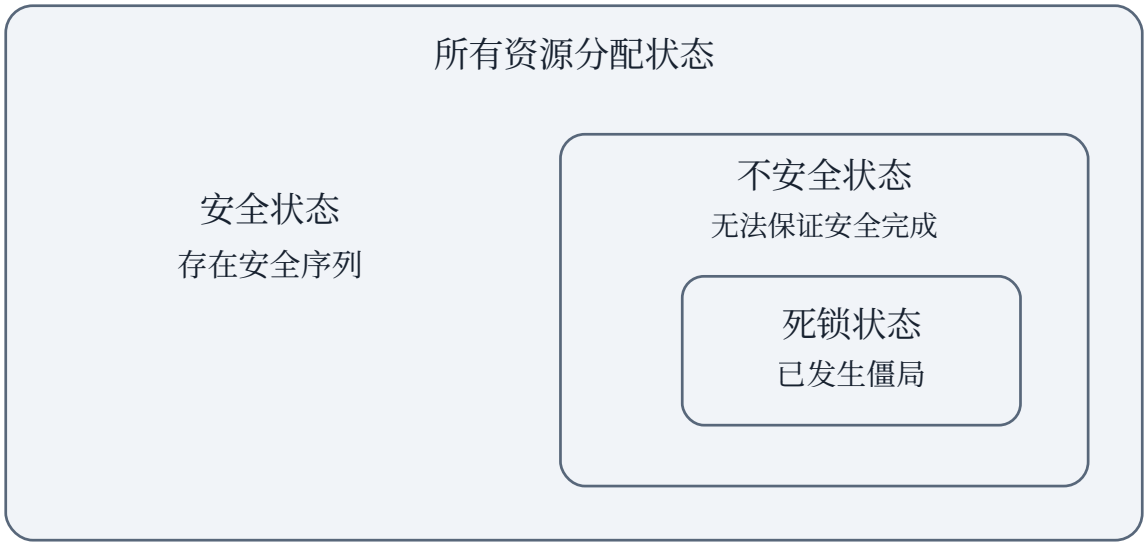
安全性检验

模拟完成时把原已分配Allocation加回，而不是把Max直接加回。Need由Work提供只是模拟临时借出后归还，净增加的是该进程原先占有的Allocation。

```
Work = Available;
Finish[i] = false;
while (存在 i 使 !Finish[i] 且 Need[i] <= Work) {
    Work = Work + Allocation[i];
    Finish[i] = true;
  将 i 追加到安全序列;
}
若所有 Finish[i] 都为 true，则安全；否则不安全。
```

机制图解

资源安全：死锁是不安全状态的子集



银行家算法适用模型：死锁 ⇒ 不安全；不安全 ⇏ 已死锁。

单独查看图片

图解与逐步推演：银行家安全性检查：Work怎样增加

银行家安全性检查：Work怎样增加



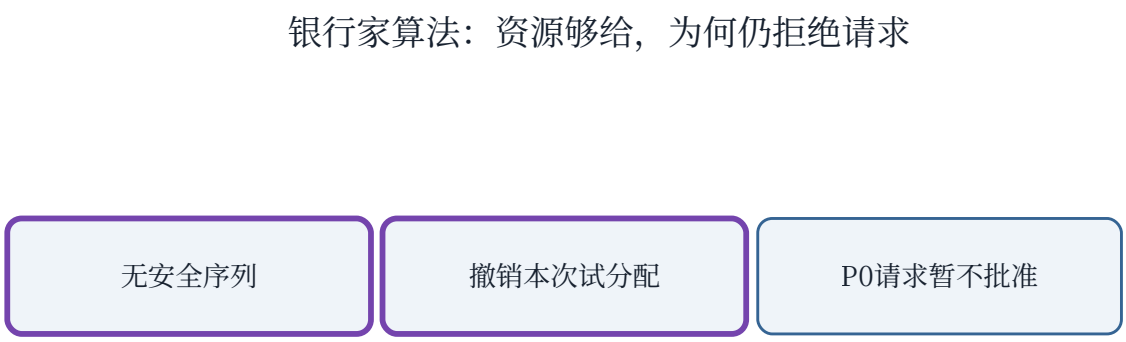
单独查看图片

银行家安全性检查：Work怎样增加

查看完整步骤（也可离线阅读）

1. 检查前：系统仅1类资源共5个：已分配2、1、1，可用1；最大需求分别3、3、2，因此Need=1、2、1。
2. 模拟完成 P0：P0的Need=1≤Work=1，可假设它完成并归还已分配的2，Work增为3。
3. 模拟完成 P1：P1的Need=2≤Work=3，可假设它完成并归还已分配的1，Work增为4。
4. 模拟完成 P2：P2的Need=1≤Work=4，可假设它完成并归还已分配的1，Work增为5。

图解与逐步推演：银行家算法：资源够给，为何仍拒绝请求



单独查看图片

银行家算法：资源够给，为何仍拒绝请求

查看完整步骤（也可离线阅读）

1. 初始安全状态：只有一类资源。P1可先完成归还1，使Work=2，再让P0完成，所以初始安全。
2. P0请求1：两项检查只说明没超需求、当前资源够给，尚未证明试分配后安全。
3. 试分配后：所有未完成进程都至少还需要1，而Work=0，找不到可假设完成的进程。
4. 回滚并等待：不安全状态不等于此刻一定已经死锁；银行家避免进入无法保证完成顺序的状态。

资源分配图、死锁检测与解除 173 检测针对当前真实等待，解除要权衡回收收益与进程损失。

资源图与简化

圆点表示进程，资源类结点可含多个实例；进程→资源是请求边，资源实例→进程是分配边。找当前请求可由空闲资源满足的进程，假设它完成并撤去相关边、归还已持资源，重复简化。

在所用教材资源请求模型下，若图能完全简化，当前不死锁；最终无法被简化的相关进程构成需要处理的死锁/受阻集合。检测按当前请求判断，不是银行家按最大剩余Need保证未来。

解除与牺牲者选择

- 资源剥夺：暂时夺取可保存恢复状态的资源，交给其他参与者；需避免同一进程反复被牺牲。
- 撤销进程：可一次终止全部参与者，也可逐个撤销并重新检测；代价可能是大量已完成工作丢失。
- 进程回退：借助检查点回到足以解除死锁的状态，回收相应资源后重新执行。
- 选择时考虑优先级、已执行时间、剩余完成量、占有资源、交互或批处理属性及回退成本。

处理策略总览

预防：破坏必要条件；避免：拒绝使系统进入不安全状态的申请；检测与解除：允许发生后发现并恢复。三者分别改变资源使用规则、动态分配判断和事后处置，不能混为同一种“银行家算法”。

结构对照图：单实例资源：请求边与分配边构成环

P1持有R2等R1，P2持有R1等R2，单实例且不可抢占时形成相互等待环。请求边从进程指向资源，分配边反向。

当各资源类型都只有一个实例时，环是死锁的充分必要条件；有多个实例时，存在环只是必要条件，仍须检查是否有进程可以完成。

单实例资源：请求边与分配边构成环



单独查看图片

内存分配、地址变换与虚拟存储

操作系统 · 17 条

编译、链接、装入与重定位 174 先得到可执行模块，再决定何时把逻辑地址绑定到物理位置。

三个阶段

编译把源程序生成目标模块，模块中包含机器指令、相对地址及待解决符号；链接把目标模块和所需库连接起来，解决外部符号与模块间地址关系；装入把可执行映像安排到内存并完成相应地址绑定。

静态链接在运行前组合模块；装入时动态链接边装入边处理所需模块；运行时动态链接在执行到相关调用时再处理模块。动态链接便于库更新和共享，但具体查找、重定位及版本规则由实现决定。

装入/重定位方式

方式	地址绑定时机	移动与限制
绝对装入	编译/汇编等阶段已确定物理地址	预先知道装入位置，位置改变需重新处理
可重定位装入/静态重定位	装入时一次把相对地址修改为物理地址	装入后通常不能任意移动，需重新重定位
动态运行时装入/动态重定位	真正访问时由硬件进行地址变换	可支持运行中移动、非连续存放与按需装入等机制

逻辑地址与物理地址

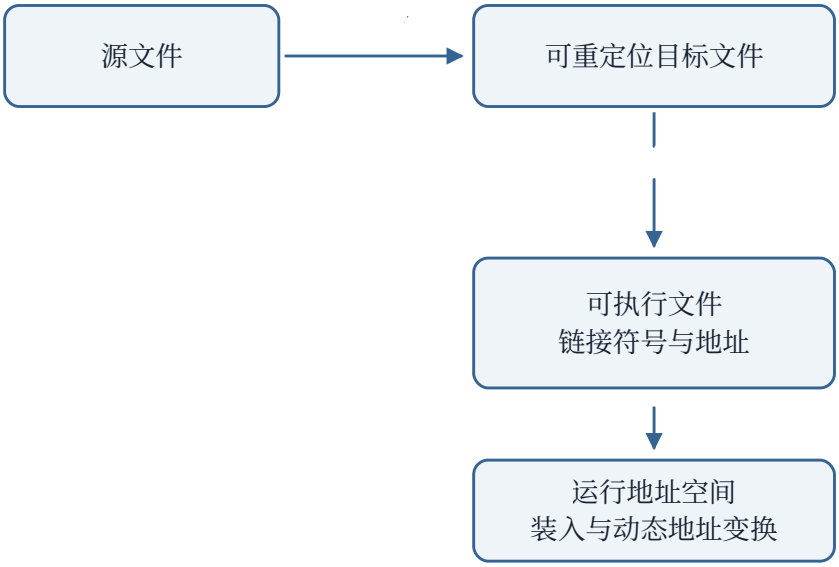
逻辑/虚拟地址是程序所用地址，物理地址是内存实际访问位置；二者偶然数值相同不代表概念相同。按字节编址时每个地址对应1B，按字编址时对应一个机器字，计算页大小和偏移位数前必须统一单位。

结构对照图：源程序到运行：各阶段负责什么

编译/汇编生成目标代码；链接把目标模块和所需库组合并处理符号引用，静态与动态链接的发生时点不同。

装入把程序映像准备到内存或虚拟地址空间。运行时地址变换由MMU等机制完成，不是每执行一条指令都重新链接整个程序。

源程序到运行：各阶段负责什么



单独查看图片

存储保护、共享与可重入代码 175 合法地址检查和访问权限检查是两道相关但不同的约束。

界限保护

上下限寄存器可规定允许的物理地址范围；基址加限长模型先检查0≤逻辑地址<长度，再计算物理地址=基址+逻辑地址。限长表示单元数量还是最大合法偏移以题设为准，不能漏掉等号边界。

这些保护寄存器由内核通过受控方式设置，用户程序不能任意改写来绕过隔离。分页、分段中还可通过表项的存在位、读写执行权限等完成更细的保护。

共享不只限于只读

可重入/纯代码在运行期间不修改自身，便于多个进程共享同一代码副本，各进程的可写数据及现场另行保存。共享只读代码较容易，但共享内存也可以是可写的，可写共享需要访问权限和同步保护，不能把“只有只读区能共享”当作普遍定义。

连续分配：单一、固定与动态分区 176 从分区是否固定、碎片在内还是在外进行比较。

分配方式

方式	基本组织	主要特点
单一连续	系统区外一次只装一个用户程序	简单，利用率和并发受限，用户区未使用部分可能成为内部浪费
固定分区	预先分成固定大小或不等大小分区，各放一作业	支持多道，实现简单；分区内部可能浪费，过大作业放不进

方式	基本组织	主要特点
动态分区	按到达进程需求临时划分，分区大小数目变化	利用更灵活，但反复申请回收产生外部碎片

内外碎片与紧凑

内部碎片在已分配范围内暂未使用，不能直接给其他请求者；外部碎片是分散空闲区，合计够大却没有足够连续空间。动态分区按精确需求分配的理想模型无内部碎片，真实实现的对齐或最小分配量仍可能引入内部浪费。

紧凑把已占分区移动到一起，合并空闲空间以缓解外部碎片，需可重定位支持和数据移动开销；不能当成完全没有成本的“把空闲块数字相加”。

动态分区的四种适应算法与回收 177 查找次序和放置位置共同决定分配结果。

算法规则

算法	空闲区组织与搜索	典型影响
首次适应FF	按地址递增，每次从低地址找首个够大的区	保留高地址大块，但低地址小碎片增加查找
邻近适应NF	按地址循环，从上次结束处继续	减少重复低址扫描，也可能更快打散高址大块
最佳适应BF	选满足请求且最小的空闲区，常按容量递增	尽量不占大块，但容易留下大量小碎片
最坏/最大适应WF	选最大的空闲区，常按容量递减	剩余块较大，但大块可能迅速被消耗

回收四种相邻关系

回收区前后都不空闲：新增一项；仅前邻空闲：并入前区；仅后邻空闲：调整后区起点并增大长度；前后均空闲：三段合并并删除多余表项。合并依据物理地址相邻，不依据它们在按大小排序链表中是否相邻。

分配对比

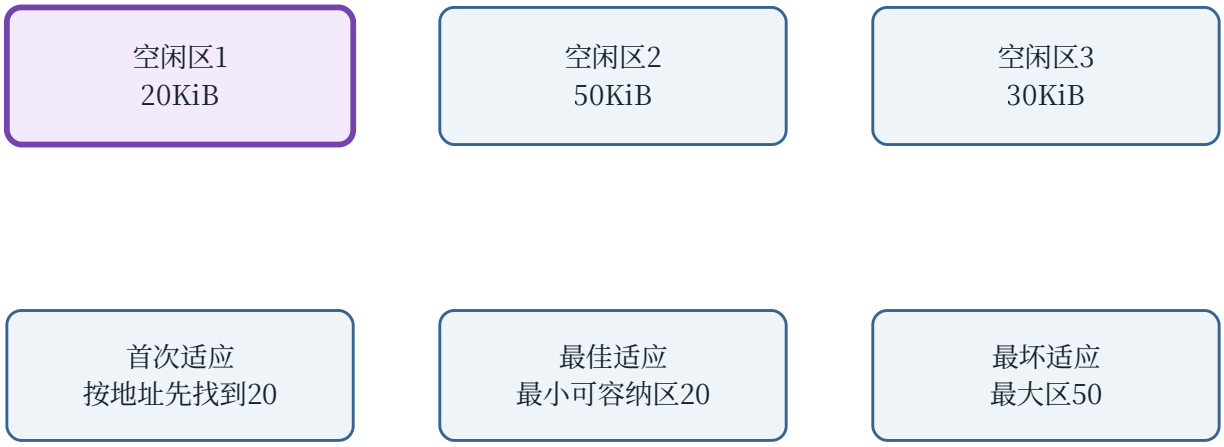
按地址排列空闲区100、500、200、300、600KB，请求212KB：FF取500，BF取300，WF取600；NF需再知道上次搜索位置。完成分割后要更新起点和剩余长度；零大小剩余区应删除。算法不存在脱离请求分布永远最优的保证。

结构对照图：申请18KiB：首次、最佳、最坏适应选谁

假设按地址排列的空闲分区大小依次20、50、30KiB，申请18KiB：首次与最佳均选20，最坏选50。

邻近适应从上次结束位置继续循环搜索，缺少上次指针就不能唯一判定。回收时须检查地址上相邻的空闲分区并合并，不能仅按分区表排列相邻就合并。

申请18KiB：首次、最佳、最坏适应选谁



单独查看图片

分页：页、页框、页表与地址拆分 178 页面属于虚拟空间，页框属于物理内存，表记录映射。

基本结构

物理内存分成等大的页框/页帧/物理块，进程逻辑空间分成同样大小的页。各页可以落在不相邻页框中，页内连续；基本分页模型通常一次装入所需全部页，请求分页则允许部分驻留。

页表项按虚页号记录对应页框及权限等信息。连续数组组织时页号可由表项位置隐含，不必逐项额外保存；页表通常作为独立内存结构存在，PCB保存其地址等管理信息，不是把整张页表塞进PCB。

地址公式

页大小为2^k个编址单元时，低k位是偏移，其余为页号。页号≥页表长度属于越界；页表项有效但页不在内存才是缺页，两者不是同一情况。

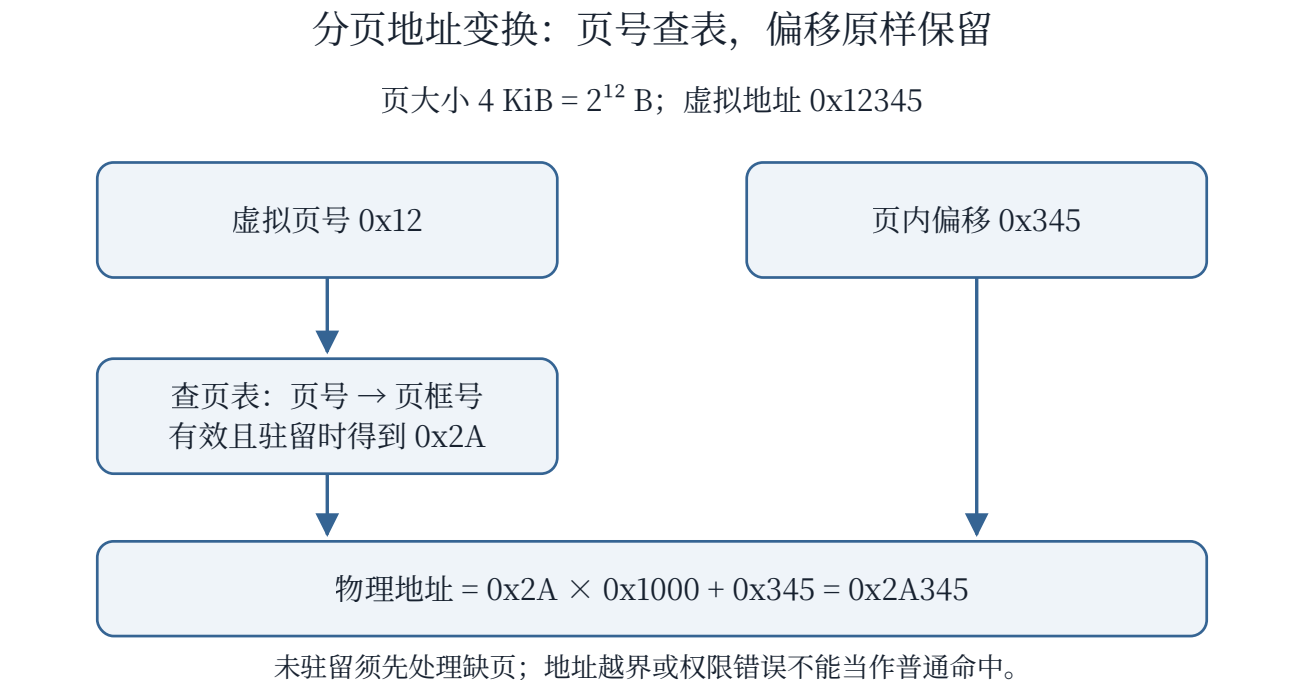
$p=\lfloor A/L \rfloor, d=A \bmod L, PA=bL+d$

A为逻辑地址，L为页大小，p为页号，d为偏移，b为查得的页框号。地址与L需使用同一编址单位。

一步步拆分

按字节编址，页4KB，逻辑地址0x12345：页号0x12，偏移0x345。若该页对应页框0x2A，则物理地址0x2A345。页内偏移保持不变，不能把页号原样当作物理块号。

机制与步骤图解



单独查看图片

页表项容量与基本地址变换 179 先算需要表示多少页框，再考虑状态位、对齐与表项规定。

容量和过程

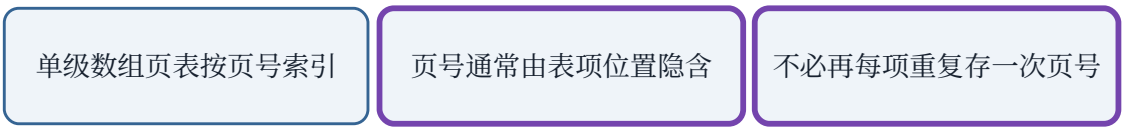
物理内存容量M、页大小L时，可用页框数为M/L，需要至少⌈$\log_2(M/L)$⌉位表示页框号；完整页表项还可能含状态、访问权限等，实际表项大小按题设给出的位数或对齐规定，不能只算页框号便直接认定表项长度。

页表基址与长度等信息在进程调入CPU时加载到相应寄存器。硬件拆出页号/偏移 → 检查页号范围 → 用页表基址+页号×表项长度定位表项 → 检查状态与权限 → 取页框号组合物理地址 → 访问目标数据。

访存次数与地址维度

不带TLB且页表驻内存、没有缓存等额外条件的单级页表模型：读表一次、取目标一次，共两次主存访问。分页地址在用户看来可用一个线性逻辑地址表示，是一维地址；拆成页号与偏移是硬件解释，不意味着用户必须显式给出两个逻辑维度。

图解与逐步推演：页表容量：虚拟页数与物理页框号位数分别算



单独查看图片

页表容量：虚拟页数与物理页框号位数分别算

查看完整步骤（也可离线阅读）

1. 32bit虚拟地址，页4KiB：按字节编址，4KiB=2^{12}B；虚拟页数由虚拟地址空间与页大小决定。
2. 物理内存1GiB：不能把20bit虚拟页号直接当成物理页框号所需位数。
3. 规定每页表项4B：题设规定的表项大小可包含有效、访问、修改和保护等字段及对齐开销。
4. 表项是否要存虚拟页号：关联式、倒排页表等其他结构可能不同，不能把所有页表都用同一存储字段假设。

TLB与有效访存时间 180 TLB缓存地址翻译，数据Cache缓存数据，命中对象不同。

TLB查询

TLB是用于缓存近期页表映射的高速结构。命中且权限有效时可直接得到页框；未命中则需查内存页表，成功后把翻译装入TLB。TLB未命中不等于缺页，页面完全可能已经在物理内存。

切换地址空间时需避免误用旧翻译：无地址空间标记的模型可能刷新TLB；有ASID等标记可保留不同空间映射。不能绝对说一切进程切换必然把全部TLB清空。

顺序查询的时间

若TLB与页表并行查询、命中时取消慢表访问，路径时间需分别按硬件重叠关系计算，不能直接套顺序公式。缺页处理时间往往远高于纳秒级内存访问，给出缺页率时应额外加权。

$$EAT=h(\varepsilon+t)+(1-h)(\varepsilon+2t)=\varepsilon+(2-h)t$$

单级页表、TLB顺序查询，命中率h、TLB查询ε、一次主存t；忽略缺页、Cache及并行重叠。

带数值核对

ε=10ns、t=100ns、h=0.9时，命中路径110ns，未命中路径210ns，加权为120ns。若错误把未命中只算一次主存，会漏掉页表读取。

多级页表：层数、地址结构和访问次数 181 把页表再分页，减少大块连续页表的需求。

为什么分级

单级页表可能很大并要求较大连续空间，而且未使用的虚页也占用表项。多级页表让各级指向下级表，未覆盖到的地址区可暂不建立相应低级表；表项的管理空间与逐级查表开销需要权衡。

拆分顺序为顶级索引、各级低级索引和页内偏移；页目录不是直接把所有虚页映射到数据页框，它的表项先指向下一层页表。

层数和次数

无TLB且各级表都在主存，N级页表访问一个数据通常需N次读表加1次取数据。若已有某级缓存、TLB命中、表页缺页或其他条件，另按题设分析。

$q=\log_2(L/E), N=\left\lceil \frac{v-k}{q} \right\rceil$

页大小L=2^k、表项E为2的幂且每级表至多一页时，完整低级表可承担q位索引；v为虚址位数，顶级剩余位可较少。

40位虚址例子

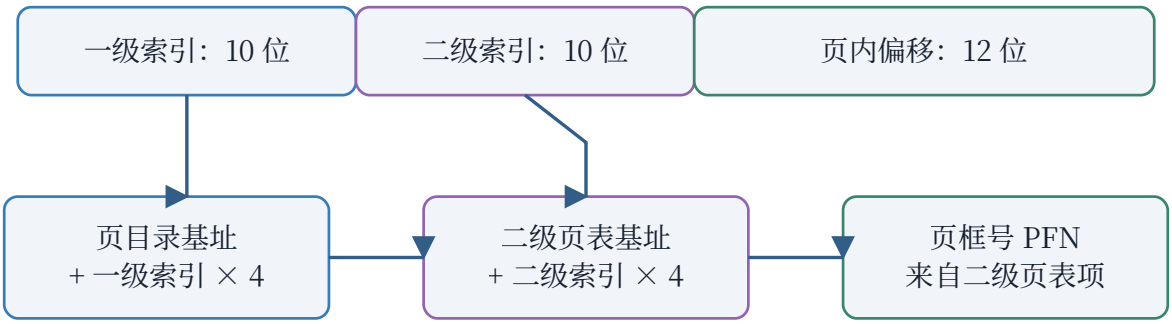
40位字节地址、页4KB、表项4B：偏移12位，每页可放1024项，低级索引每级10位；页号共28位，需3级，可拆8+10+10+12。无TLB命中时通常读三次表，再访问一次数据。

图解：两级页表的地址传递

每个4KiB页表页可装4096÷4=1024个表项，所以一层索引占10位。两级各10位加12位页内偏移恰好为32位；一级表项给出下一层表的位置，最终表项给出目标页框号。不同地址位宽、页大小和表项大小必须重新拆分，不能直接套用10/10/12。

两级页表：先找二级表，再找页框

假设：32 位虚拟地址；页大小 4 KiB；页表项大小 4 B



物理地址 = 页框号 × 4096 + 原页内偏移

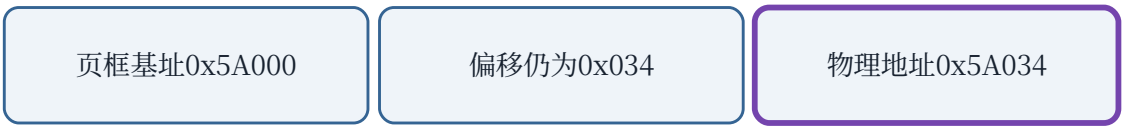
表项有效性与访问权限需逐级检查；缺页不能继续当作有效页框访问。

TLB 未命中且页表项均有效：两次查表 + 一次访问目标数据（本简化模型）。

单独查看图片

图解与逐步推演：二级页表查找：10位目录、10位页号、12位偏移

二级页表查找：10位目录、10位页号、12位偏移



单独查看图片

二级页表查找：10位目录、10位页号、12位偏移

查看完整步骤（也可离线阅读）

1. 32bit虚拟地址：本例虚拟地址=(1<<22)+(2<<12)+0x034=0x00402034；每页表项4B，一页可存1024项。

2. 先查页目录：目录项地址=页目录基址+目录号×4，不是把目录号当字节偏移直接相加。
3. 再查二级页表：二级索引2，所以项地址加8B；图中两级表项都合法、有效且驻留。
4. 拼出物理地址：无TLB、各级页表不缓存时：两次表访问后再访问数据，共3次主存访问；不计缺页等额外处理。

分段、段表与段页式 182 段贴近程序逻辑，页贴近固定大小的物理分配。

分段地址与保护

分段按代码、数据等逻辑意义划分，段长可变；逻辑地址由段号与段内位移组成，各段内连续、段间可离散。段表项保存基址、长度和相关权限，地址转换先查段号是否越界，再检查偏移是否小于该段长度，最后相加得到物理地址。

段便于按逻辑单位共享、保护和增长，但可变长段的物理安置容易出现外部碎片；分页通常没有外部碎片、最后一页可能有内部碎片，用户通常不感知页的分界。

段页式转换

先把程序分段，再将各段分页；逻辑地址分为段号、段内页号、页内偏移。段表项指向本段页表并记录页表长度；检查段号 → 查段表 → 检查页号 → 查本段页表 → 取页框号与偏移组合。无TLB的典型单层段表加单层页表模型共三次访存。

段页式用户地址仍按段及段内位置表达，段内位置再由系统拆为页和偏移。TLB可用段号与页号等信息作为查询关键字，命中时直接得到目标页框。

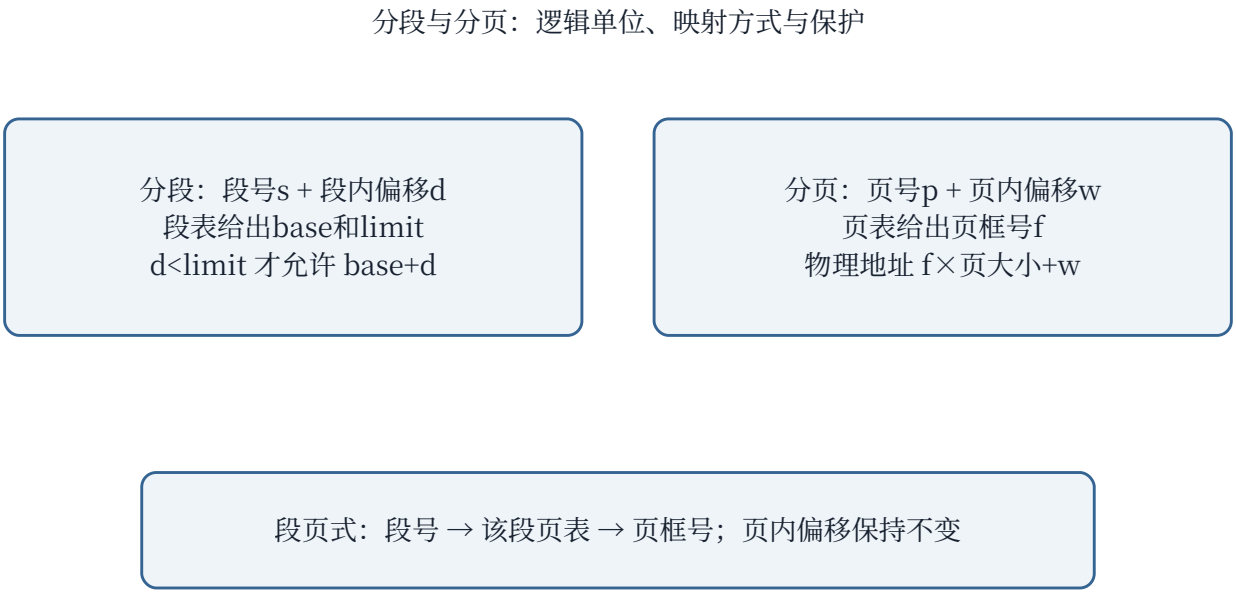
分段表项与页表项

页大小固定，所以基本分页页表项通常不需为每页记录长度；段长可变，段表必须记录边界。基本分页与基本分段都可引入TLB，不能以“是否有快表”作为两者的本质区别。

结构对照图：分段与分页：逻辑单位、映射方式与保护

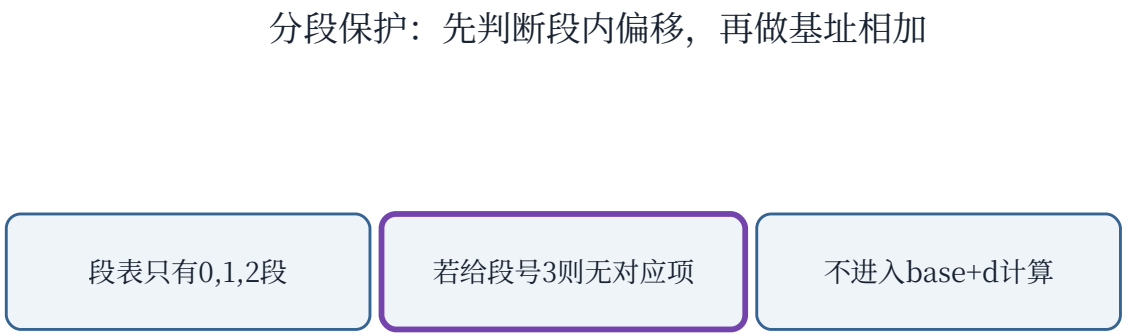
分段按逻辑模块划分，段长可变；分页按固定大小划分。段内越界先判断，不能直接拿基址加偏移而跳过保护。

段页式把段内地址继续分为页号和偏移，每段通过自己的页表映射到物理页框。所需访存次数取决于TLB及各级表缓存等条件。



单独查看图片

图解与逐步推演：分段保护：先判断段内偏移，再做基址相加



单独查看图片

分段保护：先判断段内偏移，再做基址相加

查看完整步骤（也可离线阅读）

1. 段表项：limit在本例定义为段长，因此合法偏移是0≤d<1000。若题设limit为最大合法偏移，上界比较要随之改变。

2. 访问(2,600)：段号有效且访问权限允许，先通过界限检查再形成地址。
3. 访问(2,1000)：端点偏移等于段长时已经超出本段，不能把求得数值地址当成访问合法的证据。
4. 段号越界另行检查：段号范围与段内偏移是两道检查；合法地址还必须满足读写执行权限。

虚拟内存的局部性、特征与容量 183 部分装入和按需置换打破传统一次性、驻留性的限制。

时间与空间局部性

时间局部性：近期访问的指令或数据很可能再次访问，循环是典型来源；空间局部性：访问某单元后，邻近单元也可能很快被访问，顺序指令和数组遍历体现这一规律。二者使只保留活跃部分也能有效推进成为可能。

传统全部装入模式具有一次性、运行期驻留等限制；虚拟存储通过请求调入与置换，从逻辑上提供比当前物理内存更大的地址空间。其多次性允许分批装入，对换性允许运行时换入换出，虚拟性体现逻辑扩展。

支持与边界

实现可基于请求分页、请求分段或请求段页式，需地址变换、缺失异常处理、外存后备和置换管理协同。虚拟容量受虚拟地址位数、体系结构与后备存储等约束，不是无限大；物理内存仍限制同时活跃数据的容纳量。

虚拟内存不意味着CPU直接把磁盘当内存访问；需要的页/段先调入物理内存，指令再完成原访存。

请求分页的表项、缺页处理与指令重启 184 先分清地址无效、权限不足与合法页面不在内存。

新增字段

字段	作用	用途
存在/有效状态	页面当前是否驻内存或映射是否有效	页不存在与地址非法需结合体系结构语义区分
访问字段A	近期是否访问、次数或时间信息	为置换提供依据
修改位M	调入后是否被写过	决定换出是否需保留新内容
外存位置	页后备数据位置	用于请求调页

缺页服务步骤

- 访问合法映射但页面不驻内存，触发可修复缺页故障，保存必要执行现场。
- 找到所需页的后备数据，取得空闲页框；若没有，按置换策略选可换页。
- 被换页有未保存修改时先写回；干净且后备完整者可直接丢弃其物理副本。
- 把所需页读入页框，更新页表与相关TLB状态，再恢复并重启引发缺页的指令。

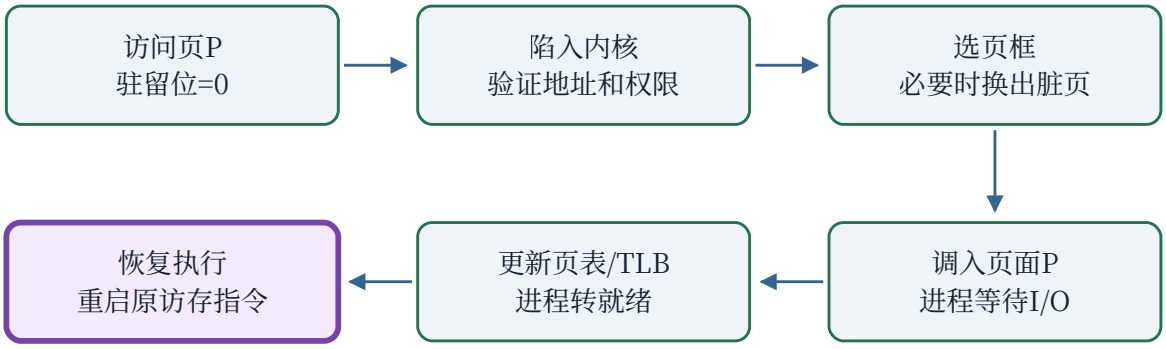
TLB与缺页边界

有效TLB翻译指向驻留页时，不应再发生“该页不在内存”的缺页；TLB miss后查到驻留页仅是翻译未缓存。置换必须使旧翻译失效，避免TLB还指向已被重用的页框。

一条指令可访问指令页、源操作数页和目标页等多个位置，因此可能在重启后又缺另一页。缺页不是等待下一条指令才处理的普通外设中断，重启语义应保证指令最终效果正确。

图解与逐步推演：缺页处理：从异常到重新执行

缺页处理：从异常到重新执行



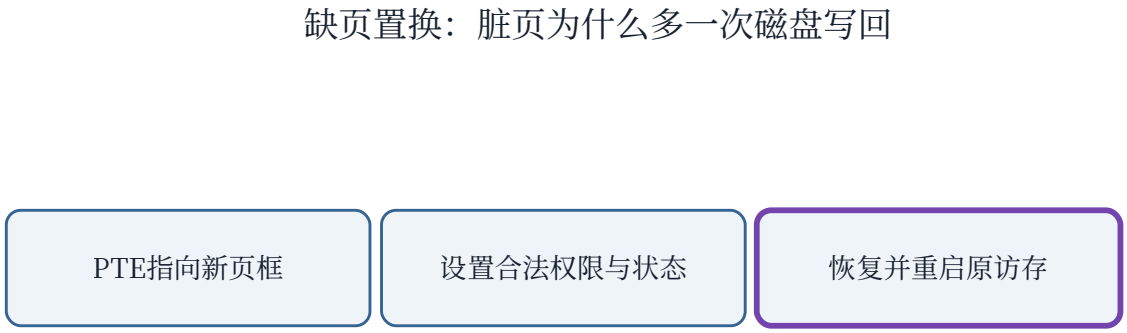
单独查看图片

缺页处理：从异常到重新执行

查看完整步骤（也可离线阅读）

1. 访问页P · 驻留位=0：合法虚拟页尚未驻留，触发缺页异常。
2. 陷入内核 · 验证地址和权限：先区分合法但不驻留与非法地址/保护错误；后者不能按正常调页继续。
3. 选页框 · 必要时换出脏页：有空闲页框就直接用；无空闲页框才选择牺牲页，脏页必要时写回。
4. 调入页面P · 进程等待I/O：访问磁盘的调页可阻塞当前进程；其他进程可以被调度。
5. 更新页表/TLB · 进程转就绪：I/O完成后更新映射和状态，并按需要使相关TLB项失效。
6. 恢复执行 · 重启原访存指令：获得CPU后重启导致缺页的指令；缺页服务时间与普通内存时间相差很大。

图解与逐步推演：缺页置换：脏页为什么多一次磁盘写回



单独查看图片

缺页置换：脏页为什么多一次磁盘写回

查看完整步骤（也可离线阅读）

1. 新页P不驻留：假定P合法但不驻留，需要把P调入一个页框。
2. Q的修改位M=1：若Q为干净页且有可复用磁盘副本，通常可省略本次写回；具体文件映射与后备存储策略另看题设。
3. 撤销旧映射并调入P：旧页框即将复用，不能让Q旧映射继续指向其中的新数据；更新顺序须保证映射一致。
4. 建立P的新映射：一次缺页服务可能包含脏页写出和新页读入两次外存传送，不能只按一次普通主存访问算。

请求分段、共享段计数与保护 185 分段也可按需调入，并按逻辑单位管理共享与访问权。

请求段表与转换

请求段表除基址、段长外，可含存取方式、访问字段、修改位、存在位、增补/增长标记和外存始址。访问合法但不驻留的段时，先调入该段并更新段表，再完成地址转换。

段保护包括段号及段内偏移的越界检查、读写执行等存取控制，也可使用环保护等体系结构机制。是否动态增长及增长后如何重定位，由管理策略决定。

共享分配与回收

系统可设共享段表记录段长、物理起点、存在状态、后备位置和共享计数；各进程在自己的段表中建立映射，所以同一物理共享段在不同进程中可以有不同段号。

首次共享者申请时建立物理段和共享表项，计数置1；后续映射成功计数增加。进程取消共享时撤其映射并减少计数；只有没有任何使用者且满足其他回收条件时才可释放共享物理区，不能因某一进程退出便删掉大家仍使用的数据。

驻留集、分配与置换范围 186 分配回答有多少页框，置换范围回答可淘汰谁。

独立的两组概念

驻留集是为进程实际保留的物理页框集合，大小可能小于完整虚拟地址空间。固定分配使配额运行期间基本不变；可变分配允许按缺页等情况增减。局部置换只在本进程驻留集中选受害页；全局置换可影响其他进程或使用全局空闲池。

常见组合

组合	缺页时处理	权衡
固定分配＋局部置换	在固定驻留集内换页	隔离较好，但初始配额难定
可变分配＋全局置换	可取空闲框，也可换其他进程的可换页	适应性强，进程间相互影响
可变分配＋局部置换	缺页时局部替换，同时另按策略调整配额	按缺页频率增减框数，控制复杂度更高

固定与全局的教材约定

典型教材不采用“固定配额且任意全局夺取”作为独立稳定组合，因为夺取会改变其他进程驻留集。具体系统可有更复杂配额约束，做题按给出的固定/可变和局部/全局定义判断。

调页时机、工作集与抖动 187 调得太少会频繁缺页，调得太多又挤占其他进程。

何时与从何处调入

请求调页在确实缺页时调入，信息明确但发生等待；预调页根据预测提前装入可能需要的页，减少连续缺页但有预测失败和多余I/O风险。首次装入可选预调，运行期也可按实现预取，不把两者机械限定为只能发生在某一阶段。

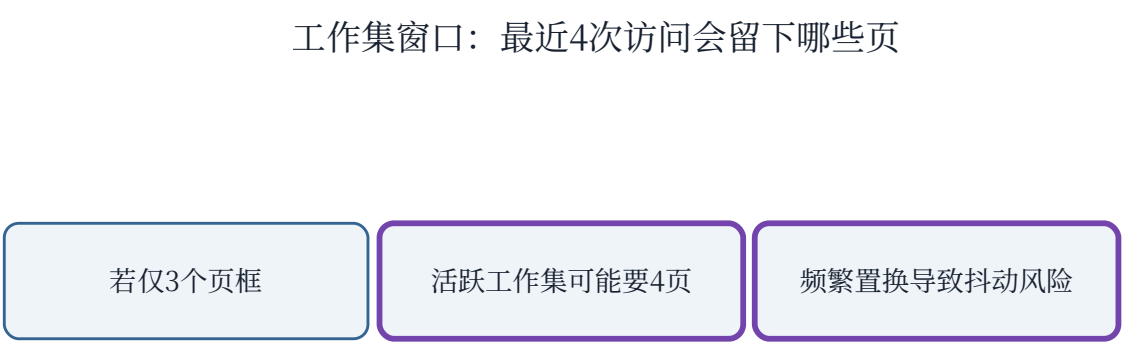
后备可来自原文件或交换区。已有完整未修改文件副本的干净页可重新从文件读入；匿名或修改后需要保留的数据通常要有适当交换/后备位置。对换区是否连续、更快等属于教材模型或具体布局，不是磁盘分区名称本身赋予的性能。

工作集与抖动

工作集W(t,Δ)是最近一个窗口内实际引用过的页面集合；驻留集是当前实际留在物理内存的集合。二者可不同，工作集大小也随执行阶段和窗口变化。

分给进程的页框不足以容纳活跃工作集时，刚换出的页很快又要调回，形成频繁调页/换出的抖动。CPU利用率下降若盲目增加多道程序度，会进一步挤压内存，恶化抖动；可降低多道度、增配框数、按工作集或缺页率控制。

图解与逐步推演：工作集窗口：最近4次访问会留下哪些页



单独查看图片

工作集窗口：最近4次访问会留下哪些页

查看完整步骤（也可离线阅读）

1. 最近访问1,2,3,1：工作集按访问历史窗口定义，不是简单最近新装入的页。
2. 下一次访问4：窗口向前推进一次，取窗口中的不同页集合。
3. 下一次再访问4：页2退出窗口；窗口中访问次数不等于不同页个数。
4. 驻留集小于活跃需求：工作集用于判断局部性与内存需求；具体是否抖动仍取决于实际访问序列和分配策略。

OPT、FIFO与LRU页面置换 188 未来、进入时间、最近访问时间，是三种不同的排序依据。

选择受害页

算法	淘汰规则	特点
OPT	未来最久才使用或不再使用的页	给定引用串下缺页最少的比较基准，运行前通常不知道未来
FIFO	最早进入内存的页	实现简单，命中通常不改变入队顺序；可能Belady异常
LRU	过去最久没有访问的页	依赖局部性，每次引用都影响最近次序；准确实现需记录访问

模拟必须记清状态

先判断命中；命中不产生换入，但LRU必须更新最近次序，FIFO不因命中把该页移到队尾。缺页而有空框时直接装入，不应计为替换；缺页数包括初次装入，置换次数不一定等于缺页次数。

FIFO可能随着页框数增加反而增加缺页，即Belady异常。OPT、LRU具有栈性质，不产生这一异常；“只有FIFO可能异常”只宜用于限定比较的常见算法，其他不具栈性质的策略也可能出现。

同一串分开记队列

引用1、2、1、3，两个空页框：前三次后FIFO进入顺序仍为1、2，LRU次序则2最旧、1最新；访问3时FIFO换1而LRU换2。若把命中后刷新顺序套到FIFO，实际上已经改成另一算法。

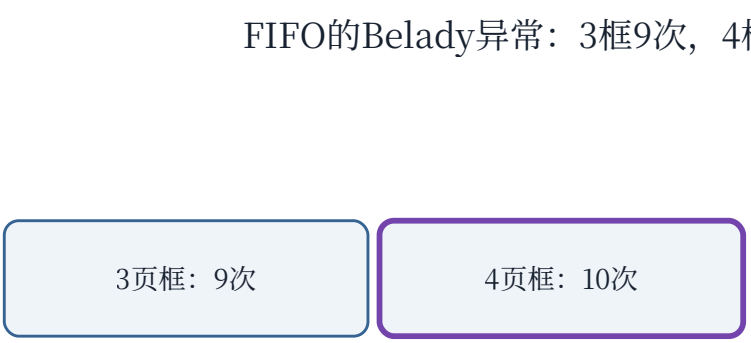
图画推演与逐步动画

例：3个页框，访问1、2、3、1、4、2

查看完整步骤（也可离线阅读）

1. 0 · 空驻留集：此图按最久未用→最近使用排列页面，位置代表新旧次序，不代表物理页框号。初始没有页面。
2. 1 · 访问1：缺页：有空页框，装入1，缺页累计1次。
3. 2 · 访问2：缺页：装入2，1比2更久未使用，缺页累计2次。
4. 3 · 访问3：缺页：装入3，3个页框已满，次序为1、2、3，累计3次缺页。
5. 4 · 再访问1：命中：页面1已在内存，不缺页，但要更新最近使用次序：2、3、1。这是LRU与FIFO的重要区别。
6. 5 · 访问4：淘汰2：4不在内存且无空页框，淘汰当前最久未用的2，装入4。次序变为3、1、4，缺页累计4次。
7. 6 · 访问2：淘汰3：2刚才已被淘汰，再访问仍缺页。淘汰最久未用的3，最终次序1、4、2，6次访问中缺页5次。

图解与逐步推演：FIFO的Belady异常：3框9次，4框为什么10次



单独查看图片

FIFO的Belady异常：3框9次，4框为什么10次

查看完整步骤（也可离线阅读）

1. 第1次访问页1：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
2. 第2次访问页2：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
3. 第3次访问页3：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
4. 第4次访问页4：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
5. 第5次访问页1：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
6. 第6次访问页2：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
7. 第7次访问页5：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
8. 第8次访问页1：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
9. 第9次访问页2：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
10. 第10次访问页3：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
11. 第11次访问页4：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
12. 第12次访问页5：FIFO只在装入时改变进入次序；命中不把页面移动到队尾。
13. 更多页框反而更多缺页：同一参考串，FIFO在3框缺页9次，在4框缺页10次，出现Belady异常。OPT和LRU具有栈性质，不出现该类异常。

Clock与改进型Clock 189 访问位给第二次机会，修改位衡量换出写回成本。

简单Clock

把驻留页组织成循环队列，设置扫描指针。页面被访问后A置1；需要受害者时，从指针处检查：A=0可选，A=1先清0并继续。若本轮全为1，先清完一圈，再次遇到0时选中；正常无并发重新置位的教材模型下至多两圈可找到。

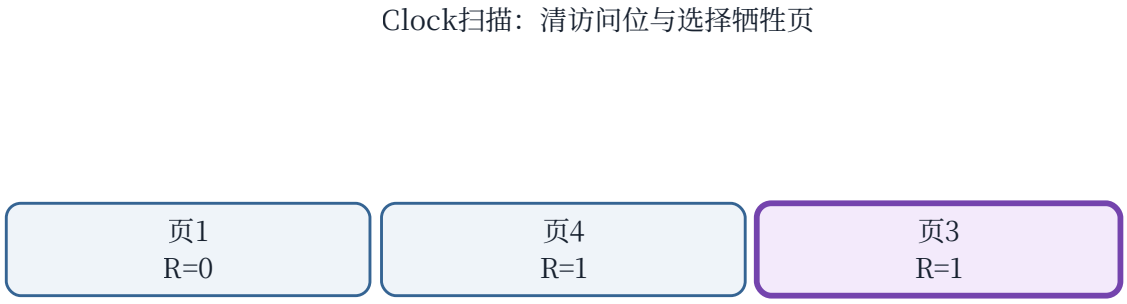
命中时设置访问位，不随意移动扫描指针；换入新页后访问位和指针下一位置依题设初始化规则，必须在模拟表里注明。

改进型Clock

同时考虑(A,M)：(0,0)近期未访问且干净，优先级最高；(0,1)近期未访问但脏；(1,0)近期访问但干净；(1,1)近期访问且脏。目标是在局部性与写回代价间折中。

常见多轮实现先找(0,0)，未找到再找(0,1)并在扫描中清访问位，必要时重复。教材也可能采用不同清位轮次，不能只按四类排序便忽略指针起点和扫描时清位动作。脏页换出写回不意味着每次写访问都立刻写盘。

图解与逐步推演：Clock扫描：清访问位与选择牺牲页



单独查看图片

Clock扫描：清访问位与选择牺牲页

查看完整步骤（也可离线阅读）

1. 检查页1：指针指页1，R=1，给第二次机会：清R并前进。
2. 选中页2：指针指页2，R=0，选它淘汰；不是继续跳到页3。
3. 装入页4：页4装入原页2的页框，完成本次访问置R=1；指针移到下一个页框。

伙伴系统与物理页框回收 190 伙伴管理空闲块，回收机制从可回收对象中补充空闲页框。

伙伴块的条件

伙伴系统按2的幂组织空闲块。请求大小向上取足够大的2^k块；若本级无空闲块，则向更大级寻找并逐级二分，分出一半供继续分配，另一半进入对应空闲链。

回收时只与同阶、相邻且来自同一父块二分的空闲伙伴合并，然后继续尝试更高阶。两个同样大且地址相邻的块不一定是伙伴；在对齐的二进制模型中可用块起址异或块大小寻找伙伴。

分配与碎片

请求20KB、最小单位1KB时，向上取32KB，产生12KB内部浪费；若32KB链为空而64KB有块，就拆成两个32KB。释放一块32KB后，只有另一伙伴也空闲才能合成64KB，不能跨父块随意拼接。

页框回收按内容处理

类型	典型处理	原因
受保护/锁定、关键内核页等	不可随意回收	不能破坏内核执行或硬件正在使用的数据
干净文件页	可丢弃物理副本，未来由文件重读	后备已包含相同内容
脏文件页	先写回适当后备，再释放	需保留未保存修改
匿名页	若可换出则写入交换后备	堆栈等没有原文件可直接重读
可回收缓存/空闲slab等	按相应缓存回收规则释放	减少可重建元数据占用

回收与置换的关系

页框回收的目标是获得可再分配的物理框；页面置换策略决定哪些虚拟页适合失去驻留副本，两者高度相关，不能硬划成互不相干的两套操作。已空闲页本来无需再“回收”；干净页丢弃虽可免写回，但未来再访问仍可能增加缺页与I/O成本。

图解与逐步推演：伙伴系统：13KiB为何要分配16KiB



单独查看图片

伙伴系统：13KiB为何要分配16KiB

查看完整步骤（也可离线阅读）

1. 初始64KiB：把物理空间抽象为[0,64)KiB一个空闲块。
2. 申请13KiB，需16KiB块：向上取2的幂：16KiB；64先拆32，再把一个32拆成两个16。
3. 申请9KiB，再取16KiB块：使用[16,32)块。两个16块互为伙伴，但有占用时不能合并。

4. 释放第一块：[0,16)变空闲，伙伴[16,32)仍占用，所以暂不能合并。
5. 释放第二块并逐级合并：两个16伙伴合为[0,32)，再与空闲伙伴[32,64)合为64KiB。

文件、目录与文件系统

操作系统 · 11 条

文件属性、FCB与索引结点 191 把文件数据、名称和管理元数据分开。

文件与属性

文件是一组具有意义的信息，支持长期保存、共享和按名访问。属性可包括名称、系统标识、类型、大小、拥有者、位置、创建/修改/访问时间和保护信息；同目录下通常不允许同名，不同目录可有同名文件。

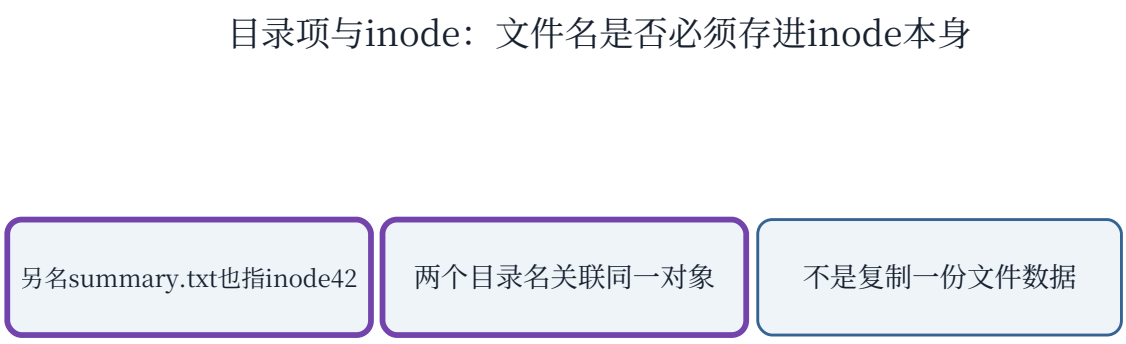
FCB保存管理文件所需的信息，包含基本属性、存取控制和使用信息。传统FCB目录中一个目录项对应一份文件控制信息；采用索引结点后，把名称以外的大部分元数据集中到inode，目录项保存名称和inode标识。

磁盘与内存索引结点

磁盘inode可记录拥有者、类型、权限、长度、地址索引、链接数和时间等；具体直接/间接地址项数量由文件系统格式规定，不能把某教材的13项当作所有inode固定长度。

内存inode还可记录引用计数、修改或锁状态、设备标识以及缓存链表/散列表指针。把目录项缩小后，一个磁盘块能容纳更多文件名，路径检索时可减少目录I/O；真正需要文件元数据时再读取相应inode。

图解与逐步推演：目录项与inode：文件名是否必须存进inode本身



单独查看图片

目录项与inode：文件名是否必须存进inode本身

查看完整步骤（也可离线阅读）

1. 目录项建立名字关联：采用常见Unix式概念模型；不同文件系统具体目录项字段可不同。
2. inode保存文件元数据：文件名属于目录命名关系，不是必须由文件inode保存的唯一属性。
3. 读取文件内容：把文件名、管理元数据与实际数据分开，可避免把FCB或inode当成文件正文。
4. 另一个硬链接名字：同一文件可以有多个目录链接；inode号的有效范围还要结合所在文件系统。

创建、打开、读写、关闭与删除 192 打开建立访问上下文，关闭清理引用，二者都不等于删除文件。

操作生命周期

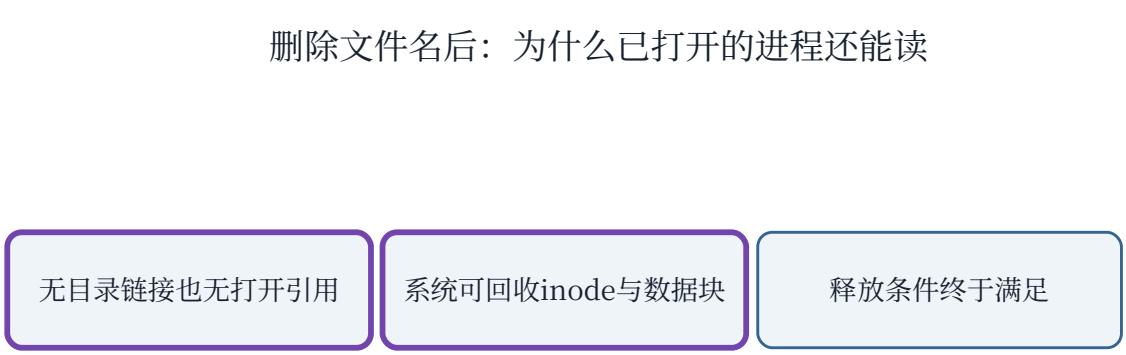
操作	主要工作	关键区别
创建	分配元数据/必要空间并建立目录项	不等于已写入大量用户数据
打开	解析路径、检查权限，建立打开状态，返回描述符/句柄	后续通常按描述符操作，不必每次重新按名检索
读/写	检查访问方式与缓冲区，按位置传送数据并更新位置	文件位置指针与数据缓存是不同对象
定位	改变当前打开上下文的文件偏移	本身不必传送文件数据
关闭	释放当前描述符及相应打开引用	一般不删除目录项和磁盘数据
删除/解除链接	移去名称引用，按链接和打开情况决定最终回收	实际空间回收可能延后

文件位置与共享边界

一个打开实例常使用同一个偏移供读和写共同推进；不同独立打开实例可有各自偏移。I/O并发是否允许、是否原子以及缓存刷新由接口规定，不能从“一个读写指针”推出全系统永远不能同时读写。

close后文件仍可存在并被再次open；unlink删除一个文件名后，已有打开引用在常见系统中仍可能继续使用文件，等最后引用消失后再回收数据。

图解与逐步推演：删除文件名后：为什么已打开的进程还能读



单独查看图片

删除文件名后：为什么已打开的进程还能读

查看完整步骤（也可离线阅读）

1. 打开已有文件：常见Unix式语义示例；忽略权限错误与并发修改。
2. 执行unlink(a)：删除目录名不等于立刻使所有打开描述符失效。
3. 进程继续通过fd3访问：重新open名字a通常找不到旧对象；它与已经打开的fd3访问路径不同。
4. 关闭最后打开引用：回收时点和实现可能受延迟分配、缓存等影响；判断题的关键是同时区分链接计数与打开引用。

进程打开表、系统打开表与inode 193 用“描述符→打开实例→文件对象”避免把计数和偏移混放。

三层关系

进程打开文件表把文件描述符映射到系统维护的打开实例；系统打开实例记录访问模式、当前位置及对文件对象的引用；inode/FCB缓存提供文件元数据和数据定位。教材有时把系统打开表与内存FCB简化为一层，分析时应先声明所用模型。

同一文件对象不等于只有一个打开实例。独立open常建立独立偏移，dup或fork继承可让多个描述符引用同一打开实例并共享偏移。

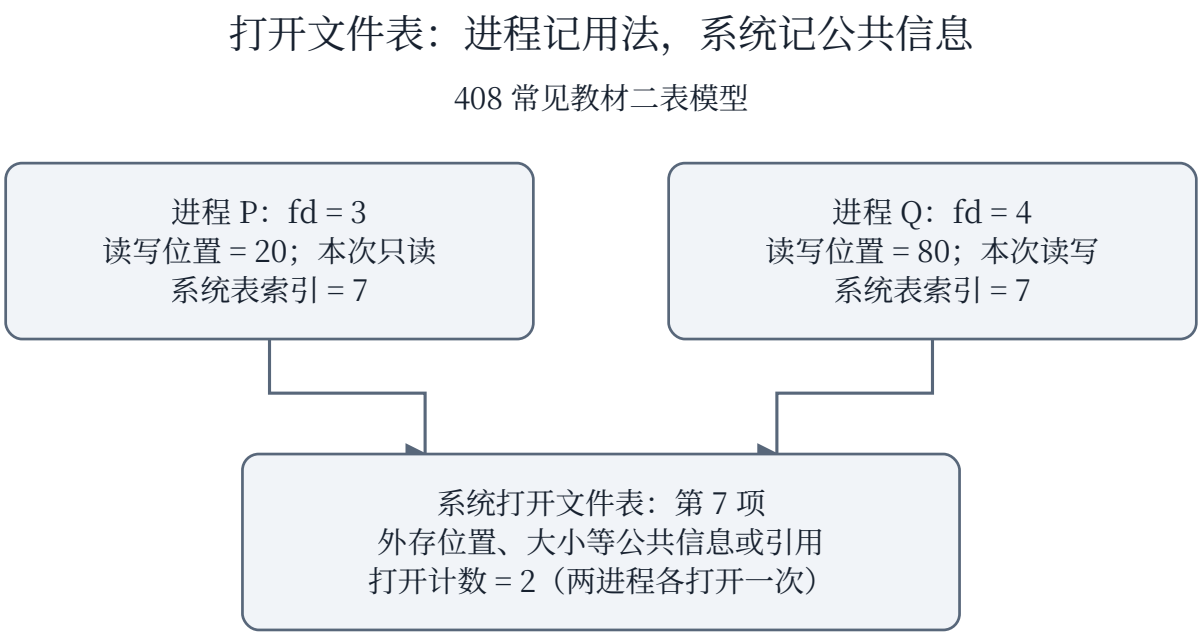
共享偏移与独立偏移

A对文件独立open两次得到fd1、fd2：二者可指向同一inode但拥有不同打开实例，fd1读10字节后fd2仍可从自己的原偏移读。若fd2由dup(fd1)得到，它们共享打开实例，fd1读后会推动fd2看到的共同偏移。

三种数量分别数

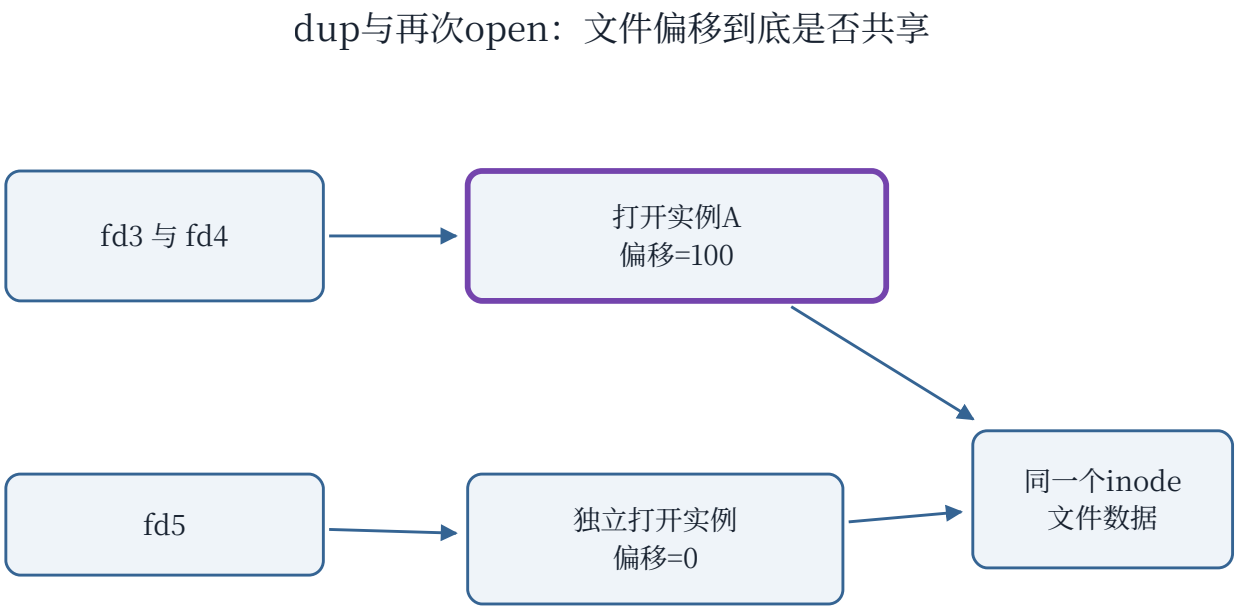
目录硬链接数反映多少目录项指向文件对象；打开实例引用数反映多少描述符等引用同一打开实例；内存inode引用/打开计数反映内核对象被使用情况。删一个名称、关一个描述符和最终回收inode/数据块不是同一个时刻，也不应只用一个count混算。

机制图解



单独查看图片

图解与逐步推演：dup与再次open：文件偏移到底是否共享



单独查看图片

dup与再次open：文件偏移到底是否共享

查看完整步骤（也可离线阅读）

1. open一次：进程open文件得到fd3，系统打开文件实例记录共享偏移0。
2. dup(fd3)→fd4：复制描述符指向同一个打开实例，因此fd3、fd4共享偏移。
3. read(fd3,100)：假设成功读100B，共享偏移从0变100，fd4随后读取也从100继续。
4. 再次open得到fd5：新open创建另一个打开实例，偏移从0开始，仍可引用同一inode。

口令、加密与访问控制 194 身份、授权和内容保密解决不同问题。

保护方式

方式	原理	优缺点和边界
口令保护	访问时提交约定凭据	实现简单但共享口令难区分用户；安全实现不应明文保存口令
加密保护	用密钥变换文件内容，授权方解密	提高保密性，增加密钥管理与编解码开销
访问控制	按主体、对象及操作权限决定是否放行	可用访问矩阵、ACL、拥有者/组/其他用户权限等组织

目录权限不是文件权限的简单复制

路径访问要通过各级目录的搜索/执行等权限检查，再检查目标文件所请求操作的权限。不能一概说“不能列目录就不能访问其任何已知名字的文件”，因为列目录和穿越目录在具体系统中可为不同权限。

加密并不自动解决删除、篡改、共享管理和密钥泄露；ACL也不保证磁盘被离线读取时内容仍保密，两者可共同使用。

文件逻辑结构：流式、顺序、索引与索引顺序 195 逻辑结构是用户如何组织信息，物理分配是系统如何安放块。

主要结构

结构	组织与查找	增删和空间
无结构/流式	看作字节序列，记录边界由应用解释	灵活，不是“没有任何内部含义”
顺序记录	串结构不按关键字，顺序结构按关键字；定长记录可计算位置	适于顺序处理，插删维护可能代价大
索引文件	每记录有索引项，表项定长并指向记录	支持变长记录定位，索引占空间
索引顺序文件	记录分组，每组一个索引项；组间关键字有序	先定位分组再组内查找，兼顾索引空间与检索速度

随机定位的条件

定长记录的偏移可由记录号×记录长度得到；变长顺序记录通常不能仅凭记录号直接算偏移，需扫描长度或辅助索引。按关键字排序且记录定位方便时可用折半等方法；链式物理组织与逻辑顺序组织不能不加区别地套同一查找复杂度。

索引表本身是定长项组织，若表太大可分组或分级。索引顺序的组内关键字可不完全有序，但组间边界要支持按关键字确定候选组。

连续文件分配与隐式链接 196 连续块便于直接定位，链式块便于增长，代价各不相同。

连续分配

FCB记录起始块号与块数。逻辑块i合法时，物理块=起始块+i，块内偏移不变；需检查0≤i<文件块数。连续区域有利于顺序和随机访问、减少机械寻道。

文件增长若相邻空间已被占用，可能要移动或增加其他扩展机制；反复增删产生外部碎片，预留过多空间又浪费。长度不一定必须永久不变，但传统单连续区域模型难以灵活扩展。

隐式链接

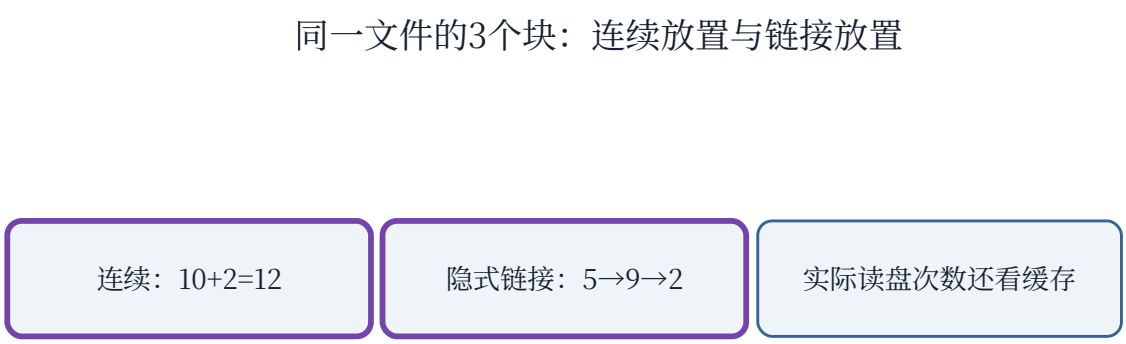
每个数据块中保存后继块指针，目录保存首块，常附尾块便于追加。文件块可离散，增长容易且无外部碎片；末块仍可能有内部浪费，指针也占数据容量。

定位第i个逻辑块需沿链依次读前面的块，不支持高效的直接随机定位。指针损坏可断链；不能把“离散无外部碎片”说成任何空间浪费都没有。

计I/O先列已有条件

要读逻辑块3（从0计），若仅首块号在内存且没有缓存，需先读块0、1、2获取后继，再读块3，共4次数据块读取；若FCB、首块或部分链已缓存，次数按题设减少。

图解与逐步推演：同一文件的3个块：连续放置与链接放置



单独查看图片

同一文件的3个块：连续放置与链接放置

查看完整步骤（也可离线阅读）

1. 连续分配：起始物理块10，共3块；逻辑块0、1、2直接映射到物理10、11、12。
2. 隐式链接分配：同样3个逻辑块可以存物理5→9→2。下一块指针随数据块保存，不是FAT统一外置表。
3. 随机访问的差异：连续分配可由起始块号加逻辑块号直接定位；隐式链接要沿前面块的指针查找，顺序访问更自然。

FAT显式链接与表容量 197 全卷共享一张分配表，但每个文件有自己的块链。

FAT的组织

显式链接把各物理块的后继指针集中到文件分配表FAT，目录项保存文件首块号，FAT[当前物理块]给下一物理块。表还可编码空闲、坏块和链末等状态。

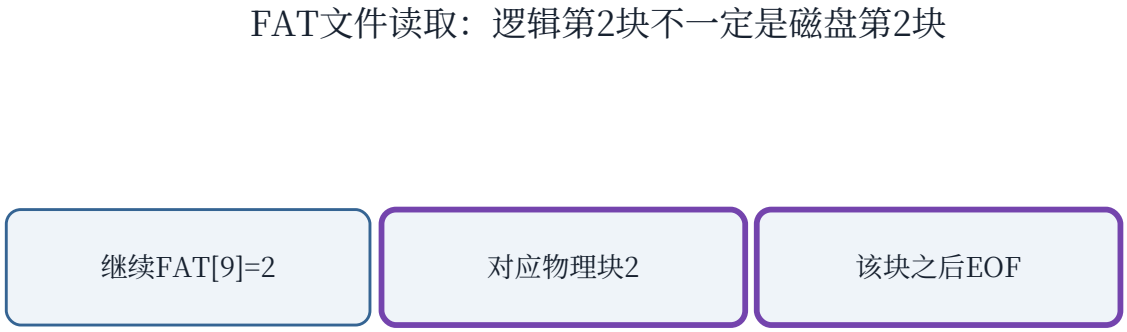
教材常假设FAT读入内存，此时沿链的地址查询不用每跳读一次磁盘，但仍要在表中逐项走链，不等于像数组那样O(1)直接算出任意逻辑块。若表本身不全在内存，还需考虑缓存和表页I/O。

容量与索引计算

若卷上有N个可编号簇，仅表达编号至少需$\lceil \log_2 N \rceil$位；实际表项还需预留特殊状态编码并按格式分配字节数。FAT空间约为表项数×每项长度，多副本和保留项由题设另计。

卷大小/簇大小决定簇数，而不是文件大小决定整张FAT长度。查某文件占多少簇，可沿其链统计；不要把整个卷FAT条目数误当作每个文件都拥有的索引项数。

图解与逐步推演：FAT文件读取：逻辑第2块不一定是磁盘第2块



单独查看图片

FAT文件读取：逻辑第2块不一定是磁盘第2块

查看完整步骤（也可离线阅读）

1. 目录起始块为5：文件块链为5→9→2→结束；逻辑块号从0起。
2. 逻辑块0：文件目录给起始块，不需要先从物理块0扫描到它。
3. 逻辑块1：FAT表项记录链中下一物理块号，不是下一字节地址。
4. 逻辑块2：逻辑连续不要求物理连续。若FAT已整体在内存，沿链查FAT不必每次都读一次磁盘；缓存条件决定实际I/O数。

索引分配、多级索引与混合索引 198 一文件一组索引映射，通过级数换取大文件容量。

索引块与扩展

索引分配在文件的索引块中按逻辑块顺序保存物理数据块号，FCB指向顶级索引。数据块可离散，随机定位比链式便捷，但索引也占盘块，小文件可能浪费索引块。

索引块链接方案允许索引表扩展，但找后面的索引块仍需沿链；多级索引通过顶级表索引下级表；混合索引同时提供直接地址和一级、二级等间接入口，让小文件用较少I/O，大文件仍可扩展。

容量公式

访问直接块不需读索引块；一级间接需读1个索引块，二级需沿路径读2个，再读目标数据。FCB、inode或索引已缓存会减少I/O，读整个文件的总索引块数不能用“每块最大级数”机械相乘。

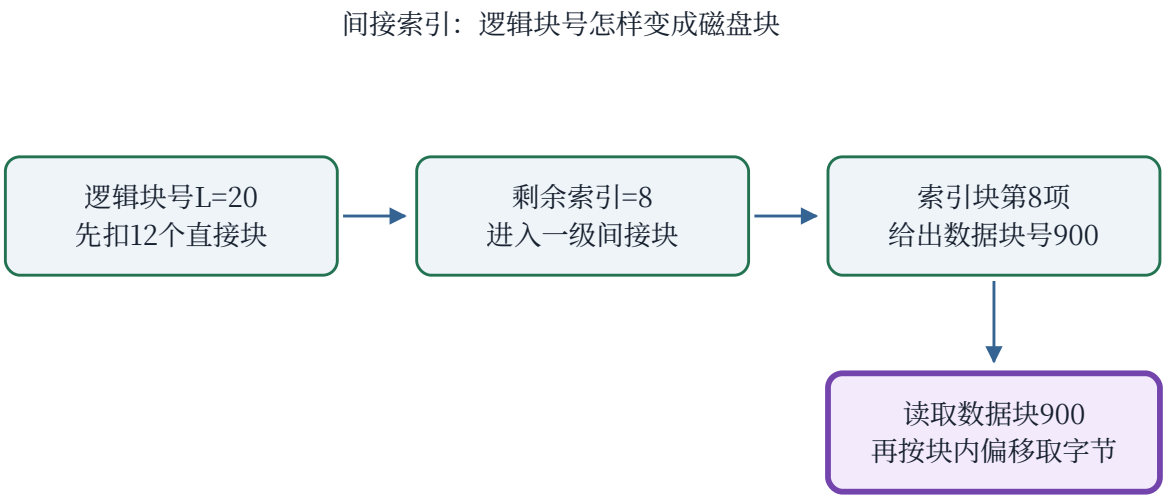
$n=B/a, C=B(d+n+n^2+n^3)$

B为块字节数，a为地址项字节数，d个直接项，另有一个一级、一个二级和一个三级间接入口；仅计数据容量，地址格式按题设。

边界定位

若块1KB、地址4B，则每索引块256项。10个直接项后，逻辑块0–9直接；10–265经一级间接；266起进入二级间接。先减去较低级覆盖量，再用商和余数确定二级路径，最后保留文件字节的块内偏移。

图解与逐步推演：间接索引：逻辑块号怎样变成磁盘块



单独查看图片

间接索引：逻辑块号怎样变成磁盘块

查看完整步骤（也可离线阅读）

1. 逻辑块号L=20 · 先扣12个直接块：示例inode有12个直接指针，一个一级间接指针；4KiB块、4B块号，每个间接块存1024项。
2. 剩余索引=8 · 进入一级间接块：L=20超出直接范围0～11；20−12=8（下标从0起）。
3. 索引块第8项 · 给出数据块号900：先读一级间接索引块，查第8项取得数据块号900。
4. 读取数据块900 · 再按块内偏移取字节：若inode已在内存而间接块和数据块都不缓存，本次需要2次磁盘块读取；题目缓存条件会改变计数。

目录结构、路径解析与目录操作 199 路径中的每一级名称，都需在相应目录中找到。

树与无环图

树形目录层次清楚、方便分类和访问控制，不同目录可重名；绝对路径从根开始，相对路径从当前工作目录开始。检索逐级进行，中间目录是否已缓存会影响磁盘访问次数。

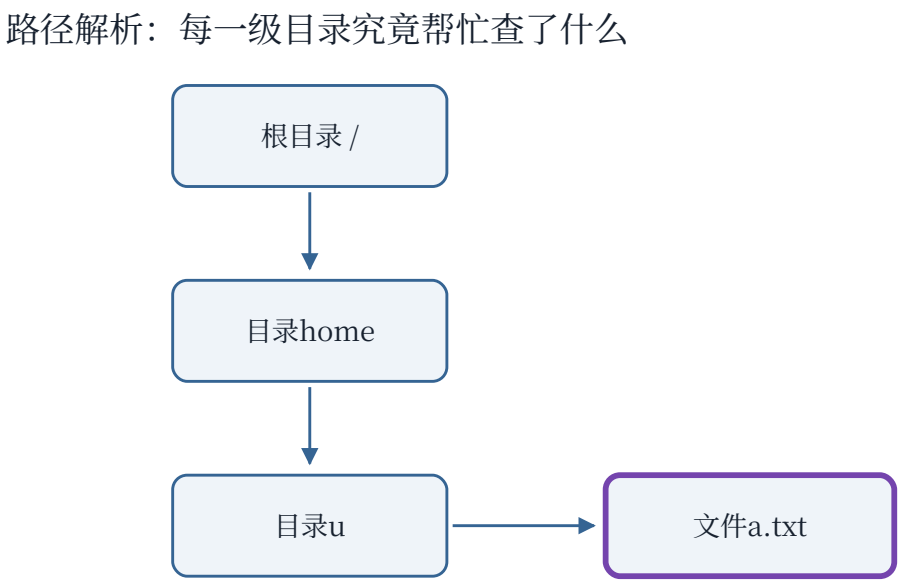
无环图目录允许多个目录项指向同一共享对象，支持共享而不复制数据。用引用计数可辅助回收；设计必须避免引入环或另配环检测/垃圾回收机制。

目录操作与缓存

搜索找到文件目录项；创建插入新条目；删除移除相应名称；显示枚举目录内容；重命名/移动改变名称或父目录关系；修改属性更新相关元数据。目录本身也是由文件系统组织的对象，不是仅存在于屏幕上的文件夹。

目录项缓存和inode缓存可以减少路径查找I/O，但缓存命中与是否已经open是两件事。计算路径代价时先列根目录、各级目录块和目标inode哪些已在内存，再逐级计数。

图解与逐步推演：路径解析：每一级目录究竟帮忙查了什么



单独查看图片

路径解析：每一级目录究竟帮忙查了什么

查看完整步骤（也可离线阅读）

1. 从根开始解析：绝对路径/home/u/a.txt先在根目录中查找名字home。
2. 逐级进入目录：在home目录查u，再在u目录查a.txt；每一级名字都由对应目录的内容解释。
3. 最终定位文件：目录项a.txt给出相应文件对象标识，再依据inode等元数据定位文件数据。

硬链接、符号链接与共享计数 200 共享保留一份数据，复制则形成独立副本。

两类链接比较

属性	硬链接	符号/软链接
指向对象	目录项直接引用同一文件对象/inode	链接自身是文件，内容通常是目标路径

属性	硬链接	符号/软链接
对象数	多个名称共享同一数据和元数据对象	链接文件与目标是两个对象
删除一个名称	减少链接数，其他硬链接仍可使用	删除目标后可留下悬空符号链接
跨文件系统	通常受同一文件系统限制	目标路径可跨文件系统
目录链接	普通用户创建常受限制，避免环	可指向目录，但解析要防循环

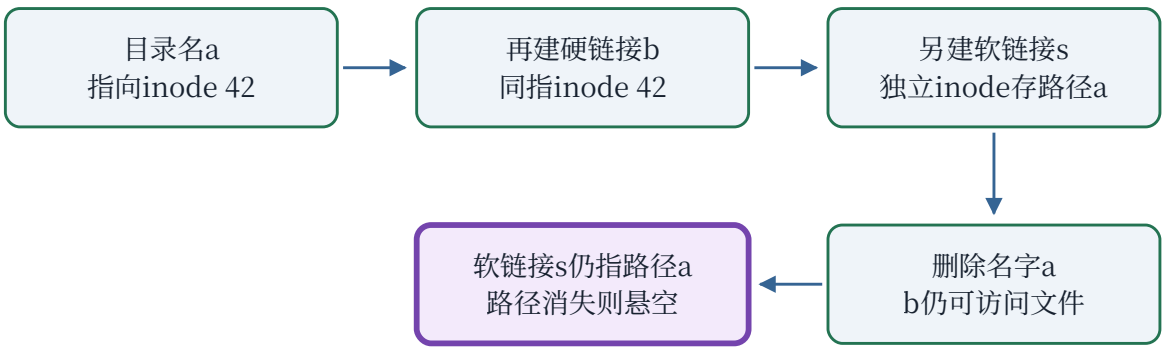
谁拥有链接数

硬链接计数统计名称引用，不是“有几个用户打开”。用户删除原文件名不表示有权立即删除其他硬链接指向的数据；系统还需考虑最后硬链接和打开引用。

符号链接不会因为目标不存在而自动消失，访问可失败而链接仍在。共享写会被其他共享访问者看到；复制后修改自己的副本通常不改变其他副本。

图解与逐步推演：硬链接和软链接：名字指向哪里

硬链接和软链接：名字指向哪里



单独查看图片

硬链接和软链接：名字指向哪里

查看完整步骤（也可离线阅读）

1. 目录名a · 指向inode 42：目录项把文件名与inode对应，文件内容由inode关联数据块。
2. 再建硬链接b · 同指inode 42：硬链接数增加；a、b是同一文件的两个名字，地位平等。
3. 另建软链接s · 独立inode存路径a：符号链接保存目标路径字符串，拥有自己的inode；不是把目标数据复制一份。
4. 删除名字a · b仍可访问文件：删除一个名字会减少链接数；只要还有b，目标文件仍有目录链接。

5. 软链接s仍指路径a · 路径消失则悬空：软链接按路径解析，删除a后s可能失效；b不因a删除而失效。最终释放还需考虑打开引用等条件。

文件系统磁盘布局、内存结构与空闲块管理 201 把持久结构、运行缓存和空闲空间记录分开。

内外存结构

文件卷可由物理分区等承载，也可通过卷管理组合多个设备。持久布局包括引导/卷描述信息、空闲空间管理信息、目录与inode等元数据，以及文件数据区域；具体不必严格固定为两块连续“目录区+数据区”。

运行时内核维护目录缓存、inode/文件对象缓存、系统打开实例和进程描述符表等。缓存是持久内容的内存副本或运行状态，关机后不能把它当作唯一长期记录。

空闲表和链表

空闲表记录各连续空闲区的起点和长度，适于连续分配；申请可用首次、最佳等规则，回收按前后相邻情况合并。空闲盘块链逐块链接，分配摘下足够块，回收重新链接；空闲盘区链按连续区链接，批量申请较高效，并在回收时尽可能合并相邻区。

位示图与编号

位示图每块一位，0空闲/1占用是本资料约定，题设相反则反转。申请找足够的空闲位，换算块号后置占用；回收按块号定位位并清空。存N块需要⌈N/8⌉字节位图，若按整字存储还需按字长向上取整。

$b=iw+j, i=\lfloor b/w \rfloor, j=b \bmod w$

b为从0编号的块，i为从0编号的机器字，j为从0编号的位，w为每字位数；若编号从1开始，应先减1换算再加回。

图解与逐步推演：空闲块位图：分配与释放到底修改哪几位

空闲块位图：分配与释放到底修改哪几位



单独查看图片

空闲块位图：分配与释放到底修改哪几位

查看完整步骤（也可离线阅读）

1. 空闲位图：本例约定0为空闲、1为已分配，块号从0起。不同系统可能反过来规定。
2. 申请两个空闲块：从低号起扫描，先找到块0与块2；它们不必连续，是否允许非连续由分配方式决定。
3. 更新位图：把0和2对应位从0改成1，防止随后重复分配。
4. 释放块4：把块4的分配位清为0，返回空闲集合。

I/O、设备、缓冲与外存

操作系统 · 13 条

设备分类、控制器与I/O端口 202 设备、接口、端口和驱动程序分别属于不同层次。

设备分类

按共享属性分独占与共享；按数据交换单位分字符设备和块设备；按用途分人机交互、存储、通信等；按速率分高速、中速、低速是相对分类。典型磁盘块设备支持块定位，典型终端字符设备按流交换，不能由“字符设备”推出所有实现都必须物理独占。

控制器的组成与作用

设备控制器连接主机总线和外设，含主机接口、设备接口和I/O逻辑，可接收译码命令、交换数据、报告状态、识别地址、缓冲和检测错误。一个控制器可以连接多个设备，设备编号与控制器端口地址不是同一个概念。

I/O端口通常指可被CPU访问的接口寄存器：数据寄存器缓冲数据，状态寄存器报告准备/错误，控制寄存器接受命令。多个端口及控制逻辑组成接口；不能只因CPU访问了状态端口便认为数据已经传完。

独立编址与内存映射

独立编址使用单独I/O端口空间及专用指令；内存映射I/O把寄存器放入统一地址空间，可用相应访存指令访问，但访问权限、缓存属性和顺序仍由系统控制。端口映射区不能同时当作普通内存使用，具体地址未必都在最高端。

轮询、中断、DMA与通道控制 203 比较CPU在哪里参与，而不只比较是否用了中断。

控制方式

方式	CPU参与	数据路径与特点
程序直接控制/轮询	发命令后反复检查状态，按单位搬数据	CPU开销大，典型模型中与设备传送串行
中断驱动	启动后可做其他工作，设备就绪后中断服务	数据常经CPU寄存器，减少忙等但有频繁中断开销

方式	CPU参与	数据路径与特点
DMA	CPU设置起址、方向、计数等，控制器搬一批，完成后通知	设备与内存直接交换，数据不逐字经过CPU寄存器
通道	CPU启动通道程序，通道执行一系列I/O控制	更强I/O处理实体，可组织多设备操作，硬件与管理更复杂

DMA寄存器与周期

命令/状态寄存器记录控制与状态，内存地址寄存器记录下一传送地址，数据计数器记录剩余字/字节数，数据寄存器可临时缓冲数据。每次传送相应更新地址和计数，结束后向CPU报告。

CPU和DMA共享总线/主存时会有总线仲裁与访存竞争。DMA请求可在适当总线周期边界处理，不等于普通中断响应只能在指令边界进入服务例程。DMA完成可发中断，所以“DMA不用CPU逐字搬”并不等于“DMA完全不使用中断”。

并行与进程阻塞

CPU发I/O后，发起进程是否阻塞取决于接口语义；它阻塞时CPU可执行别的进程。不要把“同一进程等待I/O”和“整个CPU与所有设备不能并行”混为一谈。现代控制器能力也可能超出教材简化的一控制器一设备模型。

I/O软件分层与设备独立性 204 上层提出统一请求，下层处理具体硬件差别。

自上而下的层次

层次	工作
用户层I/O软件	库函数、格式转换、用户接口，通过系统调用请求内核
设备独立性软件	统一命名、保护、分配回收、缓冲、通用差错处理及逻辑块接口
设备驱动程序	将抽象请求翻译为具体控制器命令，检查设备状态、设置寄存器
中断处理程序	对设备中断保存必要现场、识别状态、完成处理并唤醒等待者
控制器与物理设备	执行数据传送和硬件操作

逻辑设备名与统一接口

应用使用逻辑设备名或统一接口，不依赖某个物理设备的硬件细节；系统映射到可用物理设备和相应驱动，便于更换设备与I/O重定向。设备独立不表示驱动与硬件无关，而是硬件相关细节被封装在下层。

驱动与内核之间应有规范接口，新设备通过相应驱动接入，避免每来一台新设备都改全部上层程序。某些简单硬件差错由低层处理，较通用的资源、缓冲和命名由较高层处理，具体界限仍依操作系统实现。

结构对照图：I/O软件分层：请求向下，完成通知向上

图按概念层次展示，不表示每个请求都必须同步进入“中断处理程序”再访问硬件。驱动启动设备，设备完成后产生中断，由处理程序进入完成路径。

设备独立层使上层不必掌握每种控制器的寄存器细节；具体缓冲、调度和错误处理可以跨层实现。

I/O软件分层：请求向下，完成通知向上



单独查看图片

字符、块与网络接口；阻塞与非阻塞 205 接口抽象与调用返回方式是两条独立的分类轴。

应用接口

字符接口以字节/字符流为中心，常见get/put或read/write；块接口把设备抽象为按逻辑块编号的序列，将上层块请求映射到底层操作，屏蔽几何结构；网络接口常以socket建立通信端点并发送接收数据。

文件系统可在块设备接口之上提供按名文件访问，因此“对文件read”与“直接对盘块read”不能直接视为同一层。设备典型使用中断或DMA是性能选择，不是由接口名字唯一决定。

阻塞、非阻塞和异步

阻塞I/O在请求暂时不能完成时使调用线程等待，事件就绪后被唤醒；非阻塞I/O立即返回已完成量或暂不可完成状态，应用可稍后重试或借助事件通知。非阻塞不表示必然返回错误，也不等于完整的异步I/O。

阻塞是调用者状态，中断/DMA是底层传送控制；可同时存在阻塞read和DMA传送。多个线程时，一个阻塞系统调用是否拖住其他线程还取决于用户/内核线程映射。

图解与逐步推演：阻塞与非阻塞read：没数据时返回路径有什么不同

阻塞与非阻塞read：没数据时返回路径有什么不同



单独查看图片

阻塞与非阻塞read：没数据时返回路径有什么不同

查看完整步骤（也可离线阅读）

1. 缓冲区暂时无数据：以常见Unix式流接口说明，忽略信号与其他错误。
2. 阻塞调用：阻塞的是发起调用的执行流，不代表整个CPU一直忙等。
3. 非阻塞调用：非阻塞不是必定成功拿到数据，也不是自动等于异步完成通知。
4. read返回0的区别：不能把“暂时没数据”与“永久读到结束”合并为同一种状态。

磁盘缓存、缓冲目的与单缓冲 206 缓存保留可复用副本，缓冲协调传输过程，功能可以重叠。

缓存与缓冲

磁盘高速缓存通常在主存保留磁盘块数据，减少反复磁盘I/O，不是CPU与主存之间的硬件Cache；可划固定区域，也可与空闲内存、页缓存等统一管理。

缓冲用于协调CPU与设备速度、数据粒度，减少中断频率、放宽响应时限并增加重叠。既可在硬件设置小缓冲，也可在主存设置缓冲区。

单缓冲的稳态周期

缓冲区被用户提取后，设备可输入下一块，同时CPU处理上一块，所以T和C可重叠；搬运M仍是独立约束。此式是连续多块处理的平均稳态周期，不应不加首尾启动与排空时间就当作任意n块总时间。

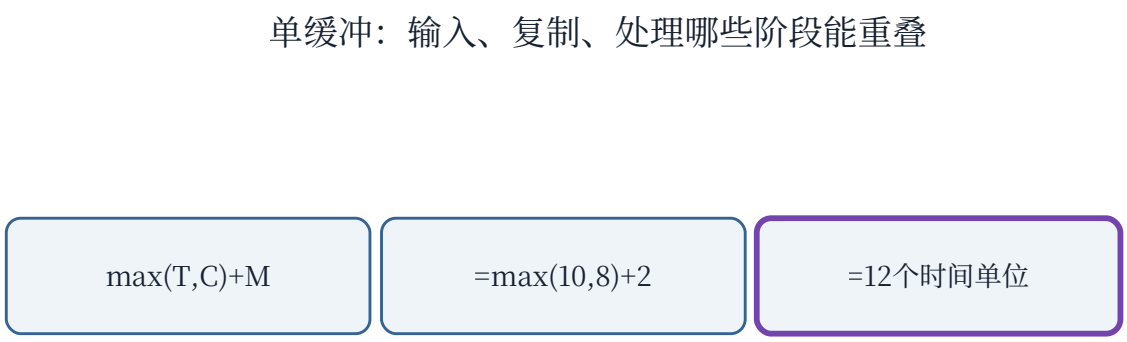
$$T_{single} = \max(T, C) + M$$

设备填满一块T，内核缓冲搬到用户区M，用户处理C；采用教材一次一块且搬运阶段不可与相应填充重叠的模型。

按事件图核对

T=8ms、M=1ms、C=3ms时，稳态单块周期9ms。若只处理第一块，从设备开始到CPU处理完需要8+1+3=12ms；把单块端到端时间和连续流水稳态周期分开，才能正确计算有限批次。

图解与逐步推演：单缓冲：输入、复制、处理哪些阶段能重叠



单独查看图片

单缓冲：输入、复制、处理哪些阶段能重叠

查看完整步骤（也可离线阅读）

1. 本例计时约定：只有一个设备输入缓冲；复制时设备不能同时覆盖该缓冲，复制完成后CPU在用户区处理。
2. 第一块输入与复制：第一块到达前不能凭空处理；复制完成后输入缓冲才空出。
3. 第二块输入与前块处理重叠：用户区处理与设备向输入缓冲写入可并行；同一个输入缓冲的输入和复制不能同时做。
4. 理想稳态每块节拍：这是上述缓冲与用户处理模型的稳态节拍。有限块总完成时间还需计算首块填充和末块处理，不可直接乘块数替代全部时序。

双缓冲、循环缓冲与缓冲池 207 增加缓冲槽能扩大重叠，但每个槽仍要遵守所有权和数据状态。

双缓冲周期

两区交替空/满，设备填一块时CPU可处理另一块。忽略M时常近似max(T,C)，但题目明确给M且不可忽略时不能擅自删去。双向同时传输还需明确发送与接收缓冲配置，不是任意多一个缓冲就自动支持全双工。

$$T_{double}=\max(T,M+C)$$

教材双缓冲流水模型：设备在一个区填充时，CPU从另一区搬运并处理；忽略额外争用，具体有限n块仍需画时间线。

循环缓冲

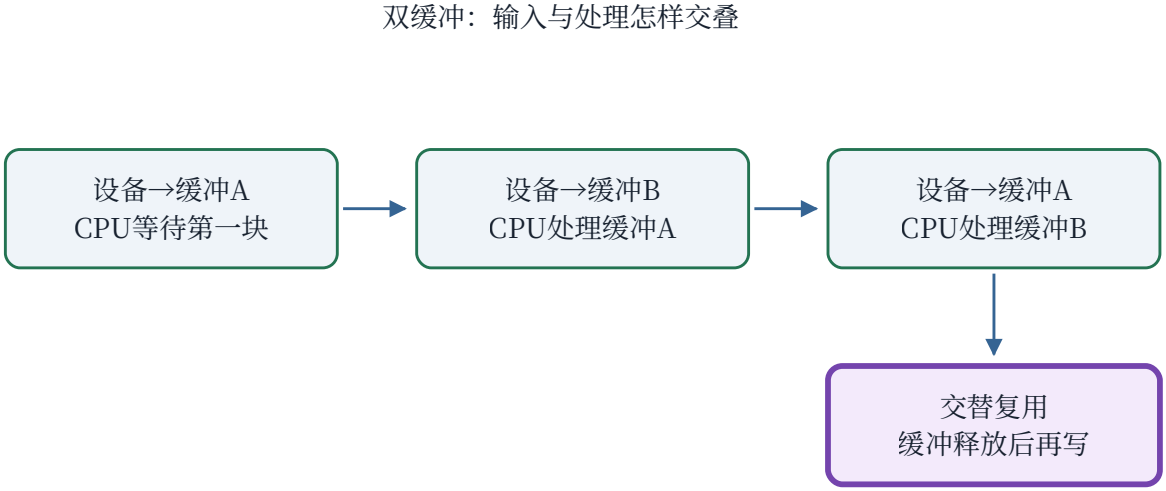
把多个等大缓冲区首尾链接，输入指针in指向下一个可填空区，输出指针out指向下一个可取满区；消费后释放，填满后发布。in==out的空/满歧义可用计数、空出一格或标志解决，采用哪种需要明确。

有多个满区时消费者可以逐区取出，生产者也可继续填其他空区；不要求所有满区一并读完。缓冲区状态和指针更新需互斥或满足相应单生产者单消费者的原子条件。

公用缓冲池

按状态组织空缓冲队列、输入满队列、输出满队列。输入进程从空队列取区，收满后挂输入队列；计算进程从输入队列取数据，使用后归空。计算进程要输出时取空区填充并挂输出队列，输出进程取出送设备后归空。四种工作缓冲角色可以由公用池中的不同区承担。

图解与逐步推演：双缓冲：输入与处理怎样交叠



单独查看图片

双缓冲：输入与处理怎样交叠

查看完整步骤（也可离线阅读）

1. 设备→缓冲A · CPU等待第一块：初始两个缓冲为空，设备先填A。
2. 设备→缓冲B · CPU处理缓冲A：A填好后交给CPU处理，设备可同时填B。
3. 设备→缓冲A · CPU处理缓冲B：A处理完成释放后，设备才可再次写A；CPU转而处理B。
4. 交替复用 · 缓冲释放后再写：理想稳态每块节拍取max(输入时间,处理时间)，首块/末块另计；若处理慢，设备仍可能等待。

设备分配数据结构与分配链 208 请求不只占设备，还可能竞争控制器和通道。

四张表

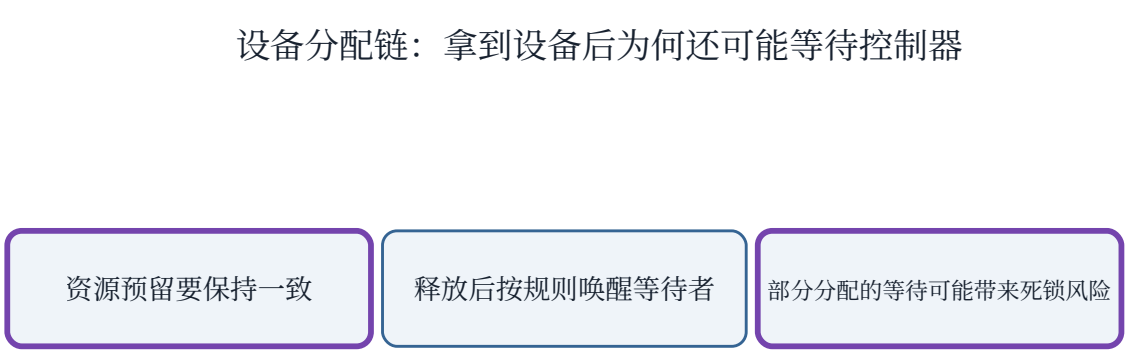
结构	范围	主要信息
系统设备表SDT	全系统设备列表	设备类型、标识、DCT指针、驱动入口
设备控制表DCT	每台物理设备	类型、状态、标识、控制器指针、等待队列、重试信息
控制器控制表COCT	每控制器	控制器状态、通道指针、等待队列
通道控制表CHCT	每通道	状态、连接控制器信息、等待队列

分配步骤

按逻辑映射得到物理设备后，查SDT定位DCT；设备忙则进入设备等待队列，空闲且分配安全才占用；再沿指针申请控制器，最后申请通道。相应层忙就挂到对应队列，三者满足后才能启动所需传送。

实际系统可能异步预约或合并这些步骤，教材分层链的意义是说明共享控制器、共享通道也会造成等待；不能因为有两台不同设备就断定其I/O一定完全并行。

图解与逐步推演：设备分配链：拿到设备后为何还可能等待控制器



单独查看图片

设备分配链：拿到设备后为何还可能等待控制器

查看完整步骤（也可离线阅读）

1. 先定位逻辑设备：具体映射可由逻辑设备表等结构完成；表名及字段按教材模型。

2. 检查设备可用性：设备空闲不自动保证整条I/O路径可用。
3. 继续检查控制器和通道：多个设备可能共享控制器或通道；图描述传统分配链概念，现代硬件组织不一定原样对应。
4. 成功分配后启动I/O：分配策略要明确能否持有部分资源继续等待，不能仅判断单个设备的忙闲位。

静态、动态与安全设备分配；逻辑设备表 209 分配时机、死锁控制与命名映射需要分别判断。

静态与动态

静态分配在作业执行前一次获得所需设备、控制器和通道，按模型直到结束再释放，破坏请求和保持可避免这一类设备死锁，但利用率低。动态分配在运行中按需请求、用后释放，提高利用率，却需正确处理持有等待与安全性。

独占设备可静态或动态分配，共享设备通常按请求动态安排；先请求先分配或按优先级分配可决定等待次序，不能仅靠队列顺序就代替死锁避免。

安全方式的适用前提

教材安全分配方式让进程在发I/O后等待、不再带着已有设备继续请求其他设备，以限制多设备占有；不安全方式允许继续运行和提出更多设备请求，并发度可能更高但有死锁风险。这里“安全”依赖所限定的持有/请求模型，不是“任何阻塞I/O天然不会死锁”。

LUT与重定向

逻辑设备表LUT记录逻辑名→物理名→驱动入口。全系统一张表时同名逻辑设备可能冲突，按用户/进程设置表可允许不同用户各自使用相同逻辑名。应用请求逻辑设备后，系统按分配结果建立映射，后续据此调用驱动，方便I/O重定向与设备替换。

SPOOLing：用磁盘队列虚拟独占设备 210 物理设备仍按序工作，但多个作业可先提交各自的逻辑请求。

假脱机结构

SPOOLing以程序模拟早期外围控制机的脱机I/O，将慢速设备操作通过高速磁盘暂存和后台进程组织起来。磁盘上设置输入井、输出井，内存中设置输入/输出缓冲区，并由输入进程、输出进程负责搬运。

输入路径可为输入设备→内存缓冲→输入井，用户需要时从井读入；输出先由用户数据写入输出井，后台在输出设备可用时经输出缓冲送出。井在外存，缓冲区在内存，不能颠倒。

共享打印机

多个进程提交打印请求，各自在输出井拥有待打印文件与队列项，感觉拥有一个逻辑打印机；后台逐个取出交给物理打印机。打印机本体没有变成能同时打印多份页面，而是通过排队与磁盘空间把独占设备虚拟成可共享服务。

效果与条件

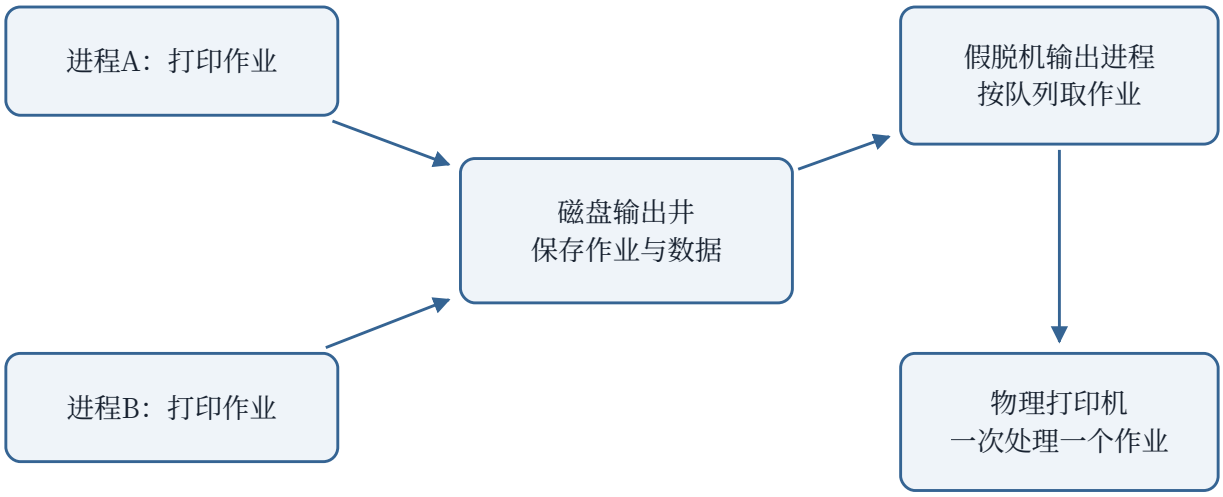
用户提交速度不再直接受慢速外设逐字符处理速度束缚，CPU与设备可更好重叠，并实现虚拟设备。并非提高物理打印头的固有速度；后台服务、磁盘空间和队列管理也占资源。

结构对照图：假脱机打印：多个进程排队共享一台打印机

用户进程把作业写入磁盘队列后可继续运行；输出进程再串行驱动物理打印机，形成逻辑上的多用户共享。

SPOOLing依赖外存中的输入/输出井和管理程序；它不等于只在主存里增加一个普通缓冲区，也不是把一台物理设备变成真正同时工作的多台硬件。

假脱机打印：多个进程排队共享一台打印机



单独查看图片

磁盘结构、格式化、分区与簇 211 几何单元、I/O块和文件系统分配单元不要混算。

结构与初始化

机械盘由盘片、盘面、磁道、扇区和磁头等组成；同半径磁道形成柱面。传统等扇区教材模型中内圈记录密度更高，现代盘区记录可改变每轨扇区数，做题按给定几何结构。

低级格式化建立物理扇区的标识、数据与校验等结构；分区规定逻辑卷的起点和范围；高级/逻辑格式化建立文件系统的空闲信息、目录等数据结构。现代分区布局不一定都是传统MBR四项模型，术语与题设一致即可。

簇与内部浪费

文件系统可把若干相邻扇区组成一个簇/分配块，按簇分配文件数据；非空文件长度s、簇大小c且无稀疏等特殊结构时，占⌈s/c⌉簇，末簇有内部浪费。

空文件不必占一个数据簇，仍可能占目录/inode等元数据；稀疏文件也不按逻辑长度全部分配数据块。遇到“0字节也一律占一簇”要先确认是否是题设简化。

磁盘调度：FCFS、SSTF、SCAN与LOOK 212 先确定当前磁头位置和方向，再列访问次序与总移动量。

规则对照

算法	下一步怎么选	边界与代价
FCFS	按请求到达顺序	公平简单，可能往返很多
SSTF	选当前距离最近请求	局部移动较短，但远端请求可能饥饿
SCAN	当前方向服务，走到磁盘边界再反向	通常无无限拖延，需计到端点的移动
C-SCAN	只单向服务，到端点后回另一端	回程不服务，回程距离是否计入必须看题设
LOOK	当前方向无待请求即转向	不必走到物理端点
C-LOOK	单向服务，至最远请求后跳到另一端最远请求	跳转不服务，仍可能有实际移动代价

总路程的计算步骤

把所有请求按磁道排序，标起点与方向；按算法写服务序列，并补SCAN/C-SCAN所需物理端点；相邻位置差取绝对值相加；平均寻道长度=总移动/请求数。仅比较访问序列不够，遗漏“无请求端点”会把SCAN误算成LOOK。

动态请求与静态集合

若所有请求已给定，可静态排序；若运行中继续到达，必须按到达时间更新候选。SSTF的饥饿来自近处持续新请求，不能只凭有限静态列表断言某请求永远无法处理。

图解与逐步推演：SSTF磁盘调度：每次重新找最近请求



单独查看图片

SSTF磁盘调度：每次重新找最近请求

查看完整步骤（也可离线阅读）

1. 50 → 43：从当前磁头50选择最近请求43，移动7个柱面；累计7。
2. 43 → 24：从当前磁头43选择最近请求24，移动19个柱面；累计26。
3. 24 → 16：从当前磁头24选择最近请求16，移动8个柱面；累计34。
4. 16 → 82：从当前磁头16选择最近请求82，移动66个柱面；累计100。
5. 82 → 140：从当前磁头82选择最近请求140，移动58个柱面；累计158。
6. 140 → 170：从当前磁头140选择最近请求170，移动30个柱面；累计188。
7. 170 → 190：从当前磁头170选择最近请求190，移动20个柱面；累计208。

磁盘访问时间：寻道、旋转与传输 213 转速决定周期，平均半圈只适用于相应随机位置假设。

时间分解

寻道是磁头移动到目标磁道，可按启动时间+每跨道时间×距离估计；旋转等待使目标扇区来到磁头下；传输是在扇区经过时搬数据。磁盘调度主要优化寻道及请求次序，并不改变硬件固有转速。

$$T_{access} = T_{seek} + T_{rotation} + T_{transfer}$$

另有控制器、排队或命令开销时按题设加上；不能把这些默认包含在转速公式里。

$$T_{avg\,rotation} = \frac{1}{2r}, T_{transfer} = \frac{b}{rN}$$

r以转/秒，N为每轨字节数，b为连续传送字节数，采用给定几何与连续传输模型。

单位换算

7200转/分＝120转/秒，一圈约8.333ms，平均半圈约4.167ms。若每轨64KB，连续传4KB占1/16圈，传输约0.521ms；再加题设寻道和其他开销。若已知当前扇区或请求位置，应算实际旋转角度，不直接套平均半圈。

图解与逐步推演：磁盘访问时间：7200转每分钟怎样换成平均半圈



单独查看图片

磁盘访问时间：7200转每分钟怎样换成平均半圈

查看完整步骤（也可离线阅读）

1. 给定硬盘模型：忽略控制器和排队开销，假设随机相位平均等半圈、扇区均匀分布。
2. 换成一圈时间：rpm的分钟要先换成秒；平均半圈来自随机等待相位假设。
3. 传1扇区：按本例均匀磁道模型估计，不计其他扇区间隙与协议开销。
4. 三项相加：连续读多个相邻扇区通常不会每扇区都重新寻道并等半圈，具体按请求组织与题设。

SSD的页读写、块擦除与磨损均衡 214 没有机械寻道，也仍有擦除、复制和写放大。

组成与读写限制

SSD由闪存芯片及控制逻辑/闪存翻译层FTL等组成，FTL把逻辑块地址映射到物理闪存页。闪存按页读写、按更大的擦除块擦除；已编程页不能像DRAM那样直接原位任意改写，需遵守先擦除与编程限制。

更新数据通常写到空闲已擦除页并修改映射，旧页失效；垃圾回收把仍有效页复制到其他位置，再整块擦除。复制和擦除使实际闪存写入量可能大于主机请求量，即写放大。

性能与寿命

无活动机械部件，随机访问通常比机械盘快，抗震、噪声和功耗有优势；但随机写、回收、可用空闲块和工作负载仍影响延迟，不能说所有访问时间都完全相同。

动态磨损均衡优先在较少擦写的空闲块写入新数据；静态磨损均衡还迁移长期不变的冷数据，让磨损较低的块有机会承担新写入、较旧块保存冷数据，尽量均衡擦除次数并延长寿命。

与磁盘算法的适用边界

SSD不需要机械磁头寻道，因此不能照搬磁道距离最短就是最快的判断。操作系统仍需I/O排队、请求合并、公平与延迟管理，但其优化目标和机械SCAN模型不完全相同。

网络体系结构与性能

计算机网络 · 7 条

网络的本质、组成与通信对象 215 先区分主机、网络和互联网，再定位边缘与核心。

互联与自治

计算机网络由互联的、自治的计算机组成：互联要求能够交换信息；自治意味着每台计算机仍能独立完成自身功能。网络可抽象为结点和连接结点的链路。共享硬件、软件和数据资源需要各系统遵守共同的协议。

“主机之间通信”是简写，实际交换数据的是不同主机上的进程。不要把网络应用限定为机器本身的通信。

三个组成视角

划分依据	组成与职责
工作方式	边缘部分由主机及其应用组成，供用户通信与共享资源；核心部分由网络及连接网络的路由器组成，向边缘提供连通和分组交换。
组成部分	硬件包括主机、链路、交换设备、网卡等；软件实现网络服务和使用工具；协议约定交换数据的规则。
功能组成	通信子网提供数据传输、交换和控制；资源子网提供可共享的计算、存储、数据与软件资源。

网络的功能

- 数据通信是最基本功能，涉及连接、传输、差错、流量、路由与复用控制。
- 资源共享使远程资源能够被访问；分布式处理把任务分给不同计算机共同完成。
- 互为备用可提高可靠性；负载均衡把任务分配到不同机器；信息综合处理可集中或分级处理分散数据。

网络分类与拓扑结构 216 分类标准不同，同一网络可以同时属于多个类别。

范围、使用者与传输方式

分类依据	类别	辨析
地理范围	PAN、LAN、MAN、WAN	个人网范围最小；局域网覆盖局部场所；城域网通常覆盖城市；广域网连接更大区域。距离是典型尺度，不能替代技术与管理边界。
使用者	公用网、专用网	公用网对符合条件的公众提供服务；专用网面向特定单位或业务。
传输技术	广播式、点对点式	广播式共享通信信道；点对点式每条链路连接一对结点，可经中间结点转发。
传输介质/交换	有线、无线；电路、报文、分组交换	不要将交换技术和拓扑形状混为同一分类。

四种常见拓扑

拓扑	优点	局限
总线形	布线较省，增减结点相对方便	共享介质，重负载性能下降，总线故障影响较大。
星形	便于集中管理和定位故障	依赖中心设备，布线和中心设备有成本。
环形	结点按环连接，可按规定次序传递访问权	链路或结点故障可能影响环的工作，需保护措施。
网状形	路径冗余多，可靠性较高	线路成本和路由控制复杂度较高。

模型边界

传统共享式局域网常用广播技术；现代交换式以太网的主机到交换机链路通常为点对点、全双工链路。局域网不等于永远只有一条共享总线。

速率、带宽、吞吐量与瓶颈 217 先分清能力上限与实际传输，再计算单位。

三个量的含义

术语	含义	常用单位
速率	单位时间传送的数据量；标称速率并不保证实际应用达到	bit/s、b/s
带宽	频域表示频带宽度；网络时域语境表示信道可提供的最高数据率	Hz；或 bit/s

术语	含义	常用单位
吞吐量	单位时间实际通过信道、接口或网络的数据量	bit/s、B/s、帧/s

瓶颈效应

端到端路径经过多个接口和链路时，持续吞吐量受其中最慢部分限制，还受竞争、协议首部、丢包与处理能力影响。不能把各段带宽相加，也不能把标称带宽直接视为有效载荷速率。

$R_{\text{path}} \le \min(R_1, R_2, \ldots, R_n)$
各段串联传输且没有其他瓶颈时，上限由最小速率决定。

单位核对

网络速率的 k、M、G 通常按十进制；1 B=8 bit。传输 1 MB 数据不能直接用“1÷100 Mb/s”，应先统一成 bit，再按题设确定 MB 是十进制还是二进制。

四种时延、往返时间和链路容量 218 发送时延与传播时延的成因不同，逐段累计才能算对。

时延分解

类型	决定因素	容易混淆的地方
发送时延	数据长度 L 与发送速率 R	从首比特发出到末比特发出所需时间，与传播距离无直接关系。
传播时延	介质长度 d 与传播速度 v	首比特沿链路传播的时间，不由数据帧长度决定。
处理时延	检错、解析首部、查表等处理	题设可忽略，不应自行加入固定数值。
排队时延	队列长度、竞争与业务负载	随机变化，拥塞时会明显增大。

计算式

$t_{\text{tx}} = L/R, t_{\text{prop}} = d/v$
总时延按实际经过的链路和结点，将发送、传播、处理、排队时延相加。

$\text{BDP} = Rt_{\text{prop}}$
单向时延带宽积表示传播管道中可容纳的比特数。

RTT 与时延带宽积

RTT 是往返时间，不能不加区分地用单程传播时延替代。具体是否包含确认发送、处理和排队时间，以题设 RTT 定义为准。高速长距离链路即使发送一个小帧很快，传播等待仍可能很长。

利用率并非越高越好

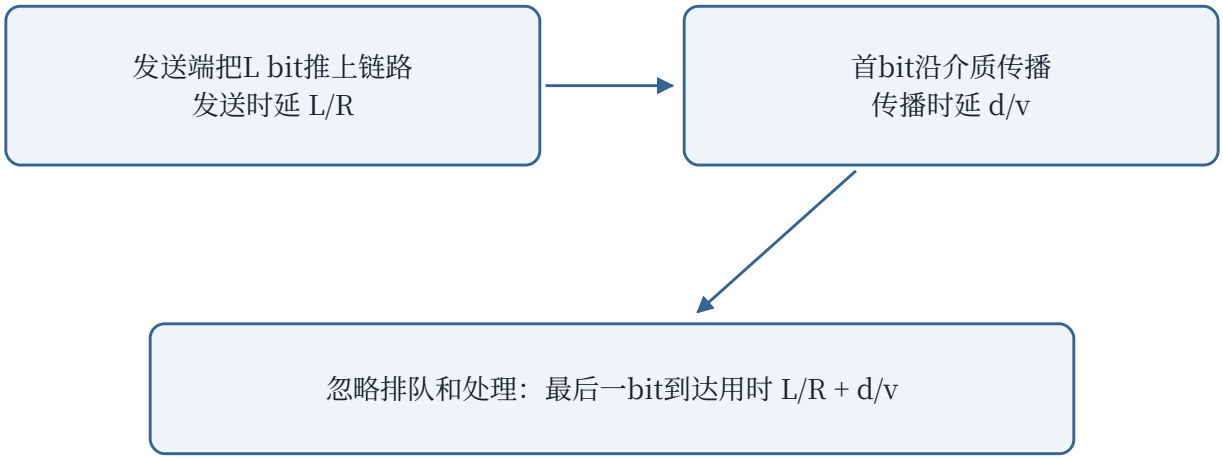
信道利用率衡量信道被使用的时间比例；网络利用率可看作各信道利用情况的综合。接近饱和时排队时延急剧增加。简单排队模型中可写 $D=D_0/(1-U)$，但该式有模型前提，不能作为所有实际网络的普遍精确公式。

结构对照图：发送与传播：最后一bit何时到达

L用bit、R用bit/s，传播距离d与速率v须使用相容单位；传播时延与报文长度无直接关系。

存储转发多跳时还要算每跳发送、传播、排队与处理；分组可流水重叠，不能把一整个报文的发送时间对所有分组和链路重复累加。

发送与传播：最后一bit何时到达



单独查看图片

协议、接口、服务与分层封装 219 协议是对等实体的约定，服务是下层给上层的能力。

先确定三个关系

协议是水平关系：不同系统的对等实体遵守的规则。协议包含语法、语义和同步：分别说明信息格式、动作及响应含义、事件时序。

服务是垂直关系：某层借助自身和下层的功能，向直接上层提供能力；服务通过接口和服务访问点使用。接口描述相邻层怎样交互，协议描述对等层怎样合作。

分层的收益与代价

- 每层尽量完成相对独立功能；相邻层界面清晰，尽量减少跨层耦合。
- 模块可分别实现和修改，便于维护、标准化及异构互联。
- 分层会引入首部、重复功能和层间处理开销；层数并非越多越好。

PDU、SDU 与 PCI

上层交下来的 PDU 到本层成为 SDU；本层加入协议控制信息 PCI 后构成本层 PDU，再交给下层。发送方向下封装，接收方向上拆封；链路层通常还有帧尾，不能把所有层都画成只有首部。

n-PDU=n-SDU+n-PCI

这里的“+”表示组成关系，不一定是简单连续首部拼接。

寄信类比

应用内容相当于信件正文，进程端点信息相当于部门收件信息，主机寻址相当于外层地址。每个层次处理自己负责的控制信息，不需要理解所有上层内容。类比只帮助理解封装，不能据此推导网络层必然可靠。

图解与逐步推演：发送到接收：数据怎样逐层封装

发送到接收：数据怎样逐层封装



单独查看图片

发送到接收：数据怎样逐层封装

查看完整步骤（也可离线阅读）

1. 应用数据 · HTTP等：应用产生消息或字节。
2. 运输层加首部 · TCP段 / UDP数据报：端口标识运输层通信端点；TCP和UDP格式不同。
3. 网络层加首部 · IP数据报：IP地址用于网络层寻址；分片等行为由协议和链路条件决定。
4. 链路层加首尾 · 以太网帧：以太网头中使用当前链路下一跳的MAC地址，帧尾含检错信息。
5. 物理层传输比特 · 跨路由逐跳转发：路由器一般解开旧链路封装，处理IP转发，再重新封装下一跳帧；并非每跳都交给应用层。
6. 接收端逆向解封装 · 交给对应应用：终点逐层检查和去除封装；首部和尾部属于哪一层应分清。

服务类型与服务原语 220 连接、确认与可靠性是不同维度。

三组服务分类

维度	一种服务	另一种服务
连接	面向连接：先建立逻辑连接，传送数据后释放	无连接：无需预建连接，各数据单元独立提交。
应答	有应答：接收方给出相应确认	无应答：本层不自动提供接收确认。
可靠性	可靠：利用检错、重传、排序等机制满足交付约定	不可靠：尽力交付，不保证正确、完整或按序到达。

不能相互替代

面向连接并不自动等于可靠，无连接也并不表示应用不能实现可靠性。TCP 以不可靠 IP 服务为基础实现可靠字节流；UDP 提供无连接数据报服务，应用可以在 UDP 之上增加自己的确认机制。

四类原语

请求由服务用户交给服务提供者；指示由提供者通知用户发生事件；响应是用户对指示的回答；证实是提供者对先前请求的确认。应沿着“谁向谁发出”理解，不能仅凭中文词义猜方向。

OSI、TCP/IP 与五层模型 221 先明确参考模型，再判断设备与协议在哪层工作。

层次对照

OSI 七层	五层教学模型	主要数据单位与任务
应用、表示、会话	应用层	应用报文；应用规则，数据表示转换、加密/压缩和会话管理在 OSI 中另分层。
运输层	运输层	TCP 报文段、UDP 用户数据报；进程到进程通信、复用分用。
网络层	网络层	分组/IP 数据报；逻辑寻址、路由与转发。
数据链路层	数据链路层	帧；成帧、差错检测、介质访问，部分协议含可靠传输。
物理层	物理层	比特；接口特性、编码、比特同步与信号传输。

TCP/IP 四层

TCP/IP 四层由应用层、运输层、网际层、网络接口层组成。网络接口层对应教学模型的数据链路层和物理层；应用层覆盖 OSI 的应用、表示与会话层功能。两种体系均分层并支持异构互联，但分层边界、服务设计和历史形成方式并不相同。

功能与具体协议实现

参考模型描述“本层可能承担什么”，不等于每个具体协议都实现全部功能。IP 不提供端到端可靠交付和流量控制；以太网 FCS 检错也不等于链路层必然重传。主机通常实现完整协议栈，路由器主要工作到网络层，交换机主要工作到链路层。

物理层与通信基础

计算机网络 · 8 条

数据、信号、码元与通信方式 222 把信息、物理表现和传输单位分开。

基本术语

数据是承载信息的符号序列；信号是数据的电气或电磁表现。模拟信号的相关参数连续变化，数字信号取离散状态。码元是在一个固定时间间隔内用来表示某种状态的基本波形，一个码元可以携带一个或多个比特。

信源产生数据，发送器编码或调制后送入信道；接收器恢复可处理的数据，再交给信宿。信道强调某方向的信息传输，一条通信线路可含发送与接收信道。

码元率与比特率

$$R_b = R_s \log_2 M$$

M 是等概率离散状态数；码元率 R_s 单位 Baud，比特率 R_b 单位 bit/s。二进制码元时二者数值相同，多进制时不同。

单工、半双工、全双工

单工只能一个方向传送；半双工可双向但不能同时；全双工可同时双向传送。通信方式与链路速率、物理线数没有简单的一一对应。同步传输按共同节拍组织，异步传输通常通过字符起止位实现字符边界同步。

奈奎斯特与香农公式的使用边界 223 先认无噪声/有噪声，再统一带宽与信噪比单位。

两种上限

$$R_{\max} = 2W \log_2 M$$

奈奎斯特公式：理想无噪声、带宽受限信道；W 以 Hz 为单位，M 是码元可区分状态数。

$$C = W \log_2 (1 + S/N)$$

香农公式：有高斯白噪声信道的理论容量；S/N 必须是功率比的线性值。

$$(S/N)_{\mathrm{dB}} = 10 \log_{10} (S/N)$$

给出 dB 时，先做 S/N=10^(dB/10)，不能直接把 dB 数值代入容量式。

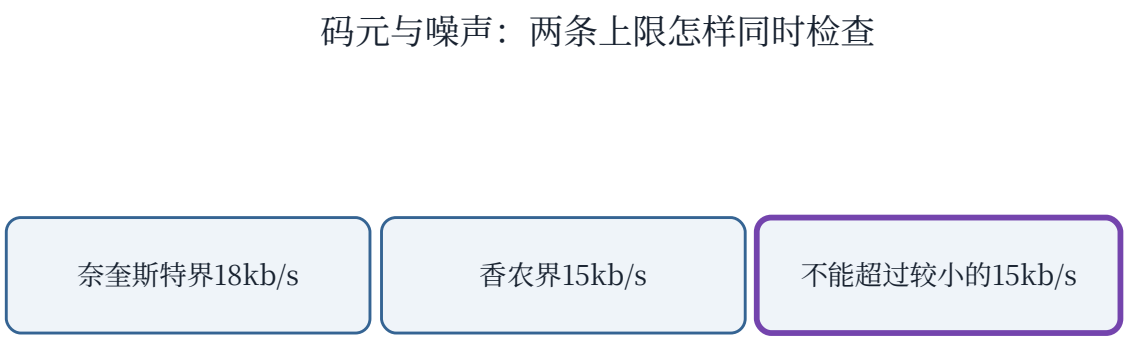
公式说明什么

奈氏上限说明不能无限提高码元率；增大 M 可提高每码元携带的信息，但状态越密越难区分。香农容量同时受带宽和信噪比约束；只要考虑噪声，增加状态数也不能突破容量上限。若题目同时给出 M、W 和 S/N，应分别计算约束并取较小上限。

计算顺序

若 W=3 kHz、M=4、S/N=1000，则奈氏上限为 $2\times3000\times2=12000$ bit/s；香农上限为 $3000\times\log_2 1001$，约 29.9 kbit/s。受两种条件同时约束时，不能超过 12 kbit/s。

图解与逐步推演：码元与噪声：两条上限怎样同时检查



单独查看图片

码元与噪声：两条上限怎样同时检查

查看完整步骤（也可离线阅读）

1. 带宽3kHz，8种状态：本例分别计算理想奈奎斯特界与有噪声香农界，区分线性信噪比和dB。
2. 奈奎斯特界：理想带限无噪声模型下$2B\log_2 M$；码元率与比特率相差每码元位数。
3. 香农界：信噪比31不是31dB；若输入为dB，须先转换为10^(dB/10)。
4. 综合约束：15kb/s是理论上限约束，不代表任意采用8状态的实际编码都能恰好达到该容量。

数字编码与调制：时钟信息在哪里 224 重点比较跳变位置及是否便于同步。

数字数据编码成数字信号

编码	规则示例	特点
不归零 NRZ	用两种持续电平表示 0/1	连续相同比特可能长时间无跳变，不便提取时钟。
归零 RZ	码元内部回到零电平	提供一定跳变，所需带宽与波形约定有关。
反向不归零 NRZI	讲义约定 0 触发翻转、1 不翻转	应明确题设规则和初始电平，不能反套另一种约定。
曼彻斯特	每个比特中间必有一次跳变；跳变方向区分 0/1	自同步，通常一个数据比特对应两个半位信号单元。
差分曼彻斯特	位中间跳变用于同步；位开始是否跳变表示数据	绝对电平不重要，首位仍须按题设初始状态读图。

编码约定不是全球唯一画法

曼彻斯特“向上为 0”或“向下为 0”存在不同约定，讲义示例使用位中心向上表示 0、向下表示 1。解题必须先读题图和编码约定；不能仅凭一个边沿就断定数据。

数字数据调制

ASK 用幅度表示离散状态；FSK 用频率；PSK 用相位；QAM 联合改变幅度和相位。若调制可稳定区分 M 种状态，每码元可表示 $\log_2 M$ bit；提高阶数会提高识别精度与信噪比要求。

结构对照图：曼彻斯特与差分曼彻斯特：先约定位值

两种编码每个码元位中间都有跳变，接收方可据此恢复定时。曼彻斯特看位中间跳变方向；差分曼彻斯特看位边界是否发生跳变。

图给出一种常见约定，不同标准或题设可能把0和1的定义反过来。不要只记某个跳变图而不读编码规则。

曼彻斯特与差分曼彻斯特：先约定位值

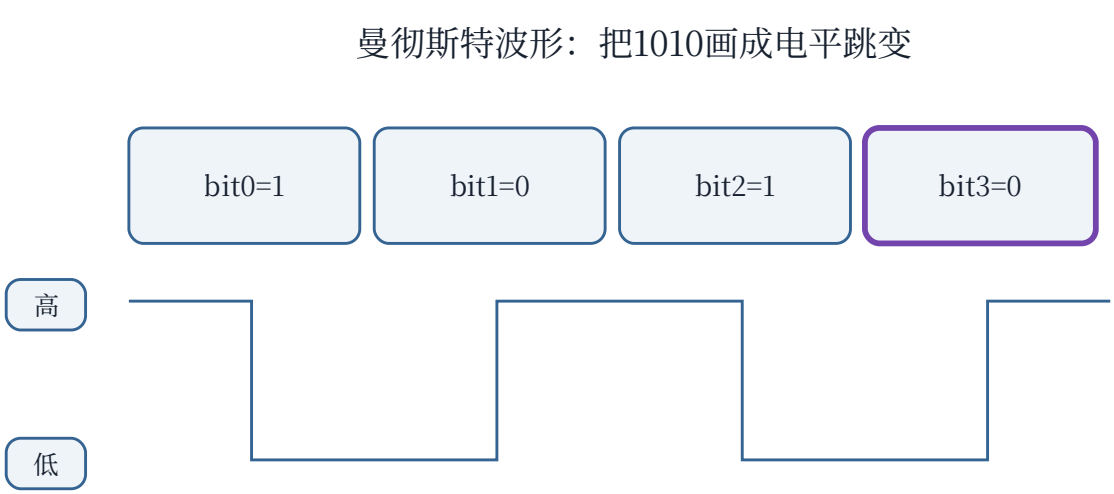
曼彻斯特示例约定
0：位中间低→高
1：位中间高→低

差分曼彻斯特示例约定
每位中间必有跳变
0：位开始跳变，1：不跳变

传数据同时携带时钟信息；具体高低电平与0/1对应应遵题设

单独查看图片

图解与逐步推演：曼彻斯特波形：把1010画成电平跳变



单独查看图片

曼彻斯特波形：把1010画成电平跳变

查看完整步骤（也可离线阅读）

1. 观察第1位：1：本例沿用正文约定：0在位中间低→高，1在位中间高→低。每位中间必有跳变，可提取时钟；位边界是否跳变取决于相邻位。
2. 观察第2位：0：本例沿用正文约定：0在位中间低→高，1在位中间高→低。每位中间必有跳变，可提取时钟；位边界是否跳变取决于相邻位。
3. 观察第3位：1：本例沿用正文约定：0在位中间低→高，1在位中间高→低。每位中间必有跳变，可提取时钟；位边界是否跳变取决于相邻位。
4. 观察第4位：0：本例沿用正文约定：0在位中间低→高，1在位中间高→低。每位中间必有跳变，可提取时钟；位边界是否跳变取决于相邻位。

模拟数据数字化与基带/带通传输 225 采样、量化、编码三个阶段不能互换。

数据与信号的四种转换

数字数据可经数字编码成为数字信号，也可经调制成为模拟信号；模拟数据可经 PCM 成为数字信号，也可经模拟调制搬到适合传输的频带。基带信号直接承载原始信息；带通信号是调制后搬移到某一通带的信号。

PCM 的顺序

- 采样：以时间间隔抽取模拟信号幅值；在满足采样定理条件时，采样频率应不低于最高频率的两倍。
- 量化：把连续幅值映射到有限个等级，会引入量化误差。
- 编码：用二进制码字表示量化等级；若每样本 n bit、每秒 f_s 个样本，原始编码速率为 n f_s bit/s。

话音编码

若采样频率为 8 kHz，每次量化编码 8 bit，则一路 PCM 速率为 64 kbit/s。这里的“两倍”约束采样频率，不是把最终码率直接设置为模拟最高频率的两倍。

图解与逐步推演：PCM：采样率、量化位数与输出码率怎样联系



单独查看图片

PCM：采样率、量化位数与输出码率怎样联系

查看完整步骤（也可离线阅读）

1. 模拟信号最高频率4kHz：这里只演示理想低通带限模型；实际系统会留过渡带等余量。
2. 每样本256种量化等级：采样离散时间，量化离散幅度，编码把等级映射为二进制位串。
3. 计算单路PCM码率：这里只算样本编码净码率；帧同步、纠错、信令等开销另加。
4. 量化位数改为10bit：增加量化位数不等于采样频率自动增加；两者对码率的作用分别相乘。

三种交换技术与分组流水传输 226 存储转发的单位决定缓存、时延和复用方式。

交换方式对照

技术	过程	优点与代价
电路交换	建立连接→传送→释放；通信期间预留端到端资源	建立后时延稳定、顺序自然保持；建立有开销，空闲时资源仍被占用，不适合突发业务。
报文交换	每一跳完整接收整个报文后再转发	无需预建电路，可动态利用线路；报文大时结点缓存需求和等待时间大。
分组交换	把报文拆成带控制信息的小分组，逐个存储转发	后继分组的接收可与前驱分组的转发重叠，缓存更易管理；新增首部开销，可能乱序、丢失、重复。

计算模型

分组交换不一定都是无连接：它还包括虚电路方式。存储转发要求接收当前分组的全部比特后才能转发该分组，而不是等整个原始文件都到齐。分组大小不必在一切网络中完全固定。

$T=(k+n-1)L/R+k\tau$

同速率、无排队、n 个等长分组经过 k 条链路时的流水时间；每组长度 L 含实际发送首部，单链路传播时延为 τ。不同速率或变长末包须画时间线。

别重复计算传播

最后一个分组到达时，发送端已连续发送 n 个分组，后续 k−1 段各补一个分组发送时间，再加各段传播。若题设给的是路由器数量 r，链路数量通常是 r+1，应先画路径再代式。

图解与逐步推演：分组存储转发：3包通过2条链路为什么只用4ms



单独查看图片

分组存储转发：3包通过2条链路为什么只用4ms

查看完整步骤（也可离线阅读）

1. 观察到t=1ms：每包必须完整到达中间路由器后，才能在下一条链路发送；不同包在不同链路上可重叠。
2. 观察到t=2ms：每包必须完整到达中间路由器后，才能在下一条链路发送；不同包在不同链路上可重叠。
3. 观察到t=3ms：每包必须完整到达中间路由器后，才能在下一条链路发送；不同包在不同链路上可重叠。
4. 观察到t=4ms：每包必须完整到达中间路由器后，才能在下一条链路发送；不同包在不同链路上可重叠。

数据报与虚电路：逻辑连接不等于独占线路 227 两者都可以采用分组交换。

数据报方式

每个分组携带独立转发所需的地址信息，中间结点逐个查转发表。不同分组可以走不同路径，因而到达时延和顺序不一定相同；发生故障后可调整路径。数据报网络不要求交换结点为每次通信预建一条连接状态。

虚电路方式

通信前通过呼叫请求与应答建立逻辑路径，中间结点建立虚电路表；传送阶段依据虚电路标识转发；结束后释放相应状态。虚电路号通常是逐链路有效，沿途可转换。

同一物理链路可同时承载多个虚电路，因此“虚电路”不能理解成电路交换那样独占的物理通路。固定路径有利于按序传送，但具体可靠性仍取决于协议机制。

比较时逐项判断

项目	数据报	虚电路
建立连接	无需事先建立	需要建立与释放
分组控制信息	通常带完整目的地址等信息	传送阶段主要带虚电路标识
路径与结点状态	各包独立选择；无需为每个连接保留虚电路表	同一虚电路走已选路径；结点维护连接状态
故障影响	可利用其他路径继续转发	路径故障可能要求重新建立虚电路

导向与非导向介质、物理接口 228 介质负责承载信号，物理层规定怎样使用介质。

有线介质

介质	构造与特点	注意
双绞线	绝缘铜线成对绞合以减轻串扰；屏蔽型另加屏蔽层	衰减与频率、长度等有关；模拟信号可用放大器，数字信号用中继器整形再生。
同轴电缆	内导体、绝缘、外屏蔽与保护层；抗干扰较强	传统 50 Ω 基带与 75 Ω 宽带用途属于典型分类，不要仅由外观判断速率。
光纤	纤芯与包层形成光波导；容量大、损耗小、抗电磁干扰	多模与单模的传播模式、色散不同；光纤不是中空管。

无线介质

无线电波、微波、红外与激光采用自由空间传输。低频无线电通常较易绕射、覆盖范围广；微波和光通信方向性强，常要求视距条件。地球同步卫星作为微波中继覆盖广，但距离造成较长传播时延；短波可利用电离层传播，质量受环境影响。

接口四特性

特性	回答的问题
机械	连接器的形状、尺寸、引脚与排列是什么?
电气	各线路的电压、电流和信号范围是什么?
功能	某条线或某电平代表什么意义?
过程	各信号的动作顺序和时序怎样协调?

中继器与集线器：物理扩展的边界 229 能再生信号，不代表能够识别帧或选择目的端口。

再生与放大

中继器恢复数字信号的电平、波形与时序后转发，不只是同时放大信号和噪声。它工作在物理层，不解析 MAC 地址，不检查 FCS，不执行路由选择。连接两侧须满足相容的物理层与速率约定。

集线器与共享介质

集线器可视为多端口中继器：物理外观是星形，逻辑上仍共享介质。收到信号会向其他端口转发，连接主机共享一个冲突域；半双工共享以太网使用 CSMA/CD，同一时刻不能让多个站成功占用同一共享信道。

旧式 10 Mb/s 以太网的 5-4-3 规则限定中继段数与可挂主机段数，属于特定模型，不能推广到现代交换式以太网。

同步与异步的语境

通信中的同步关注收发时钟和数据边界，不能直接套用程序设计中“调用是否等待”的同步/异步定义。数字基带传输直接在介质上传数字波形；频带传输经过调制；宽带复用可让多个子信道同时承载不同信号。

数据链路层与可靠传输

计算机网络 · 8 条

链路层的职责、成帧与透明传输 230 可靠链路是一种服务选择，不能把所有链路协议都当成可靠。

链路与数据链路

链路是两个相邻结点间的物理通路；数据链路还包括控制数据传输的协议和设备能力。链路层在物理层之上把比特流组织成帧，识别边界，检测差错，并按协议提供介质访问、流量控制或链路管理。

三种服务

无确认无连接服务不预建连接、不逐帧确认，如常见以太网；有确认无连接服务可在不预建连接的情况下确认帧；有确认面向连接服务先建立逻辑链路再有序可靠传送。是否允许流水发送由窗口机制决定，面向连接不等于只能逐帧停等。

透明传输

任意上层数据都应能被承载；如果数据内容恰好与帧定界符相同，就需要转义、填充或特殊编码，避免误判为边界。MTU 限制帧载荷能装的最大网络层分组，而不是把所有首尾字段一并计入。

四种组帧方法及填充的逆过程 231 先标出定界字段，再只对规定的数据范围执行填充。

组帧方法

方法	发送/接收规则	优缺点
字符计数	帧头给出帧内字符数量，按协议约定是否含计数字段本身	计数字段出错可能导致连续失去帧边界。
字节填充	用标志字节定界；数据中的标志和转义字符用转义序列替换，接收端逆变换	适合面向字节的链路，字符集与转义规则要明确。
零比特填充	规定区域每连续五个 1 后插入一个 0，接收端去掉该 0	配合 01111110 标志，处理连续比特序列，不按字符重置。
违规编码	使用正常数据编码中不会出现的信号组合定界	需要底层编码本身有冗余状态，不适用于所有编码。

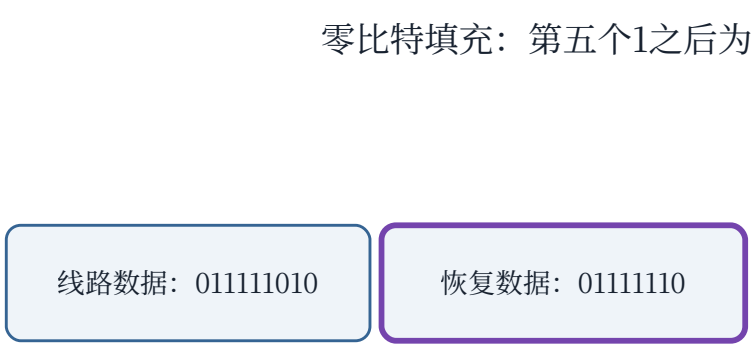
PPP 的字节转义

异步 PPP 中 0x7E 变为 0x7D 0x5E，0x7D 变为 0x7D 0x5D；对需转义的控制字符加入转义并作规定变换。接收方遇到转义符后恢复原字节，不能再把恢复出的 0x7E 当成新帧标志。

零比特填充的步骤

发送时扫描帧内容，连续读到五个 1 后立刻添 0；填入的 0 不属于原数据。接收时遇到对应的五个 1 后删去填充 0。遇到数据中的原始 0 会打断连续 1 的计数；标志本身不进行同样的填充。

图解与逐步推演：零比特填充：第五个1之后为什么插0



单独查看图片

零比特填充：第五个1之后为什么插0

查看完整步骤（也可离线阅读）

1. 扫描输入第1位：按原样输出本位；连续1的计数遇0清零。
2. 扫描输入第2位：按原样输出本位；连续1的计数遇0清零。
3. 扫描输入第3位：按原样输出本位；连续1的计数遇0清零。
4. 扫描输入第4位：按原样输出本位；连续1的计数遇0清零。
5. 扫描输入第5位：按原样输出本位；连续1的计数遇0清零。
6. 扫描输入第6位：已连续看到5个1，立即插入一个0，使数据中不会出现定界标志。
7. 扫描输入第7位：按原样输出本位；连续1的计数遇0清零。
8. 扫描输入第8位：按原样输出本位；连续1的计数遇0清零。
9. 接收端删除填充0：接收端在数据区域每看到连续5个1，就删除紧随的填充0，恢复原数据01111110。

检错、纠错与可靠交付 232 CRC 的余数为零不等于数学上保证绝对无错。

差错类型与控制

比特差错是 0/1 翻转；更高层次还可能出现丢失、重复和乱序。误码率 BER 是错误比特数与总传输比特数之比。ARQ 通过差错检测、确认和重传恢复；FEC 通过冗余编码在接收端直接纠错，能力受码距约束。

CRC 的生成与检测

- 双方约定 r 次生成多项式 G(x)，对应 r+1 位二进制除数。
- 待发送数据 M 后补 r 个 0，进行模 2 除法；余数不足 r 位时左补 0，再附到原数据后。

· 接收端用同一 G(x) 除整个码字；余数非零说明检出差错，余数零只说明通过该项校验。

· 模 2 运算用异或，没有进位和借位；检错能力取决于生成多项式。

检错通过与可靠传输

仅丢弃出错帧可减少向上交付的错误数据，但没有恢复丢失、抑制重复和按序交付等机制，仍不是可靠传输。讲义所说“无差错接收”是协议层面的理想化表述；CRC 存在漏检可能，不能宣称接受的帧绝对无比特错误。

图解与逐步推演：CRC模2长除：1101000除以1011的每次异或



单独查看图片

CRC模2长除：1101000除以1011的每次异或

查看完整步骤（也可离线阅读）

1. 补零：数据1101，生成多项式1011（次数3），先补3个0，得到1101000。
2. 对齐第1位并异或：1101000 与移位后的1011000异或，得到0110000。异或相同为0，不同为1，无借位。
3. 对齐第2位并异或：0110000 与移位后的0101100异或，得到0011100。异或相同为0，不同为1，无借位。
4. 对齐第3位并异或：0011100 与移位后的0010110异或，得到0001010。异或相同为0，不同为1，无借位。
5. 对齐第4位并异或：0001010 与移位后的0001011异或，得到0000001。异或相同为0，不同为1，无借位。
6. 得到3位余数001：原数据拼余数得到发送码字1101001；不要把7位补零后的被除数再拼上余数。

发送窗口、接收窗口与确认 233 窗口表示允许范围，可发送余量不是窗口宽度本身。

窗口与缓冲

发送窗口包含已发未确认以及允许发送但尚未发送的序号；可用窗口只是其中未发部分。发送一帧后，可用额度减少，但固定宽度滑动窗口并不因此整体变窄。接收窗口表示当前可以接受的序号范围，是否缓存乱序帧由协议决定。

发送缓存需保留未确认副本以备重传，收到有效确认后才能释放；接收缓存可保存按序但应用未读数据，以及协议允许缓存的乱序数据。

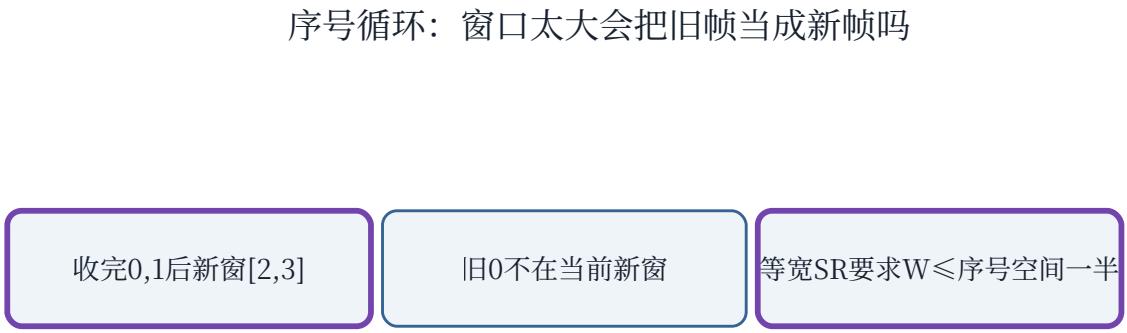
确认方式

累计确认表示某序号之前的连续数据均已正确收到；确认号究竟指“最后收到”还是“下一期望”，要看协议定义。捎带确认是把确认字段放进反向数据帧，与是否累计确认是两个不同维度。

三种 ARQ

协议	发送窗口	接收窗口	重传范围
停等	1	1	当前未确认帧
GBN	可大于 1	1	从最早未确认帧起的所有已发未确认帧
SR	可大于 1	可大于 1	单独出错或超时的帧

图解与逐步推演：序号循环：窗口太大会把旧帧当成新帧吗



单独查看图片

序号循环：窗口太大会把旧帧当成新帧吗

查看完整步骤（也可离线阅读）

1. 2bit序号，模4循环：发送方已发0,1,2，接收方都收到，但对应确认全部丢失。
2. 接收端窗口前移3格：接收端期待的新一轮序号中又出现0，而发送端还保留旧0。
3. 重传旧0产生歧义：这是窗口宽度过大造成的新旧重叠，不是只要“接收方缓存乱序”就一定安全。
4. 限制SR窗口为2：更一般要结合发送与接收窗口边界推导；GBN接收窗为1时常见发送窗上限为2^m−1。

停止等待：差错、确认丢失与重复帧 234 序号和定时器共同解决“没收到”与“收过了”。

基本运行

发送方发送一帧后保留副本并启动定时器，收到正确确认才发送下一帧。接收端检查差错：错误帧丢弃；未按时获得有效确认时，发送端重发。即使物理信道无误码，接收端处理速度仍可能比发送端慢，停等也承担流量约束。

两类故障

数据帧损坏或丢失：接收端未交付该帧，发送方超时后重传。确认损坏或丢失：接收端已交付，发送方仍会重传；接收端应通过序号识别重复帧，丢弃重复数据但再次确认，不能再交给上层一次。

序号与利用率

交替比特协议用 0、1 两个序号即可区分当前帧和最近重复帧。定时器需要足以容纳正常往返，过短会引起不必要重传。

$$U=\frac{T_D}{T_D+RTT+T_A}$$

此处 RTT 表示双向传播与题设纳入的其他等待，T_D 是数据帧发送时延、T_A 是确认发送时延；若题设 RTT 已包含某项，不可重复计入。

图解与逐步推演：停止等待：ACK丢失后如何避免重复交付



单独查看图片

停止等待：ACK丢失后如何避免重复交付

查看完整步骤（也可离线阅读）

1. 发送数据帧0：使用交替位0/1的简化可靠协议，发送方保存未确认帧并启动定时器。
2. 帧0收到并交付：ACK0在此表示确认帧0，勿与TCP确认下一个期望字节的ack混用。
3. ACK0丢失：发送方不能凭空知道接收方已收到，等待确认直至超时。
4. 超时重传帧0：接收方看到旧序号，避免再次交付，但再次确认以帮助发送方继续。
5. 收到ACK0后发1：确认使发送窗口前移；序号帮助区分新数据与重传副本。

GBN：累计确认与回退重传的完整逻辑 235 接收端只等待一个序号，发送端可流水发送多个帧。

发送方三个事件

- 上层交数据：若下一个序号在发送窗口内，封帧发送并保留副本；窗口满时等待、缓存或通知上层，具体取决于实现。
- 收到有效 ACK：按累计确认规则释放连续已确认帧，发送窗口前移；若又有可用序号，可继续发新帧。
- 最早未确认帧超时：重传全部已发送但尚未确认的帧。典型 GBN 只为最早未确认帧维持一个逻辑定时器。

接收方的规则

仅正确且正好按序到达的帧才被接受并交付；其余帧丢弃，并重复确认最近按序接收的信息。不缓存后续失序帧，因此丢失一个早期帧可能导致后续已到达帧也需要重传。

序号空间约束

$1 \le W_T \le 2^m - 1, W_R = 1$

m 位序号共有 2^m 个取值；留出一个序号，避免确认全部丢失后旧帧被误判为新一轮帧。

累计确认算重传

已发 0～7，收到对 0、2、3 的累计确认后，0～3 都已确认；超时需重发 4、5、6、7，共 4 帧。若题设 ACK 表示下一期望序号，必须先换成相同语义再计算。

图画推演与逐步动画

例：帧1丢失后，GBN从最早未确认帧重传

查看完整步骤（也可离线阅读）

1. 0 · 约定窗口与ACK含义：使用3位序号、发送窗口4、接收窗口1，忽略回绕。本例ACK n表示确认到帧n。帧0～3依次发送，先观察帧0。
2. 1 · 帧1在链路中丢失：接收方未收到帧1，仍期待1。发送方尚未收到帧1的有效确认，保留其副本。
3. 2 · 帧2到达但被丢弃：接收窗口为1，只接受当前期待的帧1。乱序帧2不缓存，接收方重复发送ACK 0。
4. 3 · 帧3也被丢弃：帧3同样不是期待的1，也丢弃并重复ACK 0。收到重复ACK不表示帧2、3已经交付。
5. 4 · 最早未确认帧1超时：按本例基本GBN超时模型，从帧1开始重传所有已发送且未确认的帧1、2、3。不能只重传丢失的1后就认为完成。
6. 5 · 重传帧1到达：现在序号与期待值相同，接收并交付1，期待值推进到2，发送ACK 1。

7. 6 · 重传帧2到达：帧2此时按序到达，交付2，期待值推进到3，发送ACK 2。
8. 7 · 重传帧3到达：交付3，期待值推进到4。有效ACK 3累计确认了0～3，发送方可滑动窗口并释放已确认副本。

SR：逐帧确认、缓存乱序与窗口滑动 236 已经确认窗口中间帧，不一定能立即移动窗口左边界。

发送方三个事件

- 上层交数据且序号落在发送窗口内时发送；窗口满时等待或缓存。
- 收到窗口内某帧的 ACK 就标记该帧已确认；只有左边界帧被确认，窗口才能越过一段连续已确认区域。
- 各未确认帧分别计时，某帧超时只重传该帧，不把后续正常帧全部重发。

接收方的规则

正确且处于接收窗口内的帧，无论是否按序都单独确认；失序帧先缓存，待左边界缺口补齐后，将连续数据按序交付并移动窗口。对仍可能由确认丢失引起的旧重复帧，应按协议范围再次发送 ACK，避免发送方一直超时。

半个序号空间

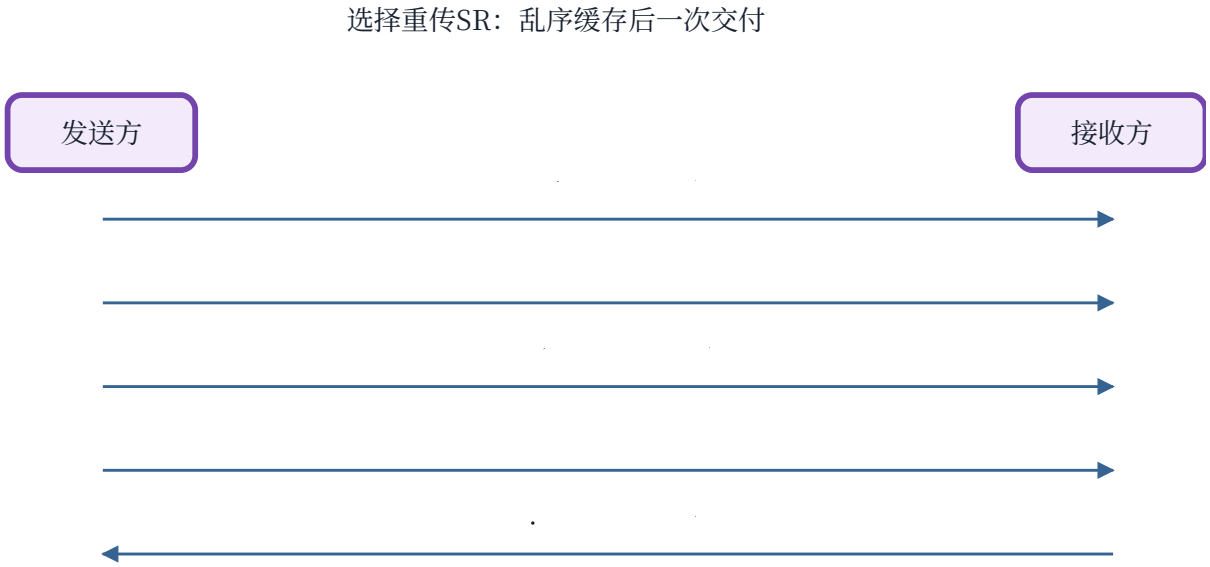
$W_T + W_R \leq m$
一般滑动窗口约束；常见 SR 取相同发送、接收窗口。

$W_T = W_R = W \leq m-1$
等窗口时各不能超过序号空间一半，避免新窗口与旧帧序号重叠。

3 位序号不能开 5 帧等窗口

序号 0～7 若窗口宽 5，接收完 0～4 后新窗口含 5、6、7、0、1。若 0～4 的确认全部丢失，发送端重发旧 0，接收端会误认作新 0。因此等窗口最多为 4。

图解与逐步推演：选择重传SR：乱序缓存后一次交付



单独查看图片

选择重传SR：乱序缓存后一次交付

查看完整步骤（也可离线阅读）

1. 起点：先确认参与者，再沿时间向下阅读。
2. 帧0到达，单独确认0：使用3bit编号、窗口4，满足窗口不超过序号空间一半；帧0按序交付。
3. 帧1丢失 ×：帧1途中丢失，接收方还在等它。
4. 帧2到达，缓存并确认2：SR可缓存窗口内乱序帧2并单独确认，区别于GBN通常丢弃乱序帧。
5. 帧1超时，单独重传1：发送方只重传计时到期的帧1；已确认的帧2不用跟着重发。
6. ACK1；接收窗口推进到3：帧1到达填补缺口，接收方向上层按序交付1和已缓存的2；图中单独ACK按帧编号约定，勿与TCP的期望字节ack混用。

流水协议的信道利用率与窗口下界 237 先算一个往返周期能放多少帧，再检查序号约束。

理想利用率模型

$$U=\min\left(1,\frac{W\cdot T_d}{T_d+RTT+T_a}\right)$$

W 为最大发送帧数；T_d 忽略出错、处理与排队，并明确 RTT 的组成。

$$W_{\min}=\left\lceil\frac{T_d+RTT+T_a}{T_d}\right\rceil$$

为避免连续发送中途停顿，发送窗口需覆盖从首帧发出到确认可用的整个时间。

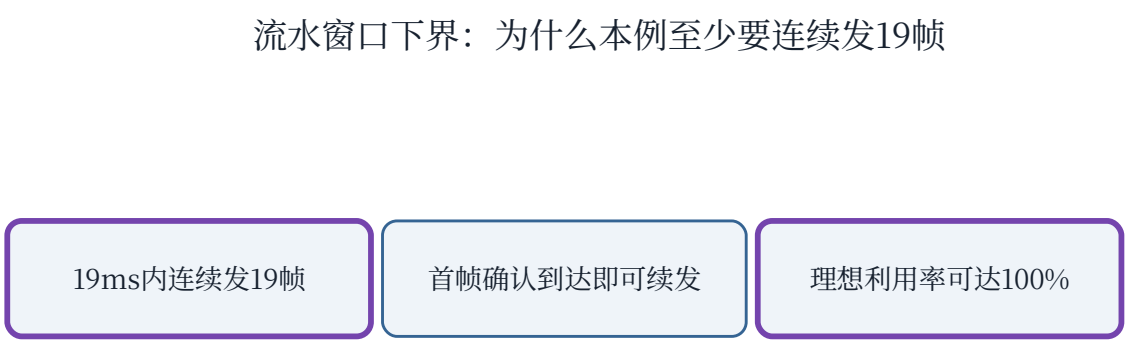
窗口与缓存不同

用“窗口帧数×每帧大小”估算允许的在途数据时，应分清大小是否包括首部以及确认是否累计。求出的理想窗口还须受 GBN/SR 序号范围、实际缓存和接收能力约束，不能只满足带宽时延积就结束。

先得到需求再选位数

若填满管道需要 W=9，GBN 要满足 2^m−1⩾9，至少 m=4；等窗口 SR 要满足 2^(m−1)⩾9，至少 m=5。SR 重传更精细，但相同序号位数允许的等窗口通常更小。

图解与逐步推演：流水窗口下界：为什么本例至少要连续发19帧



单独查看图片

流水窗口下界：为什么本例至少要连续发19帧

查看完整步骤（也可离线阅读）

1. 理想链路参数：忽略ACK发送时间、处理、差错与排队；一个首帧确认周期约Tf+RTT=19ms。
2. 停止等待W=1：发送完首帧后，链路会空等它的确认。
3. 窗口W=10：这仍没覆盖首帧确认返回前的整个等待时间。
4. 窗口W=19：下界ceil(1+RTT/Tf)=19；还必须再检查所用协议的序号空间允许这么大窗口。

介质访问、以太网与无线局域网

计算机网络 · 9 条

FDM、TDM、STDM、WDM 与 CDMA 238 区分固定分配资源和按需竞争使用。

复用方式

方式	划分资源	特征
FDM	不同频带	用户同时占不同频带，通常需要保护间隔；带宽此时是 Hz。
TDM	周期性时隙	各用户在不同时隙使用同一频带；没有数据的固定时隙可能空闲。
STDM	按需分配时隙	仅为有数据的终端安排资源，需携带标识及控制信息。
WDM	不同光波长	光纤中的频分复用；不同波长光可同时传输。
CDM/CDMA	不同码片序列	同时间同频带，通过码序列区分用户。

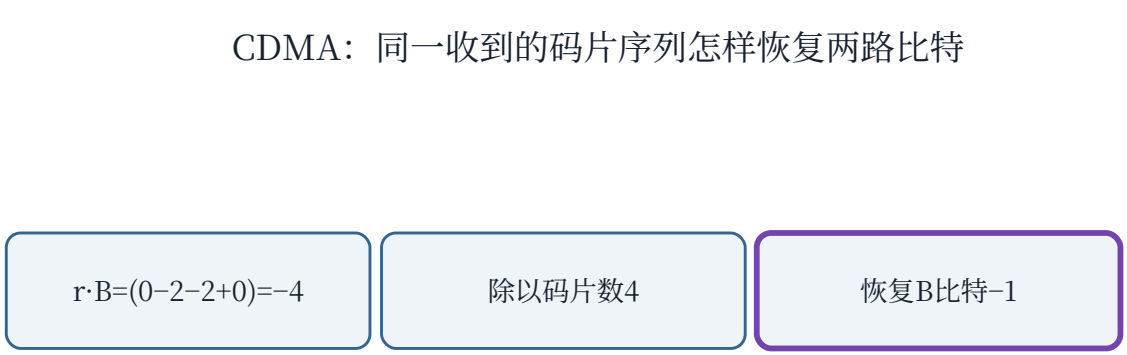
CDMA 的相关检测

把码片 0/1 映射到 −1/+1，各站码片向量相互正交且与自身归一化内积为 1。发送比特 1 用本码片，发送 0 用其反码，不发用零向量。接收叠加信号后与某站码片归一化求内积：结果 +1、−1、0 分别代表该站发 1、发 0、未发。

$$\mathbf{s}_i \cdot \mathbf{s}_j = 0 \ (i \neq j), \quad \mathbf{s}_i \cdot \mathbf{s}_i = 1$$

理想同步且功率条件满足的正交码模型；不要把复用后的和直接当成二进制字符串。

图解与逐步推演：CDMA：同一收到的码片序列怎样恢复两路比特



单独查看图片

CDMA：同一收到的码片序列怎样恢复两路比特

查看完整步骤（也可离线阅读）

1. 选择正交码片：每码片用±1表示。此例两站同步、等幅、无噪声，仅说明相关解码原理。
2. A发+1，B发−1：接收信号可在码片上相加；出现0或±2不表示一定有传输错误。
3. 用A相关：正交的B分量在与A的点积中抵消。
4. 用B相关：若只有理想静默且没有其他同码分量，相关值0可表示该站未发送；实际系统还需处理同步、功控与噪声。

ALOHA 与三种 CSMA 策略 239 先听再发仍可能碰撞，因为信号传播需要时间。

纯 ALOHA 与时隙 ALOHA

纯 ALOHA 有数据就发，未获得成功确认则随机等待后重传；一帧发送前后各一个帧时长内的其他发送都可能造成重叠。时隙 ALOHA 将时间同步划为等长时隙，只能在时隙开始发送，使冲突危险期缩短。理想泊松模型下最大吞吐量分别为 1/(2e) 与 1/e，约 0.184、0.368。

监听到忙/闲时怎样做

策略	空闲时	忙时
1-坚持	立即发送	持续监听，空闲即发送；多个等待站可能同时发。
非坚持	立即发送	随机等待后再监听；减少碰撞但可能浪费空闲时间。
p-坚持	时隙系统中以概率 p 发送，以 1−p 推到下时隙	等待忙期结束后继续按策略竞争。

共同点

载波监听只描述本地此刻看到的信道状态，远端刚发出的信号可能还未到达。因此 CSMA 降低碰撞概率，但不会自动消除碰撞。随机接入与固定时分、频分复用的资源安排不同。

CSMA/CD、争用期与最短帧 240 共享半双工以太网要在发送期间检出最远端碰撞。

运行顺序

发送前监听：忙则按 1-坚持方式继续监听；空闲则发。发送过程中继续检测碰撞；若发生碰撞，停止正常发送，发送强化碰撞信号，再随机退避后重新竞争。强化信号用于使其他站明确碰撞，不是“通知网络拥塞”的拥塞控制报文。

为什么需要最短帧

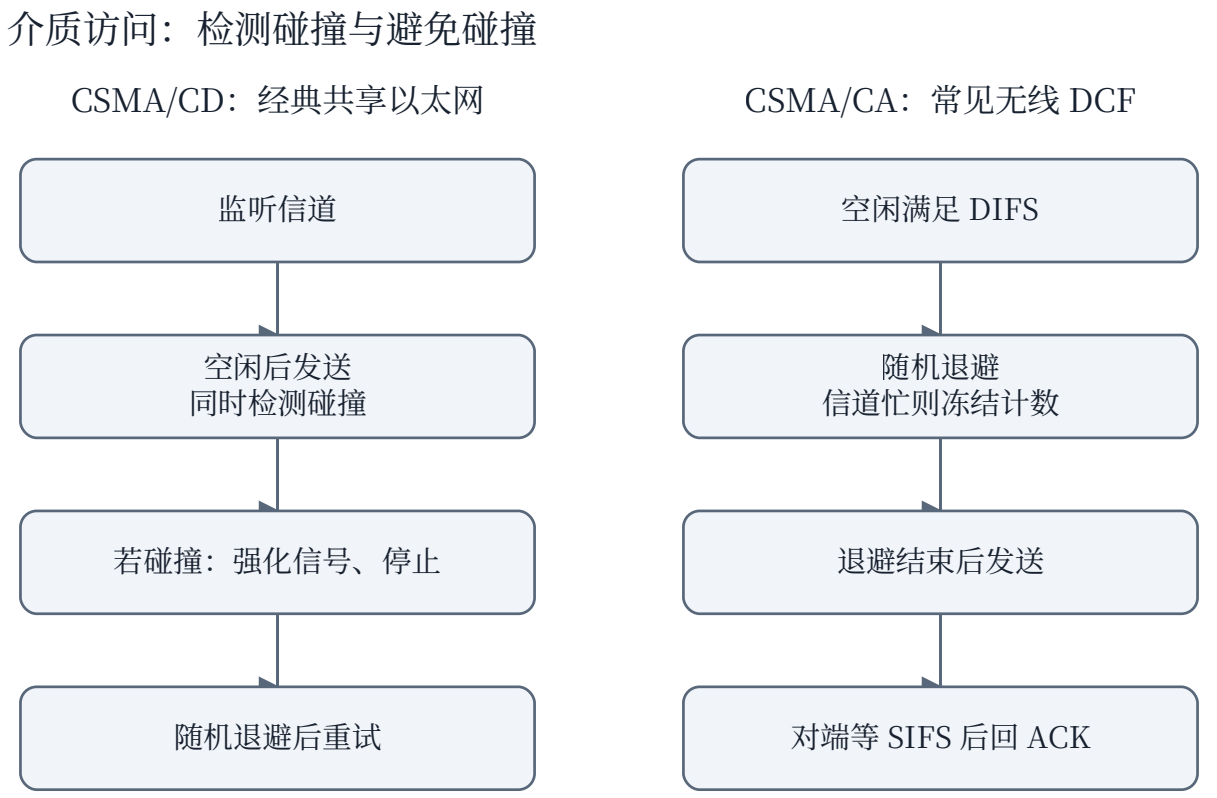
$T_{\min} \geq 2\tau, L_{\min} \geq 2\tau R$

τ 是总线两端单程最大传播时延；发送端必须保持发送直到最坏碰撞信号返回。

经典以太网的长度边界

经典以太网 MAC 帧最短 64 B，从目的 MAC 到 FCS 计，不含前导同步字段；不足载荷用填充补齐。100 Mb/s 等模型中的最小帧长和网络跨度须结合标准及题设。全双工交换链路不使用 CSMA/CD，不再依靠争用期来限制链路跨度。

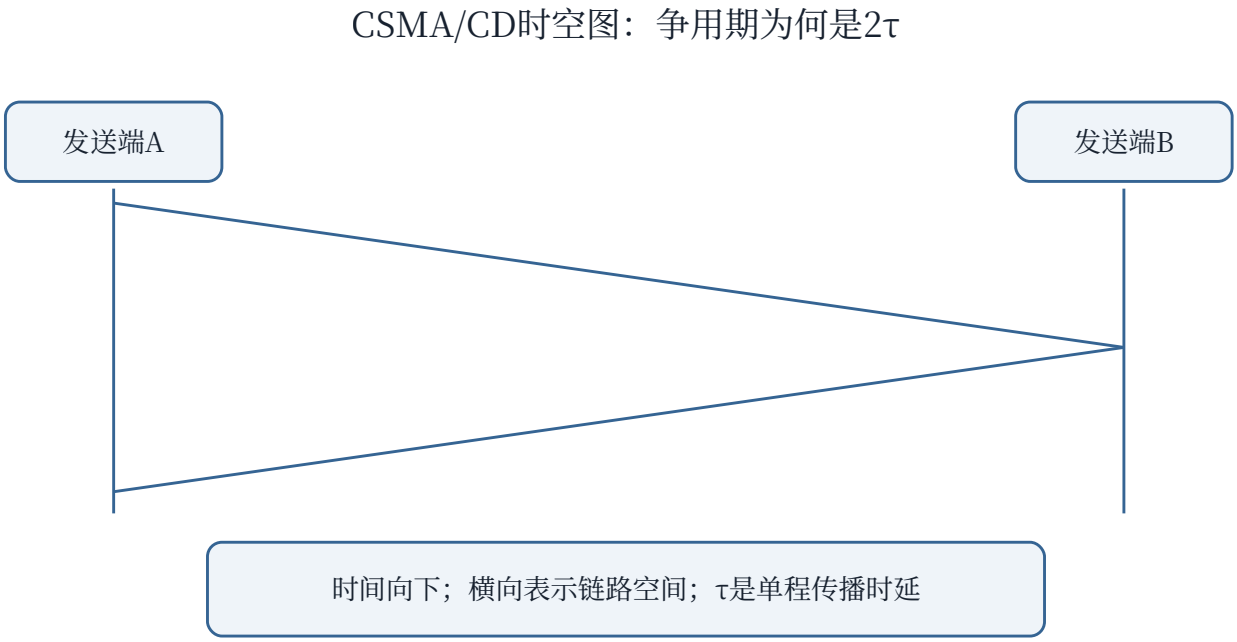
机制图解



CA 不保证绝无碰撞；SIFS < DIFS；全双工交换以太网不使用 CD 竞争。

单独查看图片

图解与逐步推演：CSMA/CD时空图：争用期为何是2τ



单独查看图片

CSMA/CD时空图：争用期为何是2τ

查看完整步骤（也可离线阅读）

1. A刚开始发送：两端传播时延τ。最坏情形中，A先发送；B在A信号即将到达前一瞬仍听到空闲，也开始发送。
2. B在接近τ时开始发送：B在t≈τ才感知碰撞；A尚需等待B的信号沿链路反向传播回来。
3. A在接近2τ时检测到碰撞：要确保A尚未发完时能检测到最远端碰撞，最短帧发送时间必须至少2τ。此为共享半双工CSMA/CD模型。

截断二进制指数退避与共享以太网效率 241 退避是随机倍数，不是每次机械翻倍的固定等待。

退避计算

设第 n 次重传采用 k=min(n,10)，从 0～2^k−1 中等概率选择整数 r，退避 r 个基本退避单位。经典以太网基本单位为争用期 2τ；达到规定的尝试上限仍失败则停止并报告。

$$k=\min(n,10),T_{\text{backoff}}=r2\tau,0\le r\le 2^{k}-1$$

题设可能把“重传次数”与“总尝试次数”分别计数，先确定 n 从哪里开始。

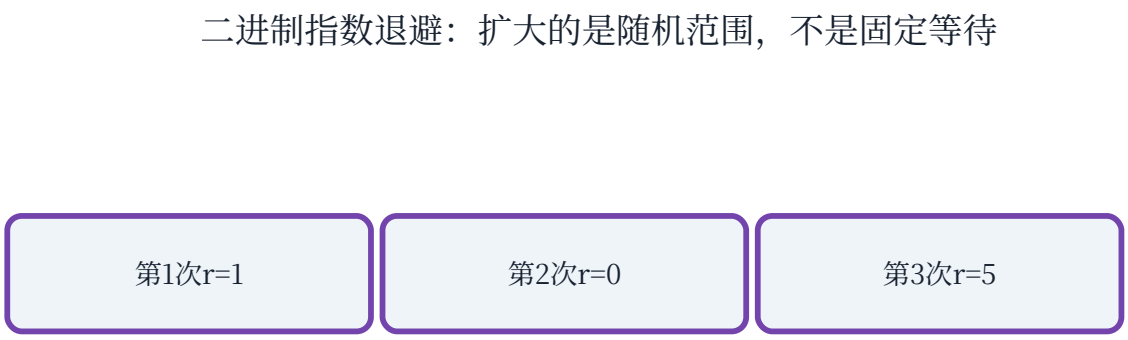
前两次选择范围

第一次重传 k=1，r∈{0,1}；第二次 k=2，r∈{0,1,2,3}。所以可以随机选择零等待，不是第 n 次一定等待 2^n 个时隙。

传播时延与效率

令 a=τ/T_D；在理想无碰撞、单站连续占用模型下，一帧占用 T_D+τ，极限有效利用率为 1/(1+a)。真实共享网络还会受竞争和碰撞开销影响。缩短介质、适当增大帧载荷可减小 a；但不能为提高效率忽略 MTU 或无限增大帧。

图解与逐步推演：二进制指数退避：扩大的是随机范围，不是固定等待



单独查看图片

二进制指数退避：扩大的是随机范围，不是固定等待

查看完整步骤（也可离线阅读）

1. 第一次碰撞后：经典以太网CSMA/CD模型；一个时隙为规定的争用期长度。
2. 第三次碰撞后：随机范围扩大不表示每次等待都必须比上一次长。
3. 超过10次时截断：按经典截断二进制指数退避模型，增长指数在10截断。
4. 具体抽样示例：等待可依次为1、0、5个时隙。达到协议规定的尝试上限后应报告失败，不能无限重传；共享半双工与现代交换全双工场景要区分。

以太网帧、MAC 地址与网卡 242 明确“以太网帧长”是否包含前导字段。

MAC 与网卡

典型 MAC 地址长 48 bit，以 6 个字节表示；网卡在主机与网络间完成串并转换、缓存、编码解码、帧封装/拆封及介质访问等，涉及物理层和链路层。硬件可完成大量收发工作，但软件驱动、DMA 和中断等仍与主机协作。

MAC 地址既可以由厂商分配，也存在本地管理和软件修改；“任何网卡地址永远不能改变”不是一般结论。接收时可按单播、多播、广播或混杂模式筛选。

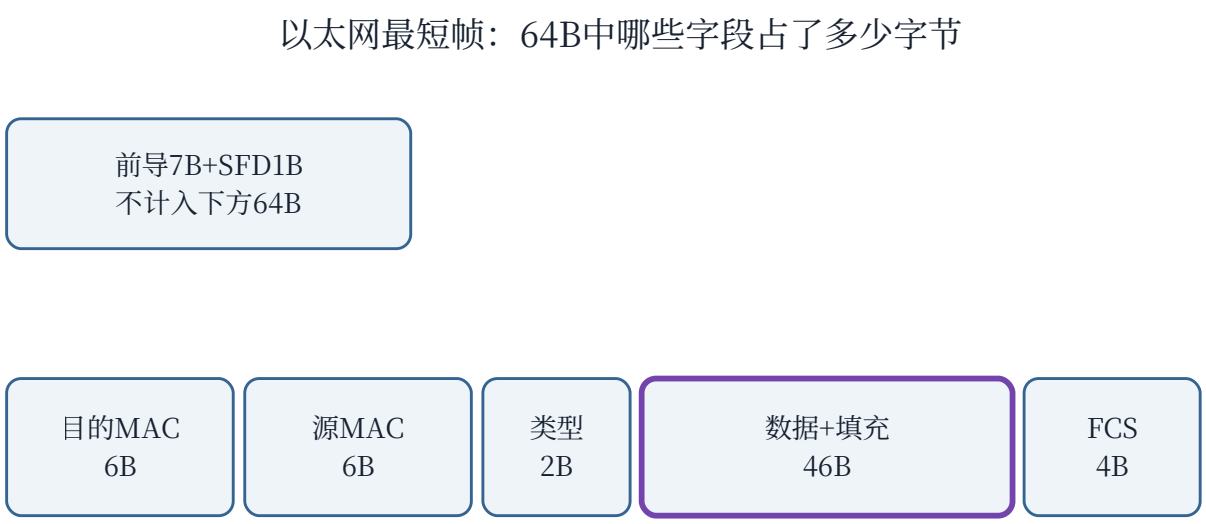
Ethernet II 格式

字段	长度	说明
目的/源地址	各 6 B	链路上的目的与源 MAC。
类型	2 B	标识上层协议，如 IPv4、ARP。
数据与填充	46～1500 B	普通以太网的载荷范围；不足最小值需填充。
FCS	4 B	CRC-32，覆盖目的地址到数据/填充末尾。
前同步码+帧开始定界符	7 B+1 B	物理同步使用，通常不计入 64～1518 B MAC 帧长。

无需确认的服务

以太网不对每帧编号和确认，FCS 出错帧通常直接丢弃；上层若需要可靠性自行恢复。10BASE-T 中 10 表示典型数据率 10 Mb/s，BASE 表示基带，T 表示双绞线；并非所有现代 BASE 以太网都采用曼彻斯特编码。

图解与逐步推演：以太网最短帧：64B中哪些字段占了多少字节



单独查看图片

以太网最短帧：64B中哪些字段占了多少字节

查看完整步骤（也可离线阅读）

1. 最短MAC帧64B：6+6+2+46+4=64B；46B是数据与必要填充的总长度，不要求应用本身必须生成46B。
2. 前导字段另计：前导码7B和SFD1B用于同步与帧起始，通常不计入64～1518B的以太网MAC帧长。

3. 应用数据只有20B：在本例无VLAN标签、普通以太网格式下，需补26B，使数据加填充达到46B；再计算FCS。

CSMA/CA、帧间间隔与无线确认 243 无线碰撞难以边发边检测，因此依靠避免和确认机制。

DCF 竞争过程

站点监听信道，满足规定的 DIFS 空闲时间并按退避规则竞争；退避计数仅在信道空闲的时隙递减，信道变忙时冻结，重新空闲并等待规定间隔后恢复。倒计时到零才发送。接收端成功收到单播数据后，通常在 SIFS 后发 ACK；未获确认则按重试规则处理。

间隔与优先级

SIFS 最短，用于 ACK、CTS 等同一次交换中的快速响应；PIFS 位于 SIFS 与 DIFS 之间，服务特定协调方式；DIFS 用于普通分布式竞争。因为等待更短，响应帧可以先于新竞争数据获得介质。

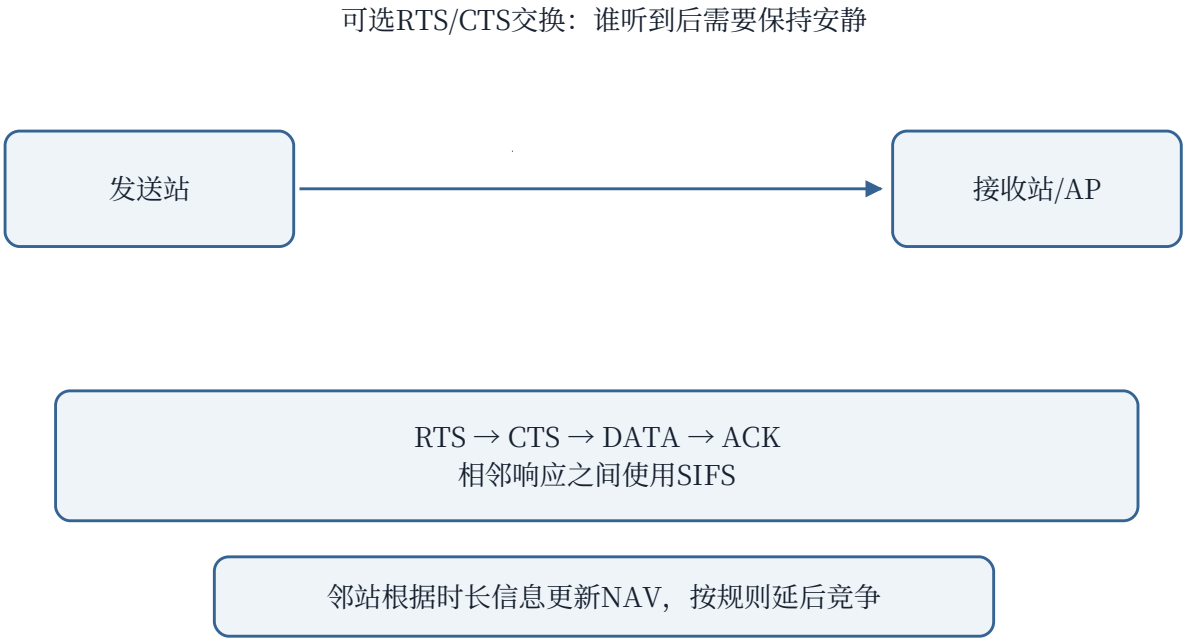
隐藏站与 RTS/CTS

听不到对方的两个发送站仍可能在共同接收端发生碰撞。可选 RTS/CTS 在发送数据前预约介质，其他站通过持续时间字段设置 NAV，进行虚拟载波监听。RTS/CTS 增加开销，不能消除所有碰撞；不能因用了 CA 就认定“永远无碰撞”。

结构对照图：可选RTS/CTS交换：谁听到后需要保持安静

普通竞争访问典型先等待DIFS并随机退避，之后可以直接发DATA，也可采用RTS/CTS保护。CTS、ACK等响应采用更短的SIFS。

RTS/CTS能缓解隐藏站问题，但不是所有802.11数据帧发送前都必须有RTS/CTS；无线无法简单等同共享以太网的边发边检测碰撞。



单独查看图片

802.11 帧类别与四地址解释 244 接收/发送地址是当前无线一跳，源/目的地址是端系统。

三类 MAC 帧

数据帧传送上层数据；控制帧辅助介质访问和确认，如 RTS、CTS、ACK；管理帧用于发现、加入、认证、关联、离开和漫游等。基本数据帧包含 MAC 首部、帧主体与 FCS，首部长度及字段随模式而异，不能把一个示例长度强行用于所有扩展格式。

To DS/From DS 地址表

To DS	From DS	地址1	地址2	地址3	地址4
0	0	目的/接收站	源/发送站	BSSID	通常不用
1	0	接收 AP（BSSID）	发送/源站	最终目的站	不用
0	1	接收/目的站	发送 AP（BSSID）	原始源站	不用
1	1	当前接收端	当前发送端	最终目的站	原始源站

A 经 AP 发给 B

A→AP：地址1=AP，地址2=A，地址3=B；AP→B：地址1=B，地址2=AP，地址3=A。若 AP 桥接到有线以太网，转成以太网帧后源/目的仍分别是原始端系统，不把 AP 的 MAC 无条件塞成源地址。

图解与逐步推演：A经AP发送给B：无线帧里的三个地址怎样变化



单独查看图片

A经AP发送给B：无线帧里的三个地址怎样变化

查看完整步骤（也可离线阅读）

1. A发向AP，ToDS=1/FromDS=0：本例A、B通过同一基础设施AP通信；以基本地址角色模型说明。
2. AP再发给B，ToDS=0/FromDS=1：当前无线一跳的发送者从A变成AP，但原始源仍是A。
3. 为什么不能只背源MAC/目的MAC：ToDS=FromDS=1的基本四地址模型中，地址3为最终目的、地址4为原始源；不要把本例三地址规则硬套所有扩展帧。

交换机自学习、转发与冲突域 245 学源地址，查目的地址；未知单播泛洪不是广播帧。

收到帧后的判断

先用源 MAC 与入端口更新转发表，条目会老化。再查目的 MAC：若目的端口已知且与入端口不同，定向转发；若目的仍在入端口侧，过滤；目的未知则在同 VLAN 的其他端口泛洪。广播帧在同广播域扩散，多播按设备支持的规则处理。

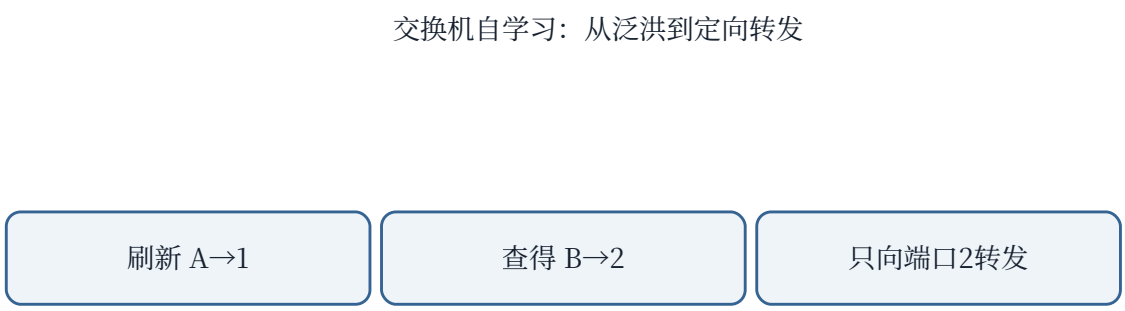
交换与集线器

交换机是多端口网桥，工作到链路层，按 MAC 转发；集线器在物理层重复信号。交换机端口划分冲突域，但默认同 VLAN 仍属一个广播域。全双工端口没有 CSMA/CD 意义上的碰撞；不能把“每端口独立冲突域”误解为全双工链路仍在运行碰撞检测。

两种交换方式

直通方式读取目的地址后尽早转发，时延低但可能转发错误帧；存储转发方式完整缓存并检查帧，再转发。交换环路会导致广播风暴和 MAC 表抖动，可用生成树协议保持无环逻辑拓扑。

图解与逐步推演：交换机自学习：从泛洪到定向转发



单独查看图片

交换机自学习：从泛洪到定向转发

查看完整步骤（也可离线阅读）

1. 空转发表：初始交换表为空。
2. A→B到达：先按源MAC学习A的位置，再查目的B；未知单播向同VLAN除入端口外的端口泛洪。
3. B→A回应：回应到达时学习B，再查A；已知单播按表定向转发。
4. 再发A→B：表项可能老化，本例尚未老化；广播、组播、环路等机制另论。

VLAN、802.1Q 与跨交换机转发 246 VLAN 划分广播域，跨 VLAN 需要三层转发。

逻辑分组

VLAN 将交换网络划成逻辑广播域，可跨多台交换机，与主机的地理位置不必一致。同 VLAN 可通过二层交换通信；不同 VLAN 即使接在同一交换机，也通常要经路由器或三层交换功能互通。

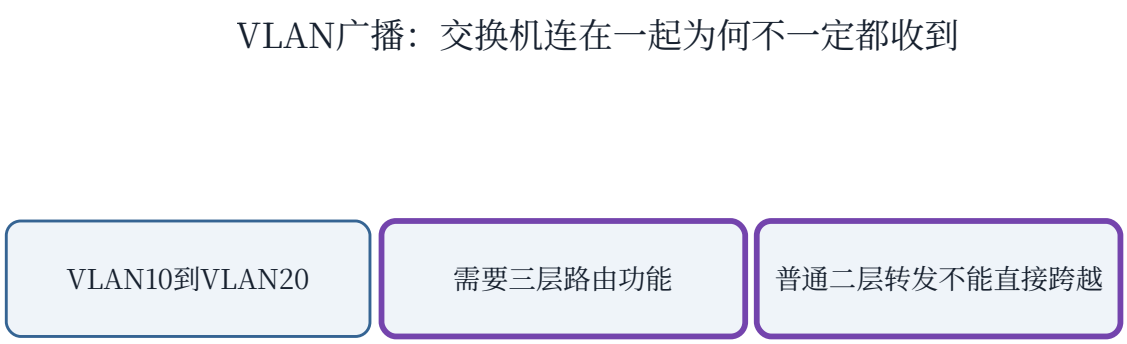
802.1Q 标签

标签插在源 MAC 与原类型字段之间，共 4 B；TPID 常为 0x8100，其后包括优先级、丢弃资格和 12 bit VID 等控制信息。VID 的 0 和 4095 有保留用途，不能直接宣称 4096 个值全部可作普通 VLAN。普通带标签 MAC 帧最大长度由 1518 B 增为 1522 B（按该基本格式计）。

接入口与干道口

终端接入口通常收发无标签帧，交换机内部依据端口归属确定 VLAN；承载多个 VLAN 的交换机间干道通常保留标签。帧到达目标终端的接入口前按配置去标签。是否存在本征 VLAN 无标签传送应看具体端口配置，不能说所有交换机间帧都必有标签。

图解与逐步推演：VLAN广播：交换机连在一起为何不一定都收到



单独查看图片

VLAN广播：交换机连在一起为何不一定都收到

查看完整步骤（也可离线阅读）

1. 两台交换机S1和S2：假设链路与trunk均允许VLAN10，忽略Native VLAN等特殊配置。
2. 在S1内泛洪：VLAN划分广播域；同一交换机上不同VLAN的接入口也不会因此收到同一广播。
3. 经trunk发往S2：通常使用802.1Q标签在共享trunk上区分VLAN，接入端口对普通主机的封装另按配置。
4. 跨VLAN通信：路由器或三层交换机可提供互联；分开广播域不等于它们永远无法通信。

点对点链路与网络互联

计算机网络 · 2 条

PPP 的组成、帧格式与透明传输 247 点到点链路无需按目的 MAC 在共享介质上选择接收站。

三个组成部分

PPP 是面向字节的点到点链路协议：封装规则把多种网络层分组放到串行链路；LCP 建立、配置、测试和终止链路；一组 NCP 分别协商不同网络层协议参数。PPP 支持差错检测、透明传输、链路状态检测、MTU 与网络层地址等配置。

基本帧字段

字段	典型值/长度	作用
Flag	0x7E，1 B	标记帧边界，连续空标志不构成有效数据帧。
Address/Control	0xFF / 0x03，各 1 B	基本格式中的固定字段，不用于普通以太网式的源/目的 MAC 寻址。
Protocol	通常 2 B	0x0021 为 IPv4，0xC021 为 LCP，0x8021 为 IPCP。
信息	按协商 MTU，默认典型最大 1500 B	无需以太网式的 46 B 最小载荷。
FCS 与结束 Flag	基本格式 FCS 通常 2 B	检错后错误帧丢弃；可存在协商扩展，按题设格式计。

PPP 不做什么

PPP 本身不提供逐帧可靠重传，不使用 CSMA/CD，没有以太网争用期规定的最短帧长。异步链路采用字节转义，面向比特的同步链路可采用零比特填充。点到点只有唯一对端，网络层下一跳已经由出接口确定时，无需 ARP 查找该 PPP 对端的 MAC 地址。

图解与逐步推演：PPP字节填充：7E与7D分别怎样转义



单独查看图片

PPP字节填充：7E与7D分别怎样转义

查看完整步骤（也可离线阅读）

1. 处理字节0x45：本字节按示例ACCM配置无需转义，直接输出。
2. 处理字节0x7E：遇特殊字节，先插入转义字节7D，再将原字节与20异或。
3. 处理字节0x7D：遇特殊字节，先插入转义字节7D，再将原字节与20异或。
4. 处理字节0x00：本字节按示例ACCM配置无需转义，直接输出。
5. 接收端逆操作：看到7D就去掉它，并把随后字节与20异或；5E还原7E，5D还原7D，恢复原数据。

PPP 工作阶段：从物理连接到网络层打开 248 先 LCP 再按需认证和 NCP，不能把 IP 配置当作物理建链。

状态推进

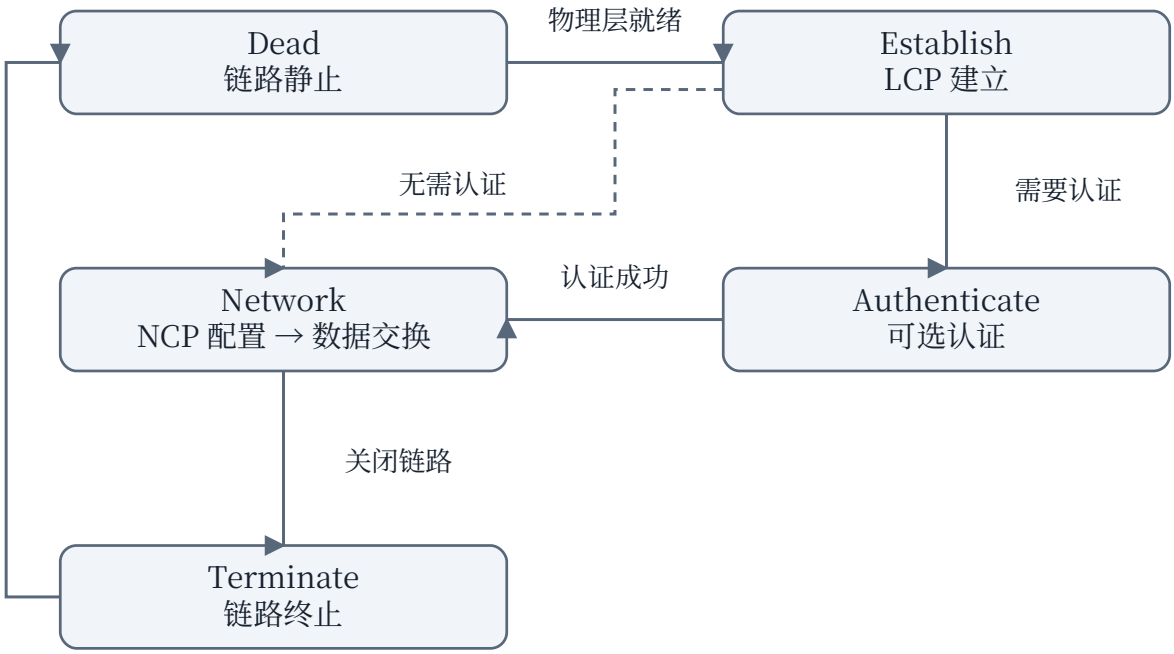
- 链路静止：尚未建立物理连接；检测到物理载波后进入建立阶段。
- LCP 协商：交换配置请求、确认、否认或拒绝，协商链路参数；“否认”通常表示能识别但值不接受，“拒绝”表示某选项不接受或不支持。
- 可选鉴别：按配置进行 PAP/CHAP 等认证；失败则终止，成功才继续。
- 网络层配置：相应 NCP 协商网络层参数，不同网络层协议可有各自的打开状态。
- 链路打开：承载已协商的网络层分组，LCP 可用回送请求/回答检查链路。
- 终止：收到终止请求、链路故障或管理关闭后释放连接，物理连接消失后回到静止。

顺序与协议字段

LCP、认证报文、NCP 和用户 IP 数据都可由 PPP 承载，但 Protocol 字段不同。某个网络层尚未配置完成，不应仅因物理线路通了就认定它的数据已经可以正常传输。

机制图解

PPP：LCP 建链，NCP 配网络层



认证可跳过；任意阶段发生失败或断链可终止，图中只画主路径。

单独查看图片

网络层、IPv4 与地址规划

计算机网络 · 16 条

异构互联、路由与转发 249 控制平面决定路径，数据平面逐包执行转发。

异构网络如何统一

不同网络可采用不同介质、编码、帧格式和接入协议；通过统一的网络层编址与转发机制形成逻辑互联网络，使主机不必理解沿途每种链路细节。物理中继器和二层网桥通常只是扩大同一网络，网络层互联依靠路由器等设备。

路由与转发

项目	路由选择	分组转发
问题	到目标应选哪条路径?	当前分组从哪个端口发出?
范围	由路由算法及协议维护网络可达性与路径	当前路由器依据转发表逐包处理
主要状态	路由表侧重路径与度量的维护	转发表侧重快速匹配、下一跳与出接口

路由器结构

输入端口恢复比特、拆帧并提取 IP 分组；交换结构将输入分组送到相应输出端口；输出端口排队并封装新的链路帧。交换结构可基于存储器、总线或互联网络。路由协议报文要交控制处理逻辑，普通过境数据主要沿转发路径处理。

端到端与逐跳

没有 NAT、隧道等特殊处理时，源/目的 IP 通常端到端保持，TTL 每过一个路由器减少，IPv4 首部校验和相应更新。MAC 地址逐链路重新封装，二层交换机在同一链路网络转发时通常不改原帧源/目的 MAC。

SDN 的分层、接口与控制集中 250 逻辑集中管理可以由多台控制器共同实现。

控制与转发分离

传统路由器通常同时承担路由计算和逐包转发。SDN 将控制逻辑与数据转发分离：控制器掌握网络状态，按策略生成或下发转发规则，交换/路由设备按规则高速处理数据。网络应用通过可编程接口提出策略需求。

三类接口

接口	连接对象	作用
北向	控制器与网络应用	给应用提供网络抽象、状态和策略编程能力。
南向	控制器与转发设备	下发规则、读取状态；OpenFlow 是一种方案，不是唯一方案。
东西向	多个控制器之间	协调状态和控制，增强扩展性及可靠性。

优势和风险

全局视角利于统一优化、自动配置与开放接口，设备转发和软件策略可独立演进；代价是控制器负载、状态一致性、故障与安全防护需要专门设计。逻辑集中不意味着必须用一台物理服务器，也不意味着每一个分组都先送控制器计算路径。

拥塞控制与流量控制：保护对象不同 251 接收方有空间，不代表网络沿途有能力承载。

两种控制

流量控制保护接收端，协调发送速率与接收缓存/应用读取能力；拥塞控制保护网络，避免对链路、交换设备、缓存和处理资源的总需求过大。拥塞可能导致排队时间增加、丢包和重传，严重时增加负载反而使有效吞吐下降。

开环与闭环

开环方法在设计和准入阶段预防拥塞，如资源预留、流量整形和调度策略；闭环方法监测拥塞，把反馈传给可采取行动的结点，再调节运行。两者强调是否利用运行反馈，并不是“开环系统永远不能修改任何配置”。

资源之间不能简单替代

增加缓存可暂缓溢出却可能加大时延；增加链路速率也可能把瓶颈推到后续处理器或链路。丢包是拥塞信号之一，也可能来自误码等原因，不能把“所有丢包都等于拥塞”作为普遍结论。

IPv4 首部：字段宽度与计量单位 252 首部长度、总长度和片偏移分别采用不同单位。

基本字段

字段	位数	解释
版本/IHL	4 / 4	IPv4 版本为 4；IHL 按 4 B 计，常见 5 即 20 B，最大 15 即 60 B。
区分服务	8	用于服务分类及相关标记，具体位义按协议约定。
总长度	16	整个 IPv4 数据报的字节数，含首部，最大 65535 B。
标识	16	分片重组关联所需信息；同一原数据报各片保持同一标识。
标志/片偏移	3 / 13	DF 决定是否允许分片；MF 标记是否还有后续片；偏移以 8 B 计。
TTL/协议	8 / 8	TTL 逐路由器递减；协议标识上层载荷，如 ICMP=1、TCP=6、UDP=17。
首部校验和	16	只检验 IPv4 首部，不覆盖整个载荷。
源/目的地址	各 32	分别占 4 B，标记发送和接收接口地址。

选项与更新

选项使首部长度可变，填充保证首部是 4 B 整数倍。路由器修改 TTL 后需更新首部校验和；分片时各片也各有自己的 IP 首部。IP 提供尽力而为的数据报服务，不为每个过境数据报进行端到端确认或重传。

MTU 的对象

链路层 MTU 限制可装入帧载荷的 IP 数据报总长。例如普通以太网 MTU 为 1500 B，包含 IP 首部但不包含以太网 MAC 首尾。IPv4 总长超过 MTU 时，若允许分片则分片；DF=1 时不能直接强行切开。

IPv4 分片与再次分片 253 片偏移始终相对原始数据部分，不随再次分片归零。

操作顺序

- 确定每片首部长度 H，最大可装数据量不超过 MTU−H。
- 除原数据报最后一片外，数据长度须是 8 B 整数倍；通常取 floor((MTU−H)/8)×8。
- 各片总长度=本片首部+本片数据；片偏移=本片第一个数据字节在原数据部分的位置/8。
- DF=0 才允许分片；MF=1 表示原数据报仍有后续片，不是“这个片将来还会不会被切分”。
- 片在最终目的主机重组，普通中间路由器不先重组再转发；一个片丢失可能使整个原数据报无法重组。

4000 B 数据报经过 MTU 1500 B

原总长 4000 B、首部 20 B，则数据 3980 B。每个前片最多装 1480 B，故分为 1480、1480、1020 B 三段数据，各片总长为 1500、1500、1040 B；片偏移分别为 0、185、370；MF 分别 1、1、0，标识均为原值 777。

再次分片与首部

某个中间片再次被分片时，子片偏移在原片偏移基础上增加；若原片 MF=1，最后一个子片也仍应保留 MF=1。存在选项时，各片首部长度可能受复制规则影响，不能未看题设就一律按 20 B。

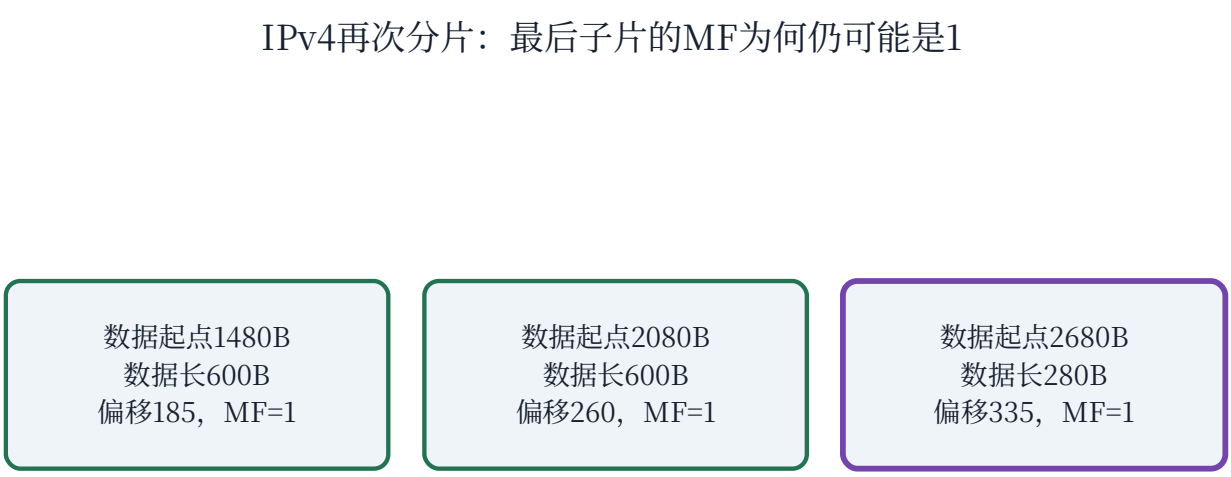
图画推演与逐步动画

例：4000B IPv4数据报通过MTU1500链路

查看完整步骤（也可离线阅读）

1. 0 · 区分首部和数据：假定原数据报总长度4000B、首部20B无选项、允许分片（DF=0）。原始数据为3980B，分片后每片都各带20B首部。
2. 1 · 求非末片最大数据量：MTU1500B包含IP首部，因此单片最多容纳1480B数据。1480能被8整除，满足非末片数据长度须为8B整数倍的规则。
3. 2 · 形成第1片：取原数据字节[0,1480)，数据1480B，总长1500B，片偏移0，MF=1，表示后面还有分片。
4. 3 · 形成第2片：取原数据字节[1480,2960)，数据1480B，总长1500B，片偏移1480/8=185，MF=1。偏移单位不是字节。
5. 4 · 形成最后一片：剩余3980−2960=1020B，总长1040B，片偏移2960/8=370，MF=0。最后一片的数据长度不必为8的整数倍。
6. 5 · 目的主机重组：三片共享原标识。目的主机按偏移恢复字节顺序，末片MF=0给出结束位置；中间路由器通常不负责重组。数据总量1480+1480+1020=3980B。

图解与逐步推演：IPv4再次分片：最后子片的MF为何仍可能是1



单独查看图片

IPv4再次分片：最后子片的MF为何仍可能是1

查看完整步骤（也可离线阅读）

1. 生成子片1：本子片数据从原始数据的1480B处开始，片偏移=1480÷8=185。原片MF=1，所以最后一个子片的MF也仍为1。
2. 生成子片2：本子片数据从原始数据的2080B处开始，片偏移=2080÷8=260。原片MF=1，所以最后一个子片的MF也仍为1。
3. 生成子片3：本子片数据从原始数据的2680B处开始，片偏移=2680÷8=335。原片MF=1，所以最后一个子片的MF也仍为1。

IPv4 地址、接口与传统分类 254 IP 标识接口及其网络位置，不是给整台多接口路由器只发一个号码。

层次地址

IPv4 为 32 bit，通常点分十进制表示；通过网络前缀与主机部分表达层次。一个路由器通常每个参与三层转发的接口各有地址；同一主机可因多网卡或配置拥有多个地址。地址唯一性的范围受公有地址、私有地址及本地网络配置影响。

传统 A/B/C 与多播

类别	前导位/首字节范围	传统默认前缀
A	0；通常可用传统网络号 1～126	/8，主机部分 24 bit

类别	前导位/首字节范围	传统默认前缀
B	10；128～191	/16，主机部分 16 bit
C	110；192～223	/24，主机部分 8 bit
D	1110；224～239	IPv4 多播，不按普通主机网络号划分
E	1111；240～255	历史保留用途，另有 255.255.255.255 特殊广播地址

传统类别不能替代 CIDR

现行前缀长度由 /n 或掩码决定，看到首字节为 172 就直接断言网络一定 /16 会出错。早期资料中“B 类或 C 类最小网络号不能分配”等说法不应当作现代 CIDR 规划规则。普通含 h 位主机号的广播子网常有 2^h−2 个主机地址，但 /31 点到点和 /32 主机路由等另有约定。

特殊 IPv4 地址与私有地址 255 先区分源地址、目的地址和路由表中的前缀。

特殊地址表

地址	主要含义	使用限制
主机部分全 0	普通子网的网络地址	通常不分配给普通主机接口。
主机部分全 1	该子网定向广播地址	作为目的使用；路由器是否转发定向广播受安全配置限制。
255.255.255.255	本地受限广播	不由路由器转发到其他网络。
0.0.0.0	未指定/初始化中的本机地址语境	DHCP 初始请求可用作源；0.0.0.0/0 在路由表中表示默认路由，语境不同。
127.0.0.0/8	环回地址块，常见 127.0.0.1	交给本机协议栈，不在外部链路传送。

三块私有地址

私有 IPv4 地址块为 10.0.0.0/8、172.16.0.0/12、192.168.0.0/16。不同私网可重复使用，公共互联网通常不为其提供全局可达路由。企业内部路由器可以正常转发私有地址；“所有路由器都不转发私有 IP”是错误的绝对化表述。

历史表示法

网络号全 0、主机号为特定值的“本网络某主机”写法属于历史地址语义，现代联网一般使用明确前缀和完整接口地址。规划地址时不能把历史分类限制与现行掩码规则混用。

NAT 与 NAPT：改写什么、怎样返回 256 普通转发通常不改 IP 端点，地址转换则有明确的改写状态。

基本转换

NAT 在私网与外网边界改写 IP 地址，并维护转换关系；NAPT/PAT 进一步使用传输层端口区分多个内部会话，使多个内部端点可以共享一个公有 IPv4 地址。狭义一对一地址 NAT 与多对一端口转换要区分。

出站与入站

内部主机发出数据报时，边界设备将源私有地址以及必要时的源端口改成外部映射；回包的目的地址/端口对应映射表，再改回内部端点转发。改写 IP 和传输层伪首部相关字段后，校验和也须相应更新。

共享公有地址

10.0.0.2:5000 与 10.0.0.3:5000 可分别映射到 203.0.113.5:40001、203.0.113.5:40002。回包依据外部端口找到正确内部主机。端口号相同并不冲突，因为内部 IP 不同；外部映射仍须能区分会话。

能力与局限

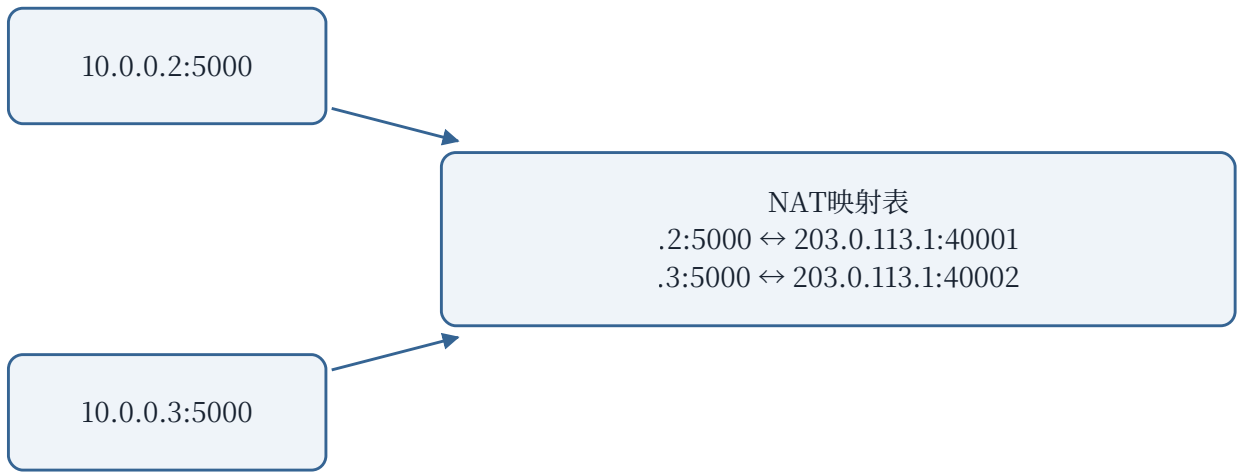
NAT 缓解 IPv4 地址不足，但破坏端到端地址透明性；外部主动发起连接通常需要预先配置映射或其他机制。NAT 不等同于完整安全防火墙，也不是所有私有网互联都必须经过 NAT。

结构对照图：NAPT：两个内网端点映射为不同公网端口

示例两个内网主机访问同一外部服务，NAPT把源IP和源端口改写为公网地址与不同端口。203.0.113.1使用文档示例地址。

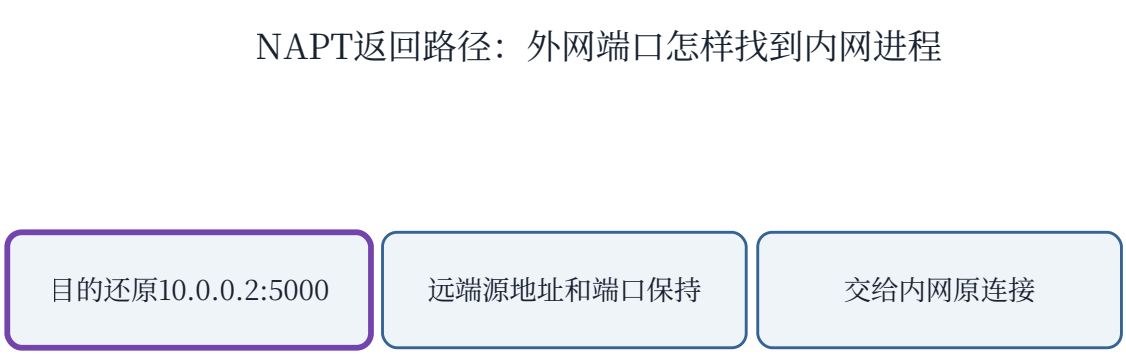
返回报文到达不同公网端口后，按转换状态还原内网目的IP与端口；相关校验和也要更新。具体实现可以把远端元组纳入映射/过滤规则。

NAPT：两个内网端点映射为不同公网端口



单独查看图片

图解与逐步推演：NAPT返回路径：外网端口怎样找到内网进程



单独查看图片

NAPT返回路径：外网端口怎样找到内网进程

查看完整步骤（也可离线阅读）

1. 内网发起连接：198.51.100.0/24与203.0.113.0/24为文档示例地址。
2. 建立转换状态：网关改写源IP和源端口，并更新相关校验和；具体映射键和过滤策略由实现决定。
3. 响应从外网返回：回复的目的端口对应外部映射，不是应用服务端的443。
4. 反向改写后转内网：返回方向通常改写目的IP/端口，不能仍向内网广播查找公网端口属于谁。

子网掩码与前缀规划 257 从需求决定主机位，再用按位与确定网络。

掩码的作用

掩码中的连续 1 对应前缀，0 对应主机位。IP 与掩码逐位 AND 得网络地址；把主机位全置 1 得普通子网广播地址。传统子网划分从原主机部分借位，子网数量增多、每个子网主机容量相应减少。

可用容量

$N_{\text{addr}} = 2^{32-p}$, $N_{\text{host}} = 2^{32-p} - 2$
普通广播子网 /p 的常见计算；/31、/32 等特殊用途不按此普通主机扣减式处理。

/24 分成两个 /25

218.75.230.0/24 使用 255.255.255.128 后，前缀为 /25。第一子网网络 218.75.230.0、广播 .127、普通主机 .1～.126；第二子网网络 .128、广播 .255、普通主机 .129～.254。两个子网都可使用，不再按古早规则把全 0、全 1 子网整体排除。

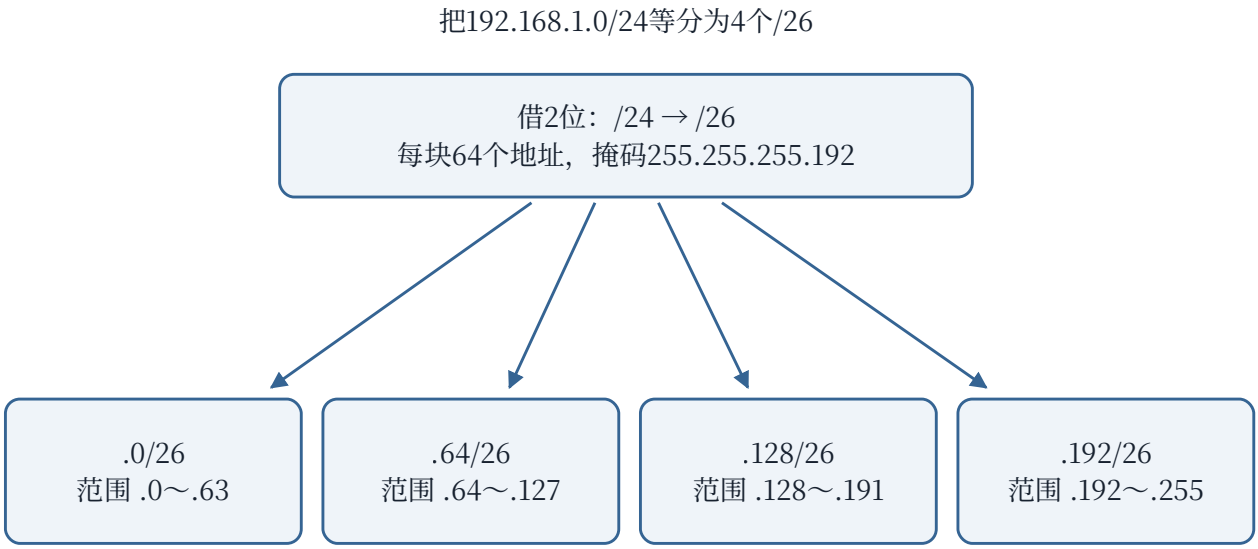
判断同网

两主机用同一掩码进行 AND，结果相同才属于同一对应子网；若各自掩码不同，双方对“是否同网”的判断可能不同。是否直连还取决于接口与链路配置，不能仅比较前三个十进制字节。

结构对照图：把192.168.1.0/24等分为4个/26

网络前缀统一为192.168.1，图中省略了前三段。每个/26地址块大小2^(32−26)=64，起点必须按64对齐。

在常规IPv4子网中，每块扣除网络地址和广播地址后可分配62个主机地址；/31点到点和/32等特殊用途不能套同一个减2规则。



单独查看图片

图解与逐步推演：子网计算：192.168.10.130/26怎样确定网络



单独查看图片

子网计算：192.168.10.130/26怎样确定网络

查看完整步骤（也可离线阅读）

1. 拆分网络与主机位：/26表示前26bit为网络位，最后6bit为主机位。
2. 按位与求网络号：前三个字节均被掩码保留，所以网络地址为192.168.10.128。
3. 将主机位全置1：本块范围.128～.191，每块64个地址，常规可分配主机为.129～.190。
4. 判断是否同子网：比较两地址各自与同一掩码相与的结果；不能仅看前三段相同就断言同一个/26子网。

CIDR 地址块、聚合与最长前缀匹配 258 地址块大小看前缀，转发优先看最具体匹配。

CIDR 的两级结构

CIDR 用网络前缀替代固定 A/B/C 类边界，以“地址/前缀长度”表示地址块。/p 对应 2^(32−p) 个地址；前缀越长，块越小、路由越具体。路由表项目至少表达目的前缀与下一跳/出接口。

128.14.35.7/20

/20 保留前 20 bit，掩码 255.255.240.0；第三字节步长 16，35 落在 32～47，故块范围为 128.14.32.0～128.14.47.255，共 4096 个地址，相当于 16 个 /24 块。

聚合规则

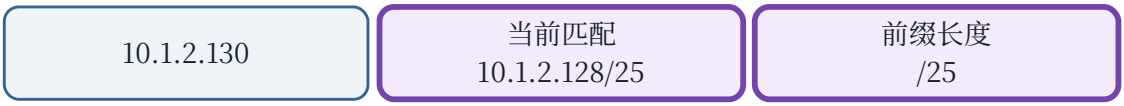
仅当网络前缀相同部分连续、地址对齐、转发策略和下一跳相容时，才能用较短共同前缀概括。若聚合包含原来没有的地址，应明确它是覆盖性汇总，可能需要更具体的丢弃路由，不能称为精确无额外覆盖的合并。

共同前缀与多余覆盖

172.1.4.0/25、172.1.4.128/25、172.1.5.0/25、172.1.6.0/25、172.1.7.0/25 的最短覆盖前缀是 172.1.4.0/22，但它还包含未列出的其他半个 /24。转发匹配多条路由时选择前缀最长的一条，默认路由 /0 仅在无更具体匹配时使用。

图解与逐步推演：最长前缀匹配：同一地址命中多条怎么办

最长前缀匹配：同一地址命中多条怎么办



单独查看图片

最长前缀匹配：同一地址命中多条怎么办

查看完整步骤（也可离线阅读）

1. 检查 默认路由：目的地址10.1.2.130落在该范围，记录它并继续比较更长前缀。
2. 检查 10.1.0.0/16：目的地址10.1.2.130落在该范围，记录它并继续比较更长前缀。
3. 检查 10.1.2.0/24：目的地址10.1.2.130落在该范围，记录它并继续比较更长前缀。
4. 检查 10.1.2.128/25：四项都匹配，选择最长的/25；不是按路由表从上到下遇到第一项就结束。

IP 转发、默认路由与聚合环路 259 先查最长匹配，再解析下一跳链路地址。

一般转发步骤

- 检查分组基本有效性，提取最终目的 IP；过境 IPv4 转发时递减 TTL 并更新首部校验和。
- 将目的 IP 与转发表各前缀匹配，选择最长前缀；/32 是特定主机路由，/0 是默认路由。
- 匹配直连路由时在相应链路直接交付目的接口；匹配远程网络时交给该路由的下一跳。
- 按出接口的链路技术获得必要的下一跳地址并重新封帧；以太网可能需要 ARP，PPP 通常无需。
- 没有可用路由时丢弃，并在符合条件时报告 ICMP 差错。

路由环路

静态路由配错下一跳可形成环路；聚合路由和默认路由互相指向时，也可能让发往不存在子网的数据报来回循环。TTL 最终使它停止，但 TTL 只是限制危害，不会自动修复路由表。可在适当汇总边界配置更具体的黑洞/丢弃路由，防止无效目的沿默认路由返回。

路由查表顺序的统一理解

一些教材把直连、主机、网络、默认分开列步骤；采用 CIDR 时应以最长前缀匹配统一理解，避免“查到第一条匹配就转发”。同前缀多路径如何选择还取决于协议优先级、度量与负载均衡配置。

ARP：解析同一链路的下一跳 260 跨网通信时不向本局域网广播询问远端目的主机的 MAC。

查找与更新

ARP 在 IPv4 以太网等链路上，把同一链路的下一跳 IP 解析为 MAC。先查本机 ARP 缓存；命中则直接封帧；未命中时发送本地广播请求，目标通常用单播应答，并更新缓存。缓存随时间老化，以适应地址和设备变化。

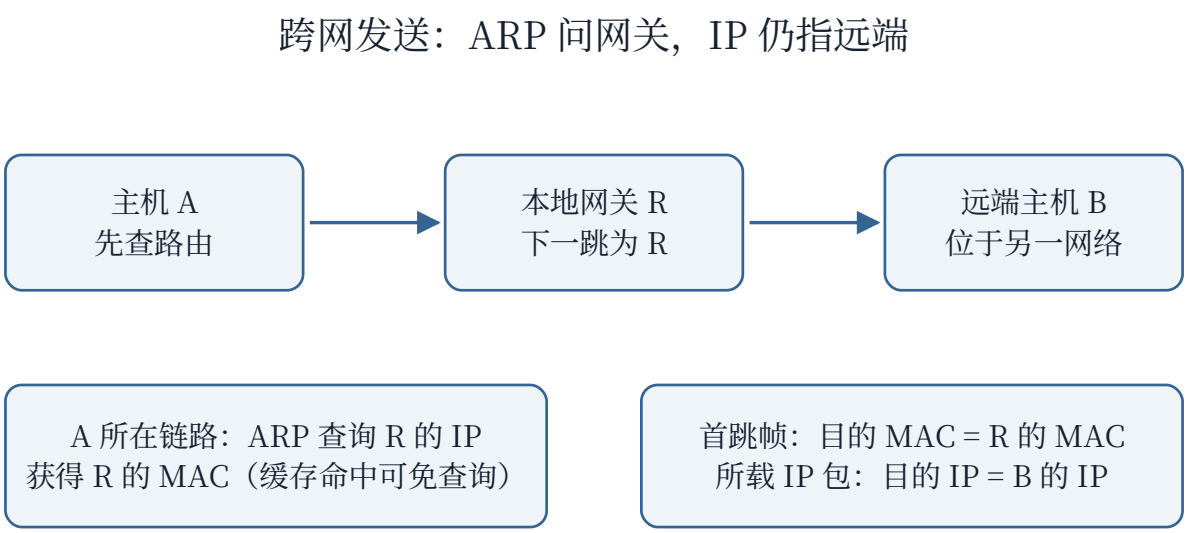
四种常见情形

发送者与目的位置	ARP 解析对象
主机→同网主机	最终目的主机接口 IP 对应的 MAC。
主机→异网主机	路由选出的本地网关接口 IP 对应的 MAC。
路由器→直连目的主机	目的主机接口的 MAC。
路由器→仍需其他路由器	下一跳路由器在本出接口链路上的 MAC。

ARP 与 DNS

DNS 把域名等名字解析成资源记录，常用于得到 IP；ARP 把本链路 IPv4 下一跳映射为硬件地址。两者既不是同一层次的查询，也不是互相替代。ARP 请求不由普通路由器跨网广播，PPP 等点到点链路则不必先运行 ARP。

机制与步骤图解



普通路由转发逐跳替换链路首部；NAT 等机制的地址改写另论。

单独查看图片

DHCP：DORA、端口与初始地址 261 初次申请通常广播，续租和其他状态可能单播。

初始四步

步骤	方向	主要信息
Discover	客户→广播	尚无可用 IPv4 时，源 0.0.0.0、目的 255.255.255.255，寻找服务器。
Offer	服务器→客户	提出可租用地址、掩码、网关、DNS 等配置，具体投递方式受客户状态和广播标志影响。
Request	客户→服务器/广播	客户表明选择的服务器与租约；初始选择阶段常广播，通知其他提供者。
ACK	服务器→客户	确认租约，客户完成必要地址冲突检查后使用。

端口和链路范围

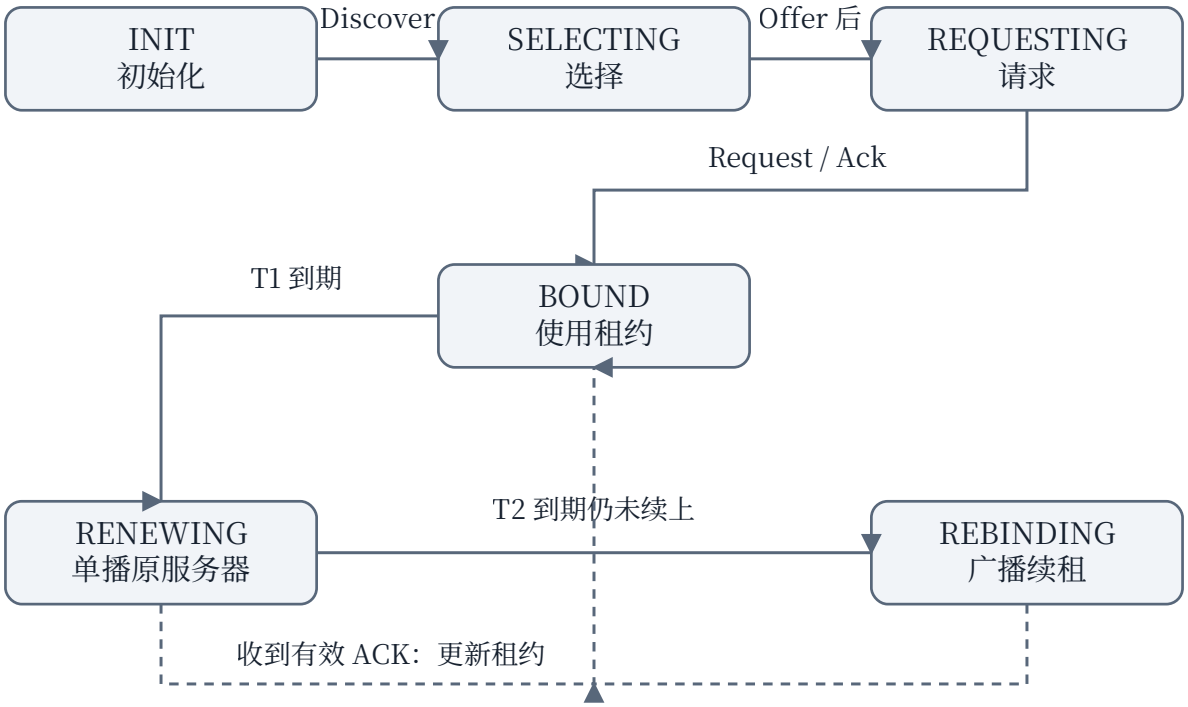
DHCP 使用 UDP，服务器端口 67、客户端端口 68。广播不能自行穿过路由器，跨子网获取配置依赖 DHCP 中继等机制；不同设备角色收到同一广播后是否交给应用，由端口与进程监听情况决定。

不要过度概括

讲义用全广播简化初始过程；不能据此断言所有 DHCP 报文、续租报文都广播。服务器可探测候选地址，客户端也可通过 ARP 检查冲突；租约不是永久静态分配，更新与释放有各自状态。

机制图解

DHCP：报文流与租约状态分开记



默认 T1 = 50% 租期，T2 = 87.5%；到期未续上则停用地址、重新初始化。

单独查看图片

图解与逐步推演：DHCP初次获取地址：DORA四步



单独查看图片

DHCP初次获取地址：DORA四步

查看完整步骤（也可离线阅读）

1. 起点：先确认参与者，再沿时间向下阅读。
2. Discover：寻找服务器：典型初次申请：客户端0.0.0.0:68向255.255.255.255:67广播；图示同一网段无中继。
3. Offer：提出可用地址：服务器提出地址和配置；回复可广播或按条件单播，不能一律认定只单播。
4. Request：选择地址/服务器：客户端典型广播Request，说明选择的服务器和请求地址，使其他服务器也获知选择。
5. ACK：确认租约：服务器ACK确认配置和租期；客户端还应按协议处理地址冲突等情况；续租路径与初次申请不同。

ICMP、Ping 与路径探测 262 ICMP 使用 IP 承载，负责报告而不保证修复差错。

作用和类型

ICMP 报文装入 IP 数据报，用于报告无法交付、超时、参数问题、重定向等网络事件，或进行回送请求/回答。它不是 TCP/UDP 的应用协议，也不为 IP 增加逐包可靠交付。源点抑制、地址掩码查询等属于已废弃或历史类型，不宜作为现代拥塞控制的正常机制。

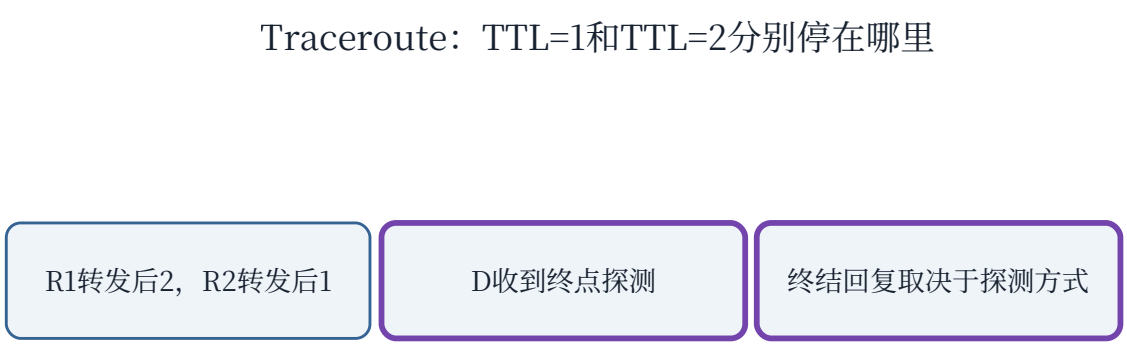
通常不再报告差错的情况

- 不对 ICMP 差错报文本身再产生 ICMP 差错，避免无限递归。
- 通常不对非首片、广播/多播以及某些无效或特殊源地址的数据报发送 IPv4 ICMP 差错；具体例外依协议种类确定。
- 差错报文会附带原数据报首部及必要前部内容，帮助发送者定位关联通信。

两个工具的机制

Ping 常用 ICMP Echo Request/Reply 判断可达性与 RTT；traceroute/tracert 通过逐渐增大的 TTL 触发沿途 Time Exceeded，再依据最终端点返回判断到达。探测报文可以是 UDP、ICMP 或 TCP，工具版本不同，不能一律说都用 UDP。

图解与逐步推演：Traceroute：TTL=1和TTL=2分别停在哪里



单独查看图片

Traceroute：TTL=1和TTL=2分别停在哪里

查看完整步骤（也可离线阅读）

1. 路径假设：简化为两台中间路由器；终点通常不再转发该探测包。
2. 发送TTL=1：通过ICMP错误报文的源信息识别R1；某些设备可能不响应或限速。
3. 发送TTL=2：逐步增大TTL暴露沿途路由器；错误报文回程路径不一定与去程相同。
4. 发送TTL=3到终点：UDP探测通常由终点返回端口不可达，ICMP探测可用回显回答；不能把所有Traceroute都限定为一种运输协议。

IPv6：基本首部、扩展首部与分片 263 基本首部固定 40 B，128 bit 地址；不是讲义误写的 8 B。

基本首部字段

字段	位数	作用
版本/通信类别/流标签	4 / 8 / 20	版本为 6，分类业务并标识数据流。

字段	位数	作用
有效载荷长度	16	基本首部以外的字节数，包括扩展首部；通常最大 65535 B，超大载荷另有扩展机制。
下一个首部	8	指明后续扩展首部或上层协议。
跳数限制	8	经路由器转发递减，为 0 时停止转发。
源/目的地址	各 128	各 16 B；基本首部合计 40 B。

相对 IPv4 的变化

基本首部取消首部校验和与 IHL，把可选功能移到扩展首部；不再在基本首部直接放 IPv4 那样的分片字段。IPv6 路由器不分片，包太大时丢弃并报告 Packet Too Big；源端可用分片扩展首部进行分片，不能说 IPv6 完全不支持任何分片。

地址与过渡

IPv6 有单播、多播和任播，没有 IPv4 式广播；任播交付到组内按路由意义较合适的一个结点。双协议栈同时运行 IPv4/IPv6；隧道把一种协议分组作为另一种协议载荷，经过不原生支持它的网络，再在出口解封装。

IPv6 地址缩写与地址类型 264 每组前导零可删，双冒号只能出现一次。

缩写规则

128 bit 分成 8 组，每组 16 bit 用十六进制表示。组内前导零可省略，例如 003a→3a；连续全零组可用 :: 压缩，但同一地址最多使用一次，以便唯一恢复组数。不能把一组中间或末尾的零任意删去。

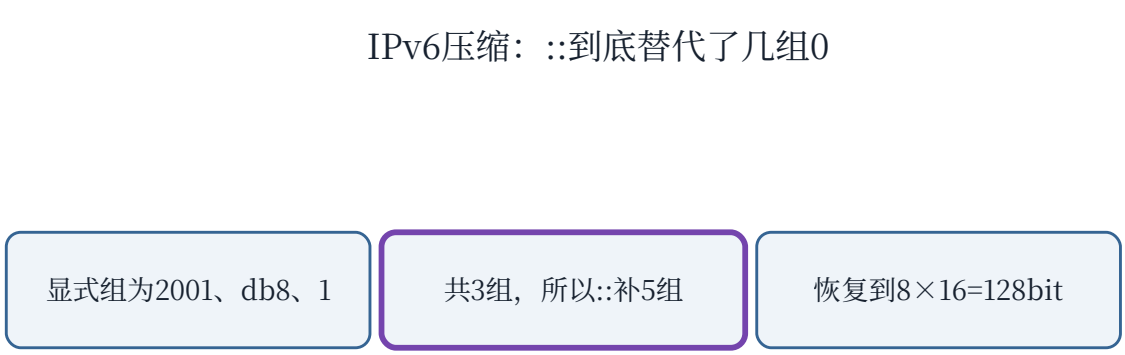
常见类型

地址/前缀	含义
::	未指定地址，不能作为普通目的地址分配。
::1	本机环回。
fe80::/10	链路本地单播；作用域为相应链路。
ff00::/8	多播地址。
全球单播	可用于全球路由的单播地址，实际分配需遵循对应地址规划。

恢复组数

2001:db8::12:0:1 已显式写出 5 组，因此 :: 表示 3 组零。地址恢复为 2001:0db8:0000:0000:0000:0012:0000:0001。链路本地地址在多接口主机上还常需要作用域标识来确定具体链路。

图解与逐步推演：IPv6压缩：::到底替代了几组0



单独查看图片

IPv6压缩：::到底替代了几组0

查看完整步骤（也可离线阅读）

1. 完整8组地址：上面因显示宽度换成两行，完整地址为2001:0db8:0000:0000:0000:0000:0000:0001。
2. 先删每组前导0：例如00f0可缩为f0，不能缩成f，否则数值改变。
3. 压缩连续零组：双冒号只能出现一次，否则省略了多少组会产生歧义。
4. 反向展开检查：零组个数=8−显式组数，适用于本例纯十六进制写法；IPv4嵌入表示需按对应规则计位。

路由协议、组播与移动 IP

计算机网络 · 9 条

静态、动态、距离向量与链路状态 265 交换的是整张路径估计，还是本地原始链路信息?

静态与动态

静态路由由管理员配置，开销小、行为可控，但故障或拓扑变化后通常需人工维护。动态路由协议让路由器交换信息并更新路径，适应性较强，代价是报文、计算、收敛和配置复杂度。

两类动态算法

项目	距离向量	链路状态
交换信息	到各目的的距离估计与下一跳相关信息	邻接关系和各本地链路的状态/代价
传播范围	通常给相邻路由器	按协议在区域内洪泛
计算依据	依赖邻居对目的的估计	建立拓扑数据库后本地运行最短路径算法
典型	RIP	OSPF

分层路由

自治系统 AS 是统一技术管理和路由策略下的网络集合。域内 IGP 解决 AS 内部路由，如 RIP、OSPF；域间协议处理 AS 之间的可达性与政策，如 BGP。链路状态数据库同步不代表“一步就立即收敛”，实际还受传播、计算和定时器影响。

RIP 更新算法与无穷计数 266 同下一跳报告变差也要更新，不是永远只接受更短。

收到邻居 X 报文后的顺序

- 将报文每条路由的距离加 1（超过可达范围按 16 处理），候选下一跳设为 X。
- 本地没有该目的网络：加入候选路由。
- 本地已有且当前下一跳就是 X：用 X 的最新信息更新，即使距离变大。
- 当前下一跳不是 X：候选更短则替换，候选更长一般不采用；等价路由是否同时保留取决于实现。
- 若长时间收不到更新，则按失效机制处理，不能把旧路由永久当真。

三个特征与规模

RIP 通常只与相邻路由器交换自己的路由信息，按周期并可触发更新。以跳数作距离，讲义模型直连网络为 1，最大可达距离 15，16 表示不可达；RIP 以 UDP 承载，标准服务端口 520。

坏消息传播慢

R1 到 N1 的链路断开后，若 R2 仍通告“可经 R1 到达 N1”，R1 可能误以为可改经 R2，随后双方距离反复增加直到 16。水平分割、毒性逆转、触发更新等可减轻问题，但不能宣称单个简单措施消除所有复杂环路。

RIP 表项更新例子与报文长度 267 先把邻居距离加一，再与原下一跳一起比较。

五类条目分别处理

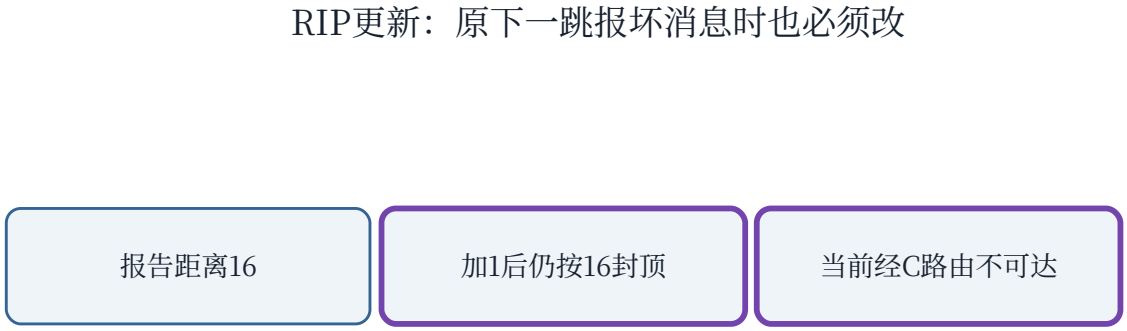
原条目	从 C 得到的候选	处理
N2：距离2，下一跳C	N2：距离5，下一跳C	同一下一跳的最新信息，更新为5。
原来没有 N3	N3：距离9，下一跳C	加入新网络。
N6：距离8，下一跳F	N6：距离5，下一跳C	新路径更短，改经C。

原条目	从 C 得到的候选	处理
N8：距离4，下一跳E	N8：距离4，下一跳C	成本相同，可按实现保留等价路径。
N9：距离4，下一跳F	N9：距离6，下一跳C	候选更差且不同下一跳，一般不改。

基本 RIPv2 格式

首部 4 B；普通路由条目每条 20 B，含地址族、路由标记、网络地址、掩码、下一跳和度量等。传统基本报文至多 25 个普通条目，因此 RIP 载荷最大 4+25×20=504 B；加 UDP 首部 8 B 为 512 B，再按题设加 IP 首部。认证条目等会占用对应空间，不能在带额外功能时机械塞满 25 条普通路由。

图解与逐步推演：RIP更新：原下一跳报坏消息时也必须改



单独查看图片

RIP更新：原下一跳报坏消息时也必须改

查看完整步骤（也可离线阅读）

1. 原路由表：路由器A当前通过邻居B到N，距离2。此处采用经典跳数模型。
2. B报告距离4：先把邻居报告的距离加1，因为还要走A到B的一跳。
3. 原下一跳变差，仍更新：由于更新来自原下一跳，必须接受变差，不能永远只保留旧距离2。
4. 另一邻居C报告距离3：来自其他下一跳时，若候选更短则替换；相等时的实现选择按题设。
5. C随后报告不可达：RIP的16为无穷，15跳是最大可达距离。若存在其他已知可用路径，应重新选择；不能把17当新可达距离。

OSPF：数据库同步、最短路径与区域 268 路由器共享链路状态，各自计算自己的路由表。

工作机制

每台路由器发现邻居并描述自身链路状态，生成 LSA；LSA 装入链路状态更新报文，按区域洪泛。区域内数据库达到一致后，各路由器从自己出发运行 Dijkstra/SPF，生成自己的路由表。因此数据库一致并不要求各路由器的下一跳表完全一样。

五种报文

类型	作用
Hello	发现、维护邻居及可达关系。
数据库描述 DD	交换链路状态数据库摘要。
链路状态请求 LSR	请求缺失或较新状态。
链路状态更新 LSU	携带一个或多个 LSA。
链路状态确认 LSAck	确认接收的链路状态信息。

区域与特点

OSPF 按链路代价选路，可支持等价多路径。划分区域可限制拓扑与状态洪泛规模，主干区域负责区域间连接。状态变化触发更新，同时协议也有刷新和维护报文，不能说“拓扑不变就绝不发任何 OSPF 包”。OSPF 直接由 IP 承载（协议号 89），不使用 TCP/UDP 端口。

图解与逐步推演：OSPF链路变化：共享拓扑后各自算路由



单独查看图片

OSPF链路变化：共享拓扑后各自算路由

查看完整步骤（也可离线阅读）

1. 本地链路成本变化：只概括协议逻辑，不逐一列出建立邻接与数据库同步的全部报文。
2. 可靠泛洪至相应范围：不是每收到一份旧通告都无限重新广播；区域与LSA类型限制传播范围。
3. 形成一致的链路状态视图：拓扑数据库相近不代表所有路由器的最终路由表完全相同，因为源点不同。
4. 更新转发表：OSPF控制过程与普通数据包转发属于不同工作；OSPF直接运行于IP之上，协议号89。

RIP、OSPF、BGP 的协议对照 269 域内度量与域间政策不能用一把“跳数最少”尺子衡量。

核心比较

协议	范围/算法	交换内容与承载
RIP	域内、距离向量，以跳数衡量	邻居间通告距离向量；UDP。
OSPF	域内、链路状态，按代价 SPF	区域内洪泛本地链路状态；直接用 IP。
BGP	域间、路径向量，重视政策	对等体交换网络前缀可达性及路径属性；TCP 179。

为什么采用层次结构

大型互联网若让每个路由器详细维护所有内部拓扑，状态和更新开销会非常大。AS 与区域把管理、策略和计算范围分层；BGP 选择的是符合策略的可行路径，不保证从物理时延、带宽或 AS 数量看都是最优。

BGP 会话、路径向量与四种报文 270 用 AS_PATH 帮助避免域间回路，用策略决定取舍。

建立与更新

BGP 发言人先建立 TCP 连接，再交换 OPEN 建立会话。初次交换可达性信息后，主要通告增量变化和撤销，而不是周期性把全部路由表广播。路径属性可包含经过的 AS 序列，若发现自己的 AS 已在路径中可据此拒绝该回路路由。

基本报文

报文	作用
OPEN	协商版本、AS、保持时间、BGP 标识和能力等，初始化会话。
UPDATE	通告可达前缀与路径属性，或撤销失效路由。
KEEPALIVE	维持邻居会话，不承载完整路由表。
NOTIFICATION	报告检测到的差错并按协议终止会话。

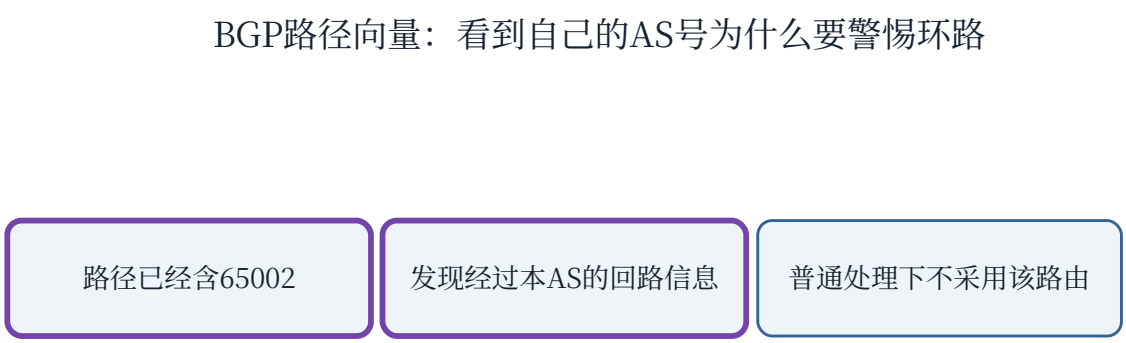
传统基本报文结构

通用首部 19 B，含 16 B Marker、2 B Length、1 B Type；基本 KEEPALIVE 仅含该首部。传统基本消息长度上限 4096 B，现代扩展可协商更大上限。OPEN 中历史 2 B AS 字段配合能力可支持 4 字节 AS，不能把所有现代自治系统号都限定为 16 bit。UPDATE 含撤销路由长度与内容、路径属性长度与内容、NLRI 等。

逻辑邻居与物理相邻

BGP 对等体依据 TCP 会话通信，不应简单理解成永远必须接在同一根物理链路。跨 AS 与 AS 内 BGP 有不同使用场景；发言人还需和域内路由可达性协作，避免出现“知道外部前缀却到不了下一跳”。

图解与逐步推演：BGP路径向量：看到自己的AS号为什么要警惕环路



单独查看图片

BGP路径向量：看到自己的AS号为什么要警惕环路

查看完整步骤（也可离线阅读）

1. AS65003发布某前缀：以普通AS_SEQUENCE、无联盟与特殊策略为例；AS号仅作示例。
2. AS65002向AS65001通告：不是把每个经过的IP路由器都塞进AS_PATH。
3. AS65001接受并继续向外通告：BGP不是永远简单选择AS跳数最少的路线。
4. 若通告又传回AS65002：这里说明基于AS_PATH的环路检测；allowas-in等特定策略不作为默认模型。

IPv4 组播与 IGMP 271 组地址代表成员集合，不是给某台固定主机独占。

组播的意义

源发送一份组播数据，网络在必要分叉处复制，交给加入该组的接收者，降低发送端重复发送与公共路径负载。主机可以加入多个组，也可以离开；组成员身份和位置可能动态改变。IPv4 组地址位于 224.0.0.0/4，其中有保留和特殊范围，并非任意地址都供普通应用自由使用。

IGMP 与组播路由

IGMP 让主机向本地组播路由器表达组成员关系；路由器间的组播路由协议负责把组流量沿适当分发树传递。IGMP 不负责计算整个互联网的组播树。组播通常基于无连接数据报，不能把 TCP 一对一连接直接当成原生一对多组播。

不要套用广播或 Ping 结论

本地硬件多播与跨路由器 IP 组播是不同范围。为避免响应风暴，IPv4 对多播等情形的 ICMP 差错响应有抑制规则，但不能从中推出所有平台对组播 Echo 都有统一的“永不响应”行为。

IPv4 多播地址映射到以太网 MAC 272 低 23 bit 映射，多对一关系需要上层再过滤。

映射步骤

以太网 IPv4 多播 MAC 的固定前缀是 01-00-5E，其后 24 bit 的最高位固定为 0，剩余 23 bit 取 IPv4 多播地址最低 23 bit。结果范围为 01-00-5E-00-00-00～01-00-5E-7F-FF-FF。

IPv4 组地址中前 4 bit 为 1110，余下 28 bit 可区分组；只映射其中 23 bit，所以有 5 bit 未映射，理论上 32 个不同 IP 组可对应同一 MAC。

224.215.145.230 的映射

第二字节 215 的二进制为 11010111，去掉最高位后为 1010111，即十六进制 57；后两字节 145、230 分别为 91、E6，因此映射为 01-00-5E-57-91-E6。

同 MAC 不同 IP 组

226.0.9.26 与 225.128.9.26 的低 23 bit 相同，均映射到 01-00-5E-00-09-1A。网卡可先按 MAC 接收候选帧，但 IP 层仍要检查目的组是否真是本机加入的组；硬件过滤通过不等于应用必定收到。

图解与逐步推演：IPv4多播映射：不同组地址为何会得到同一个MAC

IPv4多播映射：不同组地址为何会得到同一个MAC



单独查看图片

IPv4多播映射：不同组地址为何会得到同一个MAC

查看完整步骤（也可离线阅读）

1. IPv4组地址224.1.2.3：MAC的第4个字节最高位还必须是0。
2. 取出低23bit：这里只讲标准IPv4多播到以太网的映射。
3. 比较239.129.2.3：IPv4多播组地址有28个可变化位，但只映射23位，忽略的5位导致32个IP组可对应一个MAC。
4. 仍需上层过滤：链路层过滤不能单凭这一个MAC完全区分所有IPv4组。

移动 IP：归属地址、转交地址与隧道 273 移动 IP 保持会话中的归属地址，DHCP 只是分配地址。

角色

移动结点拥有相对稳定的归属地址；本地/归属代理在归属网络维护其当前位置映射；转交地址指出移动结点当前可到达的位置；外部代理可在访问网络提供辅助，也存在结点自行获得转交地址的方式。

典型通信过程

- 移动结点离开归属网，到访问网获得转交地址。
- 将“归属地址→转交地址”绑定向归属代理注册。
- 对端发往归属地址的数据到达归属网，归属代理截获后经隧道封装送往转交地址。
- 隧道出口解封装，将原始 IP 数据交给移动结点；返回方向可直接或按机制经反向隧道发送。
- 移动到另一访问网时更新绑定；回到归属网后注销转交绑定，恢复常规交付。

与动态地址的区别

DHCP 动态分配地址不自动保证主机换网后原有连接继续使用原端点。移动 IP 通过位置绑定和隧道维持归属地址语义；“不改变 IP”主要指对会话可见的归属地址，不是说所有封装层的外层 IP 都永远不变。

运输层、UDP 与 TCP

计算机网络 · 12 条

端口、套接字与复用分用 274 IP 找到主机接口，端口帮助运输层把数据交给应用端点。

逻辑通信与寻址

运输层向应用提供端到端进程通信，屏蔽下层经过的链路和路由细节。发送端把多个应用的数据复用到运输协议；接收端依据协议与端点信息分用。运输层端口是软件接口，不能与交换机网口等物理接口混为一谈。

端口号范围

范围	传统名称	说明
0～1023	系统/熟知端口	常见标准服务使用，具体可用性受系统权限规则约束。
1024～49151	用户/登记端口	可登记用于服务，不能据此说所有实际使用都必须先登记才可收发。
49152～65535	动态/私有端口	常用作临时端口，实际操作系统临时范围可配置。

端点与连接

端口为 16 bit，数字只在本机及相应运输协议语境中有意义；不同主机可用同一端口，TCP 和 UDP 也可分别使用同号端口。讲义将 Socket 简写为“IP:端口”；一个 TCP 连接通常由源 IP、源端口、目的 IP、目的端口四元组标识，在多协议比较时还需指明 TCP/UDP。一个进程可以拥有多个套接字，不能把进程、端口和连接一一等同。

UDP 与 TCP：报文边界和字节流 275 可靠性、连接与发送数据形态要逐项比较。

服务对照

项目	UDP	TCP
建立连接	不预先握手	先建立连接，再传送并正常释放
数据形态	面向报文，保留应用提交的数据报边界	面向字节流，不保留应用每次写入的边界
可靠与顺序	本身不提供确认、重传和排序保证	通过序号、确认、重传等实现可靠有序字节流

项目	UDP	TCP
流量/拥塞	UDP 首部本身不提供这两种控制	提供接收流量控制与拥塞控制
首部	固定 8 B	基本 20 B，可含选项，传统最大 60 B
通信关系	可支持单播、多播等数据报场景	每连接两个端点，全双工一对一

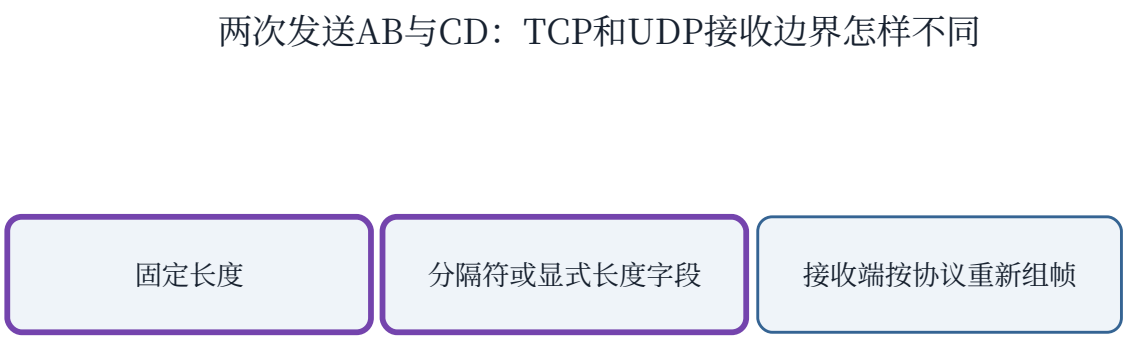
不能误解为哪个永远更快

UDP 的协议开销和启动时延通常较小，但“UDP 不拥塞控制所以网络拥塞不影响效率”是错误推论；拥塞仍会导致丢包和时延，应用应采取适当控制。TCP 可靠性是运输服务的约定，不表示任何故障下连接都永不失败。

应用读写边界

应用连续两次写入 100 B，TCP 对端可能一次读到 200 B，也可能多次读到若干部分；应自行规定消息长度或分隔。UDP 的一次交付对应一个数据报，数据报过大可能在 IP 层分片，过小又会使首部比例过高。

图解与逐步推演：两次发送AB与CD：TCP和UDP接收边界怎样不同



单独查看图片

两次发送AB与CD：TCP和UDP接收边界怎样不同

查看完整步骤（也可离线阅读）

1. 应用连续两次发送：假设UDP接收缓冲足够；网络丢失等因素另论。
2. UDP保留数据报边界：UDP接收按数据报进行；不保证它们都到达或按发送顺序到达。
3. TCP提供有序字节流：TCP按字节序交付，分段、合并和单次read返回字节数不能当作应用消息边界。
4. 应用自行定义消息格式：不能依赖“一次send对应一次recv”实现TCP应用消息协议。

UDP 首部、伪首部与反码校验 276

校验范围包含首部和数据，伪首部仅参与计算。

四个字段

字段	长度	含义
源端口	2 B	便于回送，某些 IPv4 场景不需时可置零。
目的端口	2 B	接收端用于分用。
长度	2 B	UDP 首部加数据的字节数，最小为 8。
校验和	2 B	UDP 与相应 IP 伪首部的反码校验。

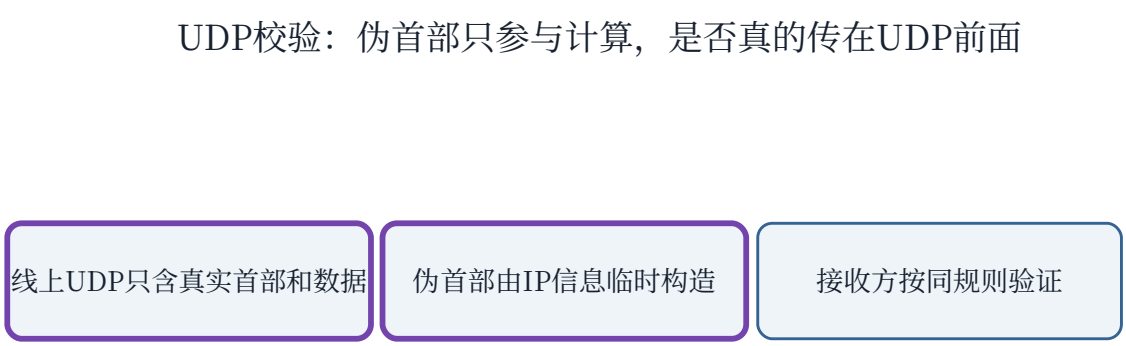
计算步骤

- 构造伪首部；IPv4 形式含源 IP、目的 IP、零、协议号 17 和 UDP 长度，共 12 B。它不作为 UDP 的真正首部在链路上单独发送。
- 发送计算时把 UDP 校验和字段先置 0；将伪首部、UDP 首部与数据按 16 bit 分组。
- 若总数据需补齐偶数字节，在末尾补 0 仅供计算，不计入 UDP 长度，也不额外传送该填充字节。
- 对各 16 bit 字作反码加法，最高进位回卷到低位；最后逐位取反写入校验和。
- 接收端连同校验和重新计算，正确的反码和应为全 1；未通过则按协议丢弃。

全零、全一与 IPv6

IPv4 UDP 中校验和字段全 0 表示未使用该校验；若计算结果恰好为全 0，发送时编码为全 1，因此 UDP 校验和值可以是 0xFFFF。IPv6 中 UDP 校验和原则上必需，少数隧道等特殊例外不能推广成一般可选。

图解与逐步推演：UDP校验：伪首部只参与计算，是否真的传在UDP前面



单独查看图片

UDP校验：伪首部只参与计算，是否真的传在UDP前面

查看完整步骤（也可离线阅读）

1. 组织校验输入：IPv4伪首部包含源IP、目的IP、0、协议号17与UDP长度；补零只用于本次计算，不增加实际UDP数据长度。
2. 16bit反码求和：反码加法把最高位进位加回低16位，不是简单丢弃进位。该小例只示意运算，不是上一帧的完整校验结果。
3. 取反写校验字段：实际计算要覆盖整个伪首部、UDP首部和数据。IPv4中计算结果为0通常以全1字段发送，字段全0另表示未计算校验。
4. 发出与接收：IPv6伪首部格式不同，UDP校验通常必须使用，特定隧道例外另有规范；不要把IPv4可不校验直接推广到IPv6。

TCP 首部与字节序号 277 seq 指向本段第一个数据字节，ack 指向下一个期望字节。

主要字段

字段	宽度	解释
源/目的端口	各16 bit	标识通信端点。
序号 seq	32 bit	本段首数据字节序号，按模 2^32 运算。
确认号 ack	32 bit	ACK 有效时，表示希望收到的下一个连续字节序号。

字段	宽度	解释
数据偏移	4 bit	以4 B计首部长度；最小20 B，传统最大60 B。
窗口	16 bit	向对方通告接收窗口；窗口扩大选项可扩展解释范围。
校验和	16 bit	覆盖伪首部、TCP首部和数据，必需。
紧急指针/选项	16 bit/可变	紧急语义由 URG 指示；选项可含 MSS、窗口扩大、SACK、时间戳等。

控制位

ACK 指示确认号有效；SYN 同步初始序号；FIN 表示发送方向数据结束；RST 重置连接；PSH 请求尽快向应用交付；URG 指示紧急指针有效。不是每个建立或释放报文都同时令 SYN、ACK、FIN 为 1。紧急数据仍在 TCP 字节流中，不是无序号的带外独立通道。

长度与确认

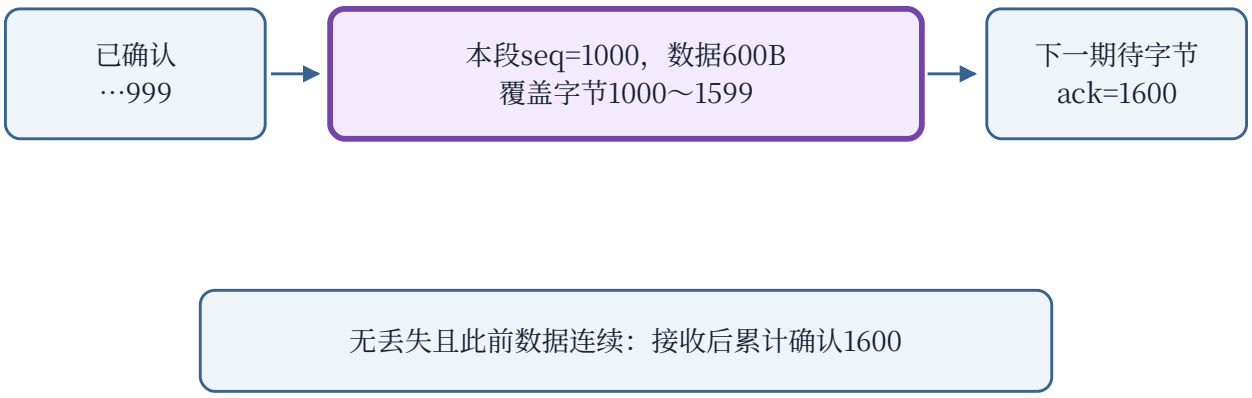
seq=301，数据100 B，对应301～400，下段新数据从401开始。累计 ack=701 表示已连续收到到700，不能据此断言701之后的乱序字节完全没到。接收窗口1000则通告可接受701～1700，计算右边界须注意包含端点。

结构对照图：TCP字节序号：seq和ack都是字节位置

seq=1000表示本段首个数据字节的序号；600字节使用1000～1599，不能把末字节写成1600。

ack=1600表示1600之前的字节都已连续收到。纯ACK不占序号，SYN和FIN各占一个序号；没有数据的段也仍有seq字段。

TCP字节序号：seq和ack都是字节位置



单独查看图片

TCP 三次握手、序号与状态 278 两方向独立选择初始序号，SYN 各消耗一个序号。

标准主动/被动打开

步骤	报文	主要状态变化
1	A→B：SYN=1，seq=x	A进入 SYN-SENT；B预先处于 LISTEN。
2	B→A：SYN=1、ACK=1，seq=y，ack=x+1	B进入 SYN-RECEIVED。
3	A→B：ACK=1，seq=x+1，ack=y+1，可携带数据	A进入 ESTABLISHED；B收到有效确认后进入 ESTABLISHED。

为什么需要第三次

建立连接不仅要知道对方存在，还要让双方确认两方向初始序号与收发能力，并建立相应连接状态。第三次确认可避免旧的连接请求使服务器错误建立无效连接。两次就简单认定双方都建立成功会留下半开或状态不一致问题，实际 TCP 还靠重传、定时器和重置等机制处理故障。

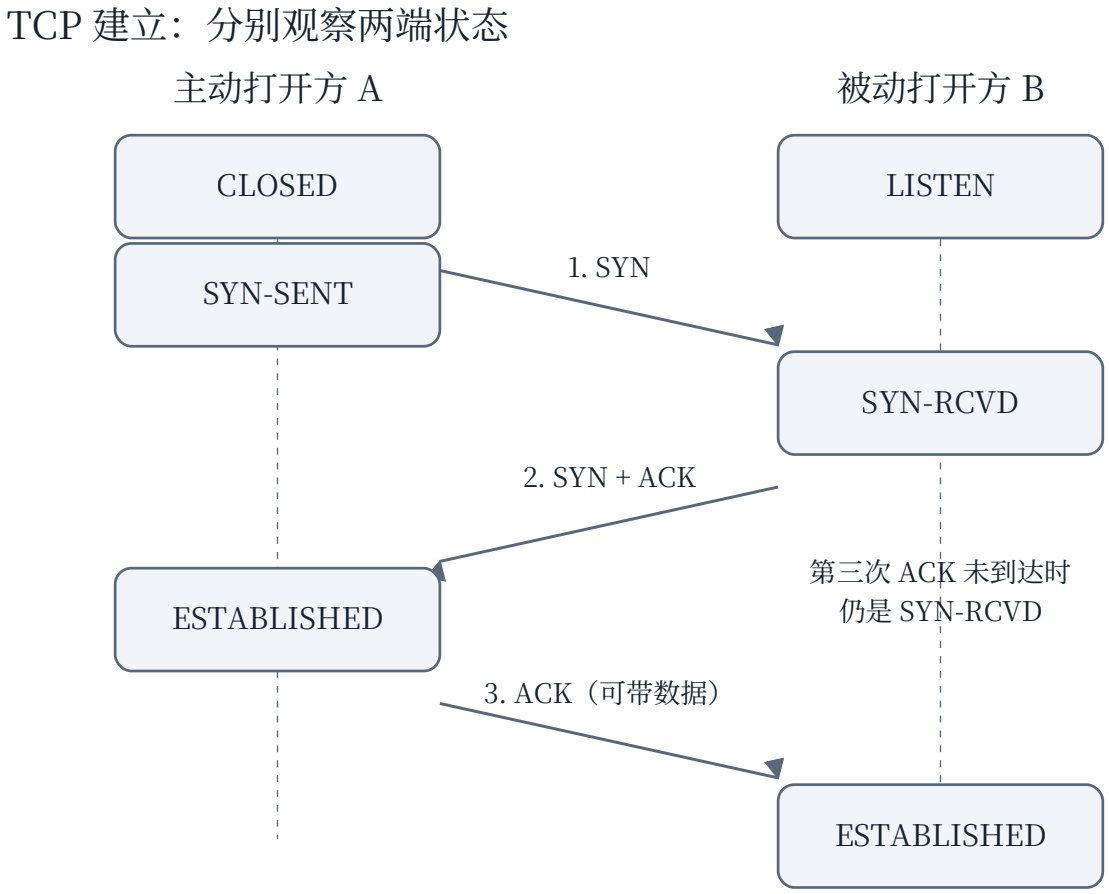
独立初始序号

A 的 SYN seq=11220，则 B 对它的有效确认是 ack=11221（未回绕时）；B 自己的 seq=y 独立选择，不必是 11221。纯 ACK 不消耗序号；SYN 即使没有应用数据仍消耗1个序号。

正常模型与选项

常见模型前两次握手不携带应用数据、第三次可带；TCP 也有特定扩展允许不同数据携带行为。计算题应使用题设的标准模型，不将扩展机制擅自加进去。

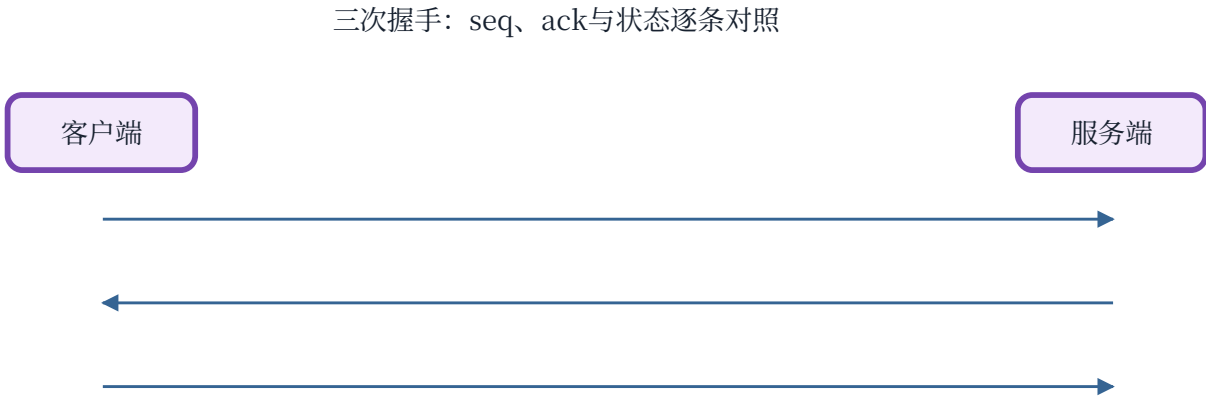
机制图解



时间向下：A 发出第三次 ACK 时已建立；B 收到有效 ACK 后才建立。

单独查看图片

图解与逐步推演：三次握手：seq、ack与状态逐条对照



单独查看图片

三次握手：seq、ack与状态逐条对照

查看完整步骤（也可离线阅读）

1. 起点：先确认参与者，再沿时间向下阅读。
2. SYN seq=100：客户端发送SYN后为SYN-SENT；服务端收到后确认101，发送自己的SYN，进入SYN-RECEIVED。
3. SYN+ACK seq=500 ack=101：SYN各占一个序号。客户端收到合法SYN+ACK后发送第三次ACK并进入ESTABLISHED。
4. ACK seq=101 ack=501：服务端收到第三次ACK后进入ESTABLISHED；纯ACK不消耗序号。示例无数据、无丢包。

TCP 连接释放、半关闭与 TIME-WAIT 279 一个方向结束发送，不代表另一个方向也没有剩余数据。

典型四段释放

报文	内容	状态
A→B FIN	FIN=1、seq=u，通常ACK=1	A：FIN-WAIT-1
B→A ACK	ack=u+1，seq=v	B：CLOSE-WAIT；A收到后：FIN-WAIT-2
B→A FIN	B应用完成发送后，FIN=1、seq=w，ack=u+1	B：LAST-ACK

报文	内容	状态
A→B ACK	ack=w+1，seq=u+1（A之后无新数据时）	A：TIME-WAIT；B收到确认后：CLOSED

半关闭与序号

A 发 FIN 后不能再在该方向发送新应用数据，但仍可接收 B 尚未完成的数据。B 的最终 FIN 序号 w 可以不同于先前确认的序号 v，因为半关闭期间 B 还能发送。FIN 本身消耗一个序号，纯 ACK 不消耗。

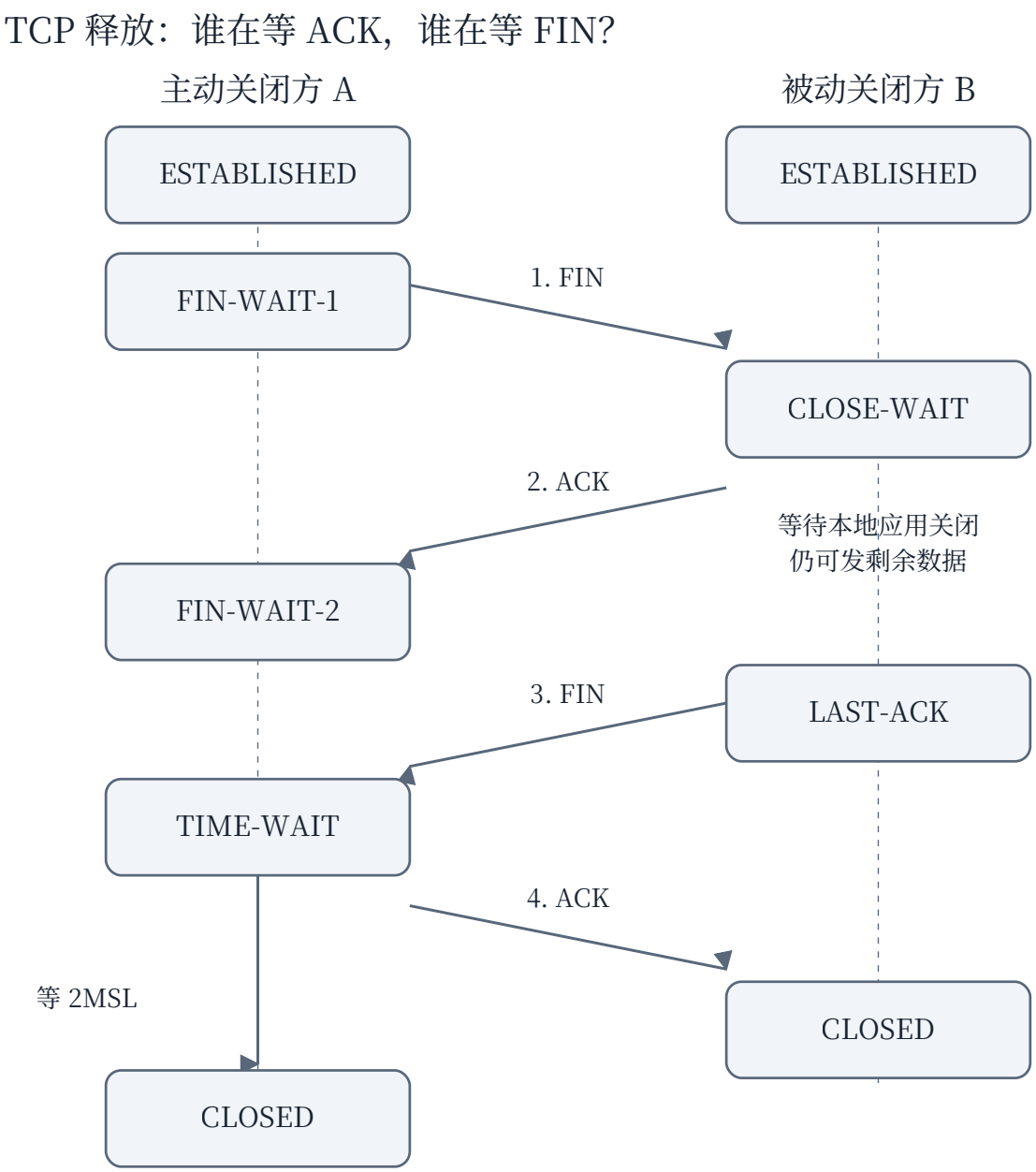
等待 2MSL 的两个目的

让最后确认丢失后对方重传的 FIN 仍能得到确认；同时使旧连接残留报文有机会在网络中消失，减少新连接混入旧报文的风险。TIME-WAIT 方若收到重传 FIN，会按规则重发 ACK 并重启等待。

不必永远正好四个报文

ACK 与 FIN 可以合并，同时关闭也有不同状态路径；“四次挥手”是典型独立关闭过程。CLOSE-WAIT 表示本机已经收到对端 FIN、等待本地应用关闭；TIME-WAIT 则是另一种等待，不能混为同一故障原因。

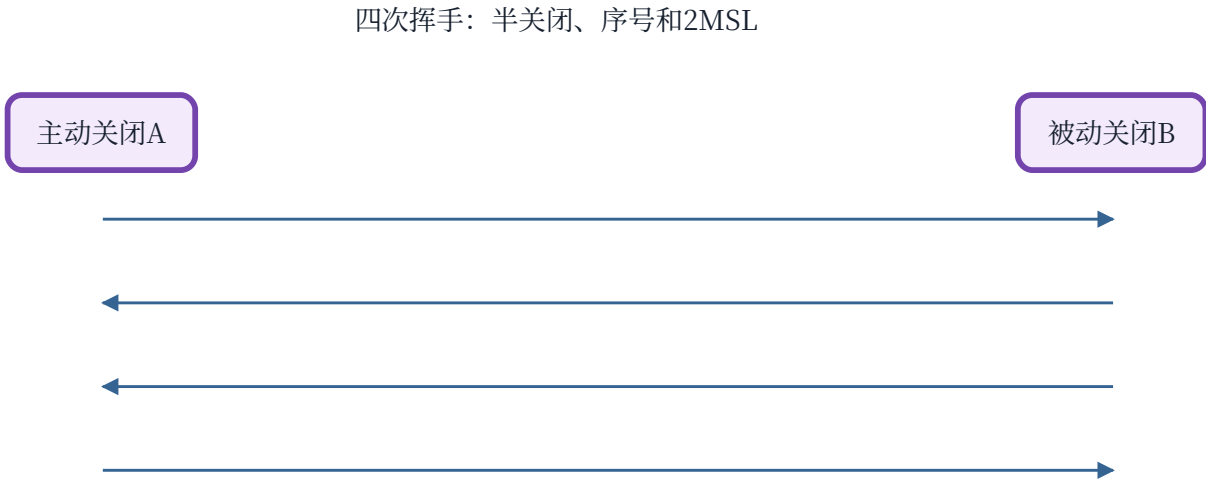
机制图解



典型四报文过程；ACK 与 FIN 可合并，特殊分支见下一图。

单独查看图片

图解与逐步推演：四次挥手：半关闭、序号和2MSL



单独查看图片

四次挥手：半关闭、序号和2MSL

查看完整步骤（也可离线阅读）

1. 起点：先确认参与者，再沿时间向下阅读。
2. FIN seq=100 ack=500：双方已建立，假设A下个seq=100、B下个seq=500且无在途数据。A发送FIN进入FIN-WAIT-1，B接收后进入CLOSE-WAIT。
3. ACK seq=500 ack=101：B确认A的FIN；A收到ACK后进入FIN-WAIT-2。B的发送方向仍可继续。
4. FIN seq=500 ack=101：本例B没有再发数据；B发送FIN进入LAST-ACK，A收到后确认501并进入TIME-WAIT。
5. ACK seq=101 ack=501：B收到最终ACK后CLOSED；A通常等待2MSL再关闭。ACK和FIN有时可合并；此处演示典型分离情形。

TCP 可靠交付、RTO 与快速重传 280 超时重传与三个重复 ACK 是两种不同触发。

可靠性的组成

TCP 用校验检测损坏，字节序号和累计确认识别缺口，发送缓存保存未确认副本，重传恢复丢失，并抑制重复、按序向应用提交。接收端收到乱序数据可缓存，但累计确认仍停留在第一个缺失字节，除非另有 SACK 等附加信息。

自适应超时

测量报文发送到相关确认到达的 RTT，平滑估计往返时间及其波动，RTO 需留出适当余量。重传后的确认可能无法判定对应哪次发送，估计时须避免采样歧义；超时通常采用退避。RTO 不能固定等于最近一次 RTT，也不能只看平均值而完全忽略波动。

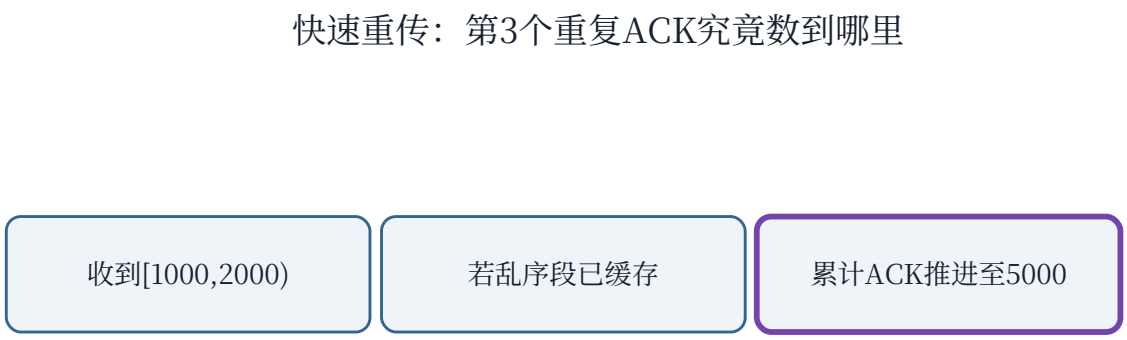
快速重传示例

M1到达后，M2丢失；随后M3、M4、M5分别乱序到达，接收端都重复确认缺口之前的连续数据。发送端收到三个重复 ACK后可提前重传M2，无需等RTO到期。重传定时器仍保留，并未被“快重传”取代。

确认语义

TCP 的 ack 值是下一期望字节，图中“确认M1”通常是教材对段的简写。要计“三个重复”需在原确认之后再收到三次相同确认，不能把原来的那次当成三个重复中的一个。

图解与逐步推演：快速重传：第3个重复ACK究竟数到哪里



单独查看图片

快速重传：第3个重复ACK究竟数到哪里

查看完整步骤（也可离线阅读）

1. 已有一次正常ACK：设MSS=1000B，接收方已经发过ACK1000。原ACK不计入后面的3个重复ACK。
2. 1000段丢失，2000段到达：假设接收端对每个乱序段立即回复同一累计ACK，无其他复杂选项或窗口变化干扰。
3. 3000段到达：即使乱序数据已缓存，累计确认也不能跨过缺口。
4. 4000段到达：发送方收到3个符合定义的重复ACK后触发快速重传，无须等RTO到期。
5. 重传填补缺口：图采用缓存乱序的接收方；拥塞窗口如何调整取决于Reno、NewReno等算法，快速重传与快恢复不能混成同一步规则。

TCP 接收窗口、在途数据与发送额度 281 窗口上限减去在途数据才是还能发多少。

流量控制

接收端根据缓存空余和应用读取情况，在 TCP 窗口字段中通告 rwnd；发送方据此限制未确认数据。接收端的流量控制是端到端的，与相邻结点的链路层流控范围不同。

$W_{\text{send}} \le \min(\text{rwnd}, \text{cwnd})$

rwnd保护接收端，cwnd保护网络；单位必须统一成字节或 MSS 数。

$\text{NewData} = \max\{0, W_{\text{send}} - \text{FlightSize}\}$

FlightSize是已经发送但尚未确认的在途数据；实际还受应用有无待发数据等约束。

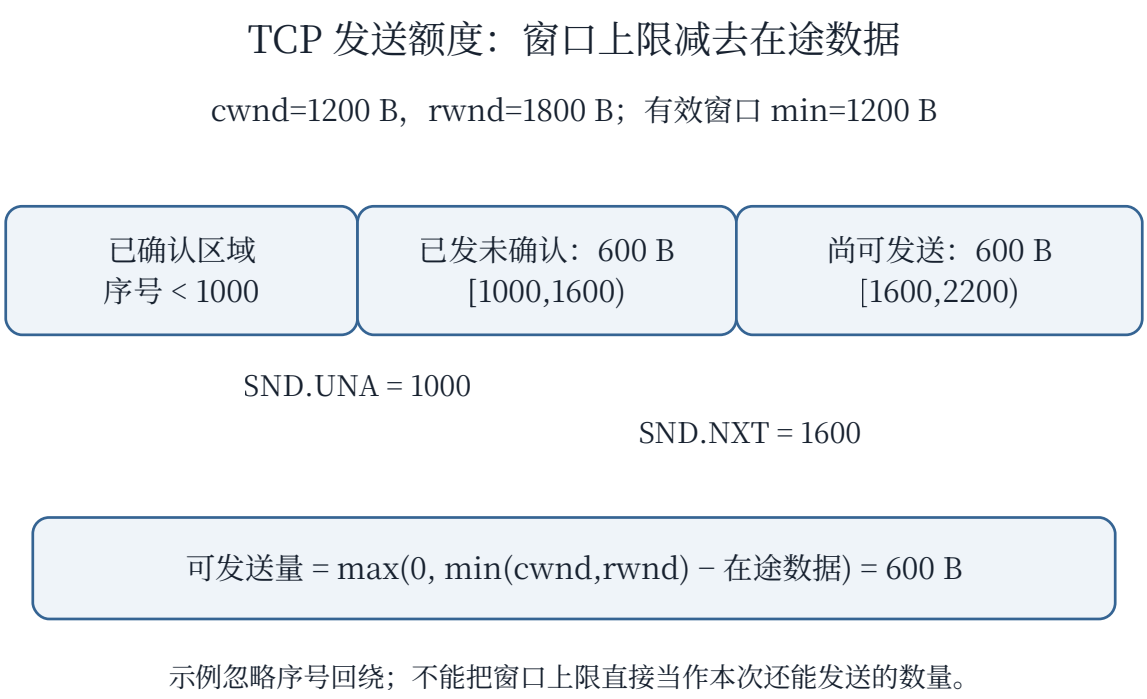
窗口不等于每次都再发同样多

ack=201、rwnd=300，允许范围为201～500；若201～300已经发出但未确认，可新增发送301～500共200 B，而不是再发300 B新数据。收到确认后左边界移动，已确认副本才能从发送缓存释放。

MSS 与 MTU

MSS 是一个 TCP 报文段数据部分的最大长度，不含 TCP/IP 首部；MTU 是链路可承载的 IP 分组总长度上限。典型无选项 IPv4/以太网中 MSS 可取1500−20−20=1460 B，但实际选项、隧道和路径条件须另算。TCP 可因 MSS、应用推送或定时策略形成不同长度报文段。

机制与步骤图解



单独查看图片

逐步操作与动画

例：发送600字节，再收到累计确认

查看完整步骤（也可离线阅读）

1. 0 · 初始窗口：忽略序号回绕，cwnd=1200B、rwnd=1800B，SND.UNA=1000、SND.NXT=1600。在途600B，最多还能发送600B。
2. 1 · 发送600字节：发送序号[1600,2200)，NXT推进到2200；UNA仍1000，在途增至1200B。接收窗口虽然更大，但拥塞窗口已用满。

3. 2 · 收到ACK=1600：有效累计确认意味着1600以前的数据已确认，UNA推进至1600；NXT保持2200，在途降到600B。为隔离变量，此例假定cwnd、rwnd不变。
4. 3 · 接收窗口缩为400B：假设携带新窗口的有效ACK已被接受，rwnd变为400B，而在途仍600B。可发送量max(0,400−600)=0，已发数据不能被“撤回”。
5. 4 · ACK=2200并通告rwnd=1000：全部在途数据确认，UNA=NXT=2200。在cwnd仍为1200B时，有效窗口1000B，可以发送1000B新数据。

TCP 窗口变化的完整例子 282 保留未确认副本，累计确认到来后再移动左边界。

初始窗口400 B

B 通告 rwnd=400，A可发1～400。A发送1～100、101～200、201～300，第三段丢失；A又收到 ack=201、rwnd=300，于是窗口覆盖201～500。201～300仍是未确认旧数据，A还能发301～400、401～500。

补齐缺口后再扩大连续确认

A窗口中201～500都已发送且未全部确认时，不能继续发送501及之后的新数据。若201～300超时，A重传该旧段；B已缓存301～500，缺口补齐后可累计确认 ack=501，并通告rwnd=100。此后A可发501～600。

接收端关闭窗口

B确认到ack=601并通告rwnd=0，表示序号600及之前的连续数据已收到，普通新数据发送额度为零。A已确认的数据可以移出缓存，但这不意味着A可无视零窗口继续发送后续普通数据。

此例的约定

上面只展示接收流量控制，假设拥塞窗口与其他条件不构成更小限制；若题设还给 cwnd，应取两种限制较小者。序号是字节序号，不是“第几个报文段”。

零窗口死锁与持续计时器 283 窗口更新可能丢失，探测机制用于恢复双方进展。

为什么会互相等待

B通告零窗口后，应用读走数据腾出空间，B发非零窗口更新，但更新丢失。A仍以为不能发送，等B通知；B以为已通知，等A数据。若没有额外机制，这种等待可持续。

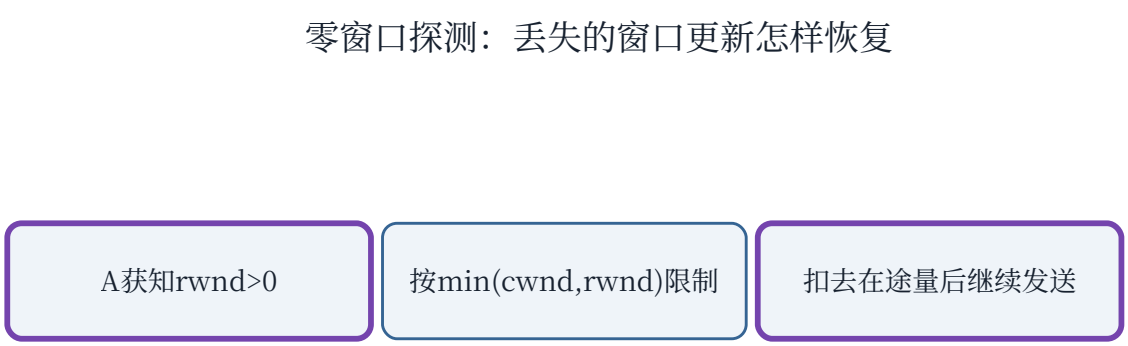
持续计时器的作用

A收到零窗口信息后，按持续计时器机制发送零窗口探测，促使B回复当前窗口。若仍为零则继续按机制探测；若已非零，A可恢复发送。探测报文是处理窗口僵局的特殊控制行为，不能据此说零窗口时普通新数据可随意发送。

与其他计时器区别

重传计时器解决未确认数据的可靠交付；持续计时器解决零窗口更新丢失；保活计时器探测长时间空闲连接的对端状态；TIME-WAIT 的2MSL等待处理关闭与旧报文。中文都叫“计时器”，但启动条件和目的不同。

图解与逐步推演：零窗口探测：丢失的窗口更新怎样恢复



单独查看图片

零窗口探测：丢失的窗口更新怎样恢复

查看完整步骤（也可离线阅读）

1. 接收方缓冲区满：零窗口表示接收方暂时无可用空间，与网络拥塞不是同一原因。
2. B释放缓冲并通告：纯ACK通常不可靠重传，单靠这次更新可能导致双方一直等待。
3. A持续计时器到期：发送方通过专门探测触发接收方报告当前窗口，不是直接把拥塞窗口翻倍。
4. 窗口更新到达：非零窗口报告到达后恢复发送；零窗口探测与超时重传、保活计时器的用途不同。

慢开始、拥塞避免与门限 284 按轮次翻倍或加一都依赖题设的理想化确认模型。

两个变量

发送端用 cwnd 控制网络在途负载，用 ssthresh 分隔慢开始与拥塞避免。教材常以1 MSS起始并假设每段均被立即确认，此时慢开始大约每RTT翻倍；真实实现的初始窗口和ACK行为可不同。

增长与故障反应

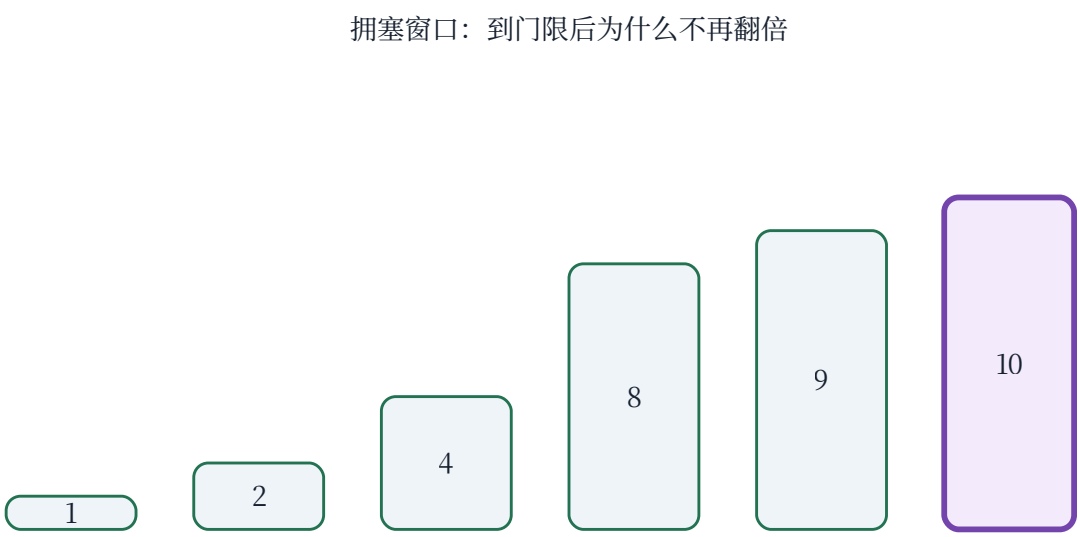
阶段/事件	典型教材规则	辨析
慢开始	每个对新数据的确认增加一定窗口，理想轮次1、2、4、8…	“慢”指从较小窗口开始，增长本身是指数式。
拥塞避免	约每个RTT增加1 MSS	线性增长，不是每收到一个ACK都再加整整1 MSS。

阶段/事件	典型教材规则	辨析
超时	降低门限，cwnd回小初始值，重新慢开始	门限常按故障前在途量一半并设下限；题目可简化为原cwnd一半。
三重复ACK	快速重传，并按算法快恢复	不应与RTO超时后直接回1的路径混同。

门限16、超时时窗口24

理想慢开始序列1、2、4、8、16；达到门限后按17、18…线性增长。若24时超时，题设按窗口一半取新门限12，cwnd回1，再1、2、4、8，达到/跨至门限附近时切换规则。具体临界轮次须看题设“达到门限”的更新约定，不能机械多翻一次至16后才处理。

图解与逐步推演：拥塞窗口：到门限后为什么不再翻倍



单独查看图片

拥塞窗口：到门限后为什么不再翻倍

查看完整步骤（也可离线阅读）

1. 第1轮开始 cwnd=1 MSS：理想轮次模型：ssthresh=8 MSS；慢开始按轮次翻倍，达到门限后按拥塞避免每轮约增加1 MSS。
2. 第2轮开始 cwnd=2 MSS：理想轮次模型：ssthresh=8 MSS；慢开始按轮次翻倍，达到门限后按拥塞避免每轮约增加1 MSS。
3. 第3轮开始 cwnd=4 MSS：理想轮次模型：ssthresh=8 MSS；慢开始按轮次翻倍，达到门限后按拥塞避免每轮约增加1 MSS。
4. 第4轮开始 cwnd=8 MSS：理想轮次模型：ssthresh=8 MSS；慢开始按轮次翻倍，达到门限后按拥塞避免每轮约增加1 MSS。

5. 第5轮开始 cwnd=9 MSS：理想轮次模型：ssthresh=8 MSS；慢开始按轮次翻倍，达到门限后按拥塞避免每轮约增加1 MSS。
6. 第6轮开始 cwnd=10 MSS：理想轮次模型：ssthresh=8 MSS；慢开始按轮次翻倍，达到门限后按拥塞避免每轮约增加1 MSS。

快恢复与拥塞曲线逐轮判读 285 先认超时还是重复确认，再确认窗口值对应轮初还是轮末。

快恢复的想法

连续重复ACK说明仍有数据穿过网络，与完全无反馈的超时相比，可用较温和的窗口缩减后继续。教材简化常设新门限为原窗口一半、cwnd降到新门限，再进入拥塞避免。标准Reno恢复期间会临时把cwnd置为ssthresh+3 MSS，并随额外重复ACK调整；收到确认新数据的ACK后退出恢复。

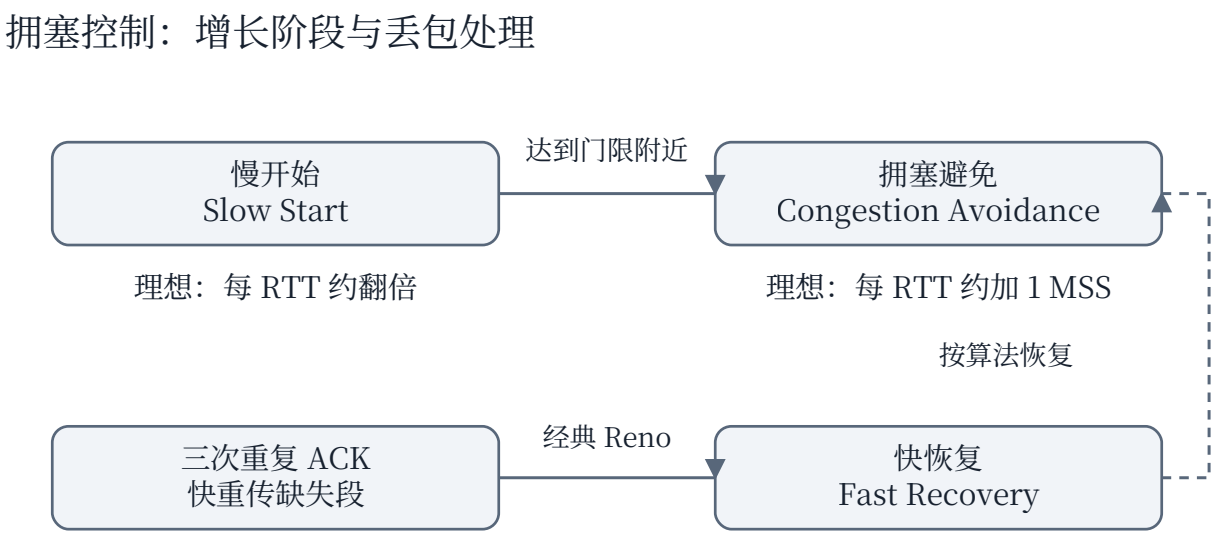
读图步骤

- 写出每轮开始可用的cwnd与rwnd，以较小者计算本轮能发多少。
- 收到新数据确认后更新窗口；区分“第n轮发送时的窗口”和“第n轮结束后的窗口”。
- 指数增长段对应慢开始，线性段对应拥塞避免；突然回很小通常表示超时，约减半通常对应快速恢复模型。
- 累计发送段数按各轮实际发送量求和；首段属于哪个轮次必须与图中时间轴一致。

别把简化模型当唯一实现

TCP拥塞控制存在多种算法。计算题若明确Tahoe/Reno、是否快恢复、初始窗口、门限及延迟确认，应严格使用给定模型；没有这些信息时，不能对所有TCP实现给出唯一的逐轮曲线。

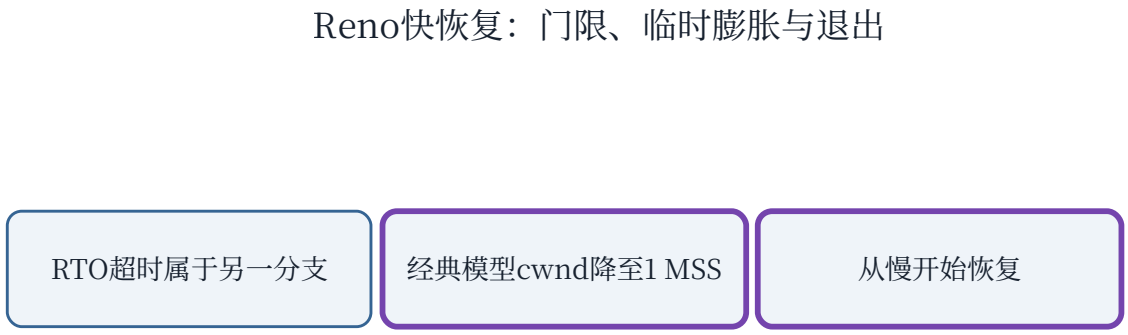
机制图解



重传超时：经典模型重新慢开始；窗口数值与门限规则以题设为准。

单独查看图片

图解与逐步推演：Reno快恢复：门限、临时膨胀与退出



单独查看图片

Reno快恢复：门限、临时膨胀与退出

查看完整步骤（也可离线阅读）

1. 收到3个重复ACK：采用RFC5681的经典Reno模型，SMSS相同；FlightSize是在途数据量，不必总等于cwnd。
2. 进入快恢复：额外3MSS是对触发恢复的三个重复ACK造成网络中已离开数据量的估计，不是将门限设为9。
3. 又收到一个重复ACK：窗口临时膨胀服务于恢复期间的ACK时钟；实际发送还受rwnd和在途量限制。
4. 收到确认新数据的ACK：本例只有一次丢段且ACK覆盖所需恢复数据；多丢段与部分ACK可涉及NewReno/SACK等规则，不能直接混用。
5. 若走超时分支：超时与三个重复ACK对cwnd的处理不同。门限按该次检测的在途量计算，不沿用这里临时膨胀后的10去机械对半。

应用层与综合通信流程

计算机网络 · 10 条

客户/服务器与 P2P 模型 286 角色描述进程关系，不只看机器硬件。

客户/服务器

服务器进程等待请求并提供服务，客户进程主动请求；服务器可并发处理多个客户，通常具有相对稳定的服务入口。建立通信后双方都可以传数据，客户先发起连接不等于服务器以后只能被动应答单个字节。

P2P

结点可同时作为服务请求者和资源提供者，直接共享内容或任务。新增结点也增加可用资源，因此有扩展优势；但结点在线状态、资源发现、信任和一致性管理更复杂。P2P也可能使用中心化的索引、登录或发现辅助，不能一概说任何P2P系统都完全没有服务器。

选择模型

集中式管理便于权限和一致性控制，但中心性能/可用性可能成为瓶颈；P2P减轻单点负载、利用分散资源，代价是动态管理。Web、FTP、邮件多采用C/S交互，文件分发可用P2P；一个系统也可混合使用。

DNS 的层次、服务器与资源记录 287 根服务器通常给出下一步线索，不保存全部主机的最终地址。

域名空间

域名由点分隔的标号组成，从右到左由高到低；最高层是根，其下包括国家/地区顶级域、通用顶级域及基础设施域等。域名本身通常不区分字母大小写。域是命名层次，区是实际由某套权威服务器负责的数据范围，两者不总相同。

服务器角色

角色	主要职责
根服务器	提供顶级域等转介信息。

角色	主要职责
顶级域服务器	给出对应下级权威服务器等信息。
权威服务器	负责其权威区的资源记录，给出权威回答或相应转介。
本地递归解析器	代表客户查询、缓存结果，不属于客户域名必经的某个固定层级。

记录与冗余

DNS不仅查IPv4地址；A对应IPv4，AAAA对应IPv6，MX用于邮件交换，NS标记权威服务器，CNAME表示别名等。主/辅助权威服务器通过区域传送等机制保持数据，提供冗余；不能把“每台主机所有信息都登记在本地递归服务器”当成规则。DNS通常使用53端口，既可用UDP也可用TCP。

DNS 递归、迭代与缓存 288 网络访问需要知道地址，不一定每次都发DNS查询包。

典型组合流程

应用先查本机缓存、配置等可用信息；需网络查询时，主机通常向本地递归解析器提出递归请求。解析器若无可用缓存，可向根、顶级域、权威服务器依次作迭代查询：每次拿到下一步线索或最终结果，再返回给客户。

递归与迭代

递归要求被询问者负责继续查询并给出最终回答或错误；迭代允许被询问者只给出下一步应找谁。不是因为“查询目标更远”就自动变成递归，关键在该次请求的责任分工。

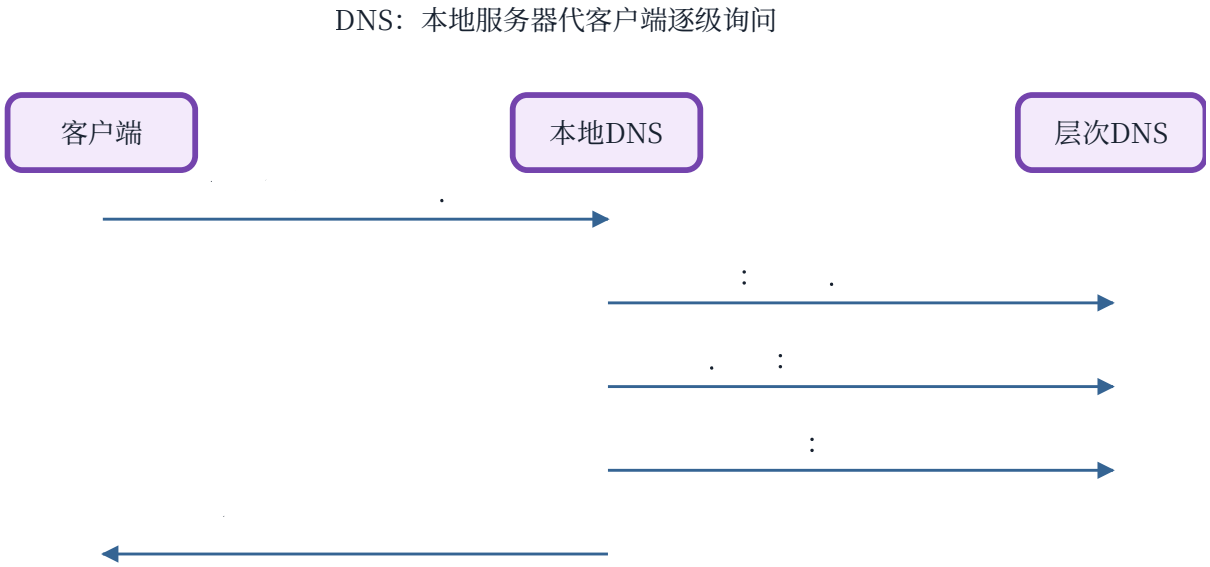
缓存与TTL

缓存减少时延和查询流量，但记录会变化，因此按TTL等有效期使用；命中后无需再逐级访问根和权威。直接使用IP、hosts或已有缓存都可能避免本次DNS网络交互。若计算访问网页总时延，必须先明确哪些记录已缓存、查询是否串行及DNS使用的RTT。

跨层辨析

DNS得到服务器IP后，若目标不在本地链路，仍需路由选择及可能的网关ARP；DNS不会顺带给出沿途每跳MAC。反过来，ARP缓存命中也不能替代域名到IP的解析。

图解与逐步推演：DNS：本地服务器代客户端逐级询问



单独查看图片

DNS：本地服务器代客户端逐级询问

查看完整步骤（也可离线阅读）

1. 起点：先确认参与者，再沿时间向下阅读。
2. 递归请求 example.com：假设客户端和本地服务器均无可用缓存，客户端向本地递归解析器请求最终答案。
3. 问根：获得 .com 服务器线索：右侧“层次DNS”合并表示根、顶级与权威服务器；向根的查询与响应在本步完成。
4. 问 .com：获得权威服务器线索：本地解析器继续发起新查询，顶级服务器通常返回权威服务器线索。
5. 问权威：获得目标记录：权威服务器返回相关记录；遇CNAME等情况还可能继续解析。
6. 返回结果并按TTL缓存：本地DNS返回结果；以后命中有效缓存时可以跳过部分或全部上游查询。图中一行代表一次问答或最终返回，不按箭头数量计算报文数。

FTP 的控制连接与主/被动数据连接 289 21是控制端口，20只属于典型主动数据连接的服务器端。

两个TCP连接

FTP以C/S方式提供文件传输、目录与权限管理，可支持匿名资源访问。控制连接由客户连到服务器TCP21，承载命令/应答，通常跨越整个会话；数据连接另行建立，传送文件或目录内容，传送结束可关闭。不能把命令和文件内容都塞入同一条“FTP21连接”模型。

主动与被动

方式	协商	数据TCP连接发起方
主动PORT	客户向服务器告知自己监听的数据地址/端口	服务器典型从TCP20连到客户指定端口。
被动PASV	服务器给客户一个监听的数据地址/端口	客户连到服务器所提供的端口；该端口通常不是20。

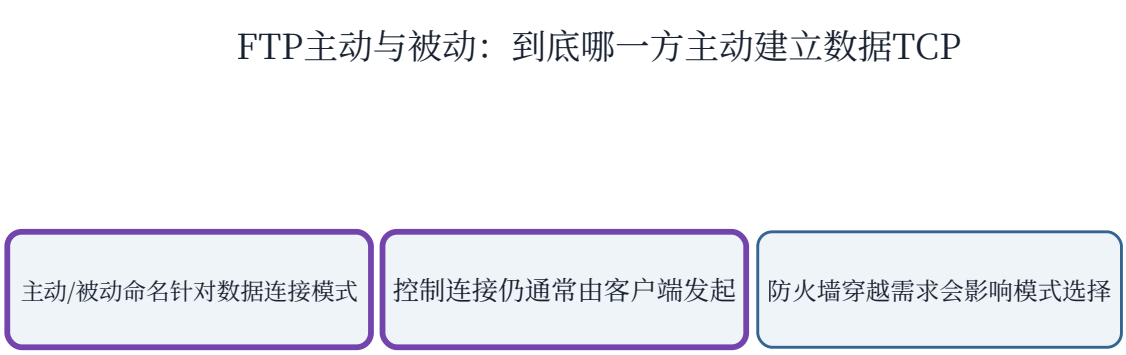
连接与传输方向不同

“主动/被动”描述数据连接由哪一侧发起，不描述文件是上传还是下载。两种模式的数据TCP连接都可以按应用请求传送相应方向的文件。NAT和防火墙对回连可能有限制，被动模式因此常更易穿过客户侧网络边界。

修正讲义笔误

讲义一处将被动模式写成使用TCP20；应结合前后流程理解为：主动模式典型使用服务器源端口20，被动模式使用协商端口。

图解与逐步推演：FTP主动与被动：到底哪一方主动建立数据TCP



单独查看图片

FTP主动与被动：到底哪一方主动建立数据TCP

查看完整步骤（也可离线阅读）

1. 控制连接先建立：FTP控制连接不直接替代每次文件传输所需的数据连接。
2. 典型主动模式：这里说典型主动FTP数据连接；客户端不是所有连接都只接服务器21。
3. 典型被动模式：被动模式的数据连接由客户端发起，服务器数据端口不固定为20。

4. 区分两个“主动”：图只演示基础FTP，FTPS、EPSV及现代扩展的额外规则另看协议与题设。

电子邮件系统与发送/读取路径 290 SMTP负责推送，POP3/IMAP负责读取；服务器可同时扮演两种角色。

组成与角色

用户代理用于撰写、显示和管理邮件；邮件服务器排队、转发、存储并报告结果。发送服务器A向接收服务器B传邮件时，A是SMTP客户、B是SMTP服务器；方向反过来角色也会互换，不能把客户与某台终端硬件永久绑定。

基本收发顺序

- 用户在代理中撰写邮件，通过配置的提交机制送到发送服务器。
- 服务器将邮件放入发送队列，按目标域的邮件路由信息寻找下一邮件服务器。
- 服务器间使用SMTP建立连接并推送邮件；接收服务器保存到收件人邮箱。
- 收件人稍后用POP3、IMAP或Web界面读取，不要求发信人与收信人同时在线。

Web邮件与端口语境

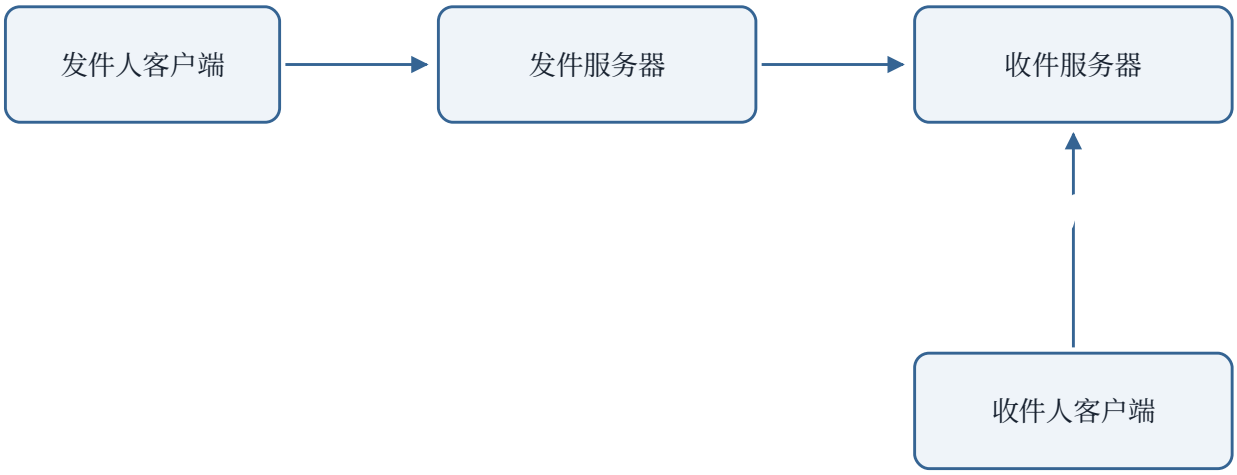
浏览器与Web邮件站点之间通常使用HTTP/HTTPS，站点后端与邮件系统之间仍可用SMTP/IMAP等。教材服务器间SMTP常用TCP25；客户提交与加密服务可能用其他标准端口。不能把“用户发送邮件”和“SMTP服务器间投递”一律混成同一条TCP25连接。

结构对照图：邮件投递与读取：SMTP、POP3、IMAP各做什么

发件客户端通常通过SMTP提交到发送服务器，服务器之间通过SMTP投递。收件人用POP3或IMAP访问其邮箱；网页邮箱的浏览器到Web服务通常使用HTTPS。

图中POP3/IMAP箭头表示客户端发起访问请求，邮件数据响应方向相反。POP3是否删除服务器副本取决于客户端会话命令和保留设置；IMAP支持服务器端状态同步。

邮件投递与读取：SMTP、POP3、IMAP各做什么



单独查看图片

邮件格式、MIME 与 SMTP 会话 291 信封负责投递，内容首部负责展示；MIME扩展内容而非取代SMTP。

邮件的层次

邮件传输有信封和内容两层：SMTP的MAIL FROM、RCPT TO等信封信息用于投递；内容含首部与主体，两者用空行分隔。To、From、Subject等是内容首部，不能假定它们与实际信封收件人永远完全一致，例如密送。

MIME解决的限制

传统SMTP的基础文本传输模型不直接适合任意二进制、多语言和多媒体内容。MIME定义内容类型、传输编码、多部分结构等，常见首部有MIME-Version、Content-Type、Content-Transfer-Encoding、Content-ID、Content-Description；它使编码后的内容可以通过邮件系统传送，并不替代SMTP投递。

SMTP三个阶段

TCP连接建立后进行SMTP问候与会话协商；MAIL FROM声明信封发件人，RCPT TO逐个指定收件人，DATA阶段传内容；服务器返回相应状态码。完成后QUIT并收到结束应答，再关闭连接。提前检查收件人可避免在明确拒收的情况下发送整封大邮件。

中继与可靠性

SMTP可以经中继服务器传递，不应把讲义“直接连接目标”示例误解为绝无中间服务器。成功接收邮件表示服务器接受后续处理责任，不等于收件人已经打开或阅读。

POP3、IMAP 与邮件保留 292 “拉取”是访问方式，是否删除由会话与客户端策略决定。

POP3

POP3使用C/S方式，教材常见TCP110，客户请求下载邮箱邮件。可配置下载后保留服务器副本，也可在协议删除和会话提交后删除；不能说所有POP3收件都必然立即删除服务器文件。

IMAP

IMAP通常把邮箱状态保留在服务器，支持文件夹、标记、搜索与部分内容获取，便于多个设备协同访问；教材常见TCP143，加密服务有不同端口。只取首部或所需部分有利于低带宽访问。

协议目的

SMTP是发送/转送；POP3与IMAP是访问已存储邮件。邮件服务器可以同时运行这些服务。应用界面中的“发件箱”“已发送”“删除”是产品行为，还需结合客户端和服务端同步策略，不能仅凭协议名字确定全部实现细节。

WWW、URL 与 HTTP 请求流程 293 URL指定访问方法和资源位置，路径的大小写由服务端语义决定。

三个基础

URL标识可访问的资源位置与方法；HTTP定义客户与服务器的请求/响应交互；HTML描述文档结构并提供超链接。WWW是运行在互联网之上的应用体系，不是互联网本身的同义词。

URL 分解

典型结构为 scheme://host:port/path，可带查询与片段。scheme与主机名通常大小写不敏感，但路径、查询等可能区分大小写；不能把“整个URL不区分大小写”当普遍规则。省略端口时依方案采用默认值，如传统HTTP80、HTTPS443。

一次访问可能经历的步骤

- 解析URL，先检查浏览器/系统缓存及是否已有可复用连接。
- 若需域名解析且未命中缓存，获取服务器IP；再由IP路由确定出接口与下一跳。
- 必要时进行ARP等本链路解析；建立所需运输/安全会话。
- 发送HTTP请求，接收响应；解析HTML时可能继续请求图像、样式、脚本等对象。
- 连接可能继续复用，不要求每收完一个网页就立刻释放TCP。

模型版本

教材HTTP/1.x通常以TCP为基础；HTTPS还涉及TLS；HTTP/3基于QUIC/UDP。做题先按明确的协议模型，不把不同版本的RTT开销混算，也不把“所有现代HTTP永远TCP”说成无例外事实。

HTTP 报文、无状态与 Cookie 294 无状态不等于服务器不能通过额外机制识别会话。

HTTP/1.x 文本报文

部分	请求	响应
开始行	方法、请求目标、版本，如 GET / index.html HTTP/1.1	版本、状态码、原因短语，如 HTTP/1.1 200 OK
首部	Host、Accept、Cookie等控制信息	Content-Type、Content-Length、Set-Cookie等元数据
空行与主体	空行分开首部与可选主体	主体可承载资源，也可能依状态/方法没有主体

无状态的准确含义

HTTP请求语义本身不要求服务器记住同一客户此前每次请求的历史。服务器仍可使用数据库、会话标识、Cookie等保存业务状态；同一客户第二次请求也可能因资源改变或鉴权不同而得到不同响应，“无状态”不意味着每次响应必须字节相同。

Cookie流程

服务器响应用Set-Cookie下发标识及属性；浏览器按域、路径、有效期和安全属性保存；后续符合条件的请求用Cookie首部带回，服务器据此关联会话数据。Cookie不等于整份用户活动数据库，往往只是关联索引；HTTP/1.x的文本格式也不能推广到HTTP/2、HTTP/3的所有线格式。

HTTP 持久连接与 RTT 计算 295 建立连接、请求响应和对象传输分开记账。

三种常见模型

模型	请求方式	典型代价
非持久	每个对象单独建立连接	若无DNS/TLS、忽略请求发送时延，一个对象约2RTT+对象发送时间。
持久非流水线	复用同一连接，前一响应到达后再发下一请求	初始建立一次，后续每个串行请求仍需往返等待。
持久流水线	同一连接可连续发多个请求而不等前一响应	可减少等待，但HTTP/1.1响应顺序等仍可能形成队头等待。

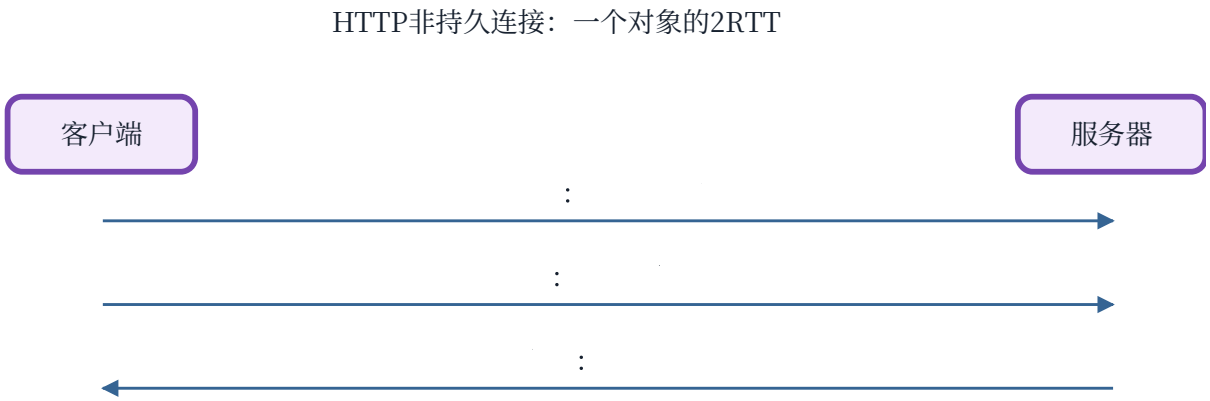
先写假设再算

2RTT模型通常把TCP建立约1RTT、请求到首响应约1RTT合计；若第三次握手可捎带HTTP请求，不应再额外给它一个完整RTT。对象发送、DNS、TLS、排队、并行连接、慢开始和缓存命中是否计入，都须按题设。

HTML 与内嵌对象

若先下载基础HTML才知道N个内嵌对象地址，不能把所有对象都从t=0并发请求。先完成HTML阶段，再依据是否持久、是否流水、并行连接上限安排后续对象。总时间取各并行支路完成的最大值，而不是不加区分地把所有时间相加。

图解与逐步推演：HTTP非持久连接：一个对象的2RTT



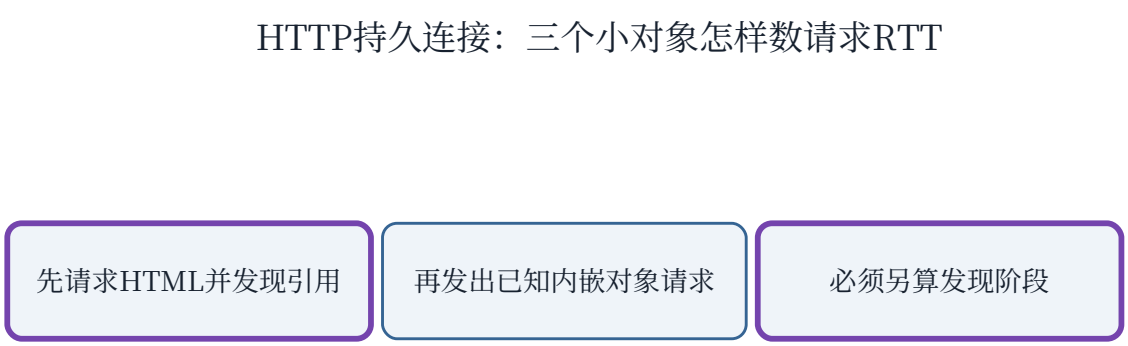
单独查看图片

HTTP非持久连接：一个对象的2RTT

查看完整步骤（也可离线阅读）

1. 起点：先确认参与者，再沿时间向下阅读。
2. 0→RTT：建立TCP连接：以普通TCP建立连接为例，忽略DNS、TLS和握手处理开销；客户端约1RTT后可发请求，第三次ACK可携带请求。
3. RTT→2RTT：HTTP请求与首字节：请求到服务器，响应首字节返回，约再用1RTT；服务器处理时间忽略。
4. 2RTT之后：传输对象数据：对象传输时间另加。复用已建连接时通常无需再算TCP握手RTT；并行、流水线和TLS会改变总时间模型。

图解与逐步推演：HTTP持久连接：三个小对象怎样数请求RTT



单独查看图片

HTTP持久连接：三个小对象怎样数请求RTT

查看完整步骤（也可离线阅读）

1. 计时前提：只比较理想化HTTP/1.1请求方式，不把建立连接的RTT重复算到每个对象。
2. 不采用流水线：后一个请求等前一个响应完成后才发，串行共约3RTT。
3. 理想流水线：在忽略数据传输与处理、请求可紧接发送的模型下，三个响应可在约1RTT后开始返回；实际完成时间仍受对象大小与队头阻塞影响。
4. 对象先依赖基础HTML：若对象URL必须先从HTML中解析，不能在收到HTML前就假定全部请求已发出；HTTP/2多路复用等模式需另建模型。